

应用密码学讲义

授课教师：林东岱 助教：王昭月

讲议初稿，请将错误发送至 ddlin@iie.ac.cn

2024 年 12 月

中国科学院大学
University of Chinese Academy of Sciences

前言

计算机和通信技术的发展与应用，方便了人们的工作、学习和生活。另一方面，大量的数据包括个人信息、商务信息等各种重要信息都需要经过网络传输达到对方。在传输的过程，任何的泄露和差错都可能造成重大的损失。如何保证信息的保密性就是一个亟待解决的问题。

信息的保密是信息安全的一个重要方面。保密的目的是防止敌手破译信息系统中的机密信息。加密是实现信息保密的一种手段。加密技术可使一些重要数据存储在一台不安全的计算机上，或在不安全的信道上传输。

密码学是研究如何保障信息的机密性、真实性、不可否认性等的一门学科，因此密码学越来越受到人们的重视。密码学的研究与应用已有几千年的历史，但作为一门科学是以 1949 年 Shannon 发表的《保密通信的信息理论》为标志开始的。目前很多国家和地区都成立了密码学学会，这些学会定期召开学术会议进行学术交流，促进了密码学的研究与应用。

我国政府十分重视密码技术和产业的发展，并把它列入我国优先发展的领域。特别是随着计算机和计算机网络的广泛应用，密码学已经从战场走向了商场，尤其是电子政务、电子商务的发展给密码学开拓了广泛的应用空间，从而使密码学进入了空前繁荣的阶段。而且，我国于近年在高等院校开设了信息安全专业和密码学专业。为促进密码学的教学，使学生掌握密码学的主要理论和技术，特编写这本教材。

全书共 11 章，包括：经典密码、密码学信息理论、序列密码、分组密码体制、公开密钥体制、数字签名以及密码协议等内容。为贴近教学实际，每章后面附有习题，本书可作为计算机、信息安全和密码学专业的本科生教材。本书不仅适用于教学，也可以供相关读者参考。

由于编者水平有限，对一些问题的理解和叙述或有肤浅和错误之处，诚挚欢迎读者提出宝贵意见。

编 者

目录

1 密码学概述 1
1.1 密码学的基本概念 1
1.2 密码体制 2
1.3 密码分析 3
1.3.1 攻击密码系统的方法 4
1.3.2 破译密码的类型 5
1.4 密码体制的安全性 6
第 1 章 习题 7

2 经典密码学 8
2.1 替换密码体制 8
2.1.1 单表替换密码 9
2.1.2 多表替换密码 13
2.2 置换密码体制 16
2.3 经典密码体制的分析 17
2.3.1 统计特性 17
2.3.2 单表密码体制的统计分析 . 19
2.3.3 多表密码体制的统计分析 . 21
第 2 章 习题 24

3 密码学的信息论基础 27
3.1 概率论基础 27
3.2 完全保密性 28
3.3 信息的度量 – 信息熵 30
3.3.1 信息论的相关概念 30
3.3.2 信息的度量 31
3.4 熵的基本性质 34
3.5 伪密钥与唯一解距离 38
3.6 乘积密码体制 40
第 3 章 习题 42

4 序列密码 43
4.1 序列密码的基本结构 43
4.1.1 序列密码的加解密方式 . . . 43
4.1.2 密钥流与密钥流生成器 . . . 44
4.2 线性反馈移位寄存器序列 46
4.2.1 线性移位寄存器的幂级数表示 48
4.2.2 随机性概念与 m-序列的伪随机性 51
4.3 滤波生成器和非线性组合生成器 . . 53
4.3.1 非线性滤波生成器 53
4.3.2 非线性组合生成器 54
4.4 序列密码算法举例 55
4.4.1 E0 序列密码算法 55
4.4.2 RC4 算法 56
4.4.3 Trivium 算法 57
4.4.4 Grain 型序列密码算法 . . . 60
4.4.5 ZUC 序列密码算法 62
4.5 序列密码攻击方法简介 65
4.5.1 仿射逼近攻击 66
4.5.2 相关攻击 66
4.5.3 代数攻击 67
第 4 章 习题 68

5 分组密码 70
5.1 分组密码的基本概念 70
5.2 数据加密标准 DES 71
5.2.1 DES 加密算法概述 71
5.2.2 DES 加密过程描述 71
5.2.3 DES 解密过程 74
5.2.4 DES 子密钥生成 77
5.2.5 DES 的安全性 77
5.2.6 三重 DES 79
5.3 高级加密标准 AES 79
5.3.1 AES 的加密变换 80
5.3.2 AES 的解密变换 84
5.3.3 AES 密钥编排 86
5.4 国际数据加密算法 IDEA 86
5.4.1 IDEA 算法描述 87
5.4.2 IDEA 算法的解密 88
5.4.3 IDEA 密钥生成 88
5.5 SMS4 密码算法 89
5.5.1 算法描述 89
5.5.2 密钥扩展 91
5.6 分组密码的工作模式 92
5.6.1 电子密码本模式（ECB） . . 93
5.6.2 密码分组链接模式（CBC） 94
5.6.3 密码反馈模式（CFB） . . . 94
5.6.4 输出反馈模式（OFB） . . . 96

5.6.5 记数器模式（CTR） 96
5.7 分组密码分析技术 98
5.7.1 代换-置换网络 98
5.7.2 线性密码分析 99
5.7.3 差分密码分析 109
第 5 章 习题 115

6 Hash 函数 117
6.1 Hash 函数的性质 117
6.1.1 Hash 函数的性质 117
6.1.2 生日攻击 117
6.1.3 迭代 Hash 函数的结构 . . . 119
6.2 Hash 函数实例 120
6.2.1 MD5 散列函数 120
6.2.2 安全 Hash 算法 126
6.3 Hash 函数的应用举例 129
第 6 章 习题 130

7 消息认证码 131
7.1 消息认证码的构造 131
7.1.1 基于分组密码的 MAC 131
7.1.2 基于带密钥的 Hash 函数的 MAC 133
7.2 MAC 函数的安全性 134
7.3 消息认证码的应用 135
第 7 章 习题 135

8 公钥密码 137
8.1 公钥密码的基本概念 137
8.1.1 公钥密码体制的原理 137
8.1.2 公钥密码算法应满足的要求 139
8.1.3 对公钥密码的攻击 139
8.2 RSA 密码体制 140
8.2.1 加密算法描述 140
8.2.2 RSA 算法中的计算问题 . . . 142
8.2.3 对 RSA 的攻击 143
8.2.4 RSA-OAEP 加密标准 146
8.3 ElGamal 密码体制 147
8.3.1 ElGamal 算法 147
8.3.2 ElGamal 公钥密码体制的安全性 148
8.4 椭圆曲线密码体制 148
8.4.1 Diffie-Hellman 公钥系统 . . 149
8.4.2 Menezes-Vanstone 公钥密码体制 149
8.4.3 椭圆曲线密码体制的优点 . 151
8.5 基于身份的加密体制 151
8.5.1 8.5.1 基于身份的密码学概述 151
8.5.2 基于身份的加密方案的定义 154
8.5.3 BF-IBE 方案 154
第 8 章 习题 155

9 数字签名 156
9.1 数字签名的基本概念 156
9.2 RSA 签名 157
9.2.1 利用 RSA 密码实现数字签名 157
9.2.2 对 RSA 数字签名的攻击 . . 157
9.2.3 RSA 签名标准 PSS 158
9.3 数字签名标准 DSS 160
9.3.1 DSS 的基本方式 160
9.3.2 数字签名算法 DSA 160
9.4 其他数字签名方案 161
9.4.1 离散对数签名体制 161
9.4.2 利用椭圆曲线密码实现数字签名 165
9.5 基于身份的签名方案 166
9.5.1 Shamir 的基于身份的数字签名方案 166
9.5.2 Cha-Cheon 的基于身份的数字签名方案 167

10 密码学侧信道攻击 168
10.1 基本概念 168
10.2 入侵型攻击 168
10.2.1 一般的篡改方法 168
10.2.2 保护措施 169
10.3 错误攻击 169
10.3.1 简单错误分析攻击 169
10.3.2 差分错误分析（DFA）攻击 170
10.3.3 错误引入 171
10.3.4 错误攻击的对策 171
10.4 时间攻击 172
10.4.1 对平方-乘算法的时间攻击 . 172
10.4.2 对多位窗口平方-乘算法的时间攻击 173
10.4.3 时间攻击的对策 175

10.5 能量攻击 175
10.5.1 简单能量分析（SPA）攻击 . 175
10.5.2 差分能量分析（DPA）攻击 176
10.5.3 能量攻击的对策 177
10.6 电磁攻击 178
第 10 章 习题 178

11 密码协议 179
11.1 什么是密码协议 179
11.2 密码协议的安全性 180
11.3 身份认证协议 181
11.3.1 身份认证概述 181
11.3.2 基于口令的认证 182
11.3.3 基于对称密码的认证 186
11.4 秘密共享 187
11.4.1 秘密共享的思想 187
11.4.2 Shamir 门限秘密共享方案 . 187
11.5 阈下信道 188
11.5.1 阈下信道的基本原理 188
11.5.2 ElGamal 签名的阈下信道 . . 189
11.6 比特承诺 190
11.6.1 什么是比特承诺 190
11.6.2 使用对称密码算法的比特承诺 191
11.6.3 使用单向函数的比特承诺 . 191
11.7 零知识证明 192
第 11 章 习题 194

A 密码数学基础 195
A.1 数论基础 195
A.2 抽象代数初步 205
A.2.1 群 205
A.2.2 环和域 206
A.2.3 多项式环 209
A.3 有限域初步 210
A.3.1 有限域的特征性质 210
A.3.2 不可约多项式的根 212
A.3.3 元素的迹 213
A.3.4 多项式的阶和本原多项式 . 214
第 A 章 习题 214

第 1 章 密码学概述

密码技术是一门古老的技术，大概自人类社会出现战争便产生了密码，它用于保护军事和外交通信可以追溯到几千年以前。但自从密码技术诞生至第二次世界大战结束，对于公众而言，密码技术始终处于一种未知的保密状态，常与军事、机要、间谍等工作联系在一起，让人在感到神秘之余，又有几分畏惧。然而，信息技术的发展迅速改变了这一切，随着计算机和通信技术的迅猛发展，大量的敏感信息通过公共通信设施或计算机网络进行交换。特别是 Internet 的广泛应用、电子商务和电子政务的迅速发展，越来越多的信息需要严格保密，如：银行账号、个人隐私等。正是这种对信息的机密性和真实性的需求，密码学才逐渐揭去了神秘的面纱。

密码学的发展历史大致可分为四个阶段：

- 第一个阶段从古代到 1949 年。这一时期通常称为古典密码时期，这时期的密码技术可以说是一种艺术，而不是一种科学，密码学家通常凭直觉和信念来进行密码设计和分析，没有科学的理论基础和推理证明。
- 第二个阶段从 1949 年到 1976 年。1949 年 Shannon（香农）发表的“保密系统的信息理论”一文产生了信息论，信息论为私钥密码系统建立了理论基础，从此密码学成为了一门科学。但密码学并没有因为科学理论的产生而丧失艺术的一面，一直到今天，密码学仍是一门具有艺术性的科学。
- 第三个阶段为 1976 年到 1984 年。1976 年 Diffie 和 Hellman 发表了《密码学新方向》一文导致了密码学上的一场革命。他们首次证明了在发送端和接收端无密钥传输的保密通信是可能的，从而开创了公钥密码学的新纪元。
- 第四个阶段为 1984 年至今。1984 年 Goldwasser 和 Micali 首次提出了可证明安全的思想。他们将概率引入了密码学，强调多项式时间和可忽略的成功概率等，认为在公认的计算复杂度理论假设下，安全性是可以证明的。随着对可证明安全性的进一步研究，密码学界意识到它不但对密码学的理论影响重大，且对密码学实践也有重要的意义，目前已成为了国内外密码学界最为关注的问题之一，同时可证明安全性也已成为了设计密码协议的公认要求。

1.1 密码学的基本概念

研究密码编制的科学称为密码编码学 (Cryptography)，研究密码破译的科学称为密码分析学 (Cryptanalysis)，密码编码学和密码分析学共同组成密码学 (Cryptology)。

密码技术的基本思想是伪装信息，使未授权者不能理解它的真实含义。所谓伪装就是对数据进行一组可逆的数学变换。伪装前的原始数据称为明文，伪装后的数据称为密文，伪装的过程称为加密，加密在加密密钥的控制下进行。用于对数据加密的一组数学变换称为加密算法。发送者将明文数据加密成密文，然后将密文数据送入通信网络传输或存入计算机文件，而且只给合法接收者分配密钥。合法接收者接收到密文后，施行与加密变换相逆的变换，去掉密文的伪装恢复出明文，这一过程称为解密。解密在解密密钥的控制下进行。用于解密的一组数学变换称为解密算法。加密算法和解密算法通常都是在一组密钥的控制下进行的，分别称为加密密钥和解密密钥。



图 1.1: 加密和解密

图1.1是加密和解密的示意图，通常明文用 P 或 M 表示，密文用 C 表示。加密函数 E 作用于 P 得

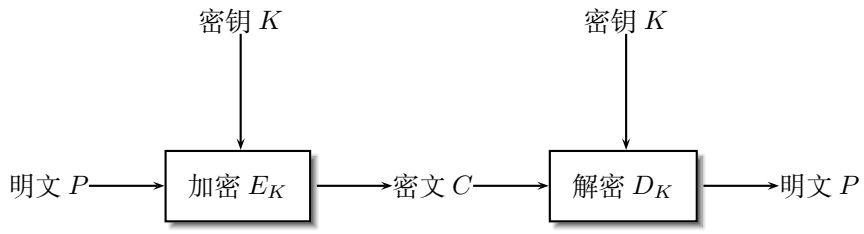


图 1.2: 使用一个密钥的加/解密

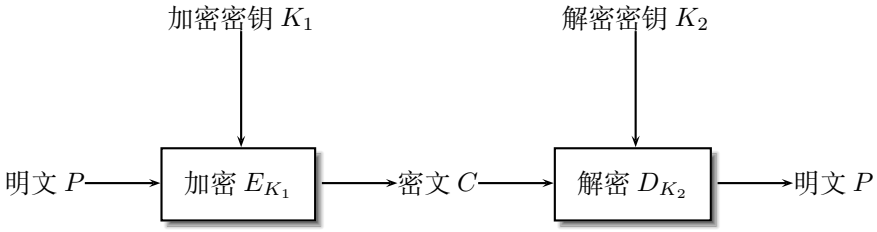


图 1.3: 使用两个密钥的加/解密

到密文 C，可以表示为：$E(P) = C$。相反地，解密函数 D 作用于 C 产生 P：$D(C) = P$。先加密后再解密消息，原始的明文将恢复出来，故有：$D(E(P)) = P$。

加密时可以使用一个参数 K，称此参数 K 为加密密钥。K 可以是很多数值里的任意值。密钥 K 的可能值的范围叫做密钥空间。如果加密和解密运算都使用这个密钥（即运算都依赖于密钥，并用 K 作为下标表示），如图1.2所示。这样，加/解密函数现在变成：

$$E_K(P) = C$$
$$D_K(C) = P.$$

这些函数具有下面的特性：$D_K(E_K(P)) = P$。

有些算法使用不同的加密密钥和解密密钥（见图1.3），即加密密钥 K_1 与解密密钥 K_2 不同，在这种情况下：

$$E_{k_1}(P) = C,$$
$$D_{k_2}(C) = P,$$
$$D_{k_2}(E_{k_1}(P)) = P.$$

1.2 密码体制

一个密码系统，通常简称为密码体制 (Cryptosystem)，由五部分组成 (如图1.4所示):

(1) 明文空间 M，它是全体明文的集合；
(2) 密文空间 C，它是全体密文的集合；
(3) 密钥空间 K，它是全体密钥的集合。其中每一个密钥 K 均由加密密钥 K_e 和解密密钥 K_d 组成，即 $K = \langle K_e, K_d \rangle$；
(4) 加密算法 E，它是一族由 M 到 C 的加密变换；
(5) 解密算法 D，它是一族由 C 到 M 的解密变换。

对于每一个确定的密钥，加密算法将确定一个具体的加密变换，解密算法将确定一个具体的解密变换，而且解密变换就是加密变换的逆变换。对于明文空间中的每一个明文 M，加密算法 E 在密钥 K_e 的

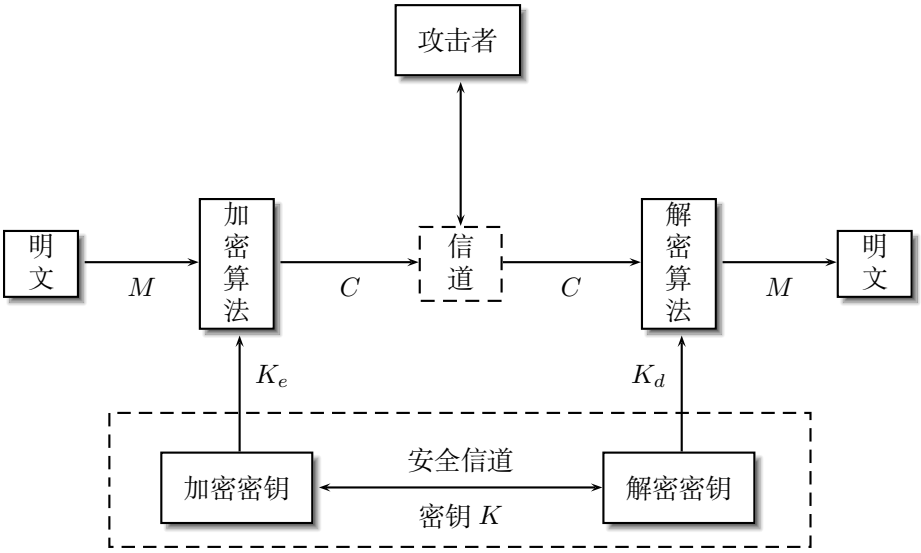


图 1.4: 密码体制

控制下将明文 M 加密成密文 C:

$$C = E(M, K_e).$$

而解密算法 D 在密钥 K_d 的控制下将密文 C 解密出同一明文 M:

$$M = D(C, K_d) = D(E(M, K_e), K_d).$$

如果一个密码体制的 $K_d = K_e$，或由其中一个很容易推出另一个，则称为单密钥密码体制或对称密码体制或传统密码体制，否则称为双密钥密码体制。如果在计算上 K_d 不能由 K_e 推出，这样将 K_e 公开也不会损害 K_d 的安全，于是便可将 K_e 公开，这种密码体制称为公开密钥密码体制。公开密钥密码体制的概念于 1976 年由 Diffie 和 Hellman 提出，它的出现是密码发展史上的一个里程碑。

根据对明文和密文的处理方式和密钥的使用不同，可将密码体制分为分组密码和序列密码体制。

序列密码又称为流密码，是将明文消息字符串逐位的加密成密文字符。以二元加法序列密码为例，设 $m_1, m_2, \ldots, m_k, \ldots$ 是明文字符，$z_1, z_2, \ldots, z_k \ldots$ 是密钥流；那么加密变换为

$$c_k = m_k \oplus z_k, \ \ k \geq 1.$$

$c_1, \ldots, c_k, \ldots$ 是密文字符序列。

分组密码就是将明文消息序列 $m_1, m_2, \ldots, m_k, \ldots$ 分成等长的消息组

$$(m_1, m_2, \ldots, m_n), (m_{n+1}, \ldots, m_{2n}), \ldots$$

在密钥的控制下。按固定的加密算法 E_k，一组一组地进行加密。加密后输出等长的密文组

$$(c_1, c_2, \ldots, c_n), (c_{n+1}, \ldots, c_{2n}), \ldots$$

分组密码和序列密码的本质区别在于其加密方式：序列密码是逐比特加密，而分组密码是按照消息组一组一组的加密，每一组加密的变换是相同的，因而相同的明文组对应的密文组必然是相同的。

1.3 密码分析

在信息的传输和处理过程中，除了意定的接收者外，还有非授权接收者，他们通过各种办法（如搭线窃听、电磁窃听、声音窃听等）来窃取信息。他们虽然不知道系统所用的密钥，但通过分析，可能从截

获的密文中推断出原来的明文，这一过程称为密码分析。密码分析就是破解密码。

对一个保密系统采取截获密文进行分析的这类攻击称为被动攻击 (passive attack)。被动攻击本质上是在传输中偷听或监视，其目的是从传输中获得信息。两类被动攻击分别是析出消息内容和通信量分析。

析出消息内容容易理解：电话交谈、电子邮件消息和传送的文件可能包括敏感或机密信息，我们希望防止对手从这些传输中得知相关内容。

第二种被动攻击是通信量分析，它更为微妙。假定我们用某种方法屏蔽了消息内容，即使敌手获取了该信息也无法从消息中提取信息。屏蔽内容的常用技术是加密。如果我们已经用加密进行保护，对手也许还能观察这些消息的模式。该对手能够测定通信主机的位置和标识，能够观察被交换消息的频率和长度。这些信息对猜测正在发生的通信的性质是有用的。

密码系统还可能遭受的另一类攻击是主动攻击 (active attack)。非法入侵者主动向系统干扰，采用删除、更改、增添、重放和伪造等方法向系统加入假消息。主动攻击还能进一步划分为四类：伪装、重放、篡改消息和拒绝服务。

- 伪装，即一个实体被另一个实体假冒。这种攻击引诱受害者相信与之通信的是另一实体。
- 重放涉及一个数据单元的被动获取以及后继的重传，以产生一个未授权的效果。
- 消息的篡改只不过意味着一个合法消息的某些部分被改变，或消息被延迟或改变顺序，以产生一个未授权效果。
- 拒绝服务，即长时间的阻止服务。拒绝服务会阻止通信设施的正常使用或管理。这种攻击可能具有一种特定目标，例如，一个实体可能抑制所有的消息指向某个特殊的目的地 (如安全审计服务)。另一种形式的拒绝服务是使整个网络崩溃，或者通过使网络不能工作的手段，或者滥发消息使之过载，以达到降低性能的目的。

1.3.1 攻击密码系统的方法

对密码进行分析的尝试称为攻击。Kerckhoffs 最早在 19 世纪阐明密码分析的一个基本假设，这个假设就是秘密必须完全寓于密钥中。Kerckhoffs 假设密码分析者已有密码算法及其实现的全部详细资料。

密码分析者攻击密码系统的方法主要有以下三种。

（1）穷举攻击

所谓穷举攻击是指密码分析者采用依次试遍所有可能的密钥对所获密文进行解密，直至得到正确的明文；或者用一个确定的密钥对所有可能的明文进行加密，直至得到所获得的密文。显然，理论上对于任何实用密码只要有足够的资源，都可以用穷举攻击将其攻破。

穷举攻击所花费的时间等于尝试次数乘以一次解密 (加密) 所需的时间。显然可以通过增大密钥量或加大解密 (加密) 算法的复杂性来对抗穷举攻击。当密钥量增大时，尝试的次数必然增大。当解密 (加密) 算法的复杂性增大时，完成一次解密 (加密) 所需的时间增大。从而使穷举攻击在实际上不能实现。穷举攻击是对密码的一种最基本的攻击。

（2）统计分析攻击

所谓统计分析攻击就是指密码分析者通过分析密文和明文的统计规律来破译密码。统计分析攻击在历史上为破译密码作出过极大的贡献。许多古典密码都可以通过分析密文字母和字母组的频率和其他统计参数而破译。对抗统计分析攻击的方法是设法使明文的统计特性不带入密文。这样，密文不带有明文的痕迹，从而使统计分析攻击成为不可能。能够抵抗统计分析攻击已成为近代密码的基本要求。

（3）数学分析攻击

所谓数学分析攻击是指密码分析者针对加解密算法的数学基础和某些密码学特性，通过数学求解的方法来破译密码。数学分析攻击是对基于数学难题的各种密码的主要威胁。为了对抗这种数学分析攻击，应当选用具有坚实数学基础和足够复杂的加解密算法。

1.3.2 破译密码的类型

此外，根据密码分析者可利用的数据资源来分类，可将破译密码的类型分为以下四种：

（1）惟密文攻击 (Ciphertext-only attack)

所谓惟密文攻击是指密码分析者有一些消息的密文，这些消息都用同一加密算法加密。密码分析者的任务是恢复尽可能多的明文，或者最好是能推算出加密消息的密钥来，以便可采用相同的密钥解出其他被加密的消息。因为密码分析者所能利用的数据资源仅为密文，因此这是对密码分析者最不利的情况。

> 已知：$C_1 = E_K(P_1), C_2 = E_K(P_2), \ldots, C_i = E_K(P_i),$
> 推导出：$P_1, P_2, \ldots, P_i, K$，或者找出一个算法从 $C_{i+1} = E_K(P_{i+1})$ 推出 P_{i+1}。

（2）已知明文攻击 (Known-plaintext attack)

所谓已知明文攻击是指密码分析者不仅可得到一些消息的密文，而且也知道这些消息的明文。分析者的任务就是用加密信息推出用来加密的密钥或导出一个算法，此算法可以对用同一密钥加密的任何新的消息进行解密。

> 已知：$P_1, C_1 = E_K(P_1), P_2, C_2 = E_K(P_2), \ldots, P_i, C_i = E_K(P_i),$
> 推导出：密钥 K，或从 $C_{i+1} = E_K(P_{i+1})$ 推出 P_{i+1} 的算法。

例如，密码分析者可能知道从用户终端送到计算机的密文数据从一个标准词 “LOGIN” 开头。又例如，加密成密文的计算机程序文件特别容易受到这种攻击。这是因为诸如 “BEGIN”，“END”，“IF”，“THEN”，“ELSE” 等词的密文有规律地在密文中出现，密码分析者可以合理地猜测它们。近代密码学认为，一个密码仅当它能经得起已知明文攻击时才是可取的。

（3）选择明文攻击 (Chosen-plaintext attack)

所谓选择明文攻击是指密码分析者不仅可得到一些消息的密文和相应的明文，而且他们也可选择被加密的明文。这比已知明文攻击更有效。因为密码分析者能选择特定的明文块去加密，那些块可能产生更多关于密钥的信息。这是对密码分析者十分有利的情况。分析者的任务是推出用来加密消息的密钥或导出一个算法，此算法可以对用同一密钥加密的任何新的消息进行解密。

> 已知：$P_1, C_1 = E_K(P_1), P_2, C_2 = E_K(P_2), \ldots, P_i, C_i = E_K(P_i)$，其中 $P_1, P_2, \ldots, P_i$ 是由密码分析者选择的。
> 推导出：密钥 K，或从 $C_{i+1} = E_K(P_{i+1})$ 推出 P_{i+1} 的算法。

如果分析员能够设法在源系统插入由该分析员选择的消息，则可进行选择明文攻击。一般而言，如果分析员能够选择消息来加密，那么他可能故意选择能够期待揭露出密钥结构的模式。

计算机文件系统和数据库系统特别容易受到这种攻击，因为用户可以随意选择明文，并获得相应的密文文件和密文数据库。

（4）自适应选择明文攻击 (adaptive-chosen-plaintext attack)

这是选择明文攻击的特殊情况。密码分析者不仅能选择被加密的明文，而且也能基于以前加密的结果修正这个选择。在选择明文攻击中，密码分析者还可以选择一大块被加了密的明文。而在自适应选择密文攻击中，他可选取较小的明文块，然后再基于第一块的结果选择另一明文块，以此类推。

另外还有至少三类其它的密码分析攻击。

（5）选择密文攻击 (Chosen-ciphertext attack)

所谓选择密文攻击是指密码分析者能够选择不同的密文，并获得相应的解密的明文。例如密码分析者存取一个防窜改的自动解密盒，密码分析者的任务是推导出密钥。

> 已知：C_1, $P_1 = D_K(C_1)$, C_2, $P_2 = D_K(C_2)$, $\ldots$, C_i, $P_i = D_K(C_i)$，其中 $C_1, C_2, \ldots, C_i$ 是由密码分析者选择的。
> 推导出：密钥 K。

（6）选择密钥攻击 (Chosen-key attack)

这种攻击并不表示密码分析者能够选择密钥，它只表示密码分析者具有不同密钥之间的关系的有关知识。这种方法有点奇特和晦涩，不是很实际。

（7）软磨硬泡攻击

密码分析者威胁、勒索，或者折磨某人，直到他给出密钥为止。行贿有时称为购买密钥攻击。这些是非常有效的攻击，并且经常是破译算法的最好途径。

1.4 密码体制的安全性

通常一个安全的密码体制应该具有如下几条性质：

(1) 从密文恢复明文应该是难的，即使分析者知道明文空间（如明文是英语）。
(2) 从密文计算出明文部分信息应该是难的。
(3) 从密文探测出简单却有用的事实应该是难的，如相同的信息被发送了两次。

从敌手对密码体制攻击的效果看，敌手可能达到以下结果：

(1) 完全攻破。敌手找到了相应的密钥，从而可以恢复任意的密文。
(2) 部分攻破。敌手没有找到相应的密钥，但对于给定的密文，敌手能够获得明文的特定信息。
(3) 密文识别。如对于两个给定的不同明文及其中一个明文的密文，敌手能够识别出该密文对应于哪个明文；或者能够识别出给定明文的密文和随机字符串。

评价密码体制安全性有不同的途径，包括无条件安全性、计算安全性、可证明安全性。

(1) 无条件安全性。如果密码分析者具有无限的计算能力，密码体制也不能被攻破，那么这个密码体制就是无条件安全的。例如只有单个的明文用给定的密钥加密，移位密码和代换密码都是无条件安全的。一次一密乱码本（One-time Pad）对于唯密文攻击是无条件安全的，因为敌手即使获得很多密文信息、具有无限的计算资源，仍然不能获得明文的任何信息。如果一个密码体制对于唯密文攻击是无条件安全的，我们称该密码体制具有完善保密性（perfect secrecy）。如果明文空间是自然语言，所有其它的密码系统在唯密文攻击中都是可破的，因为只要简单地一个接一个地去试每种可能的密钥，并且检查所得明文是否都在明文空间中，这种方法叫做蛮力攻击（Brute Force Attack）。
(2) 计算安全性。密码学更关心在计算上不可破译的密码系统。如果攻破一个密码体制的最好的算法用现在或将来可得到的资源都不能在足够长的时间内破译，这个密码体制被认为在计算上是安全的。目前还没有任何一个实际的密码体制被证明是计算上安全的，因为我们知道的只是攻破一个密码体制的当前的最好算法，也许还存在一个我们现在还没有发现的更好的攻击算法。实际上，密码体制对某一种类型的攻击（如蛮力攻击）是计算上安全的，但对其他类型的攻击可能是计算上不安全的。
(3) 可证明安全性。另一种安全性度量是把密码体制的安全性归结为某个经过深入研究的数学难题。例如如果给定的密码体制是可以破解的，那么就存在一种有效的方法解决大数的因子分解问题，而因子分解问题目前不存在有效的解决方法，于是称该密码体制是可证明安全的，即可证明攻破该密码体制比解决大数因子分解问题更难。可证明安全性只是说明密码体制的安全与一个问题是相关的，并没有证明密码体制是安全的，可证明安全性也有时候被称为归约安全性。

第 1 章 习题

1. 你是如何理解密码学，密码编码学和密码分析学的?
2. 什么是主动攻击，什么是被动攻击?
3. 攻击密码系统的方法有哪些?
4. 一个密码体制由哪五部分组成?
5. 什么是计算安全? 可证明安全?

第 2 章 经典密码学

经典密码体制是加密和解密时使用相同密钥的密码系统，在这种系统里，对所有 $E_k \in E$，$k \in K$，都有 $D_k = E_k^{-1}$，其中 $D_k \in D$。

最为人们所熟悉的古典加密方法，莫过于隐写术。它通常将秘密消息隐藏于其它消息中，使真正的秘密通过一份无伤大雅的消息发送出去。隐写术分为两种，语言隐写术和技术隐写术。技术方面的隐写比较容易想象：比如不可见的墨水，洋葱法和牛奶法也被证明是普遍且有效的方法（只要在背面加热或紫外线照射即可复现）。语言隐写术与密码编码学关系比较密切，它主要提供两种类型的方法：符号码和公开代码。

符号码是以可见的方式，如手写体字或图形，隐藏秘密的书写。在书或报纸上标记所选择的字母，比如用点或短划线，这比上述方法更容易被人怀疑，除非使用显隐墨水，但此方法易于实现。

一份秘密的信件或伪装的消息要通过公开信道传送，需要双方事前约定，也就是需要一种公开代码。这可能是保密技术的最古老形式，公开文献中经常可以看到。东方和远东的商人和赌徒在这方面有独到之处，他们非常熟练地掌握了手势和表情的应用。在美国的纸牌骗子中较为盛行的方法有：手拿一支烟或用手挠一下头，表示所持的牌不错；一只手放在胸前并且跷起大拇指，意思是“我将赢得这局，有人愿意跟我吗”。“右手手掌朝下放在桌子上”，表示“是”，手握成拳头表示“不”。

特定行业或社会阶层经常使用的语言，往往被称为行话。一些乞丐、流浪汉及地痞流氓使用的语言还被称为黑话，它们是这些社会群体的护身符。其实也是利用了伪装，伪装的秘密因此也称为专门隐语。

公开代码的第二种类型就是利用虚码和漏格进行隐藏：隐藏消息的规则比较常见的有：“某个特定字符后的第几个字符”，比如空格后的下一个字母；更好一点的还有空格后的第三个字母，或者标点符号后的第三个字母。

漏格方法可以追溯到卡达诺（Cardano，1550 年）时代，这是一种容易掌握的方法，但不足之处是双方需要相同的漏格，特别是战场上的士兵，使用时不太方便。卡尔达诺漏格板是一张用硬质材料 (如硬纸、羊皮、金属等) 做成的板，上面挖了一些长方形的孔，即漏格。如图2.1：

大	风	渐	起	，	寒	流	攻	击	着	我	们	的	肌	体	，	雪	花	从	天
空	中	落	下	，	预	示	明	天	5	点	的	活	动	，	开	始	时	会	有
困	难	。																	
							攻	击											
							明	天	5	点					开	始			
		。																	

图 2.1: 卡尔达诺漏格板

本节简单介绍了几种古典密码体制，这些密码现在已经很少使用了，但研究这些密码的原理，对于理解、构造和分析现代实用密码还是很有用的。

2.1 替换密码体制

替换密码就是将明文字母表中的每个字符替换为密文字母表中的字符，代替后的各字母保持原来位置。这里对应密文字母可能是一个，也可能是多个。接收者对密文进行逆向替换即可得到明文。有四种类型的文字替换密码体制。

2.1.1 单表替换密码

单表替换密码又称为单字母代替，明文字母表中的一个字符对应密文字母表中的一个字符。加密过程中是从明文字母表到密文字母表的一一映射。这是所有加密中最简单的方法。即令明文 $m = m_0, m_1 \ldots$，则相应密文为

$$c = E(m) = c_0 c_1 \cdots = f(m_0) f(m_1) \cdots$$

单表替换密码是很容易破译的，因为它没有把明文的不同字母的出现频率掩盖起来。可以使用统计攻击。

以下是几种常用的单表替换密码。

1、加同余密码：也叫移位代换密码 (shift substitution cipher)，是最简单的一类代换密码，其加密变换为：

$$E_k(i) = (i + k) \mod q = j, \ 0 \le i, j < q, \ K = \{k \mid 0 \le k \le q\}.$$

式中 $\mod q$ 表示以 q 除得的余数。密钥空间元素个数为 q，其中有一恒等变换，即 $k = 0$，解密变换为：

$$D_k(j) = E_{q-k}(j) = (j + q - k) \mod q = (j - k) \mod q = i.$$

例 2.1 Kaiser 密码

Kaiser 密码是每一字母向前移位。例如密钥为 5 便有明文和密文对应关系如下：

明文：	a	b	c	d	e	f	g	h	i	j	k	l	m
密文：	F	G	H	I	J	K	L	M	N	O	P	Q	R
明文：	n	o	p	q	r	s	t	u	v	w	x	y	z
密文：	S	T	U	V	W	X	Y	Z	A	B	C	D	E

于是对于明文：

data security has evolved rapidly

可加密得密文：

I F Y F X J H Z W N Y D M F X J A T Q A JI W F U N I Q D

不同的 k 可得不同的密文。

若令 26 个字母分别对应于整数 $0 \sim 25$，如表2.1所示。则 Kaiser 加密变换实际上是：

$$c = m + k \mod 26$$

其中 m 是明文对应的数据，c 是与明文对应的密文数据，k 是加密用的参数，也称为密钥。

表 2.1: 英文字母与整数的对应

a	b	c	d	e	f	g	h	i	j	k	l	m
1	2	3	4	5	6	7	8	9	10	11	12	13
n	o	p	q	r	s	t	u	v	w	x	y	z
14	15	16	17	18	19	20	21	22	23	24	25	0

例如：data security 对应于数据序列

4 1 20 1 19 5 3 21 18 9 20 25.

$k=5$ 时得密文序列

$$9\ 6\ 25\ 6\ 24\ 10\ 8\ 0\ 23\ 14\ 25\ 4.$$

对应的密文为

$$\mathrm{I\ F\ Y\ F\ X\ J\ H\ Z\ W\ N\ Y\ D.}$$

2、乘数密码 (multiplicative cipher)：加密变换为

$$E_k(i)=ik \mod q=j,\ 0\le j<q.$$

这种密码又称为采样密码 (decimation cipher)，因为密文字母表是将明文字母表按下标每隔 k 位取出一个字母排列而成（字母表首尾相接）。显然，仅当 $(k,q)=1$ 时，即 k 与 q 互素时明文字母和密文字母才是一一对应的。

例 2.2 英文字母表 $q=26$, 取 $k=9$，则有如下明文密文字母对应（表2.2）：

表 2.2: 乘数密码明密文对应

明文:	a	b	c	d	e	f	g	h	i	j	k	l	m
密文:	A	J	S	B	K	T	C	L	U	D	M	V	E
明文:	n	o	p	q	r	s	t	u	v	w	x	y	z
密文:	N	W	F	O	X	G	P	Y	H	Q	Z	I	R

对明文

$$M=\text{multiplicative cipher,}$$

有密文

$$C=\text{EYVPUFVUSAPUHK SUFLKX.}$$

3、线性同余密码：将移位密码和乘数密码进行组合就可以得到更多的选择方式，也叫仿射密码 (affine cipher)。若选取 k_1,k_2 两个参数，其中 $(k_1,26)=1$，即 k_1 和 26 互素，令

$$c=k_1m+k_2 \mod 26.$$

$k_1=1$ 时便是 Kaiser 变换。

例 2.3 $k_1=7,\ k_2=10$，则明文

$$\text{please send moneys}$$

的对应数据为

$$16\ 12\ 5\ 1\ 19\ 5\ 19\ 5\ 14\ 4\ 13\ 15\ 14\ 5\ 25\ 19.$$

通过变换 $c=7m+10 \mod 26$ 可得

$$18\ 16\ 19\ 17\ 13\ 19\ 13\ 19\ 4\ 12\ 23\ 11\ 4\ 19\ 3\ 13.$$

对应的密文为

$$\mathrm{R\ P\ S\ Q\ M\ S\ M\ S\ D\ L\ W\ K\ D\ S\ C\ M.}$$

例 2.4 Playfair 密码

在第一次世界大战中英国人和德国人分别使用 Playfair 和 ADGX 密码。

Playfair 密码将明文中的双字母组合作为一个单元对待，并将这些单元转换为密文双字母组合。Playfair 密码的密钥是一个 5×5 矩阵，该矩阵使用一个关键词构造。该密码体制的密钥是一个单词，比如 FIVESTARS，将单词中重复的字母去掉一个，可以得到 FIVESTAR，将剩下的字母排列成 5×5 矩阵的起始部分，矩阵的剩余部分则用 26 个字母表中未出现的字母来填充，I 和 J 作为一个字母来对待。例如下面的矩阵便是由密钥 FIVESTARS 导出的，其中 J 当做 I 处理。

F	I	V	E	S
T	A	R	B	C
D	G	H	K	L
M	N	O	P	Q
U	W	X	Y	Z

对每一对明文 m_1, m_2，加密方法如下：

1) 若 m_1 和 m_2 在同一行，则密文 c_1 和 c_2 分别紧靠 m_1, m_2 右端的字母，其中第一列看做是最后一列的右方。
2) 若 m_1 和 m_2 在同–列，则密文 c_1 和 c_2 分别是紧靠 m_1, m_2 下方的字母，这里第一行看做是最后一行的下方。
3) 若 m_1 和 m_2 不在同一行，也不在同一列，则 c_1 和 c_2 是出 m_1, m_2 确定的矩形的其他两角的字母，并 c_1 和 m_1，c_2 和 m_2 同行。
4) 若 m_1=m_2，则插入空字母 (比如 Q) 于重复字母之间。
5) 若明文字母数为奇数，将空字母 Q 加在明文的末端。

例如：m=P1ayfair cipher wa s actually invented by whratstone。先将明文 m 分解为两个字母一对：

pl ay fa ir ci ph er wa sa ct ua lq ly in ve nt ed by wh ea ts to ne.

通过加密可得密文：

QK BW IT VA AS OK VB IG IC TA WT QZ KZ AW ES MA FK KE ZG IB CF RM PI.

而要解密，就是上述过程的逆过程。

如果用频率攻击的方法来破解，该体制是可以被攻破的. 因为在英语中各种连字 (即两个字母的组合) 已经被制成了表格。当然我们要看一下最常用的连字；在英语中对应的最常用的连字有：th，he，an，in，re，es, …，而且，对这些连字稍做变动又很快会产生新的结果，比如，连字 re 和 er 都很常见。如果在密文中像 IG 和 GI 这样的字母对出现得比较多的话，一个很自然的猜测就是 e，i，r，g 这些字母组成了矩阵的一个矩形形式。该密码体制的另外一个弱点是每一个明文字母在密文中仅对应 5 种可能的字母，除非这个密钥很长，否则矩阵的剩余行是可以预测出来的，观察到了这些，对该体制来说仅用一段密文都可以攻破它。

例 2.5 ADFGX 密码

ADFGX 密码过程如下：将字母表中的字母组成 5×5 的矩阵，字母 i 和 j 被认为是同一个字母，矩阵的行和列用字母 A, D, F, G, X 标记，例如，矩阵可能是

	A	D	F	G	X
A	p	g	c	e	n
D	b	q	o	z	r
F	s	l	a	f	t
G	m	d	v	i	w
X	k	u	y	x	h

每一个明文字母用它所在行和列的标记代替，如 s 变成了 FA，z 变成了 DG，假设明文是

Kaiser Wilhelm,

第一步的结果就是

XA FF GG FA AG DX GX GG FD XX AG FD GA.

迄今为止可以看到，这不过是一种变相的替换密码，请注意到下一步增加了它的复杂性，选择一个关键字，比如 Rhein，用关键字中的字母来标记矩阵的列，将第一步的结果组成如下矩阵：

R	H	E	I	N
X	A	F	F	G
G	F	A	A	G
D	X	G	X	G
G	F	D	X	X
A	G	F	D	G
A				

现在重新调整列，使列的标记按字母表的顺序排列：

E	H	I	N	R
F	A	F	G	X
A	F	A	G	G
G	X	X	G	D
D	F	X	X	G
F	G	D	G	A
				A

最后，通过按列向下读字母 (忽略标记) 可得到密文如下：

FAGDFAFXFGFAXXDGGGXGXGDGAA.

只要知道了关键字，解密是很容易的，从关键字的长度和密文的长度可以确定列的长度，字母被放置到了列中，重新安排顺序可以与关键字匹配，然后用初始的矩阵来恢复明文。

初始矩阵和关键字要经常改变，这样可使密码分析更困难，因为对任意一次组合仅有有限数量的密文。然而，这个体制还是成功地被法国的密码专家乔治 · 潘威 (Georges Painvin) 和布鲁 · 都 · 切佛瑞 (Bureau du Chiffre) 破解。这里使用了一个技巧，假设同时截取的两份密文头几个字符是相同的，有理由猜测两份明文具有相同的单词，这意味着一份密文行和列的前几个输入与另一份相同，继续研究密文寻找相同的

其他地方，这些很可能表示列的开始，如果正确的话，我们就可以知道列的长度，使用这个长度把密文划分成若干列，对第一份密文，一些列会有一个长度而另一些列会更长些，长的列就很可能靠近开始的列；其他列则靠近结束的列。对第二份密文重复上述步骤，如果一个列对两份密文都长，那它就更靠近开始列，如果列只对一份密文长而对另一份密文不长，它就是中间列，如果对两份都短，它就是结束列。按照这种方法，尝试各种列的顺序并遵照上述的限制规则，每一种顺序都与一种潜在的替换密码相对应，利用频率分析尽可能解决这些问题，最终可以得出明文和原始的加密矩阵。

之所以选择字母 ADFGX 是因为这些符号在莫尔斯 (Morse) 电码 (· - ，- · ·，· · · ·，- - ·，- · · -) 中是不易混淆的，从而可以避免发生传输错误，这是早期试图将纠错与密码学结合的标志之一。后来，ADFGX 密码被 ADFGVX 取代，使用 6×6 的初始矩阵，它允许使用 26 个字母加 10 个数字。

2.1.2 多表替换密码

多表替换密码是以一系列 (两个以上) 替换表依次对明文消息的字母进行替换的加密方法。比如使用有 5 个简单替换表的替换密码，明文的第一个字母用第一个替换表，第二个字母用第二个表，第三个字母用第三个表，以此类推，循环使用这五张替换表。多表替换密码由莱昂. 巴蒂斯塔于 1568 年发明，著名的维吉尼亚密码和博福特密码均是多表替换密码。

令明文字母表为 Z_q ，令 $\pi = (\pi_1, \pi_2, \ldots$ 为替换序列，明文字母序列为 $m = m_1, m_2, \ldots$，则相应的密文字母序列为

$$c = E_k(m) = \pi(m) = \pi_1(m_1), \pi_2(m_2), \ldots$$

若 π 为非周期无限长序列，则相应的密码为非周期多表替换密码。这类密码对每个明文字母都采用不同的替换表或密钥进行加密，称作是一次一密钥密码 (one-time pad cipher)。这是一种在理论上唯一不可破的密码。这种密码对于明文的特征可实现完全隐蔽。但由于需要的密钥量和明文消息长度相同而难以广泛使用。

为了减少密钥量，实际中多采用周期多表替换密码，即替换表个数有限，重复使用。第一个密钥加密明文的第一个字母，第二个密钥加密明文的第二个字母等等。在所有的密钥用完后，密钥又再循环使用，若有 20 个单个字母密钥，那么每隔 20 个字母的明文都被同一密钥加密，这叫做密码的周期。在经典密码学中，密码周期越长越难破译，使用计算机就能够轻易破译具有很长周期的替换密码。此时代换序列

$$\pi = \pi_1\pi_2 \ldots \pi_d\pi_1\pi_2 \ldots \pi_d \ldots$$

相对应于明文字母序列 m 的密文为

$$c = E_k(m) = \pi(m) = \pi_1(m_1)\pi_2(m_2) \ldots \pi_d(m_d)\pi_1(m_{d+1}) \ldots \pi_d(m_{2d}) \ldots$$

当 $d = 1$ 时就退化为单表替换。

比较有名的多表替换密码有维吉尼亚 (Vigenere)、博福特 (Beaufort)、滚动密钥 (runningkey) 和弗纳姆 (Vernam) 等。

1、维吉尼亚密码

一种以移位替换为基础的周期替换密码，为 1858 年法国密码学家维吉尼亚提出。首先构造一个维吉尼亚方阵（表2.3）。

它的基本阵列是 26 行 26 列的方阵。方阵的第一行是 a 到 z 按正常顺序排列的字母表，第二行是第一行左移循环一位得到的，第三行又是第二循环左移一位得到的，依此类推，得到其余各行。然后在基本方阵的最上方附加一行，最左侧附加一列，分别依序写上 a 到 z 共 26 个字母，表的第一行与附加列上的的字母 a 相对应，表的第二行与附加列上的字母 b 相对应，...，最后一行与附加列上的字母 z 相对应。如果把上面的附加行看作是明文序列，则下面的 26 行就分别构成了左移 0 位，1 位，2 位，...，25 位的 26 个单表替换加同余密码的密文序列。加密时，按照密钥字的指示，决定采用哪一个单表。例如密钥字

表 2.3: 维吉尼亚方阵

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
b	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
c	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
d	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
e	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
f	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
g	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
h	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
i	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
j	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
k	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
l	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
m	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
n	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
o	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
p	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
r	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
s	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
t	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
u	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
v	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
w	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
x	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

是 bupt，加密时，明文的第一个字母用与附加列上字母 b 相对应的密码表进行加密，明文的第二个字母用与附加列的字母 u 相对应的密码表进行加密，依此类推。

下面我们用一个实例来说明维吉尼亚密码加密和解密的原理。设密钥字是

encryption，

待加密的明文是

public key distribution。

由于密钥字比明文短，所以要重复书写密钥字以得与明文等长的密钥序列。

现在按上表对明文进行加密代换。第一个密钥字母是 e ，对第一个明文字母 p 进行加密时，选用左边附加列上的字母 e 对应的那一行作为代替密码表，查出与 p 相对应的密文字母是 T，第二个密钥字母是 n ，用附加列上字母 n 所对应的一行作为代替密码表，与明文 u 进行替换，对应的密文是 H 。同理将所有的密文字母替换完毕就可以得到表2.4 中所示的密文 THDCGRDMMQMFVRGQNBWBR。由于在上述加密变换中，明文的每个字母是按照密钥字的指示，选用不同的加同余密码替换而成的。因此同一个字母在明文中的位置不同，对应的密文字母就不同。例如上例明文中有四个 i，但分别对应密文字母 G、M、W。反之密文中的三个相同字母 M 却分别对应明文中的三个不同字母 e 、y 、i。在这种体制中，所用到的单表数目与所用的密钥字长有关，并按密钥字长对这些单表轮流使用，因而可呈现固定的周期性，周期长度就是密钥字长。显然，密钥字越长，破译时就越困难。

维吉尼亚密码加密和解密的数学模型可以表示如下：令英文字母 a, b, . . ., z 对应于从 0 到 25 的整数。

表 2.4: 维吉尼亚密码加密解密举例

密钥	e	n	c	r	y	p	t	i	o	n	e	n	c	r	i	p	t	i	o	n	e
明文	p	u	b	l	i	c	k	e	y	d	i	s	t	r	i	b	u	t	i	o	n
密文	T	H	D	C	G	R	D	M	M	Q	M	F	V	R	G	Q	N	B	W	B	R

设明文是 n 个字母组成的字符串，即:

$$m = m_1 m_2 \dots m_n.$$

密钥字周期性地延伸就给出了明文加密所需的工作密钥:

$$k = k_1 k_2 \dots k_n.$$

设密文为

$$E(m) = C = c_1 c_2 \dots c_n.$$

则加解密算法为:

加密: $c_i = m_i + k_i \mod 26,$
解密: $m_i = c_i - k_i \mod 26,\ i = 1, 2, 3 \dots, n.$

类似可构造福特加密算法如下:

加密: $c_i = k_i - m_i \mod 26,$
解密: $m_i = k_i - c_i \mod 26,\ i = 1, 2, 3 \dots, n.$

2、Hill 密码

Hill 加密算法的基本思想是将 l 个明文字母通过线性变换. 将它们转换为 l 个密文字母。解密只要作一次逆变换就可以了。密钥就是变换矩阵本身。即

$$M = m_1 m_2 \dots m_l,$$

$$E_k(M) = c_1 c_2 \dots c_l,$$

其中

$$\begin{aligned} c_1 &= k_{11} m_1 + k_{12} m_2 + \dots + k_{1l} m_l \mod n, \\ c_2 &= k_{21} m_1 + k_{22} m_2 + \dots + k_{2l} m_l \mod n, \\ &\vdots \\ c_t &= k_{l1} m_1 + k_{l2} m_2 + \dots + k_{ll} m_l \mod n. \end{aligned}$$

或写成

$$C = KM \mod n,$$

其中

$$C = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_l \end{pmatrix}, \quad M = \begin{pmatrix} m_1 \\ m_2 \\ \vdots \\ m_l \end{pmatrix},$$

$K=(k_{ij})_{l\times l}$ 为 $l\times l$ 的模 n 可逆矩阵。显然有

$$M=K^{-1}C \mod n.$$

例如, $l=4$, $n=26$,

$$K=\begin{pmatrix}8&6&9&10\\6&9&5&10\\5&8&4&9\\10&6&11&4\end{pmatrix},$$

$$K^{-1}=\begin{pmatrix}23&20&5&1\\2&11&18&1\\2&20&6&25\\25&2&22&25\end{pmatrix}.$$

不难验证

$$\begin{pmatrix}8&6&9&10\\6&9&5&10\\5&8&4&9\\10&6&11&4\end{pmatrix}\cdot\begin{pmatrix}23&20&5&1\\2&11&18&1\\2&20&6&25\\25&2&22&25\end{pmatrix}=\begin{pmatrix}339&416&312&364\\416&339&442&390\\364&286&339&338\\364&494&332&131\end{pmatrix}=\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&0&1\end{pmatrix} \mod 26.$$

设明文为 M = Hill。若采用从 A 到 Z 的 26 个字母依顺序从 0 到 25 编号，M 可分别得数字化后 4 个数字：7，8，11，11。这样

$$c_1=8\times7+6\times8+9\times11+5\times11=258\equiv24 \mod 26$$

$$c_2=6\times7+9\times8+5\times11+10\times11=279\equiv19 \mod 26$$

$$c_3=5\times7+8\times8+9\times4+9\times11=242\equiv8 \mod 26$$

$$c_4=10\times7+6\times8+11\times11+4\times11=283\equiv23 \mod 26.$$

因此密文 C = YTIX。

反之，已知 C = YTIX，则有

$$m_1=23\times24+20\times19+5\times8+23=995\equiv7 \mod 26,$$

$$m_2=2\times24+11\times19+18\times8+23=995\equiv8 \mod 26,$$

$$m_3=2\times24+20\times19+6\times8+25\times23=1051\equiv11 \mod 26,$$

$$m_4=25\times24+2\times19+22\times8+25\times23=1389\equiv11 \mod 26.$$

故得 M = Hill。

当然字母和数字的对应还可以改变，使得更不容易攻击成功，一般说来，Hill 密码能比较好地抵抗频率法的分析。

2.2 置换密码体制

置换加密（换位密码）是将明文字母互相换位，明文字符集保持不变，但顺序被打乱了。

线路加密法是一种换位加密。在线路加密法中，明文的字母按规定的次序排列在矩阵中，然后用另一种次序选出矩阵中的字母，排列成密文。例如在纵行换位密码中，明文以固定的宽度水平地写出，密文按垂直方向读出。这种加密方法也可以按下面的方式解释：明文分成长为 m 个元素的块，每块按照 n 来排列。这意味着一个重复且宽度为 m 的单字母的多表加密过程，即分块换位是整体单元的换位。简单

的换位可用纸笔容易实现，而且比分块换位出错的机会少。尽管它跑遍整个明文，但它并不比整体单元置换提供更多的密码安全。

设 m 为固定的正整数。令 $P = C = (Z_{26})^m$，K 为 $\{12 \dots m\}$ 的所有置换的全体。对任意密钥 $\pi \in K$，定义

$$e_\pi(x_1, \dots, x_m) = (x_{\pi(1)}, \dots, x_{\pi(m)})$$

和

$$d_\pi(y_1, \dots, y_m) = (y_{\pi^{-1}(1)}, \dots, y_{\pi^{-1}(m)}),$$

其中π^{-1} 是π 的逆置换。

例如：明文为 COMPUTERGRAPHICSMAYBESLOWBUTATLEASTITSEXPENSIVE，按如方式排列：

COMPUTERGR
APHICSMAYB
ESLOWBUTAT
LEASTITSEX
PENSIVE

密文按列读出，即为: CAELPOPSEEMHLANPIOSSUCWTITSBIVEMUTERATSGYAERBTX。

这种线路形式很多，矩阵的大小也可变化。

无论怎么换位置，密文字符与明文字符保持相同，对密文字母的统计分析很容易决定字母的准确顺序。

置换密码中最简单的是栅栏技术。在该密码中以对角线顺序写下明文，并以行的顺序读出。例如，为了用深度 2 的栅栏密码加密消息 meet me after the toga party。写出如下形式：

m e m a t r h t g p r y
e t e f e t e o a a t

加密后的消息是：MEMATRHTGPRYETEFETEOAAT。

2.3 经典密码体制的分析

2.3.1 统计特性

任何自然语言都有许多固有的统计特性。如果自然语言的这种统计特性在密文中有所反映，则密码分析者便可以通过分析明文和密文的统计规律而将密码破译。许多古典密码都可以用统计分析的方法破译。

随便阅读一篇英文文献，立刻就会发现，其中字母 e 出现的次数比其他字母都多。如果进行认真统计，并且所统计的文献的篇幅足够长，便可以发现各字母出现的相对频率十分稳定。而且，只要文献不特别专门化，对不同的文献进行统计所得的频率大体相同。表2.5给出了英文字母的频率，同时显示出英文字母的频率的分布模式。

表 2.5: 英文字母频率的分布

字母	频率	
A	8.176	* * * * * * * * * * * * * * * *
B	1.492	* * *
C	2.782	* * * * * *
D	4.253	* * * * * * * *
E	12.703	* *
F	2.228	* * * *
G	2.015	* * * *
H	6.094	* * * * * * * * * * * *
I	6.996	* * * * * * * * * * * * * *
J	0.153	
K	0.772	* *
L	4.025	* * * * * * * *
M	2.406	* * * * *
N	6.749	* * * * * * * * * * * *
O	7.507	* * * * * * * * * * * * * * *
P	1.929	* * * *
Q	0.095	
R	5.987	* * * * * * * * * * * *
S	6.327	* * * * * * * * * * * *
T	9.056	* * * * * * * * * * * * * * * * * *
U	2.758	* * * * *
V	0.978	* *
W	2.36	* * * * *
X	0.15	
Y	1.974	* * * *
Z	0.074	

注：* 代表频率 0.5

进一步，根据各字母频率的大小可将英文字母分为几组。表2.6说明这一分组情况。

表 2.6: 英文字母组频率分布

极高频率字母组	E
次高频率字母组	T A O I N S H R
中等频率字母组	D L
低频率字母组	C U M W F G Y P
甚低高频率字母组	V K J X Q Z

不仅单字母以相当稳定的频率出现，而且双字母组（相邻的两个字母）和三字母组（相邻的三个字母）同样如此。出现频率最高的 30 个双字母组依次是：

TH HE IN EB AN RE ED ON ES ST EN AT TO NT HA ND
OU EA NG AS OR TI IS ET IT AR TE SE HI OF

出现频率最高的 20 个三字母组依次是：

THE ING AND HER EBE ENT THA NTH WAS ETH
FOR DTH HAT SHE ION HIS STH ERS VER

特别值得注意的是，THE 的频率几乎是排在第二位的 ING 的 3 倍，这对于破译密码是很有帮助的。此外，统计资料还表明：

1) 英文单词以 E，S，D，T 为结尾的超过一半；
2) 英文单词以 T，A，S，W 为起始字母的约占一半。

以上所有这些统计数据. 对于密码分析者来说都是十分有用的信息。除此之外，密码分析者的文学、历史、地理等方面的知识对于破译密码也是十分重要的因素。

最后指出，上述统计数据是对非专业性文献中的字母进行统计得到的。如果考虑实际文献中的标点、间隔、数字等符号，则统计数据将有所不同。例如，计算机程序文件的字符频率分布与报纸政治评论的字符领率分布有显著不同。

2.3.2 单表密码体制的统计分析

以下分析单表替换密码。对于加同余密码，密钥整数 k 只有 $n-1$ 个不同的取值。对于明文字母表为英文字母表的情况，k 只有 25 种可能的取值。即使是对于明文字母表为 8 位扩展 ASCII 码而言，k 也只有 255 种可能的取值。因此，只要对 k 的可能取值逐一穷举就可破译加同余密码。乘数密码比加同余密码更容易破译。密钥整数 k 要满足条件 $(n,k)=1$，因此，k 只有少 $\varphi(n)$ 个（$\varphi(n)$ 表示小于 n 且与 n 互素的整数的个数）不同的取值。去掉 $k=1$ 这一恒等情况，k 的取值只有 $\varphi(n)-1$ 种。这里 $\varphi(n)$ 为 n 的欧拉函数。对于明文字母表为英文字母表的情况，k 只能取 3，5，7，9，11，15，17，19，21. 23，25 共 11 种不同的取值，比加同余密码弱得多。仿射密码的保密性能好一些。但可能的密钥也只有 $n\varphi(n)-1$ 种。对于明文字母表为英文字母表的情况，可能的密钥只有 $26\times 12-1=311$ 种。这一数目对于古代密码分析者企图用穷举全部密钥的方法破译密码，可能会造成一定的困难，然而对于应用计算机进行破译来说，这就是微不足道的了。

本质上，密文字母表实际上是明文字母表的一种排列。设明文字母表含 n 个字母，则共有 $n!$ 种排列，对于明文字母表为英文字母表的情况，可能的密文字母表有 $26!\approx 4\times 10^{26}$。由于密钥词组代替密码的密钥词组可以随意地选择，故这 26! 种不同的排列中的大部分被用做密文字母表是完全可能的。即使使用计算机，企图用穷举一切密钥的方法来破译密钥词组代替密码也是不可能的。那么，密钥词组代替密码是不是牢不可破呢？其实不然，因为穷举并不是攻击密码的惟一方法。这种密码仅在较短的消息时是保密的，一旦消息足够长，密码分析者便可利用其他的统计分析的方法迅速将其攻破。

字母和字母组的统计数据对于密码分析者来说是十分重要的。因为它们可以提供有关密钥的许多信息。例如，由于字母 E 比其他字母的频率都高得多，如果是简单替换密码，那么可以预计大多数密文都将包含一个频率比其他字母都高的字母。当出现这种情况时，完全有理由猜测这个字母所对应的明文字母就是 E。进一步比较密文和明文的各种统计数据及其分布模式，便可确定出密钥，从而攻破单表替换密码。例如，加同余密码的密文字母频率分布是其明文字母频率分布的一种循环平移。而乘数密码的密文字母频率分布是其明文字母频率分布的某种等间隔抽样。由于多表替换密码和多名替换密码的每一个明文字母都有多个不同的密文字母来代替，因此它们的密文字母频率分布是比较平坦的，所以它们的保密性比单表替换密码高。但是仍然有其他一些统计特性在密文中留下痕迹，因此仍然是可以攻破的。

下面举例说明一般单表替换替密码统计分析过程。

密文： YKHLBA JCZ SVIJ JZB LZVHI JCZ VHJ DR IZXKHLBA VSS RDHEI DR YVJV LBXSKYLBA YLALJVS IFZZXC CVI LEFHDNZY EVBTRDSY JCZ FHLEVHT HZVIDB RDH JCLI CVI WZZB JCZ VYNZBJ DR ELXHDZSZXJHDBLXI JCZ XDEFSZQLJT DR JCZ RKBXJLDBI

JCVJ XVB BDP WZ FZHRDHEZY WT JCZ EVXCLBZ CVI HLIZB YHVEVJLXVSST VI V HZIKSJ DR JCLI JCVJ PZHZ DBXZ XDBILYZHZY IZXKHZ VHZ BDP WHZVMVWSZ。

首先统计密文的单字母频率数，并将字母分组。

- 单字母频率数：

A	B	C	D	E	F	G	H	I	J	K	L	M
5	24	19	23	12	7	0	24	21	29	6	20	1
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
3	0	3	1	11	14	9	0	27	5	17	12	45

- 字母分组：

极高频率字母组	Z
次高频率字母组	J V B H D I L C
中等频率字母组	X S E Y R
低频率字母组	T F K A W N P
甚低高频率字母组	M Q G O U

由于密文太少，故统计结果与明文统计数据不尽相同。尽管如此，已足以破译该密文。

密文字母 Z 的频率最高，它一定是明文字母 E。在英语中只有一个单字母单词 A，因此可以断定密文字母 V 对应于明文字母 A。三字母 JCZ 的频率最高，因此它一定就是 THE。密文字母 J 对应于明文字母 T，密文字母 C 对应于明文字母 H。密文字母 J 的频率处于第二位，进一步证明了其对应于明文字母为 T。考察双字母单词 VI。因为已知 V 对应于 A，根据英语知识，只可能是 AN，AS，AM，AT。首先它不是 AN. 否则因其后有冠词 A 而语法不通。又因 J 对应于 T，故又不是 AT，只能是 AS 或 AM。明文字母 M 属于低频字母，而密文字母 I 后于高频字母，因此密文字母 I 对应于明文字母 S，密文 VI 的明文为 AS。考察三字母单词 VSS。因为已知 v 的明文为 A，在英语中 A 后面接两个相同字母的单词只有从 L，因此密文字母 S 对应于明文字母 L。在三字母单词 VHZ 中，因为已知 V 的明文为 A，Z 的明文为 E，根据英语知识它只能是 ARE 或 AGE。因为 H 在密文中属于高频字母，G 在明文字母中属于低频字母，故 H 的明文为 R。仿此分析三字母单词 JZB，可知密文字母 B 对应于明文字母 N，JZB 的明文为 TEN。分析四字母单词 JCLI，可知密文字母 L 对应于明文字母 I。分析四字母单词 WZZB，可知密文字母 W 对应于明文字母 B。由双字母单词 WT 可知密文字母 T 对应于明文字母 Y。由密文 HZVIDB 可推出密文字母 D 对应于明文字母 O。双字母单词 DR 的频率很高，已知 D 的明文是 O，则 R 的明文一定是 F。由三字母组 BDP 可推出密文字母 P 对应于明文字母 W。在密文 DBXZ 中，因为已知 D，B，Z 的明文，故可推出密文字母 X 对应于明文字母 C。从密文 EVBT 可推出密文字母 E 对应于明文字母 M。从密文 IFZZXC 可推出密文字母 F 对应于明文字母 P。从密文 FZHRDHEZY 可推出密文字母 Y 对应于明文字母 D。从密文 JZXCBDSDAT 可推出密文字母 A 对应于明文字母 G。同时注意到三字母词尾 LBA 的频率较高。进一步证明这一推断是正确的。从密文 YKHLBA 可推出密文字母 K 对应于明文字母 U。从密文 LEFHDNZY 可推出密文字母 N 对应于明文字母 V。最后从 WHZVMVPZHZ 可知 M 对应于 K。至此，整个密文全部译出：

DURING THE LAST TEN YEARS THE ART OF SECURING ALL FORMS OF DATA INCLUDING DIGITAL SPEECH HAS IMPROVED MANYFOLD THE PRIMARY REASON FOR THIS HAS BEEN THE ADVENT OF MICROELECTRONICS THE COMPLEXITY OF THE FUNCTION THAT CAN NOW BE PERFORMED BY THE MACHINE HAS RISEN DRAMATICALLY AS A

RESULT OF THIS RECENT DEVELOPMENT IN TECHNOLOGY MANY OF THE CIPHER SYSTEM THAT WERE ONCE CONSIDERED SECURE ARE NOW BREAKABLE

从以上例子可以看出，破译单代替密码的大致过程是：首先统计密文的各种统计特征，如果密文量比较多，则完成这步后便可确定出大部分密文字母；其次分析双字母、三字母密文组，以区分元音和辅音字母；最后分析字母较多的密文，在这一过程中大胆使用猜测的方法，如果猜对一个或几个词，就会大大加快破译过程。

2.3.3 多表密码体制的统计分析

多表替换密码的破译要比单表替换密码的破译难得多，因为在单表替换下，字母的频度、重复字母模式、字母结合方式等统计特性除了字母名称改变以外，都未发生变化，依靠这些不变的统计特性就能破译单表替换。而在多表替换下，原来明文中的这些特性通过多个表的平均作用而被隐蔽了起来。已有事实表明，用唯密文攻击法分析单表和多表替换密码是可行的，但用唯密文攻击法分析多字母替换密码，如 Hill 密码是比较困难的。分析多字母替换密码多用已知明文攻击法。下面我们以 Vigenere 密码为例来说明多表替换密码的一些分析方法。

分析 Vigenere 密码的第一步是确定密钥字的长度 d。确定密钥子的长度 d 的方法最常用的有两种，即 Kasiski 测试法和重合指数 (index of coincidence) 法。

Kasiski 测试法是由普鲁士军官 Kaisiki 在 l863 年提出的一种重码分析法。这种方法的基本原理是：若用给定的 d 个密钥表周期地对明文字母加密，则当明文中有两个相同字母组在明文序列中间隔的字母数为 d 的倍数时，这两个明文字母组对应的密文字母组必相同。但反过来，若密文中出现两个相同的字母组，它们所对应的明文字母组未必相同，但相同的可能性很大。如果我们将密文中相同字母组找出来，并对其相间字母数综合研究．找出它们的相同字母数的最大公因子，就有可能提取出有关密钥字的长度 d 的信息。

下面我们来介绍确定 Vigenere 密码的密钥字的长度 d 的另一种方法—重合指数法。

定义 2.1

设 $x = x_1x_2\dots x_n$ 是 n 个字母的一个串，x 的重合指数定义为 x 中的两个随机元素相同的概率，记为 $I_c(x)$。

♣

假定 $f_0, f_1, \dots, f_{25}$ 分别表示 x 中字母 A，B，C，…，Z 出现的频率。我们能以 $\binom{n}{2}$ 种方法选择 x 中的两个元素，对每一个 i, $0 \le i \le 25$，x 中的两个元素都被选择为 i 的方法有 $\binom{f_i}{2}$ 种，因此

$$I_c(x) = \frac{\sum_{i=0}^{25}\binom{f_i}{2}}{\binom{n}{2}} = \frac{\sum_{i=0}^{25} f_i(f_i-1)}{n(n-1)}.$$

现在假定 x 是一个英文明文串。记字母 A，B，…，Z 出现的期望概率分别为 p_0，p_1，…，p_{25}。通过对大量的小说、杂志、报纸等的汇编统计，人们已经获得了英文的 26 个字母的概率分布的一个估计。

我们将期望 $I_c(x) \approx \sum_{i=0}^{25} p_i^2 = 0.065$，这是因为两个随机元素都是 A 的概率为 p_0^2，两个随机元素都是 B 的概率为 p_1^2，等等。如果 x 是利用任何多表替换密码获得的一个密文，那么在这种情况下，各个概率将只是被作了一个置换，但量 $\sum_{i=0}^{25} p_i^2$ 是不变的。因此，对用多表替换密码获得的一个密文 x，也有 $I_c(x) \approx \sum_{i=0}^{25} p_i^2 = 0.065$。

假定 $y = y_1y_2\dots y_n$ 是通过 Vigenere 密码所获得的一个密文串。把 y 分成 d 个长为 n/d 的子串，记为 $Y_l, Y_2, \dots, Y_d$。如果 d 的确是密钥字长度，那么每个 $I_c(Y_i)$ $(1 \le i \le d)$ 都将大概等于 0.065。如果 d 不是密钥字的长度，那么对于串 Y_i 将看起来更随机些，因为它们将是采用不同的密钥移位加密获得的。而对一个完全随机的串 x，$I_c(x) \approx 26(1/26)^2 = 1/26 = 0.038$。因为两个值 0.065 和 0.038 间隔的充分远，

表 2.7: 期望的重合指数

相对位移	MI_c 的期望值
0	0.065
1(25)	0.039
2(24)	0.032
3(23)	0.034
4(22)	0.044
5(21)	0.033
6(20)	0.036
7(19)	0.039
8(18)	0.034
9(17)	0.034
10(16)	0.038
11(15)	0.045
12(14)	0.039
13	0.043

所以我们通常能确定正确的密钥字的长度。

例：假定有一段来自用 Vigenere 密码加密的密文：

CHREEVOAHMAERATBIAXXWTNXBEEOPHBSBQMQEQERBW
RVXUOAKXAOSXXWEAHBWGJMMQMNKGRFVGXWTRZXWIAKLX
FPSKAUTEMNDCMGTSXMXBTUIADNGMGPSRELXNJELXVRVPRT
ULHDNQWTWDTYGBPHXTFAUHASVBFXNGLLCHRZBWELEKMSJI
KNBHWRJGNMGJSGLXFEYPHAGNRBIEQJTAMRVLCRREMNDGLX
RRIMGNSNRWCHRQHAEYEVTAQEBBIPEEWEVKAKOEWADREMXM
TBHHCHRTKDNVRZCHRCLQOHPWQAIIWXNRMGWOIIFKEE

首先我们用 Kasiski 测试法确定密钥字的长度。密文串 CHR 在密文中的四处出现，起始位置在 1，166，236 和 286。从第 1 个到其它 3 个的距离分别为 165，235 和 285，这三个数的最大公因子是 5，所以 5 可能是密钥字的长度。

其次我们计算一下重合指数，看是否与 Kasiski 测试法所得的结果一致。当 $d = 1$ 时，重合指数为 0.0455；当 $d = 2$ 时，两个重合指数分别为 0.046 和 0.041；当 $d = 3$ 时，三个重合指数分别为 0.043，0.050 和 0.047；当 $d = 4$ 时，四个重合指数分别为 0.042，0.039，0.046 和 0.040；当 $d = 5$ 时，五个重合指数分别为 0.063，0.068，0.069，0.061 和 0.072，这个也为密钥字的长度是 5 提供了有力的证据。

分析 Vigenere 密码的第二步是确定密钥字 (密钥字的长度已由第一步确定)。通常采用重合互指数 (mutual index of coincidence) 法来确定密钥字，重合互指数的精确定义如下。

定义 2.2

假定 $x = x_1x_2\ldots x_n$ 和 $y = y_1y_2\ldots y_n$ 分别是长为 n 和 n' 的字母串。x 和 y 的重合互指数定义为 x 的一个随机元素等于 y 的一个随机元素的概率，记为 $MI_c(xy)$。 ♣

如果我们将 x 和 y 中的字母 A，B，C，…，Z 出现的频率分别表示为 f_0，f_1，…，f_{25} 和 f'_0，f'_1，…，f'_{25}，那么

$$MI_c(x,y) = \sum_{i=0}^{25} \frac{f_i}{n} \cdot \frac{f'_i}{n'} = \frac{\sum_{i=0}^{25} f_i f'_i}{nn'}.$$

现在假定我们已经确定了 d 的值，子串 Y_i 是通过明文的移位加密获得的，并假定 $K = (k_1, k_2, \ldots, k_d)$ 是密钥字。我们来估计 $MI_c(Y_i, Y_j)$。考虑 Y_i 中的一个随机字母和 Y_j 中的一个随机字母，两个字母都是

表 2.8

i	j	$MI_c(Y_i, Y_j^g)$ 的值								
1	2	0.028	0.027	0.028	0.034	0.039	0.037	0.026	0.025	0.052
		0.068	0.044	0.026	0.037	0.043	0.037	0.043	0.037	0.028
		0.041	0.041	0.034	0.037	0.051	0.045	0.042	0.036	
1	3	0.039	0.033	0.040	0.034	0.028	0.053	0.048	0.033	0.029
		0.056	0.050	0.045	0.039	0.040	0.036	0.037	0.032	0.027
		0.037	0.036	0.031	0.037	0.055	0.029	0.024	0.037	
1	4	0.034	0.043	0.025	0.027	0.038	0.049	0.040	0.032	0.029
		0.034	0.039	0.044	0.044	0.034	0.039	0.045	0.044	0.037
		0.055	0.047	0.032	0.027	0.039	0.037	0.039	0.035	
1	5	0.043	0.033	0.028	0.046	0.043	0.044	0.039	0.031	0.026
		0.030	0.036	0.040	0.041	0.024	0.019	0.048	0.070	0.044
		0.028	0.038	0.044	0.043	0.047	0.033	0.026	0.046	
2	3	0.046	0.048	0.041	0.032	0.036	0.035	0.036	0.030	0.024
		0.039	0.034	0.029	0.040	0.067	0.041	0.033	0.037	0.045
		0.033	0.033	0.027	0.033	0.045	0.052	0.042	0.030	
2	4	0.046	0.034	0.043	0.044	0.034	0.031	0.040	0.045	0.040
		0.048	0.044	0.033	0.024	0.028	0.042	0.039	0.026	0.034
		0.050	0.035	0.032	0.040	0.056	0.043	0.028	0.028	
2	5	0.033	0.033	0.036	0.046	0.026	0.018	0.043	0.080	0.050
		0.029	0.031	0.045	0.039	0.037	0.027	0.026	0.031	0.039
		0.040	0.037	0.041	0.046	0.045	0.043	0.035	0.030	
3	4	0.038	0.036	0.040	0.033	0.036	0.060	0.035	0.041	0.029
		0.058	0.035	0.035	0.034	0.053	0.030	0.032	0.035	0.036
		0.036	0.028	0.046	0.032	0.051	0.032	0.034	0.030	
3	5	0.035	0.034	0.034	0.036	0.030	.0.43	0.043	0.050	0.025
		0.041	0.051	0.050	0.035	0.032	0.033	0.033	0.052	0.031
		0.027	0.030	0.072	0.035	0.034	0.032	0.043	0.027	
4	5	0.052	0.038	0.033	0.038	0.041	0.043	0.037	0.048	0.028
		0.028	0.036	0.061	0.033	0.033	0.032	0.052	0.034	0.027
		0.039	0.043	0.033	0.027	0.030	0.039	0.048	0.035	

A 的概率是 $p_{-k_i}p_{-k_j}$，两个字母都是 B 的概率是 $p_{1-k_i}p_{1-k_j}$ 等等 (注意，所有的下标都是模 26 运算)。因此，我们可估计

$$MI_c(Y_i, Y_j) \approx \sum_{h=0}^{25} p_{h-k_i}p_{h-k_j} = \sum_{h=0}^{25} p_h p_{h+k-k_j}.$$

易知，$MI_c(Y_i, Y_j)$ 的估计值只依赖于差 $(k_i - k_j) \mod 26$，我们将这个差称为 Y_i 和 Y_j 的相对移位。又 $\sum_{h=0}^{25} p_h p_{h+l} = \sum_{h=0}^{25} p_h p_{h-l}$，所以 MI_c 关于相对移位 l 和 $26-l$ 的估计值是一样的。

由表2.7知，当相对移位不是 0 时，这些估计值均在 $0.031 \sim 0.045$ 之间，而相对移位是 0 时，估计值是 0.065。我们能使用这个表来推测 Y_i 和 Y_j 的相对移位 $l = (k_i - k_j) \mod 26$。具体做法如下：假定我们固定 Y_i，考虑由长为 d 的密钥字 $e_0 = (0, 0, \ldots, 0)$，$e_1 = (1, 1, \ldots, 1)$，$e_2 = (2, 2, \ldots, 2)$，... 对加密 Y_j 的影响，将加密结果分别记为 Y_j^0，Y_j^1，Y_j^2，...。利用公式 $MI_c(x, y^g) = \sum_{i=0}^{25} f_i f_{i-g}/nn'$ 容易计算 $MI_c(Y_i,\ Y_j^g)$，$0 \le g \le 25$。当 $g = l$ 时，MI_c 将接近 0.065，因为 Y_i 和 Y_j^{-l} 的相对移位是 0。然而对 $g \ne l$ 的值，MI_c 的值将在 0.031 和 0.045 之间。通过使用重合互指数这种技术. 我们能获得子串 Y_1，Y_2，... 中的任何两个的相对移位。

我们已经假定了密钥字的长度是 5。现在极力计算相对移位。利用计算机不难算出 260 个值 $MI_c(Y_i, Y_j^g)$，$0 \le i < j \le 25$，$0 \le g \le 25$。这些值见表2.8。

对每对 (i,j)，找出接近于 0.065 的 $MI_c(Y_i,Y_j^g)$ 的值。如果对一个给定的对 (ij) 有唯一的一个这样的值. 我们就把它视作相对移位的值 (也可能不是)。在表2.8种，用方框已划出了六个这样的值，它们提供了如下的信息：Y_1 和 Y_2 的相对移位可能是 9；Y_1 和 Y_5 的相对移位可能是 16；Y_2 和 Y_4 的相对移位可能是 13；Y_1 和 Y_5 的相对移位可能是 7；Y_3 和 Y_5 的相对移位可能是 20；Y_4 和 Y_5 的相对移位可能是 11。这些关系给出了关于未知量 k_1,k_2,k_3,k_4,k_5 的如下关系式：

$$
\begin{aligned}
k_1 - k_2 &= 9,\\
k_1 - k_5 &= 16,\\
k_2 - k_3 &= 13,\\
k_2 - k_5 &= 7,\\
k_3 - k_5 &= 20,\\
k_4 - k_5 &= 11.
\end{aligned}
$$

这样

$$
\begin{aligned}
k_2 &= k_1 + 17,\\
63 &= k_1 + 4,\\
k_4 &= k_1 + 21,\\
65 &= k_1 + 10.
\end{aligned}
$$

所以密钥字的可能形式为 $(k_1,\ k_1+17,\ k_1+4\ k_1+21\ k_1+10)$，$k_1 \in Z_{26}$。不难确定密钥字是 JANET (最多穷搜索 26 种可能)。

将密文可解密为：

The almond tree was intentative blossom. The days were longer, often ending with magnificent evenings of corrugated pink skies. The hunting season was over, with hounds and guns put away lor six months. The vineyards were busy again as the well-organized farmers treated their vines and the more lackadaisical neighbors hurried to do the pruning they should have done in November.

第 2 章 习题

1. 设英文字母 A，B，C，...，Z 分别编码为 0，1，2，...，25。已知单表仿射加密变换为 C =5m+7 mod 26，其中 m 表示明文，试对明文 HELPME 加密。
2. 根据 Playfair 算法，构造基于密钥 occurrence 的矩阵，加密消息 Must see you over Cadogan West. Coming at once。
3. a. 用 Hill 加密消息 meet me at the usual place at then rather than eight o'clock，密钥为 $\left(\begin{smallmatrix}9 & 4\\ 5 & 7\end{smallmatrix}\right)$，要求写出计算过程和结果。
   b. 写出从密文恢复为明文所做的解密计算。
4. 本题讨论了 Vigenere 密码的一次版本的用途。在这种方案中，密钥是 0 至 26 间随机数流。例如，如果密钥是 3 19 5 ...，则明文的首个字母使用 3 个字母的移位加密，第二个字母使用 19 个字母的移位加密，第三个字母使用 5 个字母的移位加密，以此类推。
   a. 使用密钥流 9 0 1 7 23 15 21 14 11 11 2 8 9 加密明文 sendmoremoney。
   b. 使用 (a) 中产生的密文找到一个密钥，以便该密文为 cashnotneeded。
5. 假设有一段仅有 3 个字母 a，b，c 组成的文字，他们出现的频率分别为 7，2，1。下面的密文是通过 Vigenere 方法加密的（移位模 3)：CAAABBCACBCABACAABCCCACA。说明密钥的长度很可能是 2，并确定最可能的密钥。

6. 设 a，b，c，d，e，f 是模 26 下的整数，考虑下面 Hill 和仿射密码的组合。将一块明文表示成 (x, y) 模 26，相应的密文 (u, v) 是

$$(x, y)\begin{pmatrix} a & b \\ c & d \end{pmatrix} + (e, f) \equiv (u, v) (\mathrm{mod} 26).$$

请描述如何选择密文攻击攻击这个系统，目标是找到密钥 a，b，c，d，e，f。要明确地说明你选择的明文和如何恢复密钥。

7. 下面有四段密文，第一个使用 Substitution Cipher 加密，第二个使用 Vigenere Cipher 加密，第三个使用仿射密码变换，最后一个未指定。确定它们的明文，要求写出你解密消息的具体步骤，包括统计分析和计算。

(a). Substitution Cipher:

EMGLOSUDCGDNCUSWYSFHNSFCYKDPUMLWGYICOXYSIPJCK
QPKUGKMGOLICGINCGACKSNISACYKZSCKXECJCKSHYSXCG
OIDPKZCNKSHICGIWYGKKGKGOLDSILKGOIUSIGLEDSPWZU
GFZCCNDGYYSFUSZCNXEOJNCGYEOWEUPXEZGACGNFGLKNS
ACIGOIYCKXCJUCIUZCFZCCNDGYYSFEUEKUZCSOCFZCCNC
IACZEJNCSHFZEJZEGMXCYHCJUMGKUCY

提示：F 解密为 w。

(b). Vigenere Cipher:

KCCPKBGUFDPHQTYAVINRRTMVGRKDNBVFDETDGILTXRGUD
DKOTFMBPVGEGLTGCKQRACQCWDNAWCRXIZAKFTLEWRPTYC
QKYVXCHKFTPONCQQRHJVAJUWETMCMSPKQDYHJVDAHCTRL
SVSKCGCZQQDZXGSFRLSWCWSJTBHAFSIASPRJAHKJRJUMV
GKMITZHFPDISPZLVLGWTFPLKKEBDPGCEBSHCTJRWXBAFS
PEZQNRWXCVYCGAONWDDKACKAWBBIKFTIOVKCGGHJVLNHI
FFSQESVYCLACNVRWBBIREPBBVFEXOSCDYGZWPFDTKFQIY
CWHJVLNHIQIBTKHJVNPIST

(c). Affine Cipher:

KQEREJEBCPPCJCRKIEACUZBKRVPKRBCIBQCARBJCVFCUP
KRIOFKPACUZQEPBKRXPEIIEABDKPBCPFCDCCAFIEABDKP
BCPFEQPKAZBKRHAIBKAPCCIBURCCDKDCCJCIDFUIXPAFF
ERBICZDFKABICBBENEFCUPJCVKABPCYDCCDPKBCOCPERK
IVKSCPICBRKIJPKABI

(d). unspecified cipher:

BNVSNSIHQCEELSSKKYERIFJKXUMBGYKAMQLJTYAVFBKVT
DVBPVVRJYYLAOKYMPQSCGDLFSRLLPROYGESEBUUALRWXM
MASAZLGLEDFJBZAVVPXWICGJXASCBYEHOSNMULKCEAHTQ
OKMFLEBKFXLRRFDTZXCIWBJSICBGAWDVYDHAVFJXZIBKC
GJIWEAHTTOEWTUHKRQVVRGZBXYIREMMASCSPBNLHJMBLR
FFJELHWEYLWISTFVVYFJCMHYUYRUFSFMGESIGRLWALSWM

NUHSIMYYITCCQPZSICEHBCCMZFEGVJYOCDEMMPGHVAAUM
ELCMOEHVLTIPSUYILVGFLMVWDVYDBTHFRAYISYSGKVSUU
HYHGGCKTMBLRX

第 3 章 密码学的信息论基础

密码学的信息论基础主要是 Shannon 的保密系统的信息理论，概率论是研究信息论的主要工具，本章主要讲述概率论基础和密码学的信息论基础。

3.1 概率论基础

一个试验可能产生多个结果，每个结果称为一个简单事件，所有可能结果的集合称为样本空间。根据需要，我们只考虑有限多个可能结果的离散样本空间。

定义 3.1

样本空间 S 上的一个离散随机变量 X，用 $P(X=x)$ 表示随机变量取 x 时的概率，简记为 $P(x)$。对于任意的 $x \in S$，则有 $0 \le P(X=x) \le 1$，而且 $\sum_{x \in S} P(x) = 1$。

定义 3.2

一个事件 E 是样本空间 S 的一个子集，事件发生的概率记为 $P(E)$。特别的，当 E 是一个简单事件 x 时 $P(E) = P(x)$。事件 E 发生的概率 $P(E)$ 为：

$$P(E) = \sum_{x \in E} P(x).$$

定义 3.3

假设 X 和 Y 分别是定义在样本空间 S_1 和 S_2 上的随机变量。联合概率 $P(x,y)$ 是 X 取 x 且 Y 取 y 时的概率。条件概率 $P(x \mid y)$ 表示当 Y 取 y 时 X 取 x 的概率。如果对于任意的 $x \in S_1$ 和 $y \in S_2$，有 $P(x,y) = P(x)P(y)$，则称随机变量 X 和 Y 是统计独立的。而且对于联合概率和条件概率存在以下关系：

$$P(x,y) = P(x \mid y)P(y), \quad P(x,y) = P(y \mid x)P(x).$$

定理 3.1 (贝叶斯 (Bayes) 定理)

如果 $P(y) > 0$，那么：

$$P(x \mid y) = \frac{P(y \mid x)P(x)}{P(y)}.$$

由定义3.3的条件概率和联合概率的两个关系，直接可得贝叶斯定理（Bayes 定理）。而且由定义3.3可得推论：

推论 3.1

X 和 Y 是相互独立的随机变量，当且仅当对所有的 $x \in S_1$ 和 $y \in S_2$，有 $P(x \mid y) = P(x)$。

定义 3.4

设 S 是一个样本空间，X 是 S 上的一个随机变量，且 X 是一个从样本空间 S 到实数集 $\mathbb{R}$ 的函数；

对于每一个简单事件 $x \in S$，X 分配一个实数 $X(x)$。X 的数学期望定义为：

$$E(X) = \sum_{x \in S} X(x)P(x).$$



3.2 完全保密性

完全保密性：设 $R = (P, C, K, E, D)$ 是一个密码体制。如果对任意 $x \in P$ 和任意 $y \in C$，都有 $p(x \mid y) = p(x)$。则称密码体制 R 具有完善的保密性能，或者称为完全保密性。

假设 (P, C, K, E, D) 是一个特定的密码体制，密钥 $k \in K$ 只用于一次加密。假设明文空间 P 存在一个概率分布。这样就为明文元素定义了一个随机变量，用 X 表示。$p(X = x)$ 表示明文 x 发生的先验概率。同时假设以固定的概率分布选取密钥（通常密钥选取是随机的，因此所有的密钥都是等概率的，在这里假设不是等概率的）所以密钥也定义了一个随机变量，用 K 表示；$p(K = k)$ 表示密钥 k 发生的概率。假设密钥和明文是统计独立的随机变量。同样可以把密文看成随机变量，用 Y 表示。通过明文 X 和密钥 K 可以计算出密文 Y 的概率 $p(Y = y)$。

对于密钥 $k \in K$，定义

$$C(k) = \{E_k(x) \mid x \in P\},$$

也就是说 $C(k)$ 代表密钥是 k 时所有可能的密文。对于任意的 $y \in C$，我们有：

$$p(Y = y) = \sum_{k, y \in C(k)} p(K = k)p(x = d_k(y)).$$

同样，对于任意的 $y \in C$ 和 $x \in P$，可如下计算条件概率 $p(Y = y \mid X = x)$ (即给定明文 x，密文 y 的概率)：

$$p(Y = y \mid X = x) = \sum_{k, x = d_k(y)} p(K = k).$$

由贝叶斯公式计算 $P(X = x \mid Y = y)$ (也就是给定密文 y，明文 x 的概率)

$$\begin{aligned} p(X = x \mid Y = y) &= \frac{p(X = x)p(Y = y \mid X = x)}{p(Y = y)} \\ &= \frac{p(X = x) \times \sum_{k, x = d_k(y)} p(K = k)}{\sum_{k, y \in c(k)} p(K = k)p(x = d_k(y))}. \end{aligned}$$

由上述公式可知道，只要知道了概率分布就可以求出在给出密文情况下明文的概率。

例 3.1 假设明文 $P = \{a, b\}$ 满足 $p(a)/4, p(b)/4$，设密钥 $K = \{k_1, k_2, k_3\}$，$p(k_1)/2$，$p(k_2)/4$，$p(k_3)/4$，设密文 $C = \{1, 2, 3, 4\}$，则加密矩阵如下：

	a	b
k_1	1	2
k_2	2	3
k_3	3	4

计算密文分布如下：

$$
\begin{aligned}
p(1) &= (1/2)(1/4) = 1/8,\\
p(2) &= (1/2)(3/4) + (1/4)(1/4) = 7/16,\\
p(3) &= (1/4)(3/4) + (1/4)(1/4) = 1/4,\\
p(4) &= (1/4)(3/4) = 3/16.
\end{aligned}
$$

则明文空间上的条件概率分布为：

$$
\begin{aligned}
p(a \mid 1) &= 1, \quad & p(b \mid 1) &= 0,\\
p(a \mid 2) &= 1/7, \quad & p(b \mid 2) &= 6/7,\\
p(a \mid 3 &= 1/4, \quad & p(b \mid 3) &= 3/4,\\
p(a \mid 4 &= 0, \quad & p(b \mid 4) &= 1.
\end{aligned}
$$

可以发现，只有密文 $y = 3$ 时，$p(a) = p(a \mid 3)$，$p(b) = p(b \mid 3)$。也就是说对于密文 $y = 3$ 满足完善保密性的定义，但是对于其他的密文不满足。

定理 3.2

假设移位密码中的 26 个密钥等概率使用，则对任意概率分布的明文，移位密码都具有完全保密性。♡

证明 这里令 $P = C = K = \mathbb{Z}_{26}$，对于 $0 \leq K \leq 25$，加密函数 e_k 定义为 $e_k(x) = (x + K) \bmod 26$，$x \in \mathbb{Z}_{26}$。先计算 C 上的概率分布。假设 $y \in \mathbb{Z}_{26}$，则

$$
\begin{aligned}
p(Y = y) &= \sum_{k \in \mathbb{Z}_{26}} p(K = k) p(x = d_k(y)),\\
&= \sum_{k \in \mathbb{Z}_{26}} \frac{1}{26} p(x = y - k),\\
&= \frac{1}{26} \sum_{k \in \mathbb{Z}_{26}} p(x = y - k).
\end{aligned}
$$

现在固定 y，值 $(y - K) \bmod 26$ 构成 $\mathbb{Z}_{26}$ 的一个置换。因此有：

$$
\sum_{k \in \mathbb{Z}_{26}} p(x = y - k) = \sum_{k \in \mathbb{Z}_{26}} p(X = x) = 1.
$$

由此得到对于任意的 $y \in \mathbb{Z}_{26}$，

$$
p(y) = \frac{1}{26}.
$$

接下来，对于任意的 x 和 y，我们有

$$
p(y \mid x) = p(k = (y - x) \bmod 26) = \frac{1}{26}.
$$

这是因为对于任意的 x 和 y，满足 $e_k(x) = y$ 的惟一的密钥 $k = (y - x) \bmod 26)$。现在应用贝叶斯公式，很容易计算出：

$$
p(x \mid y) = \frac{p(x) p(y \mid x)}{p(y)} = \frac{p(x) 1/26}{1/26} = p(x).
$$

所以这个密码体制是完善保密的。

上面证明了只要每一个明文字符都用一个新的随机密钥加密，移位密码就是不可破的。下面让我们讨论一下更一般的情形。

从前面可知，利用贝叶斯定理，对于所有的 $x \in P$ 和 $y \in C$，$p(x \mid y) = p(x)$，同样也可得对于任意的 $x \in P$ 和 $y \in C$，$p(y \mid x) = p(y)$。若假设对于所有的 $y \in C$，$p(y) \geq 0$，固定任意的 $x \in P$，对于任意的 $y \in C$，则有 $p(y \mid x) = p(y) \geq 0$。因此，对于任意 $y \in C$，至少存在一个密钥 k 满足 $e_k(x) = y$，这样

就是有 $|K| \geq |C|$。在任意一个密码体制中，加密函数都是单射的，因此有 $|C| \geq |P|$。

设 (P, C, K, E, D) 是一密码体制，满足 $|P| = |C| = |K|$，该密码体制是完全保密当且仅当每一密钥被等概率的使用，且对任意明文 x 和密文 y，存在唯一密钥 k，将 x 加密成 y。

证明 假设这个密码体制是完善保密的。由上面可知，对于任意的 $x \in P$ 和 $y \in C$，一定至少存在一个密钥 k 满足 $e_k(x) = y$。因此有不等式：

$$|C| = |\{e_k(x) \mid k \in K\}| \leq |K|.$$

但是我们假设 $|C| = |K|$，因此一定有：

$$|\{e_k(x) \mid k \in K\}| = |K|.$$

也就是说，不存在两个不同的密钥 k_1 和 k_2 使得 $e_{k_1}(x) = e_{k_2}(x) = y$。因此对于 $x \in P$ 和 $y \in C$，刚好存在一个密钥 k 使得 $e_k(x) = y$。

记 $n = |K|$。设 $P = \{x_i \mid 1 \leq i \leq n\}$ 并且固定一个密文 $y \in C$。设密钥为 $k_1, k_2, \ldots, k_n$，并且 $e_{k_i}(x_i) = y$，$1 \leq i \leq n$。使用 Bayes 定理，我们有：

$$p(x_i \mid y) = \frac{p(y \mid x_i)p(x_i)}{p(y)} = \frac{P(K = k_i)P(x_i)}{P(y)}.$$

考虑完善保密的条件 $p(x_i \mid y) = p(x_i)$。在这里，我们有 $p(k_i) = p(y)$，$1 \leq i \leq n$。也就是说，所有的密钥都是等概率使用的。密钥的数目为 K，我们得到对任意的 $k \in K$，$p(k) = 1/|K|$。若两个假设的条件都是成立的，可得到密码体制且完善保密的。

定理 3.3

设 (P, C, K, E, D) 为一次一密加密体制，则其具有完全的保密性。 ♡

证明 假设 $n \geq 1$ 是正整数，$P = C = K = (\mathbb{Z}_2)^n$。对于 $k \in (\mathbb{Z}_2)^n$，定义 $e_k(x)$ 为 k 和 x 模 2 的和。因此，如果 $x = (x_1, x_2, x_3, \ldots, x_n)$ 并且 $k = (k_1, k_2, \ldots, k_n)$，则：

$$e_k(x) = (x_1 + k_1, \ \ldots, \ x_n + k_n) \bmod 2.$$

解密和加密是相同的。如果 $y = (y_1, y_2, \ldots, y_n)$，则

$$d_k(y) = (y_1 + k_1, \ \ldots, \ y_n + k_n) \bmod 2.$$

由上面给出的定理，容易看出一次一密密码体制提供了完善保密性。对一次一密密码体制，密码分析者无法只从密文获得关于明文或者密钥的任何信息，即使获得了相应的明文，也只能得到这些密文对应的密钥，而不能获得其它密文对应的明文或密钥。但一次一密密码体制要求每传送一个明文，都必须产生一个新的密钥并通过一个安全的信道传给对方，这样给密钥管理带来了很大的麻烦。

3.3 信息的度量 – 信息熵

3.3.1 信息论的相关概念

在介绍熵的概念之前，先了解信息论中几个概念：离散信源、信源空间、自信息量。

单符号离散信源： 如果信源发出的消息是离散的、有限或无限可列的符号或数字，且一个符号代表一条完整的消息，则称这种信源为单符号离散信源。

单符号离散信源的实例很多。例如，掷骰子每次只能是 1 、2 、3 、4 、5 、6 中的某一个；天气预报可能是晴、阴、雨、雪、风、冰雹等一种或其组合以及温度、污染等；二进制通信中传输的只是 1、0 两个数字等。这种符号或数字都可以看作某一集合中的事件，每个符号或数字 (事件) 都是信源的元素，它们的出现往往具有一定的概率。因此，信源又可以看作是具有一定概率分布的某一符号集合。

信源空间：若信源的输出是随机事件 x，其出现概率为 $p(x)$，则它们所构成的集合，称为信源的概率空间或简称为信源空间。

例如，有一个离散信源发出的各个符号消息的集合为 $X=\{x_1,x_2,x_3\dots x_n\}$，它们的概率分布分别为 $P=\{p(x_1),p(x_2),p(x_3),\dots,p(x_n)\}$，通常 $p(x_i)$ 称为符号 x_i 的先验概率。把它们写到一起称为概率空间：

$$\begin{pmatrix}X\\P\end{pmatrix}=\begin{pmatrix}x_1 & x_2 & x_3 & \dots & \dots & x_n\\ p(x_1) & p(x_2) & p(x_3) & \dots & \dots & p(x_n)\end{pmatrix}.$$

显然有 $p(x_i)\geq 0$，$\sum_{i=1}^{n}p(x_i)=1$。

定义 3.5 (自信息量)

一个随机事件的自信息量定义为其出现概率对数的负值。即

$$I(x_i)=-\log p(x_i).$$

♣

由定义可知，因为概率 $p(x_i)$ 越小，x_i 出现的可能性越小，一旦出现应该获得的信息量就越大。通常对数的底取 2，信息量的单位为比特 (bit)。

例 3.2 一个等概率的二进制随机序列，求任一码元的自信息量。

解 因为二进制序列只有 0 和 1，而且等概率 $P(0)=P(1)=1/2$，所以有

$$I(0)=I(1)=-\log_2(1/2)=\log_2 2=1\ \text{bit}.$$

例 3.3 对于 n 位的 2 进制数，假设每一符号的出现完全随机且概率相等，求任一符号的自信息量。

解 因为对于一个 n 位的 2 进制数的每一位可以从 0, 1 两个数字中任取一个，所以有 2^n 个等概率的可能组合。所以

$$p(x_i)=1/2^n,\ \ I(x_i)=-\log_2 P(x_i)=-\log_2(1/2^n)=n\ \text{bit}.$$

3.3.2 信息的度量

在这里引入随机事件的不确定度概念。在数学中，事件的不确定性可用不确定度来描述，它同样是事件概率的函数，同时在数值和量纲上和自信息量相等。

根据日常知识，各个出现概率不同的随机事件所包含的不确定度是有差别的。一个出现概率接近于 1 的随机事件，发生的可能性很大. 所以它包含的不确定度就很小。反之，一个出现概率很小的随机事件，很难猜测在某个时刻它能合发生，所以它包含的不确定度就很大。若是确定性事件，出现概率为 l，则它包含的不确定度为 0。显然随机事件的信息量和不确定度有很密切的联系，一个不确定度小的事件如果发生，它带来的信息量较小，如果是必然事件，则没有信息量；反之，如果一个不确定度高的事件发生，它带来的信息量很大。

虽然事件的自信息量和该事件的不确定度有如此密切的关系，但两者的含义却有本质的区别。不确定度只与事件的概率有关，是一个统计量具有某种概率分布的随机事件不管发生与否，都存在不确定度，不确定度表证了该事件的特性。而自信息量只有该随机事件出现时才给出，不出现时不给出，因此自信息量是在该事件发生后给予观察者的信息量。

定义 3.6

若有两个消息 x_i,y_j 同时出现，可用联合概率 $p(x_i,y_j)$ 表示，这时的联合自信息量定义为

$$I(x_i,y_j)=-\log_2 p(x_i,y_j).$$

♣

当 x_i 和 y_j 相互独立时，有 $p(x_i,y_i)=p(x_i)p(y_j)$，那么根据对数运算的性质，有 $I(x_i,y_j)=I(x_i)+$

$I(y_j)$。x_i、y_j 所包含的不确定度在数值上也等于它们的自信息量。

若两个消息出现不是独立的，但他们是有相互联系的，这时可用条件概率 $p(x_i \mid y_j)$ 来表示，即在事件 y_j 出现的条件下随机事件 x_i 发生的条件概率，则它的条件自信息量定义为条件概率对数的负值：

$$I(x_i \mid y_j) = -\log_2 p(x_i \mid y_j).$$

虽然信息函数 $I(x_i)$ 使得信息度量成为可能，是信息度量的有力工具，但在信息度量方面仍然存在某些不足。首先，信源产生符号 x_i 不是确定事件，是以 $p(x_i)$ 为概率的随机事件，相应的自信息量 $I(x_i)$ 也是一个以 $p(x_i)$ 为概率的随机性的量。显然，用一个随机性的量来度量信息是不方便的。其次，信息函数 $I(x_i)$ 只能表示信源发某一特定的具体符号 x_i 所提供的信息量。不同的符号 x_i，有不同的自信息量。所以它不足以作为整个信源的总体信息测度。

显然，能作为信源总体信息测度的确定的量，应是信源 x 可能发出的各种不同符号 x_i $(i = 1, 2 \ldots, r)$ 含有的自信息量 $I(x_i)$ $(i = 1, 2 \ldots, r)$，在信源的概率空间 $(p(x_1), p(x_2), p(x_3) \ldots, p(x_r))$ 中的统计平均值。为了指明是信源 x 的信息测度，我们把这个统计平均值记为 $H(x)$，定义信源的平均不确定度 $H(x)$ 为信源中各个符号的不确定度的数学期望。即

$$H(X) = E[I(X)] = \sum_i p(x_i) I(x_i) = -\sum_i p(x_i) \log(x_i).$$

其单位为比特 / 符号或者比特 / 符号序列。

定义 3.7

称 $H(x)$ 是信源 x 的“信息熵”。它表示信源 x 每发一个符号 (不论发什么符号) 所提供的平均信息量。

♣

例 3.4 设信源符号集 $X = \{x_1, x_2, x_3, x_4\}$，每个符号发生的概率分别为 $p(x_1) = 1/2$，$p(x_2) = 1/4$，$p(x_3) = 1/8$，$p(x_4) = 1/8$。求信源熵。

解

$$\begin{aligned} H(X) &= 1/2 \log 2 + 1/4 \log 4 + 1/8 \log 8 + 1/8 \log 8 \\ &= 1/2 + 1/2 + 3/8 + 3/8 \\ &= 1.75 \text{ 比特/符号}. \end{aligned}$$

例 3.5 二元信息源的符号只有两个 0 和 1，求其信息熵。

解 设输出符号 0 和 1 的概率分别为 $p(0)$, $p(1)$，则 $p(0) + p(1) = 1$。即信源的概率空间为：

$$\begin{pmatrix} X \\ P \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ p(0) & p(1) \end{pmatrix}.$$

1) 当 $p(0) = p(1) = 1/2$ 时，信源 X 的信息熵

$$\begin{aligned} H(X) &= -p(0) \log p(0) - p(1) \log p(1) \\ &= -1/2 \log(1/2) - 1/2 \log(1/2) \quad = 1/2 + 1/2 = 1 \text{ 比特/符号}. \end{aligned}$$

2) 当 $p(0) = 0$, $p(1) = l$ 时，信源 X 的信息熵

$$\begin{aligned} H(X) &= -p(0) \log p(0) - p(1) \log p(1) \\ &= -0 \log 0 - 1 \log 1 \\ &= 0 \text{ 比特/符号}. \end{aligned}$$

注意：上面计算中规定 $\log 0 = 0$。

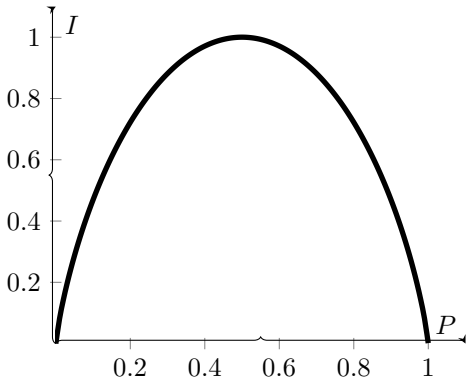


图 3.1: 二元信息源的熵函数

3) 当 $p(0)=1$, $p(1)=0$ 时，可得信源 X 的信息熵

$$\begin{aligned} H(X) &= -p(0)\log p(0) - p(1)\log p(1) \\ &= -1\log 1 - 0\log 0 \\ &= 0 \text{ 比特/符号}. \end{aligned}$$

设 $p=p(0)$，$q=p(1)$，则根据上述计算可知二元信源 X 的信息熵为

$$\begin{aligned} H(X) &= -p\log p - q\log q \\ &= -p\log p - (1-p)\log(1-p). \end{aligned}$$

在例3.4中，信源信息熵 $H(X)$ 是概率 p 的函数，通常用 $H(p)$ 表示，p 取值于 $[0,1]$ 区间。$H(p)$ 的函数如图3.1所示。从图中可看出，如果二元信息源的输出符号是确定的，即 $p=1$ 或 $q=1$，则该信源不提供任何信息。反之，当二元信息符号 0 和 1 以等概率发生时，信源熵达到极大值，等于 1 比特信息量。

熵是在平均意义上来表征信源的总体特性的。正如不确定度与自信息量的关系那样，信源熵是表征信源的平均不确定度，平均自信息是消除信源不确定度时所需要的信息的量度，即收到一个信源符号，全部解除了这个符号的不确定度。或者说获得这样大的信息量后，信源不确定度就被消除了。两者在数值上相等，但含义不同。某一信源，不管它是否输出符号，只要这些符号具有某些概率特性，必有信源的熵值；这熵值是在总体平均上才有意义，因而是一个确定值。一般写成 $H(X)$，X 是指随机变量的整体 (包括概率分布) 。而另一方面，信息量则只有当信源输出符号而被接收者收到才有意义，这就是给予接收者的信息度量。

下面给出条件熵和联合熵的概念：

定义 3.8

在给定 y_j 条件下，x_i 的条件自信息量为 $I(x_i \mid y_j)$。X 集合的条件熵 $H(X \mid y_j)$ 定义为

$$H(X \mid y_j) = \sum_i p(x_i \mid y_j) I(x_i \mid y_j).$$

进一步地，在给定 Y（即各个 y_j）的条件下，X 集合的条件熵 $H(X \mid Y)$ 定义为

$$\begin{aligned} H(X \mid Y) &= \sum_j p(y_j) H(X \mid y_j) \\ &= \sum_{i,j} p(y_j) p(x_i \mid y_j) I(x_i \mid y_j) \\ &= \sum_{i,j} p(x_i, y_j) I(x_i \mid y_j). \end{aligned}$$

即条件熵是在联合符号集合 XY 上的条件自信息量的联合概率加权统计平均值。条件熵 $H(X \mid Y)$ 表示已知 Y 后 X 的不确定度。相应地. 在给定 X (即各个 x_i)) 的条件下, Y 集合的条件熵 $H(Y \mid X)$ 定义为

$$H(Y \mid X)=\sum_{i,j} p(x_i, y_j) I(y_j \mid x_i)=-\sum_{i,j} p(x_i, y_j) \log p(y_j \mid x_i).$$

♣

定义 3.9

联合熵是联合符号集合 XY 上的每个元素对 $x_i y_j$ 的自信息量的概率加权统计平均值，定义为:

$$H(XY)=\sum_{i,j} p(x_i, y_j) I(x_i, y_j)=-\sum_{i,j} p(x_i, y_j) \log p(x_i, y_j).$$

联合熵 $H(XY)$ 表示 X 和 Y 同时发生的不确定度。

♣

3.4 熵的基本性质

信息熵是信息不确定性的一个数学度量，本节给出信息熵的一些基本性质。

命题 3.1 (非负性)

$$H(X)=H(x_1, x_2, \ldots, x_n) \geq 0.$$

其中等号只有在对任意 $1 \leq i \leq n$，$x_i=1$ 时成立。

♠

命题3.1成立是因为 $0 \leq p(x_i) \leq 1$，从而 $\log p(x_i)$ 一定是一个负数，所以 $H(X)$ 是非负的。

命题 3.2 (对称性)

熵函数所有变量可以互换，而不影响函数值。即

$$H(x_1, x_2, \ldots, x_n)=H(x_2, x_1, \ldots, x_n).$$

♠

命题3.2成立是因为熵函数只与随机变量的总体结构有关，例如下列信源的熵都是相等的:

$$\begin{pmatrix} X \\ P \end{pmatrix}=\begin{pmatrix} x_1 & x_2 & x_3 \\ 1/3 & 1/6 & 1/2 \end{pmatrix}=\begin{pmatrix} Y \\ P \end{pmatrix}=\begin{pmatrix} y_1 & y_2 & y_3 \\ 1/3 & 1/6 & 1/2 \end{pmatrix}.$$

命题 3.3 (确定性)

$$H(0,1)=H(1,0,0,\ldots,0)=0.$$

也就是说只要信源符号表中，有一个符号的出现概率为 1，信源熵就等于零。

♠

命题3.3成立是因为在概率空间中，如果有两个基本事件，其中一个是必然事件，另一个则是不可能事件，因此没有不确定性，熵必为零。当然可以类推到 n 个基本事件构成的概率空间。

下面为了讨论熵的其它性质，首先给出关于凸函数的一个结论。

定义 3.10 (凸函数)

一个实值函数 f 称为在区间 I 上是凸的，如果对于任意 $x,y\in I$，$x\neq y$，都有

$$\frac{f(x)+f(y)}{2}\leq f\left(\frac{x+y}{2}\right).$$

函数 f 称为在区间 I 上是严格凸的，如果对任意 $x,y\in I$，$x\neq y$，都有

$$\frac{f(x)+f(y)}{2}< f\left(\frac{x+y}{2}\right).$$

♣

引理 3.1 (Jensen 不等式)

设 f 是区间 I 上的一个连续的单调严格凸函数，$\sum_{i=1}^{n}a_i=1$，$a_i\geq 0$，$1\leq i\leq n$，则

$$\sum_{i=1}^{n}a_i f(x_i)\leq f\left(\sum_{i=1}^{n}a_i x_i\right).$$

其中 $x_i\in I$，$1\leq i\leq n$。上式中的等号成立当且仅当 $x_1=x_2=\cdots=x_n$。

♡

容易证明，对数函数 $f(x)=\log_2 x$ 在区间 $I=(0,+\infty)$ 上是一个连续的严格凸函数。

现在我们继续给出关于熵的几个结论。下面的定理利用了对数函数 $f(x)=\log x$ 在区间 $I=(0,+\infty)$ 上是严格凸的性质。

命题 3.4

假设 X 是一个随机变量，概率分布为 $p_1,p_2,\ldots,p_n$，其中 $p_i\geq 0$，$1\leq i\leq n$。那么 $H(X)\leq\log n$。等号成立当且仅当 $p_i=1/n$,$1\leq i\leq n$。

♠

证明 应用 Jensen 不等式，可得：

$$\begin{aligned}H(x)&=-\sum_{i=1}^{n}p_i\log_2 p_i\\&=\sum_{i=1}^{n}p_i\log_2\frac{1}{p_i}\\&\leq\log_2\sum_{i=1}^{n}p_i\times\frac{1}{p_i}\\&=\log_2 n.\end{aligned}$$

等号成立当且仅当 $p_i=1/n$,$1\leq i\leq n$。

命题 3.5

$$H(X,Y)\leq H(X)+H(Y).$$

等号成立当且仅当 X 和 Y 统计独立。

♠

证明 假设 X 取值 x_i，$1\leq i\leq n$，Y 取值 y_j，$1\leq j\leq m$。可得：

$$p(x_i)=\sum_{j=1}^{m}p(x_i,y_j),\quad 1\leq i\leq n,$$

$$p(y_j)=\sum_{i=1}^{n}p(x_i,y_j),\quad 1\leq j\leq m.$$

所以

$$\begin{aligned} H(X)+H(Y) &= -\left[\sum_{i=1}^{n} p(x_i)\log_2 p(x_i)+\sum_{j=1}^{m} p(y_j)\log_2 p(y_j)\right] \\ &= -\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 p(x_i)p(y_j). \end{aligned}$$

由联合熵的定义和 Jensen 不等式可知

$$\begin{aligned} H(X,Y)-H(X)-H(Y) &= \sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 \frac{1}{p(x_i,y_j)} \\ &\quad +\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 p(x_i)p(y_j) \\ &= \sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 \frac{p(x_i)p(y_j)}{p(x_i,y_j)} \\ &\le \log_2 \sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i)p(y_j) \\ &= \log_2 1 \\ &= 0. \end{aligned}$$

等号成立当且仅当对任意的 $1\le i\le n$ 和 $1\le j\le m$，$p(x_i)p(y_j)/p(x_i,y_j)=C$，$C$ 是一个常数。因为

$$\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)=\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i)p(y_j)=1,$$

所以 $C=1$，即对任意的 $1\le i\le n$ 和 $1\le j\le m$，$p(x_i,y_j)=p(x_i)p(y_j)$。因此，当且仅当 X 与 Y 相互独立等号成立，命题得证。

命题 3.6

$$H(X,Y)=H(Y)+H(X\mid Y)=H(X)+H(Y\mid X).$$

♠

证明

$$\begin{aligned} H(X,Y) &= -\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 p(x_i,y_j) \\ &= -\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 p(y_j)p(x_i\mid y_j) \\ &= -\sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 p(y_j) - \sum_{i=1}^{n}\sum_{j=1}^{m} p(x_i,y_j)\log_2 p(x_i\mid y_j) \\ &= -\sum_{j=1}^{m} p(y_j)\log_2 p(y_j) - \sum_{i=1}^{n}\sum_{j=1}^{m} p(y_j)p(x_i\mid y_j)\log_2 p(x_i\mid y_j) \\ &= H(Y)+H(X\mid Y). \end{aligned}$$

同理可证明 $H(X,Y)=H(X)+H(Y\mid X)$。

命题 3.7

$H(X\mid Y)\le H(X)$。等号成立当且仅当 X 与 Y 相互独立。

♠

证明 由命题3.5和3.6可得。

例 3.6 假设明文 $P=\{a,b\}$ 满足 $p(a)=1/4$, $p(b)=3/4$; 设密钥 $K=\{k_1,k_2,k_3\}$, $p(k_1)=1/2$, $p(k_2)=1/4$, $p(k_3)=1/4$; 设密文 $C=\{1,2,3,4\}$, 且加密矩阵如下:

	a	b
k_1	1	2
k_2	2	3
k_3	3	4

设 p 是明文空间 P 上的随机变量, c 是密文空间 C 上的随机变量, k 是密钥空间 K 上的随机变量。计算熵 $H(p)$, $H(k)$, $H(c)$。

解 根据熵的定义可得:

$$\begin{aligned}
H(p) &= -p(a)\times\log_2 p(a) - p(b)\times\log_2 p(b)\\
&= -(1/4)\times\log_2(1/4) - (3/4)\times\log_2(3/4)\\
&= -(1/4)\times(-2) - (3/4)\times(\log_2 3-2)\\
&= 2-(3/4)\log_2 3\\
&= 0.81,\\
H(k) &= -p(k_1)\times\log_2 p(k_1) - p(k_2)\times\log_2 p(k_2) - p(k_3)\times\log_2 p(k_3)\\
&= -(1/2)\times\log_2(1/2) - (1/4)\times\log_2(1/4) - (1/4)\times\log_2(1/4)\\
&= 1.5.
\end{aligned}$$

由于密钥的选取不依赖与明文, 所以可以假设明文和密钥是相互独立的。要求 $H(c)$ 必须先求出 $p(1)$, $p(2)$, $p(3)$, $p(4)$。可得:

$$\begin{aligned}
p(1) &= p(a)\times p(k_1)\\
&= (1/4)\times(1/2)\\
&= 1/8;\\
p(2) &= p(a)\times p(k_2) + p(b)\times p(k_2)\\
&= (1/4)\times(1/4) + (3/4)\times(1/4)\\
&= 7/16;\\
p(3) &= p(a)\times p(k_3) + p(b)\times p(k_3)\\
&= (1/4)\times(1/4) + (3/4)\times(1/4)\\
&= 1/4;\\
p(4) &= p(b)\times p(k_3)\\
&= (3/4)\times(1/4)\\
&= 3/16.
\end{aligned}$$

最终计算得

$$\begin{aligned}
H(c) &= -(1/8)\log_2(1/8) - (7/16)\log_2(7/16) - (1/4)\log_2(1/4) - (3/16)\log_2(3/16)\\
&= 1.85.
\end{aligned}$$

3.5 伪密钥与唯一解距离

在本节主要讨论在密文攻击下破译一个密码系统，密码分析者必须处理的密文量的理论下界。首先我们证明在密码体制的组成部分的熵之间存在一个基本关系。条件熵 $H(K \mid C)$ 称为密钥暧昧度，它表示在已知密文的情况下能泄露多少密钥的一种度量。

命题 3.8

设 (P, C, K, E, D) 是一个密码体制，那么

$$H(K \mid C) = H(K) + H(P) - H(C).$$

♠

证明 由命题3.6可得

$$H(K, P, C) = H(C \mid K, P) + H(K, P) = H(P \mid K, C) + H(K, C).$$

由于密钥和明文唯一确定密文，密钥和密文唯一确定明文，所以

$$H(C \mid K, P) = H(P \mid K, C) = 0.$$

密钥和明文相互独立，所以

$$H(K, P) = H(K) + H(P).$$

因此

$$\begin{aligned} H(K \mid C) &= H(K, C) - H(C) \\ &= H(K, P, C) - H(C) \\ &= H(K, P) - H(C) \\ &= H(K, P) - H(C) \\ &= H(K) + H(P) - H(C). \end{aligned}$$

假定所使用的密码系统是 (P, C, K, E, D)，明文字符串 $x_1, x_2, \ldots, x_n$ 在某个密钥的控制下被加密成密文字符串，密码分析者的目的是确定密钥。在惟密文攻击的情况下，我们假定密码分析者具有无限的计算资源，也假定分析者知道明文是某一自然语言，例如英语。一般情况下，密码分析者可能分析得到许多可能的密钥，但其中只有一个是正确的密钥，其他可能的但不正确的密钥称为伪密钥。

对于任意一个密文，用不同的密钥进行解密，如果得到的“有意义”的明文越多，则伪密钥越多。这时判断哪个密钥是正确密钥的难度就越大。当然密码分析者希望伪密钥的数目为 0。

我们的目的是得到伪密钥的期望数的下界。在此首先给出自然语言的熵和冗余度的概念。

定义 3.11

假设 L 是一种自然语言，语言 L 的熵定义为

$$H_L = \lim_{n \to \infty} \frac{H(X^{(n)})}{n}.$$

语言 L 的冗余度定义为

$$R_L = 1 - \frac{H_L}{\log_2 |X|}.$$

其中，X 表示语言 L 的字符集，$X^{(n)}$ 是 X^n 上的随机变量。

♣

H_L 表示自然语言 L 中每个字符的熵，它是明文符串中每个字符的平均信息量的度量。对于字符表

X 上的一个随机语言 L，因为 $X^{(n)}$ 是 X^n 上的均匀随机变量，所以

$$\begin{aligned} H(X^{(n)}) &= -\sum_{x\in X^n} P(x)\log_2 P(x) \\ &= -\sum_{x\in X^n} \frac{1}{|X|^n}\log_2 \frac{1}{|X|^n} \\ &= n\log_2|X|. \end{aligned}$$

随机语言 L 的熵为 $H_L=\log_2|X|$。因此，自然语言 L 的冗余度 R_L 是自然语言 L 中“多余字符”的度量。

定理 3.4

假定 (P,C,K,E,D) 是一个私钥密码体制，且 $|C|=|P|$。假设 R_L 表示自然语言的冗余度，那么给定一个充分长（长为 n）的密文串，伪密钥的期望数 $\overline{S_n}$ 满足

$$\overline{S_n} \geq \frac{2^{H(K)}}{|P|^{nR_L}} - 1.$$

♡

证明 设明文字符集为 X，明文空间为 $P=X^L$，L 为一正整数。给定 $Y\in C^n$，定义 $K(Y)=\{k\in K \mid$ 存在$x\in P^n$使得$P(x)\geq 0,\ E_k(x)=Y\}$，即 $K(Y)$ 是在给定密文串 Y 的条件下可能的密钥的集合。如果 Y 是密文串，因为在可能的密钥中只有一个是正确的，那么伪密钥的数目是 $|K(Y)|-1$，因此可得到伪密钥的期望数为

$$\begin{aligned} \overline{S_n} &= \sum_{Y\in C^n} P(Y)(|K(Y)|-1) \\ &= \sum_{Y\in C^n} P(Y)|K(Y)| - \sum_{Y\in C^n} P(Y) \\ &= \sum_{Y\in C^n} P(Y)|K(Y)| - 1. \end{aligned}$$

根据命题3.8可知

$$H(K\mid C^n) = H(K)+H(P^n)-H(C^n).$$

又因为 $H(C^n)\leq \log_2|C|$，当 n 充分大时，根据语言熵的定义可得

$$H(P^n)=H(X^{nL})\approx nLH_L = nL(1-R_L)\log_2|X| = n(1-R_L)\log_2|P|.$$

所以当 $|P|=|C|$ 时

$$H(K\mid C^n)\geq H(K)-nR_L\log_2|P|.$$

根据 Jensen 不等式和 $\overline{S_n}=\sum_{Y\in C^n}P(Y)|K(Y)|-1$ 可得

$$\begin{aligned} H(K\mid C^n) &= \sum_{Y\in C^n} P(Y)H(K\mid Y) \\ &\leq \sum_{Y\in C^n} P(Y)\log_2|K(Y)| \\ &\leq \log_2 \sum_{Y\in C^n} P(Y)|K(Y)| \\ &= \log_2(\overline{S_n}+1). \end{aligned}$$

由上面的两个式子可得结论。

定义 3.12

一个密码体制的唯一解距离是指使得伪密钥的期望数目 $\overline{S_n}$ 等于零的密文长度 n，记为 n_0。 ♣

一个密码体制的唯一解距离就是密码分析者在有足够的计算时间的情况下，能够唯一地计算出正确密钥所需的密文的平均长度。

令 $\overline{S_n}=0$。由 $\overline{S_n}\geq 2^{H(K)}/|P|^{nR_L}-1=0$ 可以得到唯一解的一个近似值，即

$$n_0\approx\frac{H(K)}{R_L\log_2|p|}.$$

这表明，当截获的密文量大于 n_0 时，原则上可以破译该密码。当截获的密文量小于 n_0 时，就存在有多种可能的密钥解，密码分析者无法从中确定哪一个解是正确的。当然 n_0 仅仅是一个理论值，一般破译密码系统所需要的密文量都远大于 n_0。

3.6 乘积密码体制

Shannon 在其 1949 年的论文《秘密系统的通信理论》中介绍的一个新思想就是通过“乘积”来组合密码体制。也就是说，为了加强密码系统的保密性，常常采用多个密码复合的方法，也就是所谓的“乘积密码”。

为简单起见,在此我们只讨论明文空间 P 和密文 C 空间相同的密码体制的情况。设 $R_1=(P,C,K_1,E_1,D_1)$ 和 $R_2=(P,C,K_2,E_2,D_2)$ 是两个具有相同明文空间和密文空间的密码体制。那么 R_1 和 R_2 的乘积密码体制定义为 (P,C,K_1*K_2,E,D)，记为 R_1*R_2。乘积密码体制 R_1*R_2 中的加密变换和解密变换都很简单。对任意的明文 $x\in P$ 和密钥 $k=(k_1,k_2)$，加密变换为:

$$E_k(x)=E_{k_2}(E_{k_1}(x)).$$

对任意的密文 $y\in C$ 和密钥 $k=(k_1,k_2)$，解密变换为

$$D_k(y)=D_{k_1}(D_{k_2}(y)).$$

也就是说，我们首先用 k_1 加密 x，然后再用 k_2 加密，则可得到密文。解密是类似的，但必须以相反的密钥顺序进行。显然有

$$\begin{aligned}D_k(E_k(x))&=D_k(E_{k_2}(E_{k_1}(x)))\\&=D_{k_1}(D_{k_2}(E_{k_2}(E_{k_1}(x))))\\&=D_{k_1}(E_{k_1}(x))\\&=x.\end{aligned}$$

ADFGVX 乘积密码是一种有名的乘积密码，同时采用代替和换位的方法，如表3.1所示。第一次世界大战期间德军曾经使用过，并被英国人破译。这种密码有一个 6 行 6 列的表，行列分别用 ADFGVX 标号，在表中随机的填入 26 个英文字母 A~Z 和 10 个数字 0 ~ 9，加密分为两步：第一步将明文字母用其所在的行和列的标号对代替，然后把中间结果按行的顺序写成一个换位长方形的形式，第二步根据密钥所示的顺序按列写出字母，就可以得到密文。

表 3.1: ADFGVX 乘积密码表

	A	D	F	G	V	X
A	K	Z	W	R	1	F
D	9	B	6	C	L	5
F	Q	7	J	P	G	X
G	E	V	Y	3	A	N
V	8	O	D	H	0	2
X	U	4	I	S	T	M

例如：待加密的明文为 PASSWORD，则首先将明文字母用其所在的行和列的标号对代替，得中间结果

FG GV XG XG
AF VD AG VF

设加密密钥为 computer，将加密密钥的顺序对于于替换后的明文，即

1	4	3	5	8	7	2	6
F	G	G	V	X	G	X	G
A	F	V	D	A	G	V	F

根据密钥顺序列出字母则得密文：FA VD GV XA GF XV GF GG。

对于乘积密码体制中密钥空间的概率分布，我们假设 K_1 中密钥的选取和 K_2 中密钥的选取是相互独立的。因此，对任意的 $k=(k_1,k_2)\in k_1*k_2$，我们有 $p(k_1,k_2)=p(k_1)p(k_2)$。

如果 $R_1*R_2=R_2*R_1$，也就是说乘积密码体制 R_1*R_2 和 R_2*R_1 是两个相同的密码体制，则我们称密码体制 R_1 和 R_2 是可交换的，当然并不是任意两个密码体制都是可交换的。

显然，密码体制的乘积运算满足结合律，即对任意的 R_1、R_2 以及 R_3，都有

$$(R_1*R_2)*R_3=R_1*(R_2*R_3).$$

这里 R_1、R_2 以及 R_3 是明文空间和密文空间相同的密码体制。

设 R 是一个明文空间和密文空间相同的密码体制，

$$R=R^n=\underbrace{R*R*\cdots*R}_{n}$$

称为迭代密码体制。如果 $R^2=R$，则称 R 是幂等的密码体制。乘法密码、仿射密码、置换密码、Hill 密码以及 Vigenere 密码等都是幂等的密码体制。当然，如果 R 是一个幂等的密码体制，则就没有必要使用迭代密码体制，因为 R^n 需要更多的密钥但其安全性能与 R 一样。如果 R 不是一个幂等的密码体制，则对 $n>1$，迭代密码体制 R^n 的安全性能可能会比 R 强。这种通过对一个密码体制进行迭代来提高其安全性能的思想被广泛应用于分组密码体制的设计之中。

典型的迭代密码定义了一个轮函数和一个密钥编排方案，对明文的加密将经过多轮迭代。设 K 是一个定长的主密钥，密钥编排方案用 K 生成 N_r 个轮子密钥 K_i。轮函数 g 以轮密钥 K_i 和当前状态 w_{i-1} 作为输入。初始状态 w_0 被定义为明文 x，密文 y 定义为经过所有 N_r 轮后的状态 w_{N_r}。加密过程是：

$$w_0=x,\ \ w_i=g(w_{i-1},K_i),\ \ i=1,2,\ldots,N_r.$$

解密过程是：

$$w_{Nr}=y,\ \ w_{i-1}=g^{-1}(w_i,K_i),\ \ i=N_r,N_r-1,\ldots,1.$$

代换一置换网络 SPN (substitution-permutation network) 是一类特殊的迭代密码，只是在某些地方有些小的变化。代换一置换网络的轮函数包括三个变换：代换、置换和密钥混合。

我们考虑一个简单的 4 轮迭代的代换一置换网络。16 位明文分成 4 个 4 位的子块，每一子块输入到一个 4×4 的 S 盒，S 盒是一个 4 位输入得到 4 位输出的代换。S 盒的一个基本性质是要求它为一个非线性映射。4 个 S 盒的输出构成了 16 位的位串，该 16 位的位串经过置换完成该轮操作，对于代换一置换网络在 5.7 节将有详细的讲述。

乘积密码的思想在现代密码体制的设计中具有重要意义，第 5 章将要讲述的数据加密标准 DES 就是一种典型的乘积密码。

第 3 章 习题

1. 考虑一个密码体制 $M=\{a,b,c\}$，$K=\{k_1,k_2,k_3\}$，$C=\{1,2,3,4\}$。假设加密矩阵为

	a	b	c
k_1	2	3	4
k_2	3	4	1
k_3	1	2	3

已知密钥概率分布为：$p(k_1)=1/2$，$p(k_2)=p(k_3)=1/4$，且明文概率分布为 $p(a)=1/3$，$p(b)=8/15$，$p(c)=2/15$。计算 $H(M)$，$H(K)$，$H(C)$，$H(M\mid C$，$H(K\mid C)$。

2. 考虑一个密码系统 $\{P,K,C\}$。
   1) 说明为什么 $H(P,K)=H(C,P,K)=H(P)+H(K)$。
   2) 假设这个系统具有完全保密性。证明 $H(C,P)=H(C)+H(P)$ 和 $H(C)=H(K)+H(K\mid C,P)$。
   3) 假设这个系统有完全保密性，并且对每一个明文密文对，最多只有一个相应的密钥能够加密。证明 $H(C)=H(K)$。
3. 假设在“一次一密”密码中，密文 y 和 y'（两个 2 进制的 n 元数组）是使用同一个密钥 K 分别加密明文 x 和 x' 得到的。证明 $x+x'=y+y' \bmod 2$。
4. 假设 S_1 是移位密码（密钥等概率），S_2 是密钥满足概率分布 P_k（不必是等概率的）的移位密码。证明 $S_1 * S_2 = S_1$。

第 4 章　序列密码

序列密码可以认为起源于 Vernam 体制，当 Vernam 体制的密钥序列是随机的 0、1 序列时，它就是“一次一密”密码体制。Shannon 已经证明“一次一密”密码体制在理论上是不可破译的。“一次一密”密码体制在理论上不可破译的这一事实使人们感觉到，如果能以某种方式仿效“一次一密”密码，则将可以得到保密性很高的密码体制。随着电子计算机技术和数学理论的发展，基于伪随机序列的序列密码得到了长足的发展与应用。目前，序列密码的理论已经比较成熟，而且具有工程实现容易、效率高等特点，序列密码是应用与各国军事和外交等领域的的主要密码体制之一。

4.1 序列密码的基本结构

4.1.1 序列密码的加解密方式

序列密码的加密和解密思想比较简单，就是用一个随机序列与明文序列进行叠加来产生密文，用同一个随机序列与密文序列进行叠加来恢复明文。

若设明文为 $m = m_0m_1m_2\ldots$，密钥为 $k = k_0k_1k_2\ldots$，加密后的密文为 $c = c_0c_1c_2\ldots$，则加密变换为 $c_i = m_i \oplus k_i$，解密变换 $m_i = c_i \oplus k_i$，其中 m, k, c 是 0、1 随机序列，$\oplus$ 表示模 2 加法运算。

序列密码的安全性主要依赖于密钥序列。当密钥序列是均匀分布的离散无记忆信源产生的随机序列时，相应的序列密码就是“一次一密”密码。在序列密码中，加密和解密所用的密钥序列是相同的。于是，序列密码的关键就是产生密钥序列的算法。密钥序列产生算法应能产生随机性和不可预测性好的密钥序列。保持通信双方的精确同步是序列密码实际应用中的关键技术。由于通信双方必须能够产生相同的密钥序列，所以这种密钥序列不可能是真随机序列，只能是伪随机序列，即具有良好随机性和不可预测性的伪随机序列。图4.1为序列密码的加密与解密示意图。

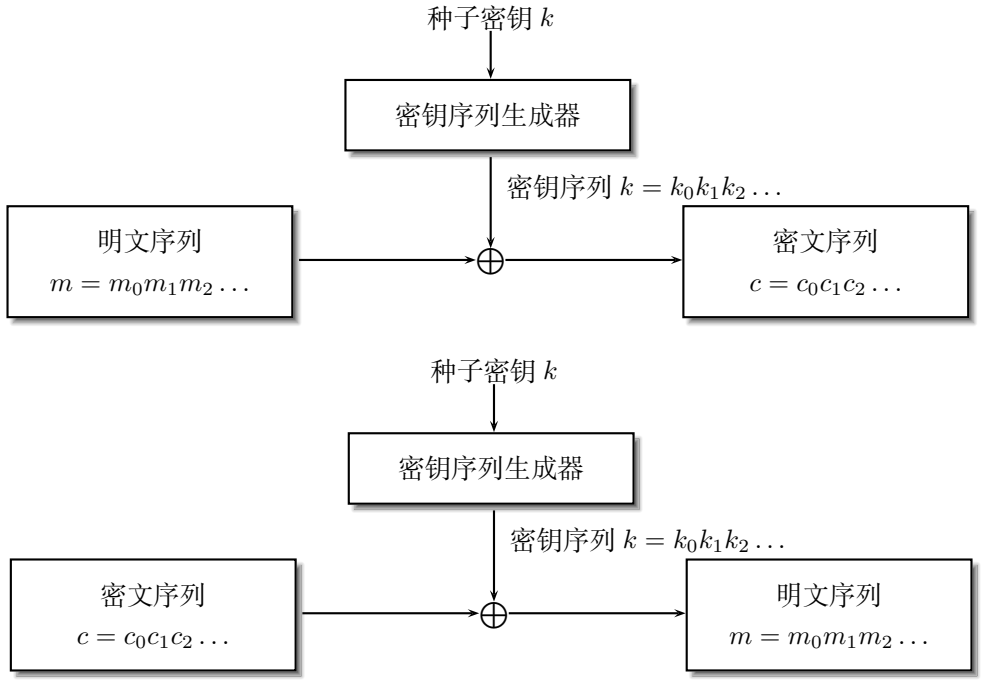


图 4.1: 序列密码的加密与解密

如果密钥序列产生算法与明文无关，则所产生的密钥序列也与明文无关，我们称这类序列密码为同步序列密码。对于同步序列密码，只要通信双方的密钥序列产生器具有相同的种子密钥和相同的初始状态，就能产生相同的密钥序列。在保密通信过程中，通信的双方必须保持精确的同步，收方才能正确解密，如果不同步收方将不能正确解密。例如，如果通信中丢失或增加了一个密文字符。则收方将一直错误，直到重新同步为止，这是同步序列密码的一个主要缺点。但是同步序列密码对失步的敏感性，使我们能够容易检测插入、删除、重播等主动攻击。同步序列密码的一个优点是没有错误传播，当通信中某些密文字符产生了错误，只影响相应字符的解密，不影响其他字符。

如果密钥序列产生算法与明文（密文）相关，则所产生的密钥序列与明文（密文）相关，我们称这类序列密码为自同步序列密码。设密钥序列产生器具有 n 位存储，则其密文位 c_i 不仅与当前的明文位 m_i 相关，而且与后面的 n 个明文位 m_{i+1}, m_{i+2} m_{i+3} $\ldots$, m_{i+n} 相关。假设在通信过程中密文位 c_i 发生了错误，则导致明文位 m_i 错误，而且还导致后 n 个明文位 m_{i+1}, m_{i+2}, m_{i+3}, $\ldots$, m_{i+n} 也发生错误，造成错误传播。

4.1.2 密钥流与密钥流生成器

一个破译密码的算法，若计算量大于或等于穷举搜索，则不会被视为一个破译方法。若一个加密算法没有比穷举搜索更好的破译方法，则被认为是不可破的。如果一个密钥流序列是完全随机的，则没有比穷搜索更好的方法破译它。

在实际中使用的密钥流序列都是按一定密钥流生成算法生成的。因而不可能是完全随机的，所以也就不可能是完善保密系统。为了尽可能提高系统的安全强度、必须要求所产生的密钥流序列尽可能具有随机序列的某些特征。一般地，序列密码中对密钥流有如下要求：

(1) 极大的周期。因为随机序列是非周期的，而按任何算法产生的序列都是周期的，因此应要求密钥流具有尽可能大的周期。

(2) 良好的统计特性。随机序列有均匀的游程分布。游程指序列中相同符号的连续段，其前后均为异种符号。连续段的比特位数称为该游程的长度，如 $\ldots, 11100001\ldots$ 中 3 个段分别为长为 3 的 1 游程、长为 4 的 0 游程、长为 1 的 1 游程。一般对周期为 2^n 的序列，可要求其在一周期内满足：同样长度的 0 游程和 1 游程的个数相等或近似相等。

(3) 不能用级数较小的线性移位寄存器近似代替，即要有很高的线性复杂度。

(4) 用统计方法由密钥序列 $k_0k_1k_2\ldots k_i\ldots$ 提取密钥生成器结构或种子密钥的足够信息在计算上是不可能的。

这些要求对于保证序列密码的安全性是必需的。因为按任何确定性算法产生的序列都是周期性的。若密钥周期 P 很短，则从两组密文

$$m_1 \oplus k_1, \ldots, m_L \oplus k_L \text{ 和 } m_{P+1} \oplus k_{P+1}, \ldots, m_{P+L} \oplus k_{P+L}$$

相加得 $m_1 \oplus m_{P+1}$, $\ldots$, $m_L \oplus m_{P+L}$，结合语言冗余度，就可获得一些关于明文的信息，因而周期长是必要的；良好的随机统计特性是为了更好地掩盖明文。高线性复杂度可用于防止从部分密钥序列通过线性关系简单推出整个密钥流序列。若知道一些明密文对 (m_i, c_i), (m_{i+1}, c_{i+1}), $\ldots$, 便可简单地确定部分密钥序列 k_i, k_{i+1}, ...。因此，安全的密码系统应能抵抗从部分密钥 k_i, k_{i+1}, ... 确定整个密钥序列的攻击。

以上要求对保证系统安全性是必要的，但不是充分的。随着对安全问题研究的深入，某种新的攻击方法的出现以及设计密钥流生成器的方法不同，为确保系统安全性还会提出一些更强的要求。

图4.2给出了密钥流生成器的一般结构。序列密码的密钥流生成器通常包含两部分：初始化部分和密钥流生成部分。序列密码的初始化部分对初始状态进行混合（Mixing），其中初始状态由原始数据构成，通常包含种子密钥 Key 和初始向量 IV 两部分。初始化之后得到的内部状态，通过密钥流生成部分产生密钥流比特。密钥流生成部分为迭代操作，可视为有限状态机，包括状态更新函数 f 和密钥导出函数 g，其

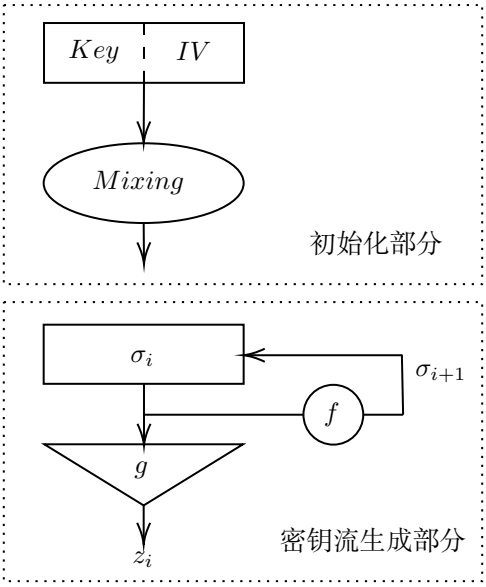


图 4.2: 密钥流生成器的一般结构

数学表示如下：

$$\sigma_{i+1} = f(\sigma_i),$$
$$z_i = g(\sigma_i).$$

在具体的密钥流生成器的设计中，通常安全性要求越高，设计越复杂。因此，在考虑安全性要求的前提下还应考虑以下两个因素：

(1) 密钥 K 易于分配、保管、更换；

(2) 易于实现、速度快。

为了满足这些要求，目前密钥流生成器大都是基于移位寄存器的，因为移位寄存器结构比较简单，易于实现且运行速度快。这种基于移位寄存器的密钥流序列称为移位寄存器序列。在序列密码发展历程中，基于移位寄存器的密钥流生成器设计经历了如下几个阶段：

(1) 基于线性移位寄存器的密钥流生成器；

(2) 基于线性移位寄存器驱动的非线性滤波和非线性组合生成器；

(3) 基于线性移位寄存器驱动的不规则钟控生成器和带记忆的生成器；

(4) 基于带进位反馈移位寄存器的密钥流生成器；

(5) 基于非线性反馈移位寄存器的密钥流生成器。

线性移位寄存器 (LFSR) 在序列密码发展过程中有着举足轻重的地位。早期的序列密码直接基于单个线性移位寄存器或多个线性移位寄存器的线性操作设计，20 世纪 60 年代，随着 Berlekamp-Massey 算法的提出，这类设计不再安全，此后一些学者提出了基于线性移位寄存器更新内部状态（称为线性移位寄存器驱动）、基于非线性函数导出密钥流的改造方式。如图4.3所示的密钥流生成器。其中，图 (a) 由一个线性移位寄存器和一个滤波器构成。图 (b) 由多个线性移位寄存器和一个组合器构成。通常将这类生成器分解成两部分，其中线性移位寄存器部分称为驱动部分，另一部分称为非线性输出部分。其工作原理都是将驱动部分即线性移位寄存器在 j 时刻的状态变量 x 作为一组值输入非线性组合部分的 f，将 $f(x)$ 的输出作为当前时刻的 k_j。驱动部分负责负责状态转移，从而提供非线性组合部分使用的周期大、统计性能好的序列，而非线性组合部分以各时刻移位寄存器的状态组合出密钥序列 j 时刻的值 k_j。为了抵抗针对此类结构的一系列攻击方法，对非线性布尔函数 f 的设计提出了很高的要求，这也导致数学上对布尔

函数安全性指标的研究。

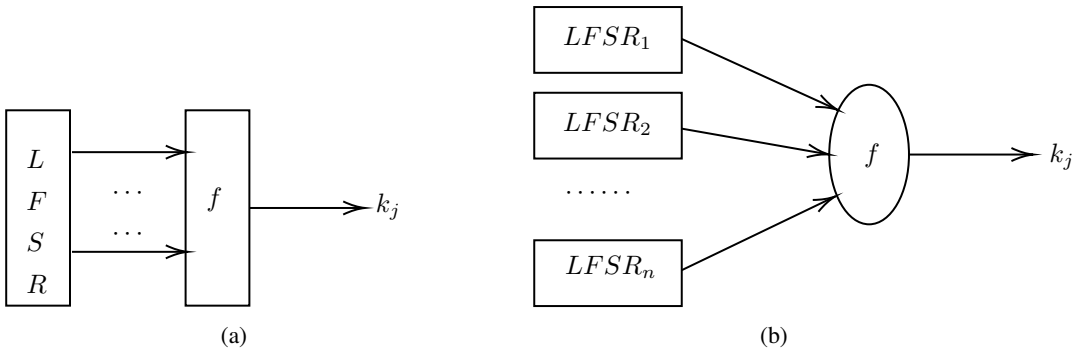


图 4.3: 密钥流生成器

进入 21 世纪，一些国家或地区发起的密码算法设计项目对序列密码的发展起到了重要的推动作用。几个典型的项目及相关序列密码算法包括：

(1) 2000 年，欧盟启动了 NESSIE 项目该项目为期 3 年，通过评估所提交的密码算法，旨在提出一个包含各种类型的、强大的密码算法组合。NESSIE 共评估了 6 个序列密码算法，分别是 BMGL、Leviathan、LILI-128、SNOW 1.0、SOBER-t16 和 SOBER-t32，但这 6 个算法在安全性方面均存在一些问题，因此最终 NESSIE 项目并未标准化任何序列密码算法；

(2) 2000 年，日本政府提出了 CRYPTREC 项目，其目的是评估和推荐一些密码算法用于日本电子政务系统。CRYPTREC 评估推荐的序列密码算法有 KCipher-2、MUGI、Enocoro、MULTI-S01 等。该项目一直持续至今，目前被采用的序列密码算法为 KCipher-2；

(3) 2004 年，由欧盟资助的 ECRYPT 宣布了 eSTREAM 项目，目的是促进设计更适合广泛应用的、高效紧凑的序列密码。eSTREAM 持续到 2008 年，共征集到 34 个算法，经过多次安全和性能评估，最终选择了其中 7 算法，包括面向软件实现的算法 Salsa20/12、SOSEMANUK、Rabbit、HC-128，以及面向硬件实现的算法 Grain v1、Trivium、MICKEY v2。

在 21 世纪，非线性移位寄存器（NLFSR）在序列密码算法的设计中得到越来越多的应用，其典型代表为 Trivium 算法和 Grain 算法，这两个算法均为 eSTREAM 项目的胜选算法，且均已被 ISO 标准化。此外，经典的基于线性移位寄存器的设计也并未被摒弃，发展为基于“字”的密钥流生成器设计方法，其典型代表为 SNOW 3G 和 ZUC 算法，这两个算法均为 3GPP 标准化的、面向 4G 时代移动通信应用的序列密码算法。其中 ZUC 算法又称祖冲之算法，是由我国自主设计的、成为国际标准的序列密码算法，同时也是中国的国家密码标准。

除了上述基于移位寄存器的密钥流生成器，在序列密码发展过程中还存在其它几类不同的设计方式，主要包括：基于随机洗牌的密钥流生成器、基于 ARX 结构的密钥流生成器、基于分组密码的密钥流生成器、基于分组密码工作模式的密钥流生成器、基于 PANAMA 结构的密钥流生成器、基于 T-函数的密钥流生成器等等。其中 RC4 算法是一个基于对状态表进行随机洗牌设计的序列密码算法，它是传输层安全协议 TLS1.2 中所采用的序列密码算法；ChaCha20 是一个基于 ARX 结构设计的序列密码算法，它取代了 RC4 成为 TLS1.3 中所采用的序列密码算法；AES-GCM 是基于分组密码的 GCM 工作模式设计的序列密码算法，它是除 SNOW 3G 和 ZUC 之外另一个面向 4G 移动通信应用的序列密码标准。

4.2 线性反馈移位寄存器序列

移位寄存器是序列密码产生密钥序列的一个主要组成部分。$GF(2)$ 上一个 n 级反馈移位寄存器由 n 个二元存储器与一个反馈函数 $f(a_1a_2.....a_n)$ 组成，如图4.6所示。每一个存储器称为移位寄存器的一级。

在任一时刻, 这些级的内容构成该反馈移位寄存器的状态。这个反馈移位寄存器的状态对应于一个 $GF(2)$ 上的 n 维向量，共有 2^n 种可能的状态。每一时刻的状态可用 n 长序列 $a_1, a_2, a_3, \ldots, a_n$ 或 n 维行向量 $(a_1, a_2, a_3, \ldots, a_n)$ 表示，其中 a_i 是第 i 级存储器的内容。

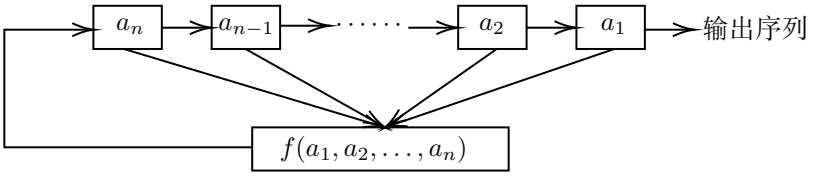


图 4.4: $GF(2)$ 上的 n 级反馈移位寄存器

每一级存储器 a_i 都将其内容向下一级 a_{i-1} 传递，并根据存储器当前状态计算 $f(a_1, a_2, a_3, \ldots, a_n)$ 作为 a_n 下一时间的内容。函数 $f(a_1, a_2, a_3, \ldots, a_n)$ 称为反馈函数，其中 $f(a_1, a_2, a_3, \ldots, a_n)$ 是 n 元布尔函数，即 n 个变元 $a_1, a_2, a_3, \ldots, a_n$ 可以独立地取 0 和 1 这两个可能的值，对 n 个变元 $a_1, a_2, a_3, \ldots, a_n$ 作逻辑与、逻辑或、逻辑补等运算，最后的函数值也为 0 或 1。如果当前反馈移位寄存器的状态为

$$S_t = (a_t, a_{t+1}, \ldots, a_{t+n-1}),$$

则

$$a_{t+n} = f(a_t, a_{t+1}, \ldots, a_{t+n-1}),$$

这个 a_{t+n} 是移位寄存器的输入。在 a_{t+n} 的驱动下，移位寄存器的各个数据向前推移一位，使状态变为

$$S_{t+1} = (a_{t+1}, a_{t+2}, \ldots, a_{t+n}),$$

同时，整个移位寄存器的输出为 a_t。由此我们得到一个反馈移位寄存器序列 $a_1, a_2, a_3, \ldots, a_n, \ldots$

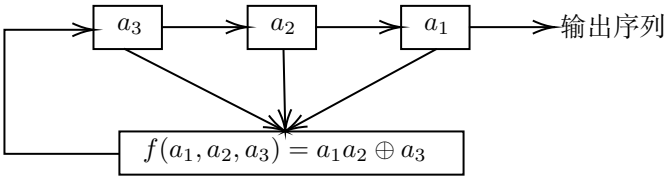


图 4.5: 一个 3 级反馈移位寄存器

图4.5是一个三级反馈移位寄存器，其初始状态为 $(a_1, a_2, a_3) = (1, 0, 1)$，输出可由下表求出，其输出序列为 $10111011101\ldots\ldots$，周期为 4。

状态 (a_3, a_2, a_1)			输出
1	0	1	1
1	1	0	0
1	1	1	1
0	1	1	0
1	0	1	1
1	1	0	0
$\cdots$	$\cdots$	$\cdots$	$\cdots$

表 4.1

如果移位寄存器的反馈函数 $f(a_1, a_2, \ldots, a_n)$ 是 $a_1, a_2, \ldots, a_n$ 的线性函数，则称之为线性反馈移位

寄存器 LFSR（Linear Feedback Shift Register）。此时反馈函数 f 可写为

$$f(a_1, a_2, \ldots, a_n) = c_n a_1 \oplus c_{n-1} a_2 \oplus \ldots \oplus c_1 a_n$$

其中常数 $c_i = 0$ 或 1，$\oplus$ 是模 2 加法。在二进制下 $c_i = 0$ 或 1 可用开关的断开和闭合来实现，如图4.6所示。

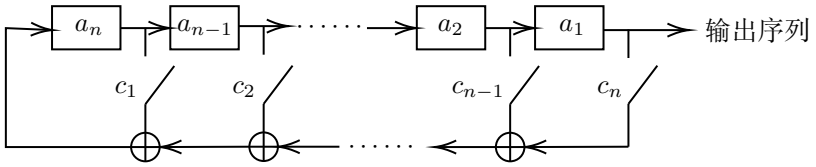


图 4.6: $GF(2)$ 上的 n 级线性反馈移位寄存器

输出序列 $\{a_t\}$ 满足

$$a_{n+t} = c_n a_t \oplus c_{n-1} a_{t+1} \oplus \cdots \oplus c_1 a_{n+t-1},$$

其中 t 为非负整数。

线性反馈移位寄存器因其实现简单、速度快、有较为成熟的理论等优点而成为构造密钥流生成器的最重要的部件之一。

例 4.1 图4.7是一个五级线性反馈移位寄存器，其初始状态为 $(a_1, a_2, a_3, a_4, a_5) = (1, 0, 0, 1, 1)$，可求出输出

$$10011010010000101011011000011111001 10\ldots\ldots,$$

周期为 31。

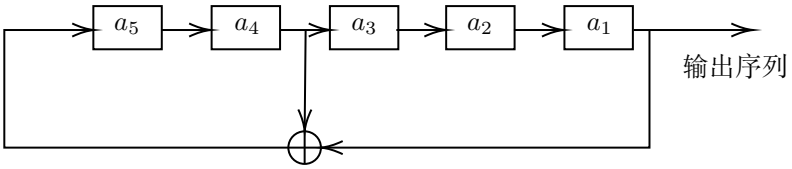


图 4.7: 一个 5 级线性反馈移位寄存器

在线性反馈移位寄存器中总是假定 $c_1, c_2, \ldots, c_n$ 中至少由一个不为 0，否则 $f(a_1, a_2, \ldots, a_n) = 0$。这样的话，在 n 个脉冲后状态必然是 $00\ldots0$，且这个状态必将一直持续下去。若只有一个系数不为 0，设仅有 c_j 不为 0，实际上是一种延迟装置。一般对于 n 级线性反馈移位寄存器，总是假定 $c_n = 1$。

线性反馈移位寄存器输出序列的性质完全由其反馈函数决定。n 级线性反馈移位寄存器最多有 2^n 个不同的状态。若其初始状态为 0，则其状态恒为 0。若其初始状态非 0，则其后继状态不会为 0。因此 n 级线性反馈移位寄存器的状态周期小于等于 $2^n - 1$。其输出序列的周期与状态周期相等，也小于等于 $2^n - 1$。只要选择合适的反馈函数便可使序列的周期达到最大值 $2^n - 1$，周期达到最大值的序列称为 m-序列。

4.2.1 线性移位寄存器的幂级数表示

设 n 级线性移位寄存器的输出序列 $\{a_i\}$ 满足递推关系

$$a_{n+k} = c_1 a_{n+k-1} \oplus c_2 a_{n+k-2} \oplus \cdots \oplus c_n a_k$$

对任何 $k \geq 1$ 成立。将这种递推关系用一个一元高次多项式

$$p(x) = 1 + c_1 x + \cdots + c_{n-1} x^{n-1} + c_n x^n$$

表示，称这个多项式为线性移位寄存器的联系多项式或特征多项式。

设一个 n 级线性移位寄存器对应于上面的递推关系，由于 $a_i \in GF(2)$ $(i = 1, 2, \ldots, n)$，所以共有 2^n 组初始状态，即有 2^n 个递推序列，其中非恒零的序列有 $2^n - 1$ 个，记 $2^n - 1$ 个非零序列的全体为 $G(p(x))$。

设取自 $G(p(x))$ 的某一序列 $\{a_i\}$，幂级数

$$A(x) = \sum_{i=1}^{\infty} a_i x^{i-1}$$

称为该序列的生成函数。

定理 4.1

设 $p(x) = 1 + c_1 x + \cdots + c_{n-1} x^{n-1} + c_n x^n$ 是 $GF(2)$ 上的多项式，$G(p(x))$ 中任一序列 $\{a_i\}$ 的生成函数 $A(x)$ 满足：

$$A(x) = \frac{\phi(x)}{p(x)},$$

其中

$$\phi(x) = \sum_{i=1}^{n} c_{n-i} x^{n-i} \sum_{j=1}^{i} a_j x^{j-1}.$$

♡

证明 在等式

$$\begin{aligned} a_{n+1} &= c_1 a_n \oplus c_2 a_{n-1} \oplus \cdots \oplus c_n a_1 \\ a_{n+2} &= c_1 a_{n+1} \oplus c_2 a_n \oplus \cdots \oplus c_n a_2 \\ &\cdots \end{aligned}$$

两边分别乘以 $x^n, x^{n+1}, \cdots$ 再求和，可得

$$\begin{aligned} A(x) - (a_1 + a_2 x + \cdots + a_n x^{n-1}) =& c_1 x \left[A(x) - \left(a_1 + a_2 x + \cdots + a_{n-1} x^{n-2}\right)\right] \\ &+ c_2 x^2 \left[A(x) - \left(a_1 + a_2 x + \cdots + a_{n-2} x^{n-3}\right)\right] \\ &+ \cdots \\ &+ c_n x^n A(x). \end{aligned}$$

移项整理得

$$\begin{aligned} &\left(1 + c_1 x + \cdots + c_{n-1} x^{n-1} + c_n x^n\right) A(x) \\ =& \left(a_1 + a_2 x + \cdots + a_n x^{n-1}\right) + c_1 x \left(a_1 + a_2 x + \cdots + a_{n-1} x^{n-2}\right) \\ &+ c_2 x^2 \left(a_1 + a_2 x + \cdots + a_{n-2} x^{n-3}\right) + \cdots + c_{n-1} x^{n-1} a_1, \end{aligned}$$

即

$$p(x) A(x) = \sum_{i=1}^{n} \left(c_{n-i} x^{n-i} \sum_{j=1}^{i} a_j x^{j-1} \right) = \phi(x),$$

注意在 $GF(2)$ 上有 $a + a = 0$。

定理 4.2

$p(x) \mid q(x)$ 的充要条件是 $G(p(x)) \subseteq G(q(x))$。

♡

证明 若 $p(x) \mid q(x)$，可设 $q(x) = p(x) r(x)$，因此

$$A(x) = \frac{\phi(x)}{p(x)} = \frac{\phi(x) r(x)}{p(x) r(x)} = \frac{\phi(x) r(x)}{q(x)}.$$

所以若 $\{a_i\} \in G(p(x))$，则 $\{a_i\} \in G(q(x))$，即 $G(p(x)) \subseteq G(q(x))$。

反之，若 $G(p(x)) \subseteq G(q(x))$，则对于多项式 $\phi(x)$，存在序列 $\{a_i\} \in G(p(x))$ 以 $A(x) = \phi(x)/p(x)$ 为生成函数。特别地，对于多项式 $\phi(x) = 1$，存在序列 $\{a_i\} \in G(p(x))$ 以 $1/p(x)$ 为生成函数。由于 $G(p(x)) \subseteq G(q(x))$，序列 $\{a_i\} \in G(q(x))$，所以存在函数 $r(x)$，使得 $\{a_i\}$ 的生成函数也等于 $r(x)/q(x)$，从而 $1/p(x) = r(x)/q(x)$，即 $q(x) = p(x)r(x)$，所以 $p(x) \mid q(x)$。

上述定理说明可用 n 级线性移位寄存器产生的序列，也可用级数更多的线性移位寄存器来产生。

设 $p(x)$ 是 $GF(2)$ 上的多项式，使 $p(x) \mid (x^p - 1)$ 的最小 p 称为 $p(x)$ 的周期或阶。

定理 4.3

若序列 $\{a_i\}$ 的特征多项式 $p(x)$ 定义在 $GF(2)$ 上，p 是 $p(x)$ 的周期，则 $\{a_i\}$ 的周期 $r \mid p$。 ♡

证明 由 $p(x)$ 周期的定义得 $p(x) \mid (x^p - 1)$，因此存在 $q(x)$，使得 $x^p - 1 = p(x)q(x)$，又由 $p(x)A(x) = \phi(x)$ 可得 $p(x)q(x)A(x) = \phi(x)q(x)$，所以 $(x^p - 1)A(x) = \phi(x)q(x)$。由于 $q(x)$ 的次数为 $p - n$，$\phi(x)$ 的次数不超过 $n - 1$，所以 $(x^p - 1)A(x)$ 的次数不超过 $(p - n) + (n - 1) = p - 1$。将 $(x^p - 1)A(x)$ 写成 $x^p A(x) - A(x)$，可看出对于任意正整数 i 都有 $a_{i+p} = a_i$。

设 $p = kr + t$，$0 \le t < r$，则 $a_{i+p} = a_{i+kr+t} = a_{i+t} = a_i$，所以 $t = 0$，即 $r \mid p$。

n 级线性移位寄存器的输出序列的周期 r 不依赖于初始条件，而依赖于特征多项式 $p(x)$。我们感兴趣的是级线性移位寄存器遍历 $2^n - 1$ 个非零状态，这时序列的周期达到最大 $2^n - 1$，这种序列就是 m-序列。显然对于特征多项式一样，而仅初始条件不同的两个输出序列，一个记为 $\{a_i^{(1)}\}$，另一个记为 $\{a_i^{(2)}\}$，其中一个必是另一个的移位，即存在一个常数 k，使得

$$a_i^{(1)} = a_{k+i}^{(2)}, \quad i = 1, 2, \cdots$$

下面讨论特征多项式满足什么条件时，LFSR 的输出序列为 m-序列。

定理 4.4

设 $p(x)$ 是 n 次不可约多项式，周期为 m，序列 $\{a_i\} \in G(p(x))$，则 $\{a_i\}$ 的周期为 m。 ♡

证明 设 $\{a_i\}$ 的周期为 r，由前面的定理可知 $r \mid m$，所以 $r \le m$。

设 $A(x)$ 为 $\{a_i\}$ 的生成函数，$A(x) = \phi(x)/p(x)$，即 $p(x)A(x) = \phi(x) \ne 0$，$\phi(x)$ 的次数不超过 $n-1$。而

$$\begin{aligned} A(x) &= \sum_{i=1}^{\infty} a_i x^{i-1} = a_1 + a_2 x + \cdots + a_r x^{r-1} + x^r \left(a_1 + a_2 x + \cdots + a_r x^{r-1}\right) \\ &\quad + (x^r)^2 \left(a_1 + a_2 x + \cdots + a_r x^{r-1}\right) + \cdots \\ &= \frac{a_1 + a_2 x + \cdots + a_r x^{r-1}}{1 - x^r} \\ &= \frac{a_1 + a_2 x + \cdots + a_r x^{r-1}}{x^r - 1}. \end{aligned}$$

于是 $A(x) = \left(a_1 + a_2 x + \cdots + a_r x^{r-1}\right) / (x^r - 1) = \phi(x)/p(x)$，即

$$p(x)\left(a_1 + a_2 x + \cdots + a_r x^{r-1}\right) = \phi(x)(x^r - 1).$$

因 $p(x)$ 是不可约的，所以 $\gcd(p(x), \phi(x)) = 1$，$p(x) \mid (x^r - 1)$，因此 $m \le r$。

综上可得 $r = m$。

定理 4.5

n 级 LFSR 产生的序列有最大周期 $2^n - 1$ 的必要条件是其特征多项式为不可约的。 ♡

证明 设 n 级 LFSR 产生的序列周期达到最大 2^n-1，除 0 序列外，每一序列的周期由特征多项式惟一决定，而与初始状态无关。设特征多项式为 $p(x)$，若 $p(x)$ 可约，可设为 $p(x)=g(x)h(x)$，其中 $g(x)$ 不可约，且次数 $k<n$。由于 $G(g(x))\subseteq G(p(x))$，而 $G(g(x))$ 中序列的周期一方面不超过 2^k-1，另一方面又等于 2^n-1，这是矛盾的，所以 $p(x)$ 不可约。

该定理的逆不成立，即 LFSR 的特征多项式为不可约多项式时，其输出序列不一定是 m-序列，可以通过下面的例子说明。

例 4.2 $f(x)=x^4+x^3+x^2+x+1$ 为 $GF(2)$ 上的不可约多项式，这可由 x，$x+1$，x^2+x+1 都不能整除 $f(x)$ 得到。以 $f(x)$ 为特征多项式的 LFSR 的输出序列可由

$$a_k=a_{k-1}\oplus a_{k-2}\oplus a_{k-3}\oplus a_{k-4}\ \ (k\geq 4)$$

和给定的初始状态求出，设初始状态为 0001，则输出序列为 $000110001100011\cdots$，周期为 5，不是 m-序列。

定义 4.1

若 n 次不可约多项式 $p(x)$ 的阶为 2^n-1，则称 $p(x)$ 是 n 次本原多项式。 ♣

定理 4.6

设 $\{a_i\}\in G(p(x))$，$\{a_i\}$ 为 m-序列的充要条件是 $p(x)$ 为本原多项式。 ♡

证明 充分性：若 $p(x)$ 是本原多项式，则其阶为 2^n-1，且 $p(x)$ 为不可约多项式，则 $\{a_i\}$ 的周期等于 2^n-1，即 $\{a_i\}$ 为 m-序列。

必要性：反之，若 $\{a_i\}$ 为 m-序列，即其周期等于 2^n-1，可知 $p(x)$ 是不可约的。且 $\{a_i\}$ 的周期 2^n-1 整除 $p(x)$ 的阶，而 $p(x)$ 的阶不超过 2^n-1，所以 $p(x)$ 的阶为 2^n-1，即 $p(x)$ 是本原多项式。

$\{a_i\}$ 为 m-序列的关键在于 $p(x)$ 为本原多项式，n 次本原多项式的个数为

$$\frac{\phi(2^n-1)}{n}$$

其中 ϕ 为欧拉函数。已经证明，对于任意的正整数 n，至少存在一个 n 次本原多项式。所以对于任意的 n 级 LFSR，至少存在一种连接方式使其输出序列为 m-序列。

例 4.3 设 $p(x)=x^4+x+1$，由于 $p(x)\mid(x^{15}-1)$，但不存在小于 15 的常数 l，使得 $p(x)\mid(x^l-1)$，所以 $p(x)$ 的阶为 15。$p(x)$ 的不可约性可由 x，$x+1$，x^2+x+1 都不能整除 $p(x)$ 得到，所以 $p(x)$ 是本原多项式。

若 LFSR 以 $p(x)$ 为特征多项式，则输出序列的递推关系为

$$a_k=a_{k-1}\oplus a_{k-4},\ \ (k\geq 4)$$

若初始状态为 1001，则输出为

$$100100011110101100100011110101\cdots$$

周期为 $2^4-1=15$，即输出序列为 m-序列。

4.2.2 随机性概念与 m-序列的伪随机性

流密码的安全性取决于密钥流的安全性，要求密钥流序列有好的随机性，以使密码分析者对它无法预测。也就是说，即使截获其中一段，也无法推测后面是什么。如果密钥流是周期的，要完全做到随机性是困难的。严格地说，这样的序列不可能做到随机，只能要求截获比周期短的一段时不会泄露更多信息，这样的序列称为伪随机序列。前面提到，如果一个序列的周期达到最大值 2^n-1，该序列称为 m-序列。为讨论 m-序列的随机性，我们先讨论随机序列的一般特性。

设 $\{a_i\}=(a_1a_2,a_3\cdots)$ 为 0、1 序列，例如 00110111，其前两个数字是 00，称为 0 的 2 游程；接着是 11，是 1 的 2 游程；再下来是 0 的 1 游程和 1 的 3 游程。

定义 4.2

$GF(2)$ 上周期为 T 的序列 $\{a_i\}$ 的自相关函数定义为

$$R(\tau)=\frac{1}{T}\sum_{k=1}^{T}(-1)^{a_k}(-1)^{a_{k+\tau}},\ 0\le\tau\le T-1.$$

♣

定义中的和式表示序列 $\{a_i\}$ 与 $\{a_{i+\tau}\}$（序列 $\{a_i\}$ 向后平移 τ 位得到）在一个周期内对应位相同的位数与对应位不同的位数之差。当 $\tau=0$ 时，$R(\tau)=1$；当 $\tau\neq 0$ 时，称 $R(\tau)$ 为异自相关函数。

Golomb 对伪随机周期序列提出了应满足的如下三个随机性公设：

(1) 在序列的一个周期内，0 与 1 的个数相差至多为 1；

(2) 在序列的一个周期内，长为 1 的游程占游程总数的 $1/2$，长为 2 的游程占游程总数的 $1/2^2$，…，长为 i 的游程占游程总数的 $1/2^i$，…，且在等长的游程中 0 的游程个数和 1 的游程个数相等；

(3) 异自相关函数是一个常数。

公设 (1) 说明 $\{a_i\}$ 中 0 与 1 出现的概率基本上相同，(2) 说明 0 与 1 在序列中每一位置上出现的概率相同；(3) 意味着通过对序列与其平移后的序列做比较，不能给出其它任何信息。

从密码系统的角度看，一个伪随机序列还应满足下面的条件：

(1) $\{a_i\}$ 的周期相当大；

(2) $\{a_i\}$ 的确定是计算上容易的；

(3) 由密文及相应的明文的部分信息，不能确定整个 $\{a_i\}$。

下一定理说明，m-序列满足 Golomb 的三个随机性公设。

定理 4.7

$GF(2)$ 上的 n 长 m-序列 $\{a_i\}$ 具有如下性质：

(1) 在一个周期内，0、1 出现的次数分别为 $2^{n-1}-1$ 和 2^{n-1}；

(2) 在一个周期内，总游程数为 2^{n-1}；对 $1\le i\le n-2$，长为 i 的游程有 2^{n-i-1} 个，且 0、1 游程各半；长为 $n-1$ 的 0 游程一个，长为 n 的 1 游程一个；

(3) $\{a_i\}$ 的自相关函数为

$$R(\tau)=\begin{cases}1, & \tau=0,\\ -\frac{1}{2^n-1}, & 0<\tau\le 2^n-2.\end{cases}$$

♡

证明 (1) 在 n 长 m-序列的一个周期内，除了全 0 状态外，每个 n 长状态（共有 2^n-1 个）都恰好出现一次，这些状态中有 2^{n-1} 个在 a_1 位是 1，其余 $2^n-1-2^{n-1}=2^{n-1}-1$ 个状态在 a_1 位是 0。

(2) 对 $n=1,2$，易证结论成立。

对 $n>2$，当 $1\le i\le n-2$ 时，n 长 m-序列的一个周期内，长为 i 的 0 游程数目等于序列中如下形式的状态数目：

$$1\underbrace{00\cdots0}_{i个0}1*\cdots*,$$

其中 $n-i-2$ 个 $*$ 可任取 0 或 1。这种状态共有 2^{n-i-2} 个。同理可得长为 i 的 1 游程数目也等于 2^{n-i-2}，所以长为 i 的游程总数为 2^{n-i-1}。

由于寄存器中不会出现全 0 状态，所以不会出现 0 的 n 游程，但必有一个 1 的 n 游程，而且 1 的游程不会更大，因为若出现 1 的 $n+1$ 游程，就必然有两个相邻的全 1 状态，但这是不可能的。这就证明了

1 的 n 游程必然出现在如下的串中：

$$0\underbrace{11\cdots1}_{n个1}0.$$

当这 $n+2$ 位通过移位寄存器时，便依次产生以下状态：

$$0\underbrace{11\cdots1}_{n-1个1},\quad \underbrace{11\cdots1}_{n个1},\quad \underbrace{11\cdots1}_{n-1个1}0.$$

由于 $0\underbrace{11\cdots1}_{n-1个1}$，$\underbrace{11\cdots1}_{n-1个1}0$ 这两个状态只能各出现一次，所以不会有 1 的 $n-1$ 游程。

0 的 $n-1$ 游程有一个：

$$1\underbrace{00\cdots0}_{n-1个0}1,$$

它产生 $1\underbrace{00\cdots0}_{n-1个0}$ 和 $\underbrace{00\cdots0}_{n-1个0}1$ 两个状态。

于是在一个周期内，总游程数为：

$$1+1+\sum_{i=1}^{n-2}2^{n-i-1}=2^{n-1}.$$

(3) $\{a_i\}$ 是周期为 2^n-1 的 m-序列，对于任一正整数 τ $(0<\tau<2^n-1)$，$\{a_i\}+\{a_{i+\tau}\}$ 在一个周期内为 0 的位的数目正好是序列 $\{a_i\}$ 和 $\{a_{i+\tau}\}$ 对应位相同的位的数目。

设序列 $\{a_i\}$ 满足递推关系

$$a_{h+n}=c_1a_{h+n-1}\oplus c_2a_{h+n-2}\oplus\cdots\oplus c_na_h.$$

故

$$\begin{aligned}a_{h+n+\tau}=&c_1a_{h+n+\tau-1}\oplus c_2a_{h+n+\tau-2}\oplus\cdots\oplus c_na_{h+\tau},\\ a_{h+n}\oplus a_{h+n+\tau}=&c_1(a_{h+n-1}\oplus a_{h+n+\tau-1})\oplus c_2(a_{h+n-2}\oplus a_{h+n+\tau-2})\\ &\oplus\cdots\oplus c_n(a_h\oplus a_{h+\tau}).\end{aligned}$$

令 $b_j=a_j\oplus a_{j+\tau}$，由递推序列 $\{a_i\}$ 可推得递推序列 $\{b_i\}$，$\{b_i\}$ 满足

$$b_{h+n}=c_1b_{h+n-1}\oplus c_2b_{h+n-2}\oplus\cdots\oplus c_nb_h.$$

$\{b_i\}$ 也是 m-序列。为了计算 $R(\tau)$，只要用 $\{b_i\}$ 在一个周期中 0 的个数减去 1 的个数，再除以 2^n-1，即

$$R(\tau)=\frac{2^{n-1}-1-2^{n-1}}{2^n-1}=-\frac{1}{2^n-1}.$$

4.3 滤波生成器和非线性组合生成器

虽然 m-序列具有良好的伪随机性和较大周期，但是因为 Berlekamp-Massey 算法的出现，导致不能直接将线性移位寄存器作为密钥流生成器，也不能利用多个线性移位寄存器的线性组合设计密钥流生成器，必须在密钥流生成器的设计中引入非线性运算。非线性滤波生成器和非线性组合生成器即为两种典型的基于线性移位寄存器驱动、基于非线性函数输出密钥流的生成器；如图4.3所示。这两种设计方式在序列密码中也被称为前馈模型。

4.3.1 非线性滤波生成器

非线性滤波生成器基于 LFSR 更新内部状态，每一节拍使用布尔函数 f 计算产生密钥流比特 k_t。设 LFSR 的初始状态为 $s_0,s_1,\ldots,s_{n-1}$，状态更新可表示为线性变换 L，则 t 时刻寄存器的内部状态可表示

为

$$(s_t, s_{t+1}, \ldots, s_{t+n-1}) = L^t(s_0, s_1, \ldots, s_{n-1}),$$

从而密钥流比特为

$$k_t = f(s_t, s_{t+1}, \ldots, s_{t+n-1}) = f(L^t(s_0, s_1, \ldots, s_{n-1})).$$

对非线性滤波生成器产生的密钥流序列 $\{k_t\}_{t\geq 0}$，我们给出如下结论。鉴于其证明比较复杂，这里不再给出，感兴趣的读者可以参考相关文献。

定理 4.8

1. 设线性移位寄存器的级数为 n 且为本原的，布尔函数 f 的代数次数为 m，则密钥流序列 $\{k_t\}_{t\geq 0}$ 的最大线性复杂度为
$$L_m = \sum_{i=0}^{m} \binom{n}{i}.$$
2. 若 n 为素数，函数 f 在代数次数为 m 的布尔函数集合中随机选取时，其产生的密钥流序列能够达到最大线性复杂度 L_m 的概率为
$$p_m \approx e^{-\frac{L_m}{n\cdot 2^n}} > e^{-\frac{1}{n}}.$$

♡

由定理4.8 (1) 能够看出，若欲使密钥流序列 $\{k_t\}_{t\geq 0}$ 具有高的线性复杂度，布尔函数 f 应具有高的代数次数。在实际设计中，为了抵抗对非线性滤波生成器的各种攻击方法，f 还应具有平衡性、高非线性度、高代数免疫度等密码学特性。

4.3.2 非线性组合生成器

非线性组合生成器通过非线性布尔函数 f 组合多个线性移位寄存器的输出比特产生密钥流比特。假设生成器包含 n 个线性移位寄存器 LFSR_1，LFSR_2，…，LFSR_n，，其反馈多项式通常选取本原多项式，反馈系数可以设计为与种子密钥无关的常数，也可以由种子密钥决定。

设 LFSR_i 产生的线性移位寄存器序列为 $\{s_i^{(j)}\}_{j\geq 0}$，则 t 时刻 LFSR 产生输出向量 $x^{(t)} = (s_1^{(t)}, s_2^{(t)}, \ldots, s_n^{(t)})$，密钥流比特 k_t 为

$$k_t = f(x^{(t)}) = f(s_1^{(t)}, s_2^{(t)}, \ldots, s_n^{(t)}).$$

对非线性组合生成器产生密钥流序列 $\{k_t\}_{t\geq 0}$ 的线性复杂度，我们不加证明地给出如下结论。

定理 4.9

设非线性组合模型中 n 个 LFSR 的级数 $L_1, \ldots, L_n$ 两两不同且均大于 2，非线性组合函数 f 的代数标准型为 $f(x_1, \ldots, x_n)$，则其产生的密钥流序列 $\{k_t\}_{t\geq 0}$ 的线性复杂度为 $f(L_1, \ldots, L_n)$，即将 f 表达式中模 2 加法和乘法替换为通常整数加法和乘法计算后的结果。

♡

如下给出一个非线性组合生成器的例子，称为 Geffe 生成器，它是 1973 年由 Geffe 提出的一类序列生成器。

Geffe 生成器包含三个 LFSR，均为本原的，即能产生 m-序列，其级数 L_1，L_2，L_3 两两互素。非线性组合函数为

$$f(x_1, x_2, x_3) = x_1x_2 \oplus (1 + x_2)x_3 = x_1x_2 \oplus x_2x_3 \oplus x_3.$$

设三个 LFSR 产生的序列分别为 $\{x_1(t)\}_{t\geq 0}$, $\{x_2(t)\}_{t\geq 0}$, $\{x_3(t)\}_{t\geq 0}$，则 Geffe 生成器产生的序列 $\{z(t)\}_{t\geq 0}$

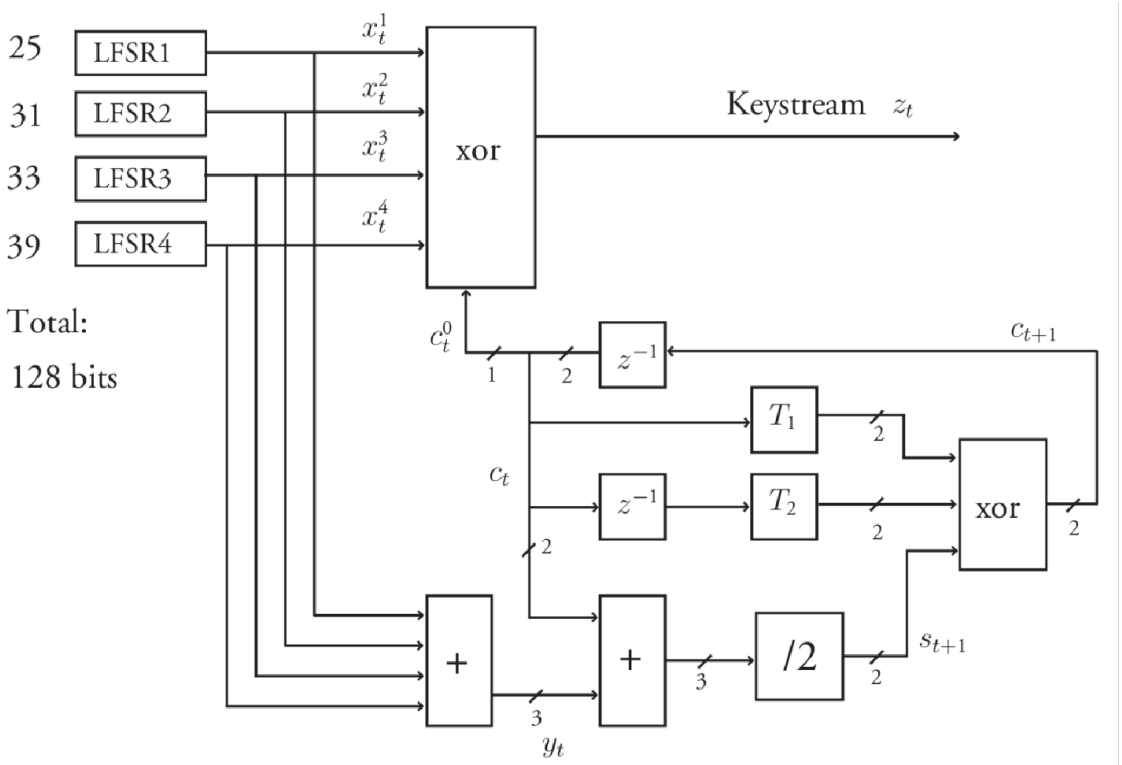


图 4.8: E0 序列密码算法结构

满足

$$z_t = \begin{cases} x_1(t) & x_2(t) = 1, \\ x_3(t) & x_2(t) = 0. \end{cases}$$

可以证明，Geffe 生成器产生序列 $\{z(t)\}_{t\geq 0}$ 的周期为 $(2^{L_1}-1)\cdot(2^{L_2}-1)\cdot(2^{L_3}-1)$，线性复杂度为 $L_1L_2 + L_2L_3 + L_3$。因此，Geffe 生成器产生序列的周期和线性复杂度与 LFSR 相比，均得到了很大提升。但是，Geffe 生成器却是一个非常弱的密钥流生成器，无法抵抗相关攻击和快速相关攻击（参看第4.5节）。

4.4 序列密码算法举例

4.4.1 E0 序列密码算法

E0 是基于带记忆单元的非线性组合生成器设计的序列密码算法，1999 年被选作蓝牙链路层协议中两个加密算法之一（另一个算法为分组密码 SAFER+）。2013 年有学者给出了针对 E0 算法的实际攻击，因此目前该算法已不再安全。

E0 算法支持 8 到 128 比特的种子密钥长度，但通常采用 128 比特密钥。初始向量 IV 长度为 64 比特，包括 48 比特蓝牙地址和 26 比特计数器值。算法整体结构如图4.8所示。

(1) 线性移位寄存器：E0 使用了 4 个线性移位寄存器，其长度分别为 25、31、33、39，连接多项式分

别为

$$f_1 = x^{25} + x^{20} + x^{12} + x^8 + 1,$$
$$f_2 = x^{31} + x^{24} + x^{16} + x^{12} + 1,$$
$$f_3 = x^{33} + x^{28} + x^{24} + x^4 + 1,$$
$$f_4 = x^{39} + x^{36} + x^{28} + x^4 + 1.$$

(2) 加法运算：假设 t 时刻四个线性移位寄存器的输出分别为 x_t^1, x_t^2, x_t^3, x_t^4，计算其整数加法

$$y_t = x_t^1 + x_t^2 + x_t^3 + x_t^4,$$

显然 $y_t \in \{0, 1, 2, 3, 4\}$。

(3) 记忆单元 c_t：记忆单元包含 2 比特，即 $c_t = (c_t^0, c_t^1)$，因此可对应整数 $c_t = c_t^0 + 2c_t^1$。c_t 按照图4.8中的方式更新为 c_{t+1}，其中 z^{-1} 表示延时操作，T_1, T_2 表示作用在两比特上的线性变换，定义为 $T_1 : (x_1, x_0) \to (x_1, x_0)$，$T_2 : (x_1, x_0) \to (x_0, x_1 \oplus x_0)$。具体地，我们有

$$s_{t+1} = (s_{t+1}^1, s_{t+1}^0) = \lfloor \frac{y_t + c_t}{2} \rfloor,$$
$$c_{t+1} = (c_{t+1}^1, c_{t+1}^0) = (s_{t+1}^1, s_{t+1}^0) \oplus T_1(c_t) \oplus T_2(c_{t-1}).$$

(4) 密钥流输出：t 时刻的密钥流比特 z_t 为

$$z_t = x_t^1 \oplus x_t^2 \oplus x_t^3 \oplus x_t^4 \oplus c_t^0.$$

E0 算法的初始化阶段基于种子密钥、128 比特随机数、64 比特 IV 以及全部置 0 的 4 比特记忆单元 c_t, c_{t-1} 产生 128 比特初始状态，将其装填进四个线性移位寄存器并生成 200 比特密钥流，将这 200 比特的后 128 比特重新装填进四个寄存器，并使用当前的记忆单元值继续运行，产生用于加密数据的密钥流序列。详细的初始化流程感兴趣的读者可以参考算法的设计文档。

4.4.2 RC4 算法

RC4 算法是由美国密码学家 Rivest 于 1987 年设计的序列密码算法，是迄今为止软件实现速度最快的密码算法之一。RC4 算法的设计最初作为商业机密并未公开，直到 1994 年才被通过 Cyperpunks 的匿名邮件列表匿名地公开于网络。该算法是实际应用最广泛的序列密码算法之一，在国际著名的安全协议标准 SSL/TLS（安全套接层协议/传输层安全协议）以及无线局域网标准 IEEE 802.11 的 WEP（有线等效保密协议）和 WPA（Wi-Fi 网络安全接入协议）等安全模块中都得到应用。

RC4 算法是一个基于对状态表进行随机洗牌设计的序列密码算法，此类设计也被称为单表驱动序列密码，设计思想简单却能带来非常高的安全性。它使用一个长度为 256 字节的驱动表作为内部状态，每一节拍使用简单的两字节互换操作实现内部状态更新，互换操作基于字节的模加运算。

RC4 算法支持可变密钥长度，密钥规模可以为 40 到 256 字节，算法无初始向量。假设密钥 K 的长度为 ℓ 字节，算法的内部状态包含的 256 字节记为 $S[0]$, $S[1]$, ..., $S[255]$，每个字节视为 0 ~ 255 间的整数。算法流程包含两部分：

(1) 密钥调度算法（KSA）：

密钥调度算法即为算法的初始化部分。首先初始化状态表的值，对 $0 \le i \le 255$，$S[i] = i$。将 j 赋值为 0，对 $0 \le i \le 255$，按如下方式更新 j 的值：

$$j \leftarrow j + S[i] + K[i \bmod \ell],$$

其中“+”表示模 256 的加法运算。然后执行 swap$(S[i], S[j])$，即交换 i 和 j 指向的值。

(2) 伪随机生成算法（PRGA）：

伪随机生成算法即为算法的密钥流输出部分，利用 KSA 产生的状态表生成密钥流序列。将 i 和 j 分

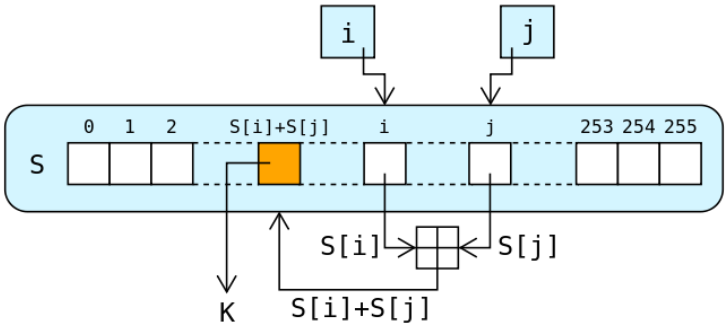


图 4.9: RC4 序列密码算法结构图

别赋值为 0，对时刻 t，令 $i \leftarrow i+1$, $j \leftarrow j+S[i]$，进而执行 swap$(S[i], S[j])$，并输出

$$z_t = S[S[i] + S[j]].$$

算法每个节拍输出一个字节。PRGA 部分的结构如图4.9 所示。

RC4 算法的也启发了其它表驱动的序列密码算法的设计，入 Py 算法、MV3 算法、HC-128 算法和 HC-256 算法。其中 HC-128 是一个多表驱动的序列密码算法，它也是 eSTREAM 项目的 7 个胜选算法之一。

4.4.3 Trivium 算法

Trivium 算法是由比利时密码学家 Cannière 和 Preneel 于 2005 年基于分组密码思想设计的面向硬件实现的同步序列密码算法，它采用非线性反馈组合模型，由三个非线性反馈移位寄存器相互驱动更新内部状态，每一节拍通过线性函数产生密钥流比特。算法结构优美，设计简洁，迄今为止尚未发现有效的攻击方法。

Trvium 算法密钥和初始向量 IV 的规模均为 80 比特，内部状态为 288 比特，包含三个级数分别为 93 比特、84 比特、111 比特的寄存器。将算法的内部状态比特记为 $s_1, \ldots, s_{93}, s_{94}, \ldots, s_{177}, s_{178}, \ldots, s_{288}$。Trvium 算法结构示意图如图4.10所示。

(1) 初始化过程：

在初始化过程中，首先将寄存器装载 80 比特种子密钥 k 和 80 比特初始向量 IV，并对剩余内部状态进行填充，除了寄存器最后 3 比特内部状态设置为 1 外，其余比特均设置为 0。具体填充方式如下：

$$(s_1, s_2, \ldots, s_{93}) \leftarrow (k_1, \ldots, k_{80}, 0, \ldots, 0),$$
$$(s_{94}, s_{95}, \ldots, s_{177}) \leftarrow (IV_1, \ldots, IV_{80}, 0, \ldots, 0),$$
$$(s_{178}, s_{179}, \ldots, s_{288}) \leftarrow (0, \ldots, 0, 1, 1, 1).$$

填充完成后，对 $i = 1 \sim 1152$ 计算:

$$t_1 \leftarrow s_{66} + s_{93},$$
$$t_2 \leftarrow s_{162} + s_{177},$$
$$t_3 \leftarrow s_{243} + s_{288},$$
$$t_1 \leftarrow t_1 + s_{91} \cdot s_{92} + s_{171},$$
$$t_2 \leftarrow t_2 + s_{175} \cdot s_{176} + s_{264},$$
$$t_3 \leftarrow t_3 + s_{286} \cdot s_{287} + s_{69},$$

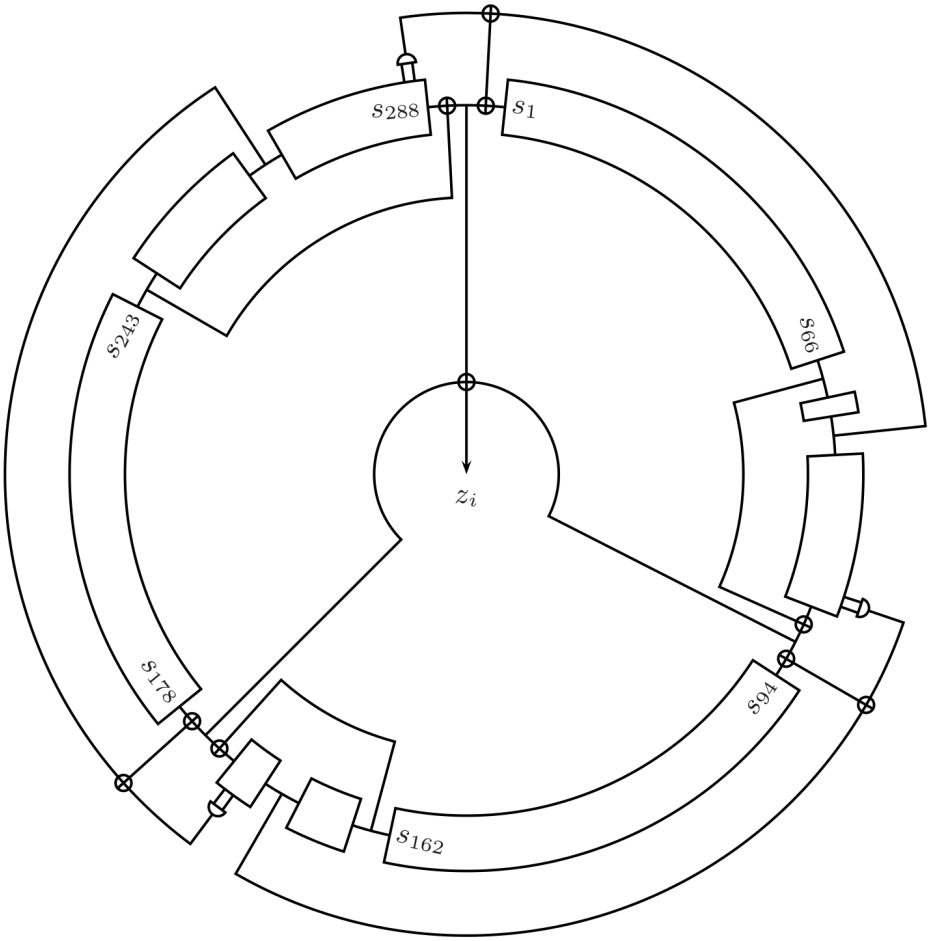


图 4.10: Trivium 序列密码算法结构示意图

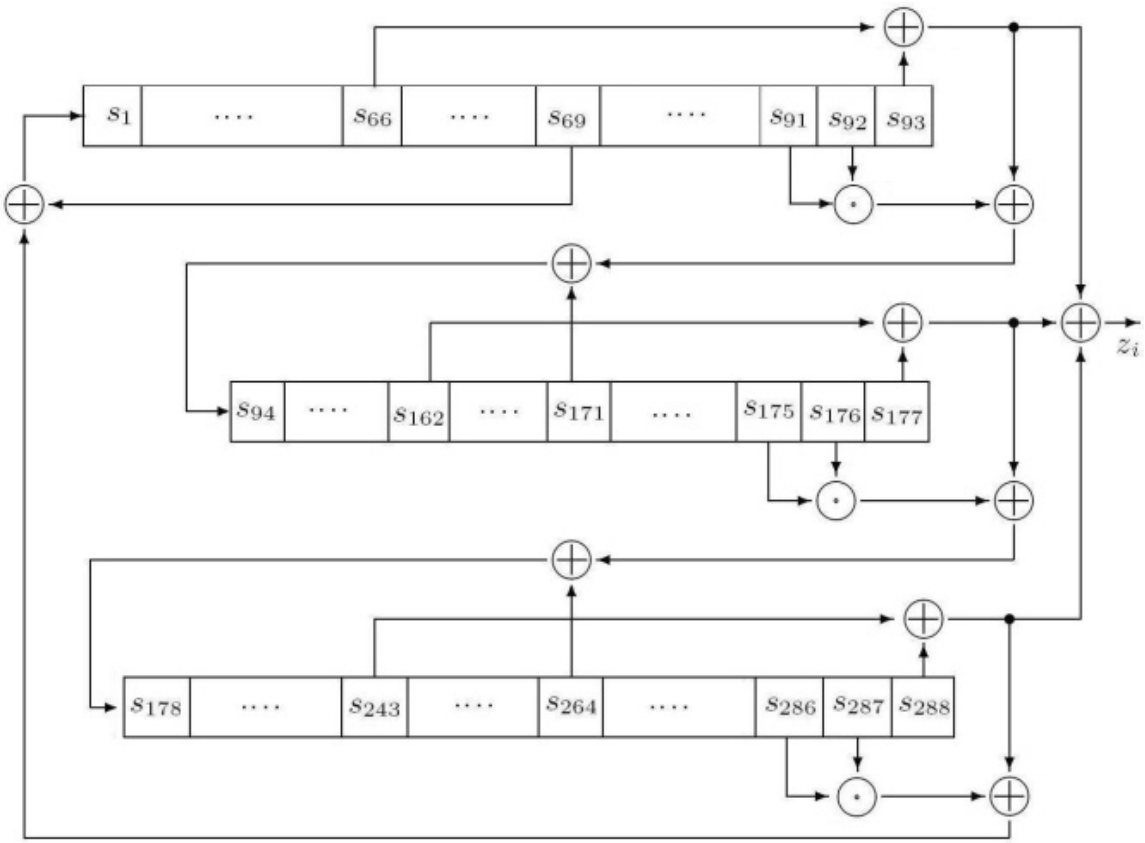


图 4.11: Trivium 序列密码算法结构图

$$(s_1, s_2, \ldots, s_{93}) \leftarrow (t_3, s_1, \ldots, s_{92}),$$
$$(s_{94}, s_{95}, \ldots, s_{177}) \leftarrow (t_1, s_{94}, \ldots, s_{176}),$$
$$(s_{178}, s_{179}, \ldots, s_{288}) \leftarrow (t_2, s_{178}, \ldots, s_{287}).$$

(2) 密钥流生成过程：

对初始化之后的内部状态 $s_1, \ldots, s_{288}$，在时刻 i，按如下方式计算产生密钥流比特 z_i：

$$t_1 \leftarrow s_{66} + s_{93},$$
$$t_2 \leftarrow s_{162} + s_{177},$$
$$t_3 \leftarrow s_{243} + s_{288},$$
$$z_i \leftarrow t_1 + t_2 + t_3,$$
$$t_1 \leftarrow t_1 + s_{91} \cdot s_{92} + s_{171},$$
$$t_2 \leftarrow t_2 + s_{175} \cdot s_{176} + s_{264},$$
$$t_3 \leftarrow t_3 + s_{286} \cdot s_{287} + s_{69},$$
$$(s_1, s_2, \ldots, s_{93}) \leftarrow (t_3, s_1, \ldots, s_{92}),$$
$$(s_{94}, s_{95}, \ldots, s_{177}) \leftarrow (t_1, s_{94}, \ldots, s_{176}),$$
$$(s_{178}, s_{179}, \ldots, s_{288}) \leftarrow (t_2, s_{178}, \ldots, s_{287}).$$

注意上面计算中，“+”表示 $GF(2)$ 上的加法运算，“$\cdot$”表示 $GF(2)$ 上的乘法运算。

Trivium 的密钥流生成过程也可参看图4.11中的结构。

4.4.4 Grain 型序列密码算法

Grain 序列密码算法是 eSTREAM 项目最终胜选的面向硬件实现的算法之一，由 Hell、Johansson 和 Meier 设计，目前包括 Grain v1、Grain-128 和 Grain-128a 三个版本。Grain 算法的设计采用了线性移位寄存器与非线性移位寄存器级联的模型。本节只介绍 Grain v1 的设计细节。

Grain v1 算法包括一个 80 比特的线性移位寄存器、一个 80 比特的非线性移位寄存器和一个非线性滤波函数 h。算法的密钥和初始向量 IV 的规模分别为 80 比特和 64 比特。将线性移位寄存器的内部状态记为 $s_0, s_1, \ldots, s_{79}$，非线性移位寄存器的内部状态记为 $t_0, t_1, \ldots, t_{63}$。线性移位寄存器的反馈多项式为 $GF(2)$ 上 80 次的本原多项式，其表达式为

$$f(x) = 1 + x^{18} + x^{29} + x^{42} + x^{57} + x^{67} + x^{80},$$

从而反馈关系形如

$$s_{t+80} = s_{t+62} \oplus s_{t+51} \oplus s_{t+38} \oplus s_{t+23} \oplus s_{t+13} \oplus s_t.$$

非线性反馈移位寄存器的更新受线性移位寄存器输出的影响，状态更新函数为

$$\begin{aligned} b_{t+80} =& s_t \oplus b_{t+62} \oplus b_{t+60} \oplus b_{t+52} \oplus b_{t+45} \oplus b_{t+37} \oplus b_{t+33} \oplus b_{t+28} \oplus \\ & \oplus b_{t+21} \oplus b_{t+14} \oplus b_{t+9} \oplus b_t \oplus b_{t+63}b_{t+60} \oplus b_{t+37}b_{t+33} \oplus \\ & \oplus b_{t+15}b_{t+9} \oplus b_{t+60}b_{t+52}b_{t+45} \oplus b_{t+33}b_{t+28}b_{t+21} \oplus \\ & \oplus b_{t+63}b_{t+45}b_{t+28}b_{t+9} \oplus b_{t+60}b_{t+52}b_{t+37}b_{t+33} \oplus \\ & \oplus b_{t+63}b_{t+60}b_{t+21}b_{t+15} \oplus b_{t+63}b_{t+60}b_{t+52}b_{t+45}b_{t+37} \oplus \\ & \oplus b_{t+33}b_{t+28}b_{t+21}b_{t+15}b_{t+9} \oplus b_{t+52}b_{t+45}b_{t+37}b_{t+33}b_{t+28}b_{t+21}. \end{aligned}$$

非线性滤波函数 h 为一个 5 变元的布尔函数，其代数标准型为

$$\begin{aligned} h(x) &= h\left(x_0, x_1, \ldots, x_4\right) \\ &= x_1 \oplus x_4 \oplus x_0x_3 \oplus x_2x_3 \oplus x_3x_4 \oplus x_0x_1x_2 \oplus x_0x_2x_3 \oplus x_0x_2x_4 \oplus x_1x_2x_4 \oplus x_2x_3x_4. \end{aligned}$$

函数 h 是一个高度非线性且具有其它多个良好密码学性质的布尔函数。

(1) 初始化过程：

将 80 比特种子密钥记为 $k_0, k_1, \ldots, k_{79}$，64 比特 IV 记为 $IV_0, \ldots, IV_{63}$，Grain v1 的密钥和 IV 装载过程如下：

$$\begin{cases} b_i = k_i, & 0 \le i \le 79, \\ s_i = IV_i, & 0 \le i \le 63, \\ s_i = 1, & 64 \le i \le 79. \end{cases}$$

Grain v1 的初始化过程如图4.12 所示。密钥和 IV 装载完成后，执行 160 轮空转，将密钥流输出函数的输出反馈回线性移位寄存器参与其内部状态更新。

(2) 密钥流生成过程：

密钥流生成过程对初始化后的 160 比特内部状态，利用密钥流输出函数输出密钥比特。在时刻 t 输出的密钥比特 z_t 为：

$$z_t = H\left(B_t, S_t\right) = \bigoplus_{j \in \mathcal{A}} b_{t+j} \oplus h\left(s_{t+3}, s_{t+25}, s_{t+46}, s_{t+64}, b_{t+63}\right),$$

其中 $\mathcal{A} = \{1, 2, 4, 10, 31, 43, 56\}$。

密钥流生成过程如图4.13 所示。

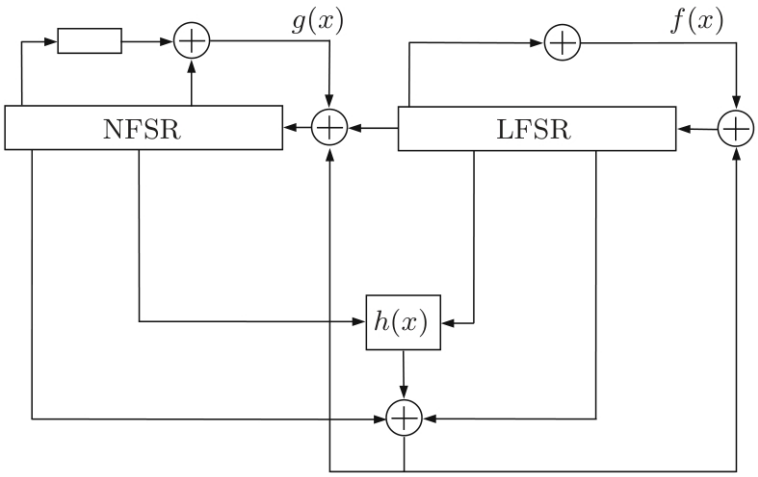


图 4.12: Grain v1 算法初始化过程

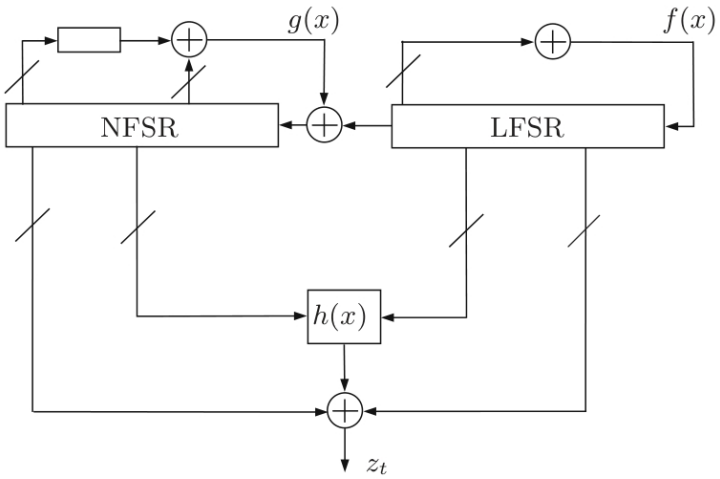


图 4.13: Grain v1 算法密钥流生成过程

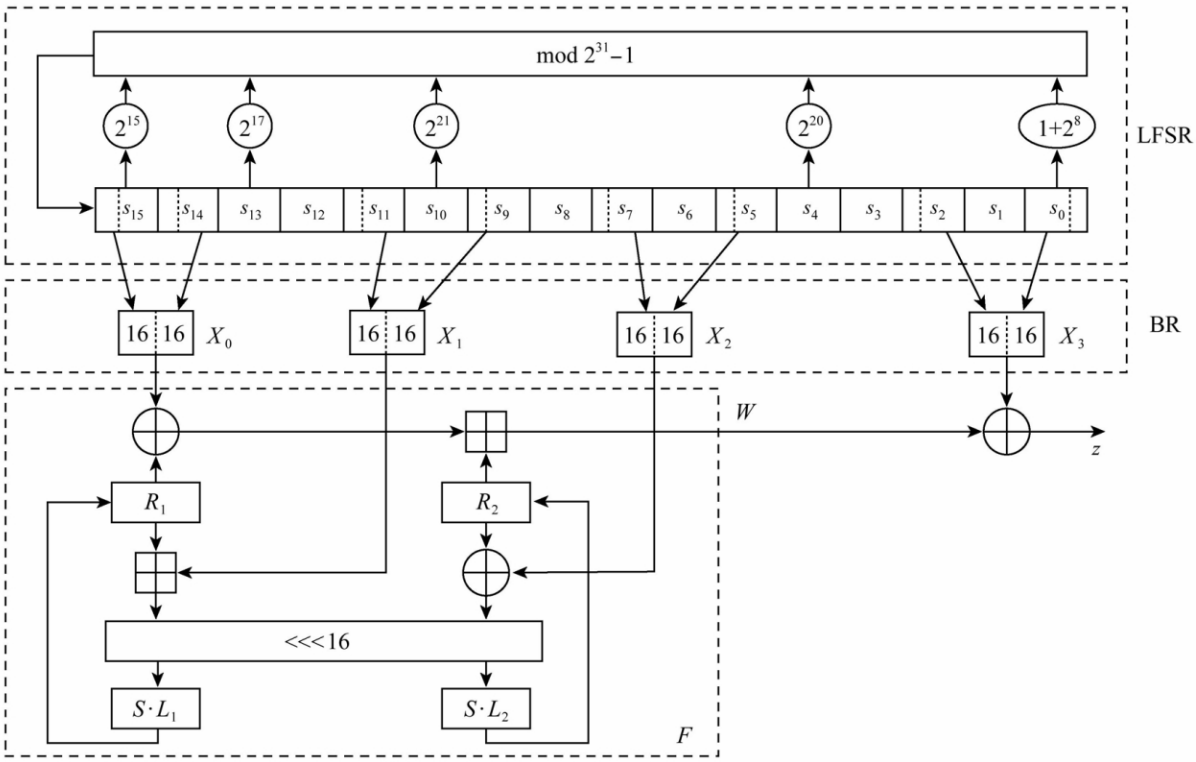


图 4.14: ZUC 序列密码算法结构图

4.4.5 ZUC 序列密码算法

ZUC 算法是我国自主研制的序列密码算法，其名字源于我国古代著名数学家祖冲之。ZUC 算法经中国通信标准化协会与工业与信息化部电信研究院推荐给 3GPP 申请国际标准，最终在 2011 年 9 月被批准成为新一代无线移动通信系统国际标准。2012 年 3 月 ZUC 算法被发布为国家密码行业标准 (标准号为 GM/T 0001—2012), 2016 年 10 月被发布为国家标准 (标准号为 GB/T 33133—-2016)，2020 年 8 月被纳入 ISO/IEC 标准体系（标准号为 ISO/IEC 18033-4: 2011）。ZUC 算法包括加密算法 128-EEA3 和完整性保护算法 128-EIA3，其密钥和 IV 规模均为 128 比特，因此也成为 ZUC-128 算法。

ZUC 算法是一个基于字的序列密码算法，每个时钟节拍输出 32 比特密钥字。算法结构如图4.14所示，包括三个部分：素域 $GF(2^{31}-1)$ 上的 16 级线性反馈移位寄存器 LFSR、比特重组 BR 和基于有限状态机的滤波函数 F。

- 线性移位寄存器：

  LFSR 由 16 个 31 比特的字单元变量 s_i $(0 \le i \le 15)$ 构成，定义在素域 GF$(2^{31}-1)$ 上（注意 $2^{31}-1$ 为素数），其特征多项式为

  $$f(x) = x^{16} - (2^{15}x^{15} + 2^{17}x^{13} + 2^{21}x^{10} + 2^{20}x^4 + (2^8+1)),$$

  设 $\{s_t\}_{t\le 0}$ 为 LFSR 生成的序列，则有

  $$s_{t+16} = 2^{15}s_{t+15} + 2^{17}s_{t+13} + 2^{21}s_{t+10} + 2^{20}s_{t+4} + (2^8+1)s_t \mod 2^{31}-1.$$

  注意如果 $a_{16+t} = 0$，则令 $a_{16+t} = 2^{31}-1$。

- 比特重组：

  BR 层从 LFSR 的内部状态抽取 128 比特，组成 4 个 32 比特的字 X_0，X_1，X_2，X_3，以供下层 F 函

表 4.2: ZUC 算法的 S_0-盒

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	3 E	72	5 B	47	CA	E 0	00	33	04	D 1	54	98	09	B 9	6 D	CB
1	7 B	1 B	F 9	32	AF	9 D	6 A	A 5	B 8	2 D	FC	1 D	08	53	03	90
2	4 D	4 E	84	99	E 4	CE	D 9	91	DD	B 6	85	48	8 B	29	6 E	AC
3	CD	C 1	F 8	1 E	73	43	69	C 6	B 5	BD	FD	39	63	20	D 4	38
4	76	7 D	B 2	A	CF	ED	57	C 5	F 3	2 C	BB	14	21	06	55	9 B
5	E 3	EF	5 E	31	4 F	7 F	5 A	A 4	0 D	82	51	49	5 F	BA	58	1 C
6	4 A	16	D 5	17	A 8	92	24	1 F	8 C	FF	D 8	AE	2 E	01	D 3	AD
7	3 B	4 B	DA	46	EB	C 9	DE	9 A	8 F	87	D 7	3 A	80	6 F	2 F	C 8
8	B 1	B 4	37	F 7	0 A	22	13	28	7 C	CC	3 C	89	C 7	C 3	96	56
9	07	BF	7 E	F 0	0 B	2 B	97	52	35	41	79	61	A 6	4 C	10	FE
A	BC	26	95	88	8 A	B0	A 3	FB	C 0	18	94	F 2	E 1	E 5	E 9	5 D
B	D 0	DC	11	66	64	5 C	EC	59	42	75	12	F 5	74	9 C	AA	23
C	0E	86	AB	BE	2 A	02	E 7	67	E 6	44	A 2	6 C	C 2	93	9 F	F 1
D	F 6	FA	36	D 2	50	68	9 E	62	71	15	3 D	D 6	40	C 4	E 2	0 F
E	8 E	83	77	6 B	25	05	3 F	0 C	30	EA	70	B 7	A 1	E 8	A 9	65
F	8 D	27	1 A	DB	81	B 3	A0	F 4	45	7 A	19	DF	EE	78	34	60

数和密钥流输出函数使用。BR 的具体计算过程如下：

$$X_0 = s_{15H}||s_{14L},$$
$$X_1 = s_{11L}||s_{9H},$$
$$X_2 = s_{7L}||s_{5H},$$
$$X_3 = s_{2L}||s_{0H},$$

其中 s_{iH} 和 s_{iL} 分别表示 s_i 的高 16 比特和低 16 比特，|| 表示比特串连接。

- 非线性函数 F：

非线性函数 F 包含 2 个 32 比特记忆单元变量 R_1 和 R_2, 其输入为比特重组 BR 输出的 3 个 32 比特的字 X_0, X_1, X_2, 输出为一个 32 比特字 W。F 的计算过程如下:

$$W = (X_0 \oplus R_1) \boxplus R_2,$$
$$W_1 = R_1 \boxplus X_1,$$
$$W_2 = R_2 \oplus X_2,$$
$$R_1 = S\left(L_1\left(W_{1\,\mathrm{L}} \| W_{2\mathrm{H}}\right)\right),$$
$$R_2 = S\left(L_2\left(W_{2\,\mathrm{L}} \| W_{1\mathrm{H}}\right)\right).$$

其中，$\oplus$ 表示比特串的按位 XOR 运算，$\boxplus$ 表示模 2^{32} 的加法运算（比特串的模加意为其整数表示的模加），L_1 和 L_2 为 32 比特的线性变换，其定义为

$$L_1(X) = X \oplus (X <<< 2) \oplus (X <<< 10) \oplus (X <<< 18) \oplus (X <<< 24),$$
$$L_2(X) = X \oplus (X <<< 8) \oplus (X <<< 14) \oplus (X <<< 22) \oplus (X <<< 30),$$

这里 <<< 表示比特串的循环移位。

S 为 32 比特的 S-盒运算。S-盒由 4 个小的 8×8 的 S 盒并置而成，即：$S = (S_0, S_1, S_2, S_3)$, 其中 $S_0 = S_2$, $S_1 = S_3$。S_0 和 S_1 的定义分别见表4.2和表4.3.

注意上述 S_0 盒和 S_1 盒数据均为十六进制表示，查表方法与 AES 算法 S-盒的查表方法相同（参看

表 4.3: ZUC 算法的 S_1-盒

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	55	C2	63	71	3B	C8	47	86	9F	3C	DA	5B	29	AA	FD	77
1	8C	C5	94	0C	A6	1A	13	00	E3	A8	16	72	40	F9	F8	42
2	44	26	68	96	81	D9	45	3E	10	76	C6	A7	8B	39	43	E1
3	3A	B5	56	2A	C0	6D	B3	05	22	66	BF	DC	0B	FA	62	48
4	DD	20	11	06	36	C9	C1	CF	F6	27	52	BB	69	F5	D4	87
5	7F	84	4C	D 2	9C	57	A 4	BC	4F	9A	DF	FE	D6	8D	7A	EB
6	2B	53	D8	5C	A1	14	17	FB	23	D5	7D	30	67	73	08	09
7	EE	B7	70	3F	61	B2	19	8E	4E	E 5	4B	93	8F	5 D	DB	A 9
8	AD	F 1	AE	2 E	CB	0 D	FC	F 4	2 D	46	6 E	1 D	97	E 8	D 1	E 9
9	4D	37	A 5	75	5 E	83	9 E	AB	82	9 D	B 9	1 C	E 0	CD	49	89
A	01	B 6	BD	58	24	A 2	5 F	38	78	99	15	90	50	B 8	95	E 4
B	D 0	91	C 7	CE	ED	0 F	B 4	6 F	A0	CC	F 0	02	4 A	79	C 3	DE
C	A 3	EF	EA	51	E 6	6 B	18	EC	1 B	2 C	80	F 7	74	E 7	FF	21
D	5 A	6 A	54	1 E	41	31	92	35	C 4	33	07	0 A	BA	7 E	0 E	34
E	88	B 1	98	7 C	F 3	3 D	60	6 C	7 B	CA	D 3	1 F	32	65	04	28
F	64	BE	85	9 B	2 F	59	8 A	D 7	B 0	25	AC	AF	12	03	E 2	F 2

第5.3.1节)。设 S-盒长度为 32 比特的输入 X 和输出 Y 分别为

$$X = x_0\|x_1\|x_2\|x_3,$$

$$Y = y_0\|y_1\|y_2\|y_3,$$

则

$$y_i{=}S_i(x_i),\ \ i{=}0,1,2,3.$$

ZUC 算法的初始化过程和密钥流生成过程描述如下:

(1) 初始化过程:

设 128 比特的初始密钥 k 和 128 比特的初始向量 IV 分别为

$$K = k_0||k_1||\cdots||k_{15},\ \ IV = IV_0||IV_1||\cdots||IV_{15},$$

这里 k_i 和 IV_i 均为字节，$0 \le i \le 15$，将第 i 个字单元 s_i 填充为

$$s_i = k_i\|d_i\|IV_i,$$

其中 d_i 为 15 比特的常量，其取值如下:

$$d_0 = 100010011010111,\ \ d_1 = 01001101011100,$$

$$d_2 = 110001001101011,\ \ d_3 = 00100110101110,$$

$$d_4 = 10101110001001,\ \ d_5 = 01101011100010,$$

$$d_6 = 111000100110101,\ \ d_7 = 000100110101111,$$

$$d_8 = 10011010111000,\ \ d_9 = 01011100010011,$$

$$d_{10} = 110101111000100,\ \ d_{11} = 00110101110001,$$

$$d_{12} = 10111000100110,\ \ d_{13} = 011110001001101,$$

$$d_{14} = 11100010011010,\ \ d_{15} = 10001110101100.$$

将记忆单元 R_0 和 R_1 的值填充为全 0 向量。

完成数据填充后，运行初始化过程 32 轮。初始化过程如图4.15所示，每一轮 F 函数的输出 W 右移

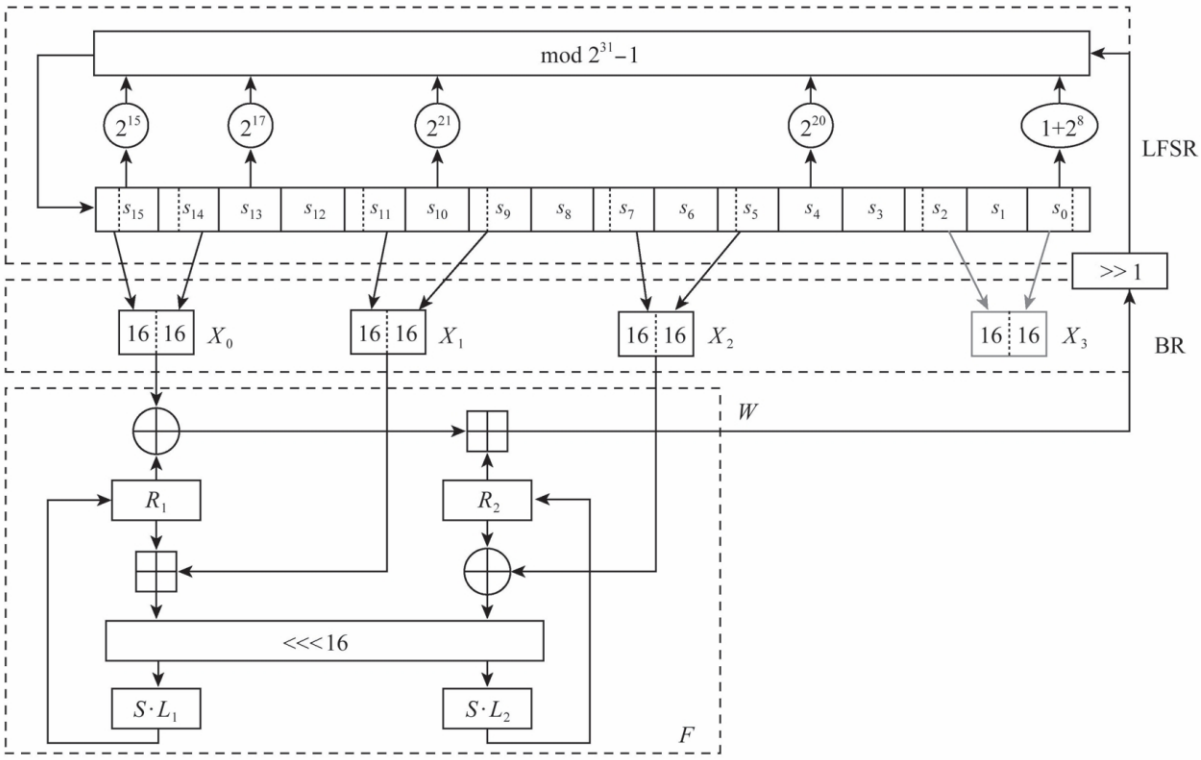


图 4.15: ZUC 算法的初始化过程

一位（即丢弃最低比特位），参与到 LFSR 的反馈计算中，也就是说初始化阶段 LFSR 的反馈关系为

$$s_{t+16} = 2^{15}s_{t+15} + 2^{17}s_{t+13} + 2^{21}s_{t+10} + 2^{20}s_{t+4} + (2^8+1)s_t + (w >> 1) \mod 2^{31} - 1.$$

(2) 密钥流生成过程：

完成初始化后，ZUC 算法按照图4.14所示的结构产生密钥字，算法每迭代一次产生一个 32 比特的密钥字 z：

$$z = W \oplus X_3,$$

其中 W 为当前节拍非线性函数 F 的输出，X_3 为比特重组 BR 的输出。

4.5 序列密码攻击方法简介

本节简要介绍一些针对序列密码的分析方法，篇幅原因，这里只介绍仿射逼近攻击、相关攻击、代数攻击三类重要攻击方法的基本思想及原理。

首先需要说明的是在序列密码攻击中攻击的目标是什么。当然，攻击的最终目标是恢复算法的种子密钥，可获得的数据为密钥流序列的片段或部分比特。对某些密钥流生成器，如果能够恢复寄存器某时刻的内部状态，也认为是成功的攻击，因为由内部状态可能能够反推得到算法的种子密钥。此外，在攻击开始前初始向量 IV（如果有的话）可以认为是可人为操作的，通过改变其值产生不同的密钥流序列，有可能能够导出关于密钥流生成器结构方面的信息，这通常称为攻击的离线阶段或预处理阶段。在此基础上，在攻击的线上阶段恢复算法的种子密钥。

4.5.1 仿射逼近攻击

仿射逼近攻击是一类针对非线性组合生成器的序列密码攻击方法。设非线性组合生成器包含 n 个线性移位寄存器 LFSR_1, LFSR_2, …, LFSR_n, , 其产生的线性序列的线性复杂度分别为 $r_1, \ldots, r_n$, 非线性组合函数为 f。线性逼近攻击的步骤和原理描述如下:

(1) 寻找非线性函数 f 的仿射逼近函数 ℓ。假设用 ℓ 替代 f 作为密钥流生成器的组合函数, 则能产生一个线性序列 S, 由于寄存器初始状态是未知的, 这里 S 也是未知的。但是根据定理4.9可知, 该线性序列的线性复杂度满足 $LC \leq \sum_{i=1}^{n} r_i$。注意, 由 f 产生的密钥流序列的线性复杂度远大于 LC;

(2) 取足够长的密钥流序列 K, 遍历其所有的长度为 $2 \cdot LC$ 的子序列, 则可以期望找到一个子序列与 S 的某个长度为 $2 \cdot LC$ 的子序列重合。所需的密钥流 K 的长度取决于 f 与 ℓ 的距离: 距离越小, 所需密钥流 K 的长度越小;

(3) 利用找到的 $2 \cdot LC$ 长的子序列, 利用 Berlekamp-Massey 算法则可以恢复出产生序列 S 的线性移位寄存器。若 f 与 ℓ 的距离较小, 则序列 S 与密钥流序列 K 具有较高的符合率, 利用 S 的初态则可以恢复寄存器初态的部分比特, 从而恢复部分种子密钥比特。

该攻击的关键是寻找 f 的仿射逼近函数 ℓ, 即 ℓ 应与 f 具有较小的距离。这里的距离是指 Hamming 距离, 即布尔函数 f 和 ℓ 的真值表向量中重合分量的个数。

为了抵抗此类攻击, 这就要求在非线性组合生成器的设计中, 函数 f 应不能具有近距离的仿射逼近函数, 这边引出了关于布尔函数非线性度的研究。

4.5.2 相关攻击

相关攻击从对基于线性反馈移位寄存器的序列密码的分析起步, 最早由 Blaser 和 Heinzmann 提出。1985 年, Siegenthaler 提出了针对非线性组合生成器的分别征服相关攻击, 展示了该攻击方法的威力。1998 年, Meier 和 Staffelbach 又给出了加速分别征服相关攻击的两个算法, 通常被称为快速相关攻击方法, 是相关攻击发展的里程碑。

分别征服相关攻击的核心思想是利用非线性组合函数 f 与其变元的相关性, 利用密钥流序列恢复单个线性移位寄存器的初态。以 Geffe 生成器为例, 其非线性组合函数为

$$f(x_1, x_2, x_3) = x_1 x_2 \oplus (1 + x_2) x_3 = x_1 x_2 \oplus x_2 x_3 \oplus x_3.$$

设 Geffe 生成器产生的序列为 $\{z(t)\}_{t \geq 0}$, 三个 LFSR 产生的序列分别为 $\{x_1(t)\}_{t \geq 0}, \{x_2(t)\}_{t \geq 0}, \{x_3(t)\}_{t \geq 0}$, 则通过简单计算可以发现

$$\begin{aligned} \Pr(z(t) = x_1(t)) &= \Pr(x_2(t) = 1) + \Pr(x_2(t) = 0) \cdot \Pr(x_3(t) = x_1(t)) \\ &= \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} \\ &= \frac{3}{4}. \end{aligned}$$

同理可得 $\Pr(z(t) = x_3(t)) = \frac{3}{4}$。这说明 Geffe 生成器产生的密钥流序列与 LFSR_1 和 LFSR_3 产生的线性序列具有较高的相关性, 由此可以通过尝试恢复 $\{x_1(t)\}_{t \geq 0}$ 和 $\{x_3(t)\}_{t \geq 0}$, 从而可恢复 LFSR_1 和 LFSR_3 的初态。

设非线性组合生成器包含 n 个线性移位寄存器 LFSR_1, LFSR_2, …, LFSR_n, ,其级数分别为 $r_1, \ldots, r_n$, 非线性组合函数为 f。显然, 若使用穷举攻击, 恢复寄存器初始状态的复杂度为

$$\prod_{i=1}^{n} (2^{r_i} - 1).$$

相关攻击的步骤和原理描述如下:

(1) 取足够长的密钥流序列 K，遍历尝试 LFSR$_j$ 的 $2^{r_j}-1$ 个可能初态，检测输出的线性序列 $\{x_j^{(t)}\}$ 与 K 的符合率；

(2) 根据 f 与变元 x_j 的相关性，找到与 K 的符合率最高的 $\{x_j^{(t)}\}$ 所对应的初态；

(3) 按照这种方式分别征服每个 LFSR。最终攻击的复杂度由 $\prod_{j=1}^{n}(2^{r_j}-1)$ 降低到 $\sum_{j=1}^{n}(2^{r_j}-1)$。

在实际攻击中，可以基于统计学手段，实现唯密文攻击。当然，如果算法中采用的 LFSR 的级数较大，这种攻击方式仍具有很高的复杂度。针对 LFSR 数量较少的非线性组合生成器，Meier 和 Staffelbach 提出了一种使用概率迭代译码算法的改进攻击, 不需要搜索整个 LFSR 的所有可能初始状态就能找出正确的初始状态，这便是快速相关攻击的内容了。

为了抵抗相关攻击，要求非线性组合生成器中的布尔函数 f 与每个变元都应具有低相关性，这引出了关于布尔函数相关免疫性的研究。

4.5.3 代数攻击

1949 年，Shannon 其在“保密系统的通信理论”一文中指出，任何一个密码系统都可以看成关于密钥的代数方程系统，破解该密码系统等价于求解其对应的代数方程系统。这本质上就是代数攻击的基本思想。因此，简单来说，代数攻击的流程分为两步：

(1) 根据密码算法的代数结构，导出关于已知数据（如明文、密文）和未知数据（密钥）的代数方程（组），方程的变量即为密钥变元；

(2) 利用求解代数方程组的技术求解步骤 (1) 中的方程组，从而恢复未知数据。

但是，密码系统的设计往往是比较复杂的，因此上面步骤 (1) 中导出的代数方程组往往具有复杂的形式，如具有高代数次数和非常稠密的单项式，而求解非线性多元多项式方程组在计算上是困难问题，这就导致直接利用上述步骤对密码算法实施代数攻击几乎是不可能成功的。但是，对特殊的密码结构，通过对其导出的代数方程组进行处理，有可能降低求解方程组的复杂度。

2003 年著名密码学家 Coutois 等人提出了针对非线性滤波生成器的代数攻击方法，其核心思想是利用布尔函数的零化子，降低密码系统所导出代数方程组的次数，从而降低其求解复杂度。此后，Coutois 等人又提出了快速代数攻击技术，在此后十余年的时间里，代数攻击成为针对序列密码算法最成功的攻击方法。

下面介绍 Coutois 等人提出的代数攻击的核心思想。

在第4.3.1节中我们指出，对非线性滤波生成器，t 时刻的密钥流比特可以表示为

$$z_t = f(L^t(s_0, s_1, \ldots, s_{n-1})),\ t \geq 0. \tag{4.1}$$

将寄存器初始状态简记为 K，则可以得到方程组

$$\begin{cases} z_0 = f(K) \\ z_1 = f(L(K)) \\ z_2 = f(L^2(K)) \\ \quad\vdots \\ z_t = f(L^t(K)) \\ \quad\vdots \end{cases} \tag{4.2}$$

在密码算法设计中，f 具有高代数次数，因此方程组(4.2)具有高代数次数（注意 $f \circ L$ 的次数与 f 的次数相同）。但是，Coutois 等人发现了一种降低方程组次数的方法，分为如下 3 种情形：

(1) 存在低次函数 g 使得 fg(布尔函数的乘积) 具有低代数次数。此时，对方程组(4.2)中任一方程，将等号左右两边分别乘以 $g(L^t(K))$，则可得到

$$(f \cdot g)(L^t(K)) = z_t \cdot g(L^t(K)),$$

这是一个低代数次数的方程；

(2) 存在低次函数 g 使得 $fg=0$。此时，对方程组(4.2)中满足 $z_t=1$ 的方程，将等号左右两边分别乘以 $g(L^t(K))$，则可得到低代数次数的方程

$$g(L^t(K))=0;$$

(3) 存在高次函数 g 使得 fg 具有低代数次数。此时，对方程组(4.2)中满足 $z_t=0$ 的方程，将等号左右两边分别乘以 $g(L^t(K))$，则可得到低代数次数的方程

$$(f\cdot g)(L^t(K))=0.$$

对降低次数的代数方程组，利用线性化、重线性化、XL、Gröbner 基、特征列等处理多项式方程组的求解技术，可以估计方程组求解的复杂度，从而评估算法代数攻击的复杂度。

如下给出一个代数攻击的实例，这也是 Coutois 等人的代数攻击技术第一个成功应用的案例。

例 4.4 Toyocrypt 是一个日本 CRYPTREC 项目中的候选序列密码算法。该算法采用非线性滤波模型，其线性移位寄存器的级数为 128，非线性滤波函数 f 是一个包含 128 个变元、代数次数为 63 的布尔函数，其代数标准型如下：

$$f(x_1,x_2,\cdots,x_{128})=x_{128}+\sum_{i=1}^{63}x_ix_{\pi(i+63)}+x_{11}x_{24}x_{33}x_{43}+$$

$$x_2x_3x_{10}x_{13}x_{19}x_{21}x_{24}x_{26}x_{27}x_{29}x_{34}x_{39}x_{42}x_{43}x_{52}x_{54}x_{59}+\prod_{i=1}^{63}x_i,$$

其中 π 为下标集合 $\{64,\ 65,\ \ldots,\ 126\}$ 上的一个置换。注意到在 f 表达式的 4 次项、17 次项、63 次项中均包含变元 x_{24}，因此存在函数 $g(x_1,x_2,\cdots,x_{128})=1+x_{24}$，使得 $g\cdot f$ 的次数降低为 3，这可以将 Toyocrypt 导出的代数方程组由 63 次降低为 3 次，从而大大降低方程组求解的复杂度。事实上，利用该攻击方法，基于并行计算技术（128 个主频 2GHz 的 CPU），可以在 27 分钟内恢复 Toyocrypt 的种子密钥。

上述三种降低方程组代数次数的情形事实上可以统一起来。在情形 (1) 我们有

$$fg=h\Rightarrow fh=f\cdot fg=fg=h\Rightarrow(f+1)h=0.$$

这等价于函数 $f+1$ 存在低次的零化函数 h。情形 (2) 相当于函数 f 在存在低次的零化函数 g。情形 (3) 我们有 $fg=h\Rightarrow fh=h$，从而归结为情形 (1)，即 $f+1$ 存在低次的零化函数 h。将一个布尔函数的零化函数称为其零化子，因此代数攻击能够成功的关键是非线性滤波函数 f 或其补函数 $f+1$ 存在低次零化子。由此可以引出布尔函数的代数免疫度这一安全性指标。

定义 4.3

$$\mathcal{AI}(f)=\min_{g\neq0}\{\deg(g)\mid fg=0\ \text{或}\ (f+1)g=0\}.$$

♣

可以证明，对任意 n-元布尔函数，其代数免疫度满足

$$\mathcal{AI}(f)\leq\min\left\{\lceil\frac{n}{2}\rceil,\quad\deg(f)\right\}.$$

达到此上界的布尔函数称为最优代数免疫函数。在序列密码算法的设计中，应选择具有高代数免疫度的函数，特别是最优代数免疫函数。

第 4 章 习题

1. 设二元域 $GF(2)$ 上的线性移位寄存器的联系多项式为 $f(x)=1+x+x^3+x^4$，初始状态为 1101，试求其输出序列及其周期。

2. 设序列 $a = (00110101\ldots)$ 是由四级线性移位寄存器所产生的连续 8 个信号，求该移位寄存器的线性递推式，和产生该序列的一个周期。
3. 假设分析者得到密文串 1010110110 和相应的明文串 0100010001。假定攻击者也知道密钥流是使用 3 级线性移位寄存器产生的，试破译该密码系统。

第 5 章 分组密码

在许多密码系统中，分组密码是系统安全的一个重要组成部分，分组密码可以用于构造伪随机数生成器、消息认证码和杂凑函数等。本章主要介绍分组密码算法的基本概念、设计原则、工作模式、分组密码的线性分析与差分分析以及具有代表性的分组密码算法包括 DES、AES、IDEA 和 SMS4。

5.1 分组密码的基本概念

分组密码是将明文消息划分成长为 L（L 的值通常为 64 或 128）的分组 $M=(m_0,m_1,m_2\ldots,m_{L-1})$，各个长为 L 的分组分别在密钥 $K=(k_0,k_1,k_2,\ldots,k_{t-1})$（密钥长为 t）的控制下变换成与明文组等长的一组密文输出序列 $C=(c_0,c_1,c_2\ldots,c_{L-1})$。分组密码的原理如图5.1所示。

定义 5.1

设 S 是一个有限集合，f 是从 S 到 S 的一个映射。如果对于任意 u、$v\in S$，当 $u\neq v$ 时，$f(u)\neq f(v)$，则称 f 为 S 上的一个置换。 ♣

对于一个分组长度为 n 的分组密码，不同的密钥应该对应不同的加密和解密变换。给定密钥 k，对于任意的 u、$v\in S$，如果 $u\neq v$，则一定有 $E_k(u)\neq E_k(v)$，因为如果 $E_k(u)=E_k(v)$，则在解密时将难以准确的恢复出明文。因此，对于给定的密钥 k，加密变换 E_k 是 S 上的一个置换，解密置换 D_k 是 E_k 的逆置换。

分组密码的加密原理是：在密钥的控制下，通过置换将明文分组映射到密文空间的一个分组。本章介绍的数据加密标准 DES（Data Encryption Standard）、高级加密标准 AES（Advanced Encryption Standard）和国际数据加密算法 IDEA（International Data Encryption Algorithm）都是典型的分组密码算法。

对于分组密码，安全性的两个设计原则是扩散和混淆。扩散和混淆是 Shannon 提出的设计密码系统的两种基本方法，其目的是为了抵抗密码分析者对密码体制的统计分析。

扩散原则：设计的密码算法应使明文中的每一位影响密文中的许多位，或者说让密文中的每一位受明文中许多位的影响，这样可以隐蔽明文的统计特性；使得密钥的每一位影响密文的许多位，防止对密钥进行逐字段破译。扩散的目的是希望密文中的任一比特 C_i 都尽可能与明文、密钥相关联，或者明文和密钥的任何位上的值的改变都会在某种程度上影响到 C_i 的值，以此防止将密钥分解成若干个孤立的小部分，然后各个击破。

混淆原则：设计的密码算法应使得密钥和明文以及密文之间的依赖关系变得尽可能复杂。因为仅仅使每个明文比特和每个密钥比特与所有的密文比特紧密相关还是不够的，还必须要求有相关的等价数学函数完成混淆功能，使得对手即使获取了关于密文的一些统计特性，也无法推测密钥。可以使用复杂的非线性代替变换来达到较好的混淆效果。

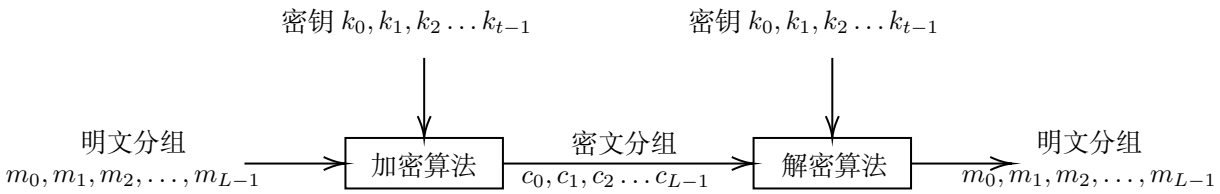


图 5.1: 分组密码的原理

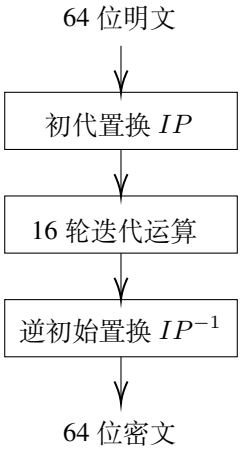


图 5.2: DES 加密的过程略图

5.2 数据加密标准 DES

数据加密标准 DES（Data Encryption Standard）是美国国家标准局（NBS，即现在的美国国家标准和技术协会 NIST）为了满足计算机通信网络的发展对安全保密的要求，实现同一水平的安全性和兼容性，降低数据加密产品的生产成本，推广使用密码算法而公开征集的一种用于政府部门及民间进行计算机数据加密的密码算法。NBS 最初于 1973 年 5 月 13 日向社会公开发起征集，IBM 公司提出的一种称为 Lucifer 的加密算法，被选做为数据加密标准。1977 年 1 月 15 日 DES 被正式批准并作为美国联邦信息处理标准，即 FIP-46，同年 7 月 15 日生效。1994 年 1 月 NSA（美国国家保密局）对 DES 进行评估后，决定 1998 年 12 月以后将不再使用 DES，同时开始征集评估和制定新的加密标准，新的加密标准称为 ASE，将在下一节介绍。

DES 是一个分组密码算法，它使用 56 位的密钥，以 64 位为单位对数据分组进行加密和解密（密文与明文的分组长度相同，均为 64 位），DES 加密与解密使用同一密钥，DES 的保密性依赖于密钥。

5.2.1 DES 加密算法概述

可以将 DES 的加密过程简单描述为三个阶段，如图5.2所示：

(1) 64 位的明文通过一个初始置换 IP，实现明文中的比特重排，同时把 64 比特的明文分成左右两个部分 L_0 和 R_0，每部分都是 32 位。L_0 是比特重排后的前 32 位，R_0 是后 32 位。

(2) 进行十六轮迭代运算。十六次迭代运算具有相同的结构，迭代运算也称为乘积变换。乘积变换包含换位和替代，将数据与密钥结合起来，是 DES 加密的核心。

(3) 将十六次迭代变换后得到的结果通过一个逆置换 IP^{-1}，最后所得到的输出就是 64 位的密文结果。

5.2.2 DES 加密过程描述

(1) 初始置换 IP

在迭代运算之前，需要将输入的 64 位明文进行初始置换 IP，DES 的初始置换表5.1所示。进行初始置换后，明文的次序被打乱，如原来放在第 58 位的数据置换后放在第 1 位。

(2) 16 轮迭代运算

图5.3所示的是 DES 单轮加密的示意图，在每一轮迭代中，每个 64 位的中间结果被分成左右两部分，而且左右两部分作为相互独立的 32 位数据进行处理。每轮迭代的输入是上轮的结果 L_{i-1} 和 R_{i-1}。

表 5.1: 初始置换 IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

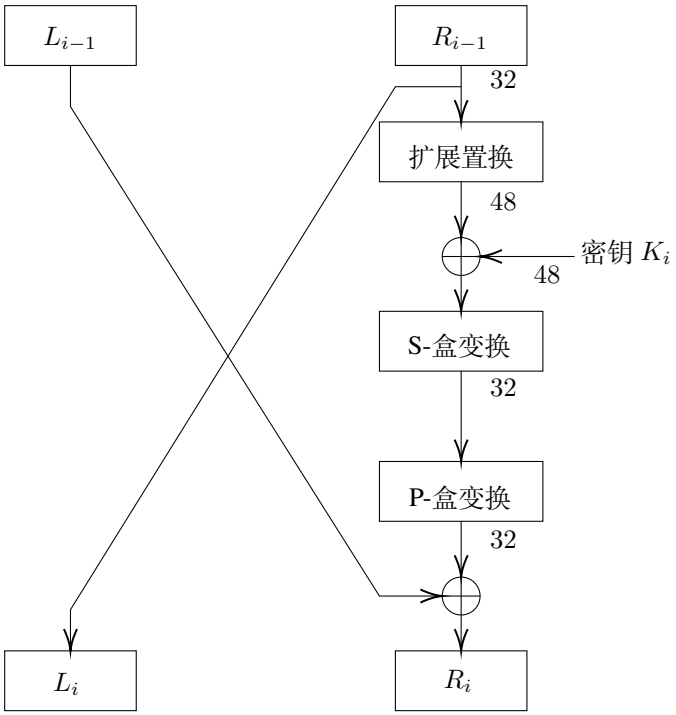


图 5.3: DES 一轮加密的原理图

过程为：R_{i-1} 先由 32 位扩展到 48 位置，扩展后的 48 位结果与 48 位的密钥 K_i 进行异或，得到的结果经过一个 S 盒替代产生 32 位的输出，然后再经过一个 P 盒置换，再将得到的结果与 L_{i-1} 进行简单异或，最后的输出结果作为 R_i。而 L_i 是不经过变换的 R_{i-1} 直接得到的。
可以将 DES 加密算法的 16 轮迭代变换形式化的表示为：

$$
\begin{aligned}
&L_i = R_{i-1}, \\
&R_i = L_{i-1} \oplus F(R_{i-1}, K_i), \\
&i = 1, 2, 3, \ldots, 16.
\end{aligned}
$$

其中 L_i 和 R_i 是表示迭代变换的左右两部分，其长度都是 32 位，F 是一个非线性加密函数，K_i 是由密钥 K 产生的一个 48 位的子密钥。函数 F 包括四个变换，分别是扩展置换，与子密钥与，S 盒变换，P 盒变换。函数 F 的输入为 32 位，产生 48 位的中间结果，并最终产生 32 比特的输出。图5.4描述了加密函数 F 的过程。

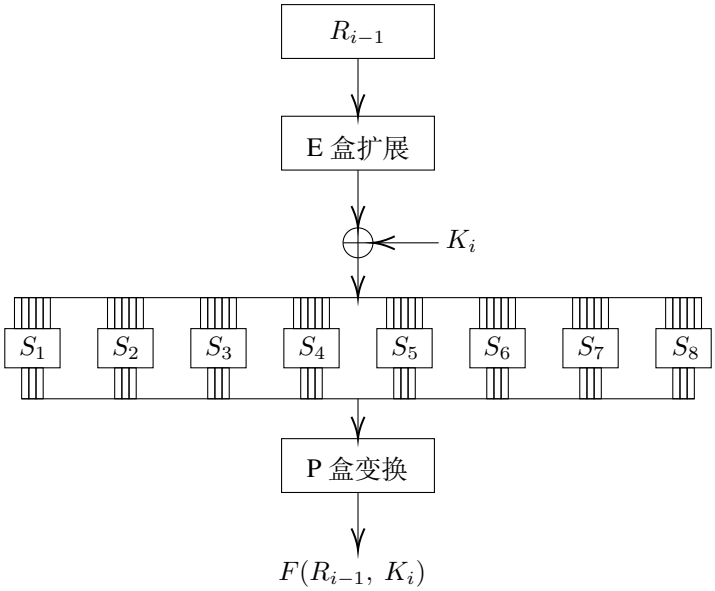


图 5.4: 加密函数 F

下面分别介绍 F 函数变换中的 4 个变化：扩展置换、与子密钥与、S 盒变换以及 P 盒变换，进行详细介绍：

(1) 扩展置换

扩展置换将前一轮迭代的结果 R_{i-1} 作为输入，根据扩展函数 E 将 32 位的比特输入扩展为 48 位。扩展函数 E 将 32 位的明文每 4 位为分成一组，共有 8 组。每个分组由 4 位扩展为 6 位，扩展方法为：每个分组的 4 位作为 6 位输出分组的中间 4 位，6 位输出分组中的第 1 位和第 6 位分别由相邻的两个 4 位小分组的最外面两位扩散进入到本分组产生。(其中第 1 个小分组的左侧相邻分组为最后一个小分组)。图5.5表示 E 盒扩展置换的过程。

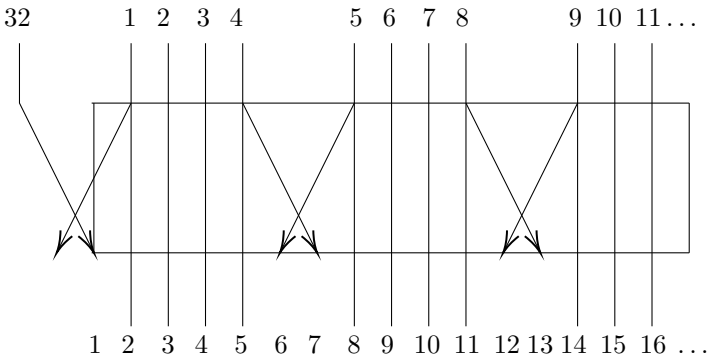


图 5.5: E 盒扩展分组的产生过程

可以将 8 个小分组扩展后的结果列成一张表，就构成了 E 置换的扩展置换表，如表5.2。

(2) 与子密钥异或

将经过 E 盒扩展置换得到的 48 位输出与子密钥 K_i 进行异或 (按位模 2 加) 运算。

(3) S 盒变换

S 盒变换是将第 (b) 步与子密钥异或得到的结果作为 S 盒变换的输入，经过变换得到 32 位的输出。DES 加密函数中共有 8 个 S 盒，每个 S 盒子有 4 行 16 列。如图5.4与表5.3所示。

S 盒变换的过程为：将第 (b) 步与子密钥异或得到 48 位，每 6 比特一组，共分成 8 组，分别作为 8 个 S 盒的输入；每一个分组将对应于一个 S 盒进行变换运算：分组 1 由 S 盒 1 进行变换

表 5.2: E 扩展置换函数表

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

操作，分组 2 由 S 盒 2 操作……而且 8 个 S 盒所表示的变换运算各不相同。

每个 S 盒都有 6 位输入，4 位输出。通过观察表 5.3，可知 S 盒的每一行是数字 0 到 15 的不同排列。那么如何根据一个 6 位的 S 盒输入，利用表 5.3 得到 4 位的输出?

如果第 i 个 S 盒的输入为：$B_i = b_1b_2b_3b_4b_5b_6$，则 S 盒变换得到 4 位输出的过程如下：

将 b_1b_6 和 $b_2b_3b_4b_5$ 作为二进制数，令 b_1b_6 和 $b_2b_3b_4b_5$ 对应的十进制数分别为 r 和 c $(0 \leq r \leq 3, 0 \leq c \leq 15)$，设第 i 个 S 盒中的第 r 行、c 列对应的整数为 N，则 N 的二进制表示就是该 S 盒在输入为 $B_i = b_1b_2b_3b_4b_5b_6$ 时对应的 4 位输出。举例说明：

若 $B_2 = 110101$，可知 $r = 3$、$c = 10$，在表5.3找到 S_2 的第 3 行、第 10 列对应的整数为 7，其二进制表示为 0111，所以 110101 对应的输出为 0111。

S 盒变换是函数 F 的核心所在，也是 DES 算法的关键步骤。除 S 盒之外，DES 的其他运算都是线性的，易于分析，而 S 盒是非线性的，它决定了 DES 算法的安全性。

48 位的比特输入（分为 8 个 6 位分组）在经过 8 个 S 盒代替运算后，得到 8 个 4 位的分组，它们重新合在一起形成一个 32 位的比特串，作为 P 盒变换的输入。

(4) P 盒变换

P 盒变换是将 S 盒输出的 32 位比特串根据固定的置换 P（也称为 P 盒）置换到相应的位置。表5.4给出了 P 盒变换表，P 盒变换运算后得到的输出即为函数 $F(R_{i-1}, K_i)$ 的最终结果。

(3) 逆初始置换 IP^{-1}

表5.5给出了 IP^{-1} 的置换表，显然可以看出 IP^{-1} 是 IP 的逆置换。

64 位的明文经过 IP 置换、16 轮迭代运算、逆初始置换 IP^{-1} 后，得到的结果为 DES 加密的密文输出。图5.6给出了 DES 对明文加密的整个过程，注意在最后一轮迭代运算后的结果并没有进行交换，即 $C = IP^{-1}(R_{16}, L_{16})$，这样做的目的是为了加密和解密使用同一个算法。

5.2.3 DES 解密过程

DES 的解密算法与加密算法共用相同的算法过程。两者的不同之处仅在于解密时子密钥 K_i 的使用顺序与加密时相反。如果加密的子密钥为 $K_1K_2 \ldots K_{16}$，那么，解密时子密钥的使用顺序为 $K_{16}, K_{15}, \ldots, K_1$。也就是说，使用 DES 解密算法进行解密时，将以 64 位的密文作为输入，第一轮迭代运算使用子密钥 K_{16}，第二轮迭代运算使用子密钥 K_{15}，……第 16 次迭代使用子密钥 K_1，而其它的运算过程与加密相同。这样输出的便是 64 位明文。与加密相对应，解密的 16 轮可以形式化的表示为：

$$
\begin{aligned}
&R_{i-1} = L_i \\
&L_{i-1} = R_i \oplus F\left(L_i, K_i\right) \\
&i = 16, 15, \ldots 2, 1
\end{aligned}
$$

表 5.3: DES 的 S 盒

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7	
2	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8	S_1
3	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0	
4	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13	
1	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10	
2	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5	S_2
3	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15	
4	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9	
1	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8	
2	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1	S_3
3	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7	
4	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12	
1	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15	
2	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9	S_4
3	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4	
4	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14	
1	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9	
2	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6	S_5
3	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14	
4	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3	
1	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11	
2	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8	S_6
3	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6	
4	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13	
1	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1	
2	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6	S_7
3	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2	
4	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12	
1	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7	
2	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2	S_8
3	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8	
4	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11	

表 5.4: P 盒变换表

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

表 5.5: IP^{-1} 的置换表

39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	68	26
33	1	41	9	49	17	67	25

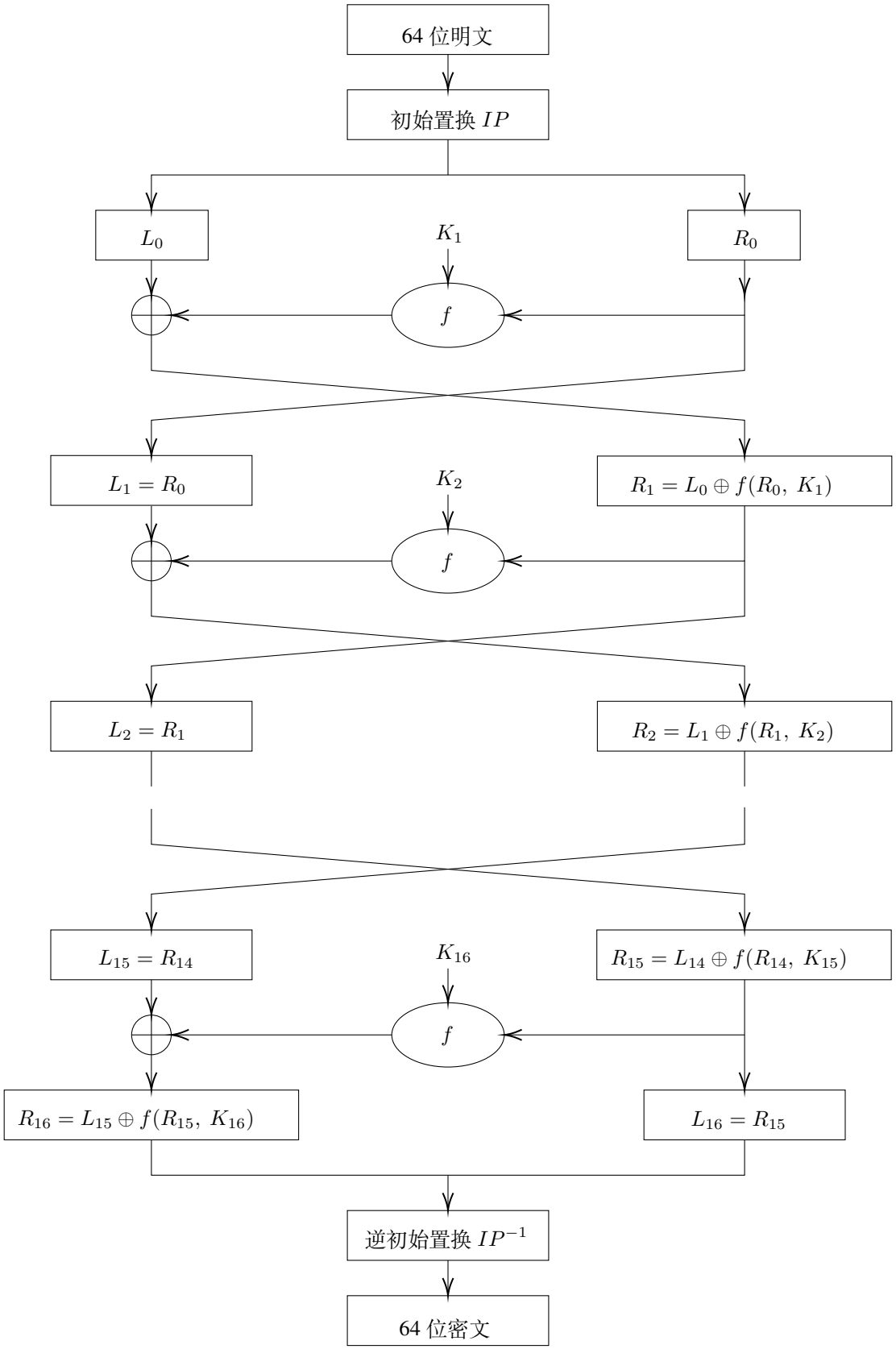


图 5.6: DES 加密 16 轮的过程

表 5.6: PC-1 置换

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

表 5.7: PC-2 置换

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

5.2.4 DES 子密钥生成

DES 算法共需进行 16 轮迭代运算，每轮迭代运算使用一个子密钥，共需要 16 个子密钥。子密钥是从用户输入的初始密钥（或称为种子密钥）产生的，用户输入的初始密钥 K 为 64 位，其中有 8 位用于奇偶校验，分别位于第 8，16，24，32，40，48，56，64 位。奇偶校验位用于检查密钥 K 在产生和分配以及存储过程中可能发生的错误，这样 DES 的密钥实际上只有 56 位。图5.7表示由初始密钥生成子密钥的过程。

(1) PC-1 置换。输入的种子密钥 K 首先经过一个置换（PC-1 选择置换）将比特重排。PC-1 置换的结果 (56 位) 的前 28 位作为 C_0，后 28 位作为 D_0。如表5.6所示，经过 PC-1 置换后不再出现第 8，16，24，32，40，48，64 位。也就是说 64 位的种子密钥 K 经过 PC-1 置换选择，奇偶校验位已经被删除，且剩余的 56 位密钥打乱重排。

(2) 在计算第 i 轮迭代所用的子密钥时，首先对 C_{i-1} 和 D_{i-1} 进行循环左移，分别得到 C_i 和 D_i。如果 $i = 1, 2, 9, 16$，循环左移动 1 位，否则循环移动 2 位。经过移位后得到的 C_i 和 D_i 作为下一个循环的输入。然后将 C_iD_i 经过 PC-2 置换，从 56 位中选取 48 位作为子密钥 K_i。选择置换 PC-2 如表5.7所示。

5.2.5 DES 的安全性

从 DES 诞生之日起，人们就对它的安全性持怀疑态度，并展开了激烈的争论，本节介绍几个主要的争论。

(1) 对 DES 的 S 盒、迭代次数、密钥长度等设计准则的争议。DES 的 S 盒、迭代次数、密钥长度都被设计成固定的参数，而且 S 盒的设计标准是保密的，至今未公布。人们担心如果在算法中嵌入了陷门，可使 NSA 使用一个简单的方法就能对消息解密。1976 年，NSA 公布了下列几条 S 盒的设计准则：

(a). S 盒的每一行是整数 $0, 1, 2 \ldots, 15$ 的一个置换；

(b). 没有一个 S 盒是它输入的线性或仿射函数；

(c). 改变 S 盒的一个输入比特至少要引起两比特的输出改变；

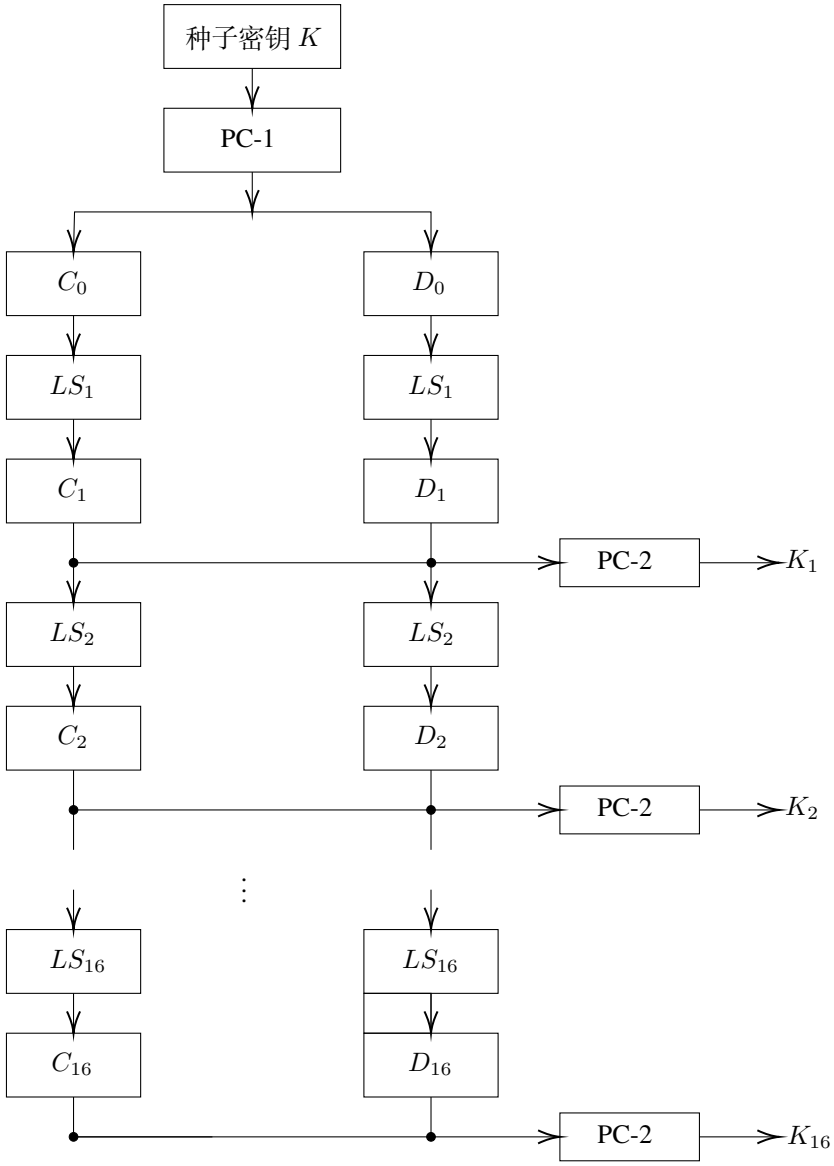


图 5.7: DES 子密钥的产生

(d). 对任何一个 S 盒和任何一个输入 x（x 为 6 位的比特串），$S(x)$ 和 $S(x \oplus 001100)$ 至少有两比特不同；

(e). 对任何一个 S 盒，如果两个输入的前两位不同，而最后两位相同，两个输出必须不同；

(f). 对任何一个 S 盒，如果固定一个输入比特保持不变而使其他 5 比特输入变化，我们观察一个固定输出比特的值，使这个输出比特为 0 的输入的数目与使这个输出比特为 1 的输入的数目总数接近相等。

但是，S 盒的设计准则还没有完全公开，我们仍然不知道 S 盒的构造中是否使用了进一步的设计准则。这也使得人们的猜测无法得到消除，关于 S 盒的争议仍然在继续。

(2) DES 存在着一些弱密钥和半弱密钥

DES 算法中各轮迭代所使用的子密钥是通过输入的种子密钥产生的。生成子密钥时，将种子密钥 (去除了奇偶校验位后) 分成两个部分 (可以分别称为 C 寄存器与 D 寄存器，分别用来保存 C_i 与 D_i 的值)，每个部分独立地进行循环移位。如果某个密钥 K，使得这两个部分的每一部分的所有位都

是 0 或 1，这时由它产生的 16 个子密钥相同，由此 $\mathrm{DES}_k(m) = \mathrm{DES}_k^{-1}(m)$。即由密钥 K 确定的加密变换和解密变换一致，这时我们称密钥 K 为一个弱密钥。也就是说由弱密钥 K 确定的加密变换与解密变换是一致的。

同时，DES 中还存在半弱密钥，这些密钥把明文加密成相同的密文，即存在一个不同于 K 的密钥 K' 使 $\mathrm{DES}_K(m) = \mathrm{DES}_{K'}(m)$。这时，密钥 K' 将能够解密由密钥 K 加密的明文。

虽然，DES 存在有弱密钥，但这并不是十分大的安全问题。相对于总数为 $2^{56} = 72057594037927936$ 的密钥空间，弱密钥与半弱密钥所占的比例实在太小了。如果随机地选择密钥，那么选中这些弱密钥中的一个的概率可以忽略不计。为了确保弱密钥不被选择做密钥，可以在密钥产生进行检查，保证不使用弱密钥作为 DES 的密钥。

(3) DES 的 56 位密钥无法抵抗穷举攻击

DES 的密钥只有 56 位，实际上，在 IBM 最初向 NSA 提交的方案当中，密钥的长度为 112 比特，但在 DES 成为一个标准时，被削减至 56 比特密钥。密钥量仅为 $2^{56} \approx 10^{17}$ 就目前的计算设备的计算能力而言，DES 不能抵抗对密钥的穷举攻击。

5.2.6 三重 DES

为了提高 DES 的安全性，可以使用多重 DES。多重 DES 就是使用多个密钥利用 DES 算法对明文进行多次加密，使用多重 DES 可以增加密钥量，从而大大提高抵抗密钥的穷索攻击的能力。

在这里介绍三重 DES，图5.8为加密和解密过程。设 k_1, k_2, k_3 是三个长度为 56 位的密钥。

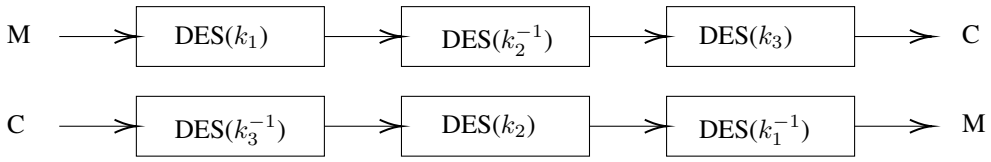


图 5.8: 三重 DES 加密与解密过程

给定明文 M，则密文 C 为:

$$C = \mathrm{DES}_{k_3}(\mathrm{DES}_{k_2{}^{-1}}(\mathrm{DES}_{k_1}(M))).$$

给定密文 C，则解密明文 M 为:

$$M = \mathrm{DES}_{k_1{}^{-1}}(\mathrm{DES}_{k_2}(\mathrm{DES}_{k_3{}^{-1}}(C))).$$

对于三重 DES，如果 $k_1 = k_2$ 或者 $k_2 = k_3$，则三重 DES 就变成了使用一个 56 位密钥的单重 DES。

在 DES 的标准报告 FIPS46-3 中推荐使用的三重 DES 是 $k_1 = k_3$ 的情形，这是一种比较受欢迎的 DES 代替方案。也可以使用 k_1, k_2, k_3 互不相同的三重 DES。

5.3 高级加密标准 AES

1997 年 1 月 2 日，美国国家标准和技术委员会 NIST（National Institute of Standard and Technology）宣布征集一个新的对称密钥分组密码算法，以取代 DES 作为新的加密标准，新的算法被命名为高级加密标准（AES，Advanced Encryption Standard)）。1997 年 9 月 12 日发布了征集算法的正式公告，要求 AES 具有 128 位分组长度，支持 128、192 和 256 位的密钥长度；比三重 DES 有效，至少要与三重 DES 一样安全；而且要求 AES 能在全世界范围内免费得到。

1998 年 8 月 20 日，NIST 宣布接受 15 个算法作为 AES 的候选算法，并提交全世界的密码学界协助分析这些算法。1999 年 8 月，确定了 5 个候选决赛算法，分别是 IBM 提交的 MARS、RSA 实验室提交的

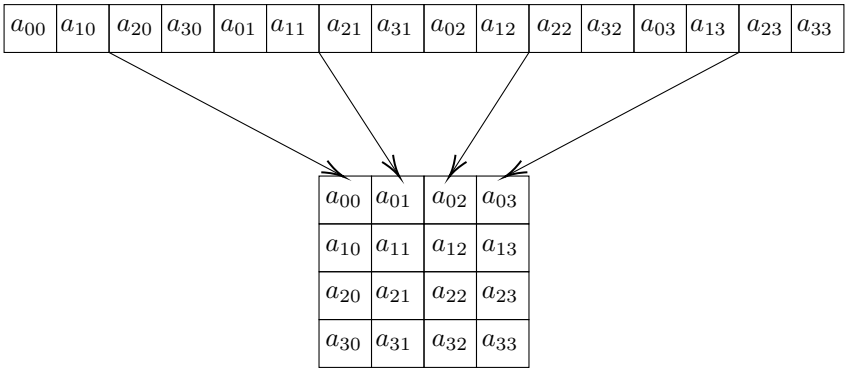


图 5.9: AES 的状态矩阵

表 5.8: 密钥矩阵

k_{00}	k_{01}	k_{02}	k_{03}
k_{10}	k_{11}	k_{12}	k_{13}
k_{20}	k_{21}	k_{22}	k_{23}
k_{30}	k_{31}	k_{32}	k_{33}

RC6、（Joan Daemen、Vincent Rijment 提交的）Rijndael、（Anderson、Biham、Knudsen 的）Serpent 以及（Schneier、Kelsy、Whiting、Wagner、Hall、Ferguson 的）Twofish。经过对决赛算法的进一步分析，NIST 决定将 Rijndael 作为 AES。

Rijndael 是由两个比利时的密码学家 Daemen 和 Rijment 共同设计的。Rijndael 算法是一个迭代型分组密码，其分组长度和密钥长度都可变，各自可以为 128 比特、192 比特、256 比特。

Rijndael 对明文以字节为单位进行处理，以 128 位的分组、128 位密钥的情况为例。首先将明文按字节分成列组，将明文的前 4 个字节组成一列，接下来的 4 个字节组成第二列，后面的字节依次组成第三列和第四列，则组成了一个 4 乘 4 的矩阵。如图5.9所示。

这样 AES 输入的 16 个字节排成了一个二维数组，称之为状态矩阵。AES 的加密和解密变换都是基于状态数组来处理的，在中间结果上的不同变换操作称为状态。AES 的状态矩阵的列数 N_b 等于的长度除以 32，矩阵中的每个元素是一个 8 位字节。

同样，密钥也可以表示成一个 $4 * N_k$ 的矩阵，N_k 等于密钥的长度除以 32，当密钥长度为 128 位时 $N_k = 4$，如表5.8所示。

5.3.1 AES 的加密变换

为表述简单起见，本节只讨论密钥长度为 128 比特、分组长度为 128 比特时的情形，对于其他分组长度和密钥长度的类型，AES 工作的原理是相同的。

与 DES 相同，AES 也是由最基本的变换单位——“轮”多次迭代而成的，当分组长度和密钥分组长度均为 128 比特时，轮数为 $N_r = 10$。我们将 AES 中的轮变换计为 Round(State, RoundKey)，State 表示消息矩阵；RoundKey 表示轮密钥矩阵。一轮的完成将改变 State 矩阵中的元素，称为改变它的状态。对于加密来说，输入到第一轮中的 State 就是明文消息矩阵，最后一轮输出的 State 就是对应的密文消息矩阵。

AES 的轮（除最后一轮外）变换由四个不同的变换组成，这些变化我们称之为内部轮函数，AES 的轮可表示成如下形式：

```
Round(State, RoundKey){
    SubBytes(State);
    ShiftRows(State):
```

MixColumns(State);
AddRoundKey(State,RoundKey);}

其中：ByteSub(State) 称为字节代替变换、ShiftRow(State) 称为行移位变换、MixColumn(State) 为列混合变换以及 AddRoundKey(State, RoundKey) 为与子密钥异或。最后一轮略微的不同，将其记作 FinalRoundKey(State, RoundKey)，相当于前面的 Round(State, RoundKey) 去掉 MixColumns(State)。

AES 的算法的执行过程为（如图5.10所示）：

(1) 给定的明文 M，将 State 初始化为 M，并进行 AddRoundKey(State,RoundKey)，将 RoundKey 与 State 进行异或运算；

(2) 对于前 N_r-1 轮中的每一轮，分别执行 Round(State, RoundKey) 过程；

(3) 执行最后一轮 FinalRoundKey(State,RoundKey) 过程，即只执行 SubBytes(State)、ShiftRows(State)、AddRoundKey(State,Ro 三个操作。

前面已知，AES 的轮函数分 4 层（或称为内部轮函数）：ByteSub(State) 字节代替变换、ShiftRow(State) 行移位变换、MixColumn(State) 列混合变换以及 AddRoundKey (State, RoundKey) 与子密钥与。下面分别介绍在 AES 加密轮变换过程中的 4 个子变换。

(1) ByteSub(State) 字节代替变换

ByteSub(State) 为非线性的、可逆的字节代替变换，它作用在状态的每个字节上，将字节利用替代表进行运算。该变换是由以下两个子变换合成：

首先，将字节看成有限域 $GF(2^8)$ 上的元素，按模 $m(x)$ 映射到自己的乘法逆，0 映射到自身；其次，做如下(5.1)所示 $GF(2)$ 的仿射变换（可逆的）。

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}. \tag{5.1}$$

可以预先将 $GF(2^8)$ 上的每个元素做 ByteSub(State) 变换，从而形成字节代替表，也称为 S 盒子，如表5.9所示。进行 $ByteSub(State)$ 变换时只需要查表操作。AES 的 S 盒是一个 16 乘 16 的矩阵。通过查表可以方便的得到 $ByteSub(State)$ 的输出，如果状态中的一个字节为 xy，则在 S 盒的第 x 行，y 列对应的字节就是 $ByteSub(State)$ 的输出。如(5.2)所示的输入状态数组与相应的输出的一个例子：

$$\begin{pmatrix} 21 & \text{a3} & 12 & \text{b7} \\ 3\text{a} & \text{db} & 04 & 9\text{c} \\ 74 & \text{ff} & \text{e5} & \text{af} \\ 8\text{c} & 48 & 5\,\text{b} & 7\text{c} \end{pmatrix} \xrightarrow{\text{ByteSub(State)}} \begin{pmatrix} \text{fd} & 0\text{a} & \text{c9} & \text{a9} \\ 80 & \text{b9} & \text{f2} & \text{de} \\ 92 & 16 & \text{d9} & 79 \\ 64 & 52 & 39 & 10 \end{pmatrix} \tag{5.2}$$

(2) ShiftRow(State) 行移位变换

ShiftRow(State) 行移位变换作用于 S 盒的输出，对状态数组的每一行循环左移不同的字节。若密钥为 128 位的情况，第 0 行保持不变，其他行内的字节循环左移，第 1 行移动 1 个字节，第 2 行移动 2 个字节，第 3 行移动 3 个字节，如图5.11所示。

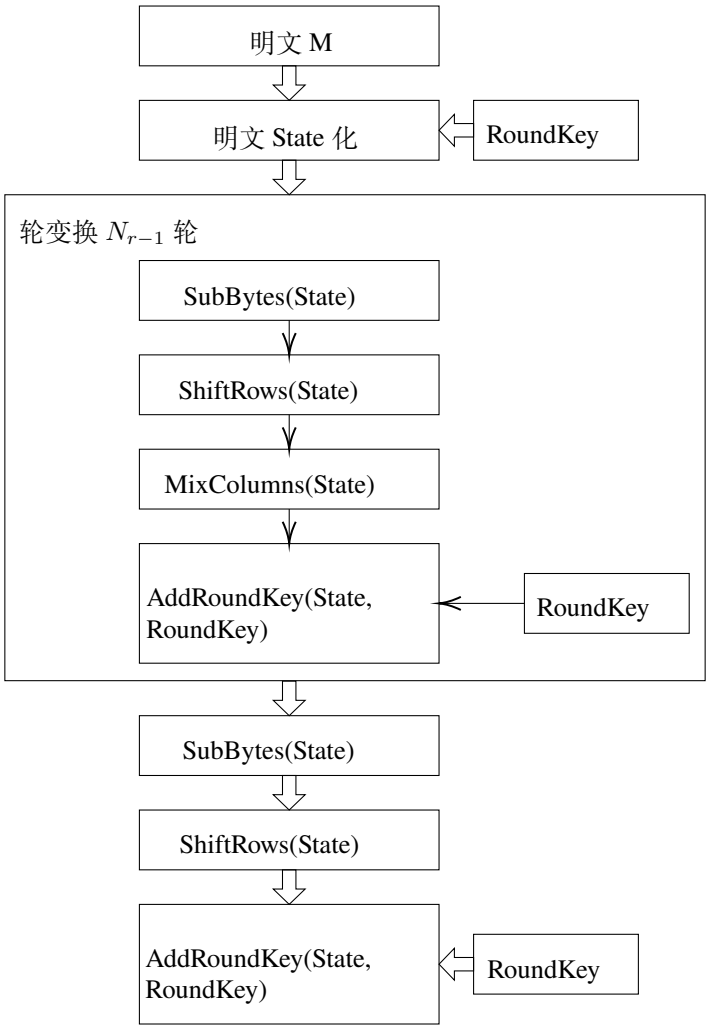


图 5.10: AES 加密过程

表 5.9: AES 的字节代替表

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16



图 5.11: ShiftRow(State) 变换

(3) MixColumn(State) 列混合变换

MixColumn(State) 将状态数组上的每一列看成 $GF(2^8)$ 上的一个多项式，且与一个固定的多项式 $c(x)$ 进行模 $M(x) = x^4 + 1$ 乘法，其中多项式 $c(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$。如图5.12所示。

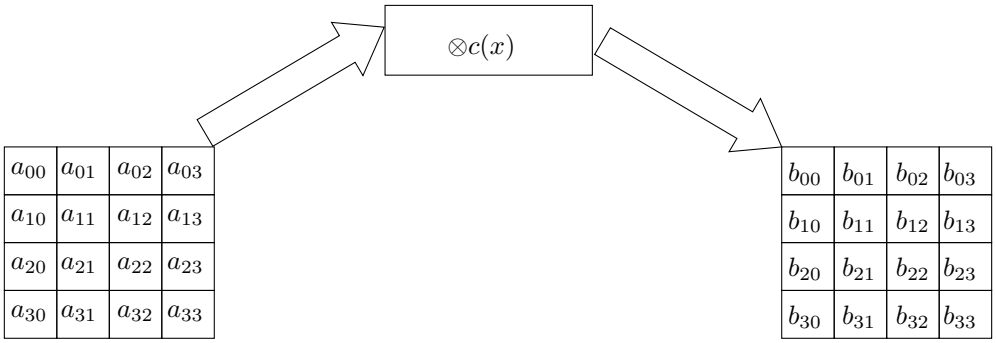


图 5.12: MixColumn(State) 变换

令 $b_j(x) = c(x) \otimes a_j(x)$，$(0 \le j \le 3$，$\otimes$ 表示模 $x^4 + 1$ 乘法$)$ 由于 $x^i \bmod (x^4 + 1) = x^{i \bmod 4}$，可以将其表示成矩阵乘法。如(5.3)式所示。

$$\begin{pmatrix} b_{0j} \\ b_{1j} \\ b_{2j} \\ b_{3j} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 01 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} a_{0j} \\ a_{1j} \\ a_{2j} \\ a_{3j} \end{pmatrix} \tag{5.3}$$

例如，如果 ShiftRow() 变换输出的某一列为 11，09，01 和 35，那么利用矩阵相乘，可得

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 01 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} 11 \\ 09 \\ 01 \\ 35 \end{pmatrix} \longrightarrow \begin{pmatrix} 73 \\ 6\mathrm{b} \\ \mathrm{ba} \\ \mathrm{a7} \end{pmatrix}$$

(4) AddRoundKey(State, RoundKey) 与子密钥异或

AddRoundKey(State, ExpandedKey) 只是简单的将密钥按位异或到一个状态上，如(5.4)式所示，每个轮密钥长度为 4 个字。每轮加密密钥按顺序取自扩展密钥，扩展密钥是由初始密钥扩展而成，128 位分组的情况，则需要 44 个字的密钥。

$$\begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix} \bigoplus \begin{pmatrix} k_{00} & k_{01} & k_{02} & k_{03} \\ k_{10} & k_{11} & k_{12} & k_{13} \\ k_{20} & k_{21} & k_{22} & k_{23} \\ k_{30} & k_{31} & k_{32} & k_{33} \end{pmatrix} \longrightarrow \begin{pmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \\ b_{20} & b_{21} & b_{22} & b_{23} \\ b_{30} & b_{31} & b_{32} & b_{33} \end{pmatrix} \tag{5.4}$$

5.3.2 AES 的解密变换

AES 的解密变换与加密变换是互逆的，轮函数也分为 4 层。分别为 InvShiftRow(State) 逆行移位变换、InvByteSub(State) 逆字节代替变换、AddRoundKey(State,RoundKey) 与子密钥位与和 InvMixColumn(State) 逆列混合变换。同样在最后一轮中，不采用列的 InvMixColumn(State) 变换。

将 AES 的解密过程的轮变换表示为：

```
InvRound(State, RoundKey){
    InvShiftRow(State);
    InvByteSub(State);
    AddRoundKey (State, RoundKey );
    InvMixColumn(State);}
```

则 AES 的算法的解密执行过程为：

(1) 给定的密文 C，进行 State 初始化，并进行 AddRoundKey(State,RoundKey)，将 RoundKey 与 State 进行异或运算；

(2) 对于前 N_r-1 轮中的每一轮，分别执行 InvRound(State, RoundKey) 过程；

(3) 执行最后一轮的 InvShiftRow(State) 变换、InvByteSub(State) 变换、与 AddRoundKey(State, RoundKey)。

对于解密轮变换中的子变换，描述如下：

(1) InvShiftRow(State) 变换

InvShiftRow 变换是 ShiftRow(State) 的逆变换。InvShiftRow(State) 对每个状态的每一行循环右移不同的字节。密钥为 128 位的情况下，第 0 行不移位保持不变，第 1 行循环右移 1 个字节，第 2 行循环右移 2 个字节，第 3 行循环右移 3 个字节，如图5.13所示：

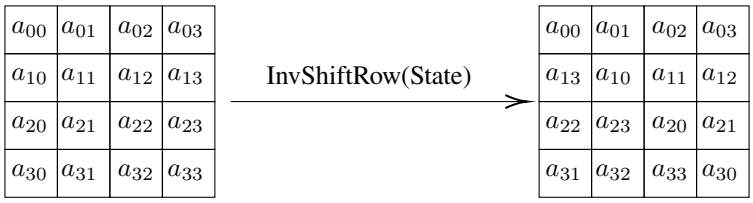


图 5.13: InvShiftRow(State) 变换

(2) InvByteSub(State) 变换

InvByteSub(State) 是 ByteSub(State) 的逆变换，它将状态中的每一个字节非线性地变换成另一个字节，首先，对字节在 $GF(2)$ 上做如(5.5)式的逆仿射变换，然后求 $GF(2^8)$ 上的逆元素。

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}^{-1} \left(\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \right). \tag{5.5}$$

同样，可以将逆字节变换 InvByteSub(State) 对各种可能的的变换结果排成一个表，称为 AES 的逆字节代替表或逆 S 盒，如表5.10所示。通过查表可以方便的得到逆字节替换的返回值。因为 InvByteSub(State) 是 ByteSub(State) 的逆变换，则如果在 AES 字节代替表中第 x 行 y 列的字节为 $x'y'$，那么 AES 逆

表 5.10: AES 的逆字节代替表

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

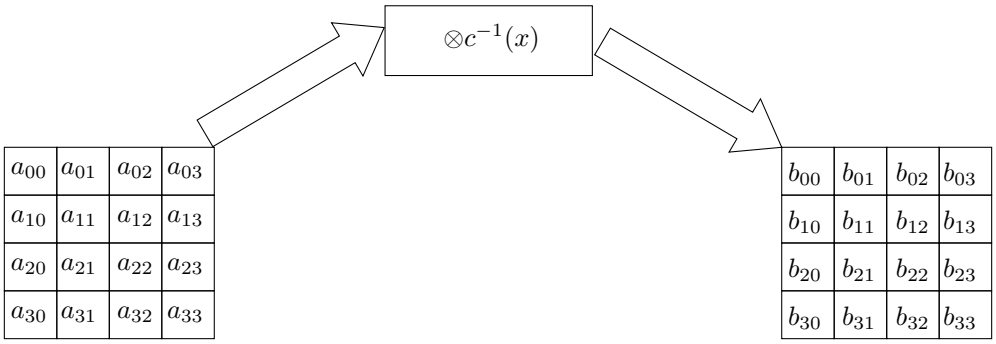


图 5.14: InvMixColumn() 变换

代替表中的第 x' 行 y' 列的字节为 xy。

例如,将 ByteSub(State) 变换中的例子的输出状态数组作为 InvByteSub(State) 的输入,可得 InvByte(State) 的输出矩阵就是 ByteSub(State) 的输入矩阵，如(5.6)式所示。

$$\begin{pmatrix} 21 & a3 & 12 & b7 \\ 3a & db & 04 & 9c \\ 74 & ff & e5 & af \\ 8c & 48 & 5\,b & 7c \end{pmatrix} \xrightarrow{\text{InvByteSub(State)}} \begin{pmatrix} fd & 0a & c9 & a9 \\ 80 & b9 & f2 & de \\ 92 & 16 & d9 & 79 \\ 64 & 52 & 39 & 10 \end{pmatrix}. \tag{5.6}$$

(3) AddRoundKey(State) 变换

与加密时的 AddRoundKey(State) 变换相同，只是将密钥与状态数组进行简单的按位异或运算。

(4) InvMixColumn(State) 变换

InvMixColumn(State) 变换是 MixColumn(State) 的逆变换。InvMixColumn(State) 将状态数组中的每一列看作有限域 $GF(2^8)$ 上，次数小于 4 的多项式，并且与一个固定的多项式 $c^{-1}(x)$ 进行模 $m(x) = x^4 + 1$ 乘法，如图5.14所示。

$$c^{-1}(x) = \{0b\}x^3 + \{0d\}x^2 + \{09\}x + \{0e\},$$

其中多项式 $c^{-1}(x)$ 是 $c(x)$ 的乘法逆多项式。也就是 $c(x) \otimes c^{-1}(x) = \{01\}$。

同样也可以将 $b_j(x) = c^{-1}(x) \otimes a_j(x)$ 写成矩阵相乘的形式为（(5.7)式）：

$$\begin{pmatrix} b_{0\mathrm{j}} \\ b_{1\mathrm{j}} \\ b_{2\mathrm{j}} \\ b_{3\mathrm{j}} \end{pmatrix} = \begin{pmatrix} 0\mathrm{e} & 0\mathrm{b} & 0\mathrm{d} & 09 \\ 09 & 0\mathrm{e} & 0\mathrm{b} & 0\mathrm{d} \\ 0\mathrm{d} & 09 & 0\mathrm{e} & 0\mathrm{b} \\ 0\mathrm{b} & 0\mathrm{d} & 09 & 0\mathrm{e} \end{pmatrix} \begin{pmatrix} a_{0\mathrm{j}} \\ a_{1\mathrm{j}} \\ a_{2\mathrm{j}} \\ a_{3\mathrm{j}} \end{pmatrix}. \tag{5.7}$$

5.3.3 AES 密钥编排

AES 的每一轮的轮密钥都是从初始的种子密钥编排得到的，密钥编排包括密钥扩展和轮密钥选取两个步骤。首先密钥扩展将种子密钥扩展成长为 $N_b(N_r + 1)$ 的扩展密钥，然后从扩展密钥中依次选取 N_b 个字长作为每一轮的密钥。

经过密钥扩展得到的扩展密钥是 4 字节长的字的线性序列，可表示为 $W[i]$（其中 $0 \le i \le N_b(N_r+1)$）。前 N_k 个字就是种子密钥本身，其余的字则由序号较小的字递归定义得到，递归的方式按照 N_k 大小分的为 $N_k \le 6$ 和 $N_k > 6$ 两种情况。

(1) $N_k \le 6$。

(a). $i \not\equiv 0 \bmod N_k$ 时，$W[i]$ 定义为 $W[i-1]$ 与 $W[i-N_k]$ 进行异或运算的结果。

(b). $i \equiv 0 \bmod N_k$ 时，先将 $W[i-1]$ 左移一个字节，然后做字节替代变换，再先后与 $((02)^i, 00, 00, 00)$ 和与 $W[i-N_k]$ 进行异或运算，其结果定义为 $W[i]$。

(2) $N_k > 6$。

(a). $i \not\equiv 0 \bmod N_k$ 且 $i \not\equiv 4 \bmod N_k$ 时，将 $W[i]$ 定义为 $W[i-1]$ 与 $W[i-N_k]$ 进行异或运算的结果。

(b). $i \not\equiv 0 \bmod N_k$ 但 $i \equiv 4 \bmod N_k$ 时，将 $W[i]$ 定义为先对 $W[i-1]$ 做字节代替变换，然后再与 $W[i-N_k]$ 进行异或运算的结果。

(c). $i \equiv 0 \bmod N_k$ 时，先将 $W[i-1]$ 左移一个字节，然后做字节替代变换，再先后与 $((02)^i, 00, 00, 00)$ 和与 $W[i-N_k]$ 进行异或运算，其结果定义为 $W[i]$。

5.4 国际数据加密算法 IDEA

IDEA 是由 X.J.Lai 和 J.L.Massey 设计。1990 年公布的第一版称为 PES（Proposed Encryption Standard，建议加密标准）。最初的设计不能抵挡差分密码分析。经过改进后第二版本称为 IPES（Improved Proposed Encryption Standard，即改进型建议加密标准），可以抗击差分分析。1992 年，设计者又将 IPES 改名为 IDEA（International Data Encryption Algorithm，国际数据加密算法）。

IDEA 的明文和密文分组都是 64 比特，密钥的长度是 128 比特，同一算法既可用于加密又可用于解密，该算法所依据的设计思想是“混合使用来自不同代数群中的运算”。该算法所需要的“混乱”可通过连续使用三个“不相容”的群运算于两个 16 比特子块来获得，并且该算法所选择使用的密码结构可提供必要的“扩散”。

IDEA 加密由 8 轮类似的运算和最后的一个输出变换组成，共应用了 3 种不同的作用于一对 16 比特子块的群运算，即：

(1) 两个 16 比特子块的按位摸 2 加法，用 $\oplus$ 表示，即 XOR 运算。

(2) 定义在模 $2^{16}(= 65536)$ 的模加运算，其操作数都可以表示成 16 位整数，用 $\boxplus$ 表示这个函数，下面简称为加法或相加。

(3) 定义在模 $2^{16}+1(= 65537)$ 的模乘运算。因为 65537 是一个素数，所以对于任何数的乘法逆元是存在的。同时，为了保证 0 出现在 16 位的操作数时也有乘法逆元存在，0 被定义为 2^{16}，用 $\odot$ 表示这

个函数，简称为乘法或相乘。

5.4.1 IDEA 算法描述

IDEA 是由 8 轮和随后的一个输出变换组成，如图5.15所示，描述了 IDEA 算法的一轮变换。

将 64 比持明文块分成 4 个 16 比持子块 X_1，X_2，X_3，X_4，这四个子块就是加密算法第一轮的输入。在每一轮中，这四个子块分别和密钥的 6 个 16 比特子块及中间结果进行 3 种不同的群运算，即 XOR、加运算和乘运算。在两圈之间，第 2 和第 3 个子块进行交换。

在每一轮中，执行的过程如下：

(1) 输入子块 X_1 和第 1 个子密钥块进行“乘”运算；
(2) 输入子块 X_2 和第 2 个子密钥块进行“加”运算；
(3) 输入子块 X_3 和第 3 个子密钥块进行“加”运算；
(4) 输入子块 X_4 和第 4 个子密钥块进行“乘”运算；
(5) 将第 (1) 步的结果和第 (3) 步的结果相异或；
(6) 将第 (2 步的结果和第 (4) 步的结果相异或；
(7) 第 (5) 步的结果与第 (5) 个子密钥块进行“乘”运算；
(8) 第 (6) 步的结果和第 (7) 步的结果进行“加”运算；
(9) 第 (8) 步的结果与第 (6) 个子密钥块进行“乘”运算；
(10) 第 (7) 步的结果和第 (9) 步的结果进行“加”运算；
(11) 第 (1) 步的结果和第 (9) 步的结果相异或；
(12) 第 (3) 步的结果和第 (9) 步的结果相异或；
(13) 第 (2) 步的结果和第 (10) 步的结果相异或；
(14) 第 (4) 步的结果和第 (10) 步的结果相异或。

第 r 轮的输出的 4 个子块 $(1 \le r \le 8)$，即 (11)、(12)、(13) 和 (14) 步的结果。将中间两个块交换 (最后一轮除外) 后，就是下一轮的输入。

在经过 8 轮运算之后，进行下述的最终输出变换：（图5.16 IDEA 的输出变换）

(1) 输入子块 X_1 和第 1 个子密钥块进行“乘运算；
(2) 输入子块 X_2 和第 2 个子密钥块进行“加”运算；
(3) 输入子块 X_3 和第 3 个子密钥块进行“加”运算；
(4) 输入子块 X_4 和第 4 个子密钥块进行“乘”运算。

最后，将这四步运算的结果连到一起，就构成长度为 64 比特的密文。

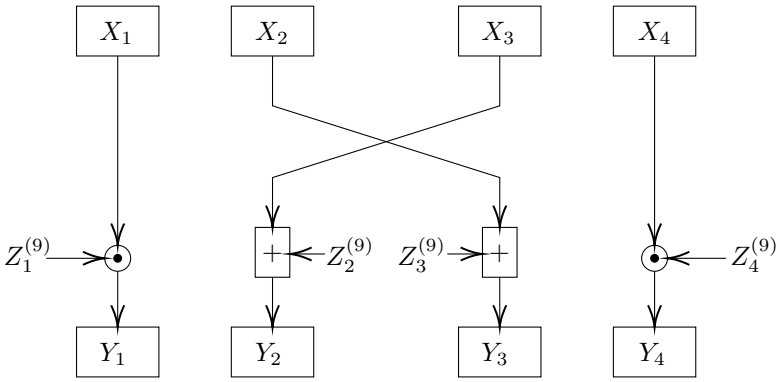


图 5.16: IDEA 的输出变换

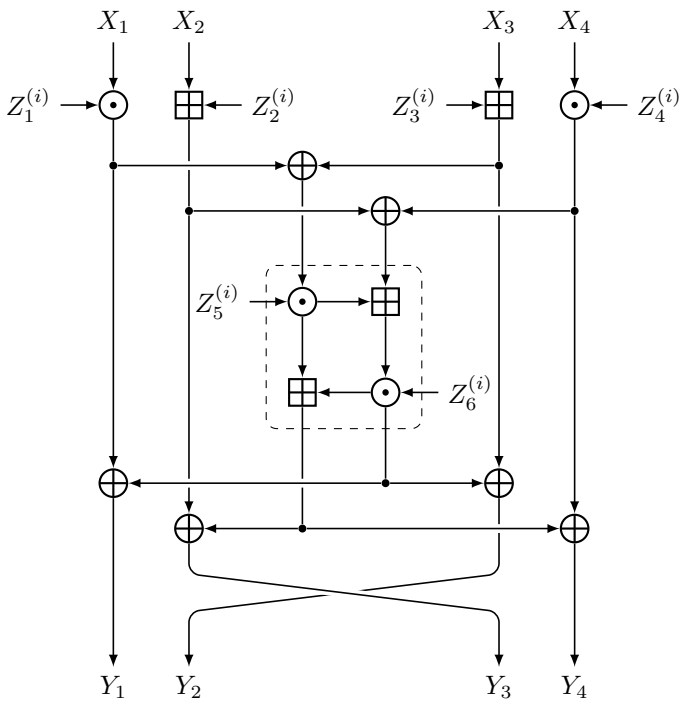


图 5.15: IDEA 的轮函数

5.4.2 IDEA 算法的解密

IDEA 的解密过程与加密过程基本相同，使用的运算逻辑相同。但两者在每一轮迭代所使用的子密钥上有较大的差别。

IDEA 的解密子密钥是加密子密钥的乘法逆 (即模 $2^{16}+1$ 乘法逆元) 或加法逆 (即模 2^{16} 加法逆元) 。IDEA 的子密钥通过如下方式从加密子密钥中产生：

(1) 解密运算的第 i 轮迭代的前 4 个子密钥是从加密运算的第 $10-i$ 轮的前 4 个子密钥中得到的，在这里将输出变换看作第 9 轮迭代。其中第 1 和第 4 个解密子密钥等于对应的第 1 个和第 4 个加密子密钥的乘法逆元。从第 2 轮到第 8 轮迭代，第 2 个和第 3 个解密子密钥等于对应的第 2 个和第 3 个加密子密钥的加法逆元。第 1 轮和第 9 轮迭代的第 2 个和第 3 个子密钥等于对应的第 2 个和第 3 个加密子密钥的加法逆元。

(2) 对于前 8 轮迭代，第 i 轮迭代的最后两个解密子密钥等于加密运算的第 $9-i$ 轮迭代的最后两个子密钥。

可以把解密子密钥与加密子密钥之间的关系表示成一张表，如表5.11所示；其中 $(k_j^{(i)})^{-1}$ 表示 $k_j^{(i)}$ 的乘法逆，$-k_j^{(i)}$ 表示 $k_j^{(i)}$ 的加法逆。

5.4.3 IDEA 密钥生成

上一小节介绍了 IDEA 的加密和解密子密钥的关系，那么 IDEA 的加密子密钥是如何产生的？IDEA 的用户子密钥为 128 位，但在 IDEA 的加密中，共用了 52 组 16 位的子密钥，每个子密钥都是通过初始密钥产生的。

IDEA 子密钥的产生过程比较简单，它主要通过对初始密钥进行移位，对得到的结果进行分组，将分组后的结果作为子密钥。其产生过程描述如下：

首先，将 128 位的用户密钥分成 8 组，每组 16 位。其中的前 6 个分组作为第 1 轮迭代的子密钥，后

表 5.11: IDEA 的加密与解密子密钥

轮数	加密子密钥	解密子密钥
1	$k_1^{(1)}k_2^{(1)}k_3^{(1)}k_4^{(1)}k_5^{(1)}k_6^{(1)}$	$(k_1^{(9)})^{-1} - k_2^{(9)} - k_3^{(9)}(k_4^{(9)})^{-1}k_5^{(8)}k_6^{(8)}$
2	$k_1^{(2)}k_2^{(2)}k_3^{(2)}k_4^{(2)}k_5^{(2)}k_6^{(2)}$	$(k_1^{(8)})^{-1} - k_2^{(8)} - k_3^{(8)}(k_4^{(8)})^{-1}k_5^{(7)}k_6^{(7)}$
3	$k_1^{(3)}k_2^{(3)}k_3^{(3)}k_4^{(3)}k_5^{(3)}k_6^{(3)}$	$(k_1^{(7)})^{-1} - k_2^{(7)} - k_3^{(7)}(k_4^{(7)})^{-1}k_5^{(6)}k_6^{(6)}$
4	$k_1^{(4)}k_2^{(4)}k_3^{(4)}k_4^{(4)}k_5^{(4)}k_6^{(4)}$	$(k_1^{(6)})^{-1} - k_2^{(6)} - k_3^{(6)}(k_4^{(6)})^{-1}k_5^{(5)}k_6^{(5)}$
5	$k_1^{(5)}k_2^{(5)}k_3^{(5)}k_4^{(5)}k_5^{(5)}k_6^{(5)}$	$(k_1^{(5)})^{-1} - k_2^{(5)} - k_3^{(5)}(k_4^{(5)})^{-1}k_5^{(4)}k_6^{(4)}$
6	$k_1^{(6)}k_2^{(6)}k_3^{(6)}k_4^{(6)}k_5^{(6)}k_6^{(6)}$	$(k_1^{(4)})^{-1} - k_2^{(4)} - k_3^{(4)}(k_4^{(4)})^{-1}k_5^{(3)}k_6^{(3)}$
7	$k_1^{(7)}k_2^{(7)}k_3^{(7)}k_4^{(7)}k_5^{(7)}k_6^{(7)}$	$(k_1^{(3)})^{-1} - k_2^{(3)} - k_3^{(3)}(k_4^{(3)})^{-1}k_5^{(2)}k_6^{(2)}$
8	$k_1^{(8)}k_2^{(8)}k_3^{(8)}k_4^{(8)}k_5^{(8)}k_6^{(8)}$	$(k_1^{(2)})^{-1} - k_2^{(2)} - k_3^{(2)}(k_4^{(2)})^{-1}k_5^{(1)}k_6^{(1)}$
9	$k_1^{(9)}k_2^{(9)}k_3^{(9)}k_4^{(9)}$	$(k_1^{(1)})^{-1} - k_2^{(1)} - k_3^{(1)}(k_4^{(1)})^{-1}$

两组作为第 2 轮迭代的前两个子密钥。

然后，将 128 的密钥循环左移 25 位，再分成 8 组，得到 8 个子密钥，其中前 4 组作为第 2 轮的子密钥，后 4 组作为第 3 轮迭代的子密钥。重复这个过程，直至产生全部 52 个子密钥。

5.5 SMS4 密码算法

SMS4 是用于 WAPI（WLAN Authentication and Privacy Infrastructure）的分组密码算法，是国内官方公布的第一个商用密码算法。SMS4 算法是一个分组算法。SMS4 算法的分组长度为 128 比特，密钥长度为 128 比特。加密算法与密钥扩展算法都采用 32 轮非线性迭代结构。解密算法与加密算法的结构相同，只是轮密钥的使用顺序相反，解密轮密钥是加密轮密钥的逆序。

5.5.1 算法描述

SMS4 算法以字为单位进行加密处理，一次迭代运算称为一轮变换，假设明文的输入为 (X_0, X_1, X_2, X_3)，密文输出为 (Y_0, Y_1, Y_2, Y_3)。SMS4 一轮迭代当前的输入为 $(X_i, X_{i+1}, X_{i+2}, X_{i+3})$，本轮的轮密钥为 rK_i，则一轮的加密变换为：

$$\begin{aligned} X_{i+4} &= F(X_i, X_{i+1}, X_{i+2}, X_{i+3}, rK_i) \\ &= X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rK_i). \end{aligned}$$

其中，$\oplus$ 表示 32 比特的字相异或，每轮的输入 X_{i+1}，X_{i+2}，X_{i+3} 以及本轮的密钥 rK_i 进行异或运算，运算的 32 位的结果进行 T 变换，T 变换包括 S 盒变换和 L 线性变换。T 变换的结果再 X_i 与进行异或运算，这时候运算的结果输出就是 X_{i+4}，如图5.17所示。

SMS4 算法共需要 32 轮迭代，第 29 轮，30 轮，31 轮和 32 轮的迭代输出 X_{32}、X_{33} 、X_{34} 和 X_{35} 经过一个 R 变换 $R(X_{32}, X_{33}, X_{34}, X_{35}) = (X_{35}, X_{34}, X_{33}, X_{32}) = (Y_0, Y_1, Y_2, Y_3)$，$Y_0$，$Y_1$，$Y_2$，$Y_3$ 就是加密的密文。其中 R 变换为反序变换：$R(A_0, A_1, A_2, A_3) = (A_3, A_2, A_1, A_0)$，其中 A_i 是 32 位的字。

S 盒变换：SMS4 的 S 盒输入为 8 位，输出为 8 位。SMS4 将 32 位的输入分成 4 个字节，四个字节分别经过 S 盒，如图5.18所示。和 DES 相同，可以通过查表的方式得到 S 盒的输出，如表5.12所示 SMS4 的 S 盒。例如 S 盒的输入为 cd，通过查表可知，其输出为 10。

L 线性变换：设输入为 B，B 为一个 32 位的字，输出为 C，则 L 变换为：$C = L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$，$\lll$ 表示 32 比特的字循环左移，如图5.19所示（CLS 表示 32 比特的字循环左移）。

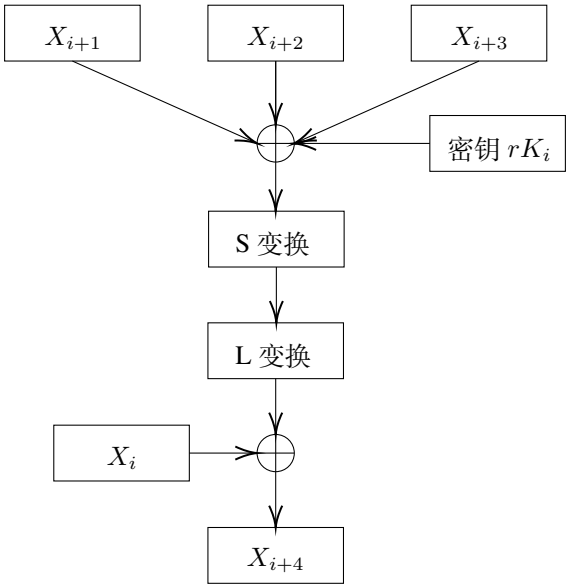


图 5.17: SMS4 的单轮加密

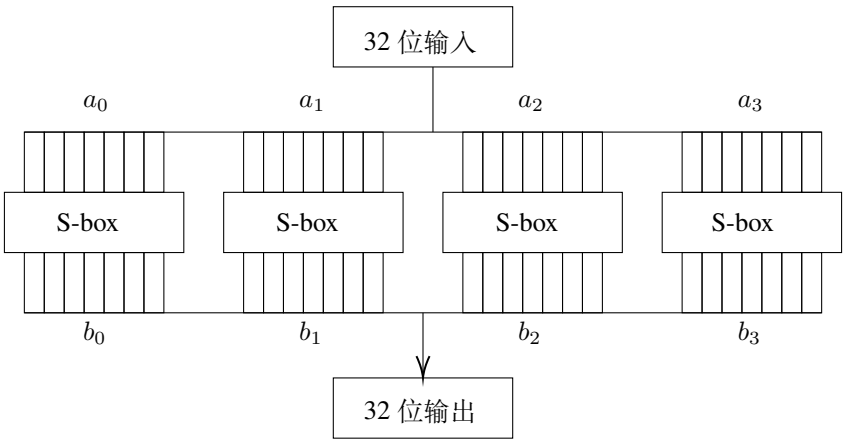


图 5.18: SMS4 的 S 盒变换

表 5.12: SMS4 的 S 盒

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	d6	90	e9	fe	cc	e1	3d	b7	16	b6	14	c2	28	fb	2c	05
1	2b	67	9a	76	2a	be	04	c3	aa	44	13	26	49	86	06	99
2	9c	42	50	f4	91	ef	98	7a	33	54	0b	43	ed	cf	ac	62
3	e4	b3	1c	a9	c9	08	e8	95	80	df	94	fa	75	8f	3f	a6
4	47	07	a7	fc	f3	73	17	ba	83	59	3c	19	e6	85	4f	a8
5	68	6b	81	b2	71	64	da	8b	f8	eb	0f	4b	70	56	9d	35
6	1e	24	0e	5e	63	58	d1	a2	25	22	7c	3b	01	21	78	87
7	d4	00	46	57	9f	d3	27	52	4c	36	02	e7	a0	c4	c8	9e
8	ea	bf	8a	d2	40	c7	38	b5	a3	f7	f2	ce	f9	61	15	a1
9	e0	ae	5d	a4	9b	34	1a	55	ad	93	32	30	f5	8c	b1	e3
a	1d	f6	e2	2e	82	66	ca	60	c0	29	23	ab	0d	53	4e	6f
b	d5	db	37	45	de	fd	8e	2f	03	ff	6a	72	6d	6c	5b	51
c	8d	1b	af	92	bb	dd	bc	7f	11	d9	5c	41	1f	10	5a	d8
d	0a	c1	31	88	a5	cd	7b	bd	2d	74	d0	12	b8	e5	b4	b0
e	89	69	97	4a	0c	96	77	7e	65	b9	f1	09	c5	6e	c6	84
f	18	f0	7d	ec	3a	dc	4d	20	79	ee	5f	3e	d7	cb	39	48

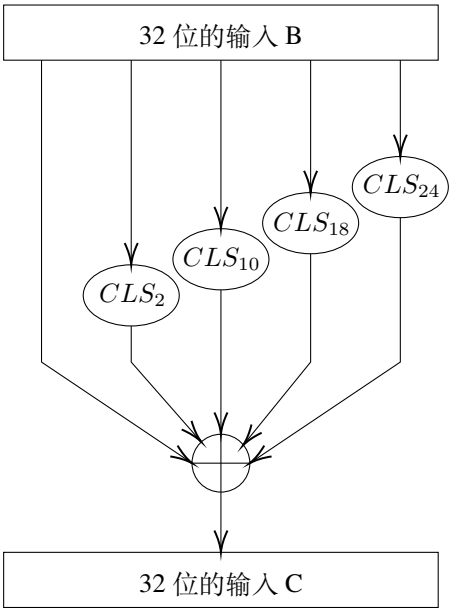


图 5.19: SMS4 的 L 变换

5.5.2 密钥扩展

SMS4 算法的解密与加密变换结构相同，只是使用轮密钥的顺序不同，如果加密的时候使用的密钥顺序为：$(rK_0, rK_1, rK_2, \ldots, rK_{31})$，则解密时使用的顺序为 $(rK_{31}, rK_{30}, rK_{29}, \ldots, rK_0)$。

SMS4 的加密密钥为 128 位，但每轮迭代的密钥为 32 位，由于算法需要迭代 32 轮，所以共需要 32 个子密钥，则需要从 128 位的用户密钥扩展出 32 个子密钥。其密钥扩展算法如下：

设加密密钥为 $MK = (MK_0, MK_1, MK_2, MK_3)$，$MK_i$ 为 32 位的字，$i = 0, 1, 2, 3$；则轮密钥的生成算法为，如图5.20：

首先，$(K_0, K_1, K_2, K_3) = (MK_0 \oplus FK_0, MK_1 \oplus FK_1, MK_2 \oplus FK_2, MK_3 \oplus FK_3)$，其中，$FK_i$ 为系统参数，如表5.13所示。

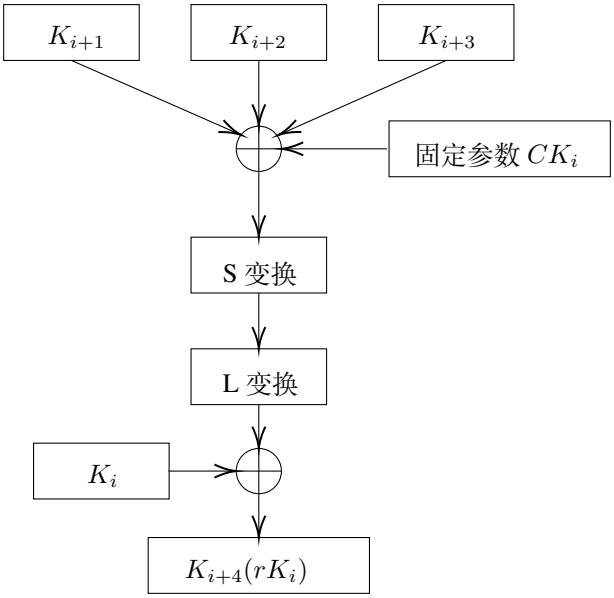


图 5.20: SMS4 算法轮密钥的生成

表 5.13: SMS4 的系统参数

FK_0	A3B1BAC6
FK_1	56AA3350
FK_2	677D9197
FK_3	B27022DC

然后，对 $i = 0, 1, 2, \ldots, 31$，$rK_i = K_{i+4} = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$，其中 T' 变换与加密算法中的迭代变换基本相同，只是线性变换 L 变为：$L(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$，如图5.21所示，其中 CK_i 为固定参数，$(i = 0, 1, 2, \ldots, 31)$，如表5.14所示。

5.6 分组密码的工作模式

分组密码的工作模式就是以该分组密码算法为基础构造的各种密码系统。分组密码常用的工作模式有 5 种：电子密码本模式 ECB、密码分组链接模式 CBC、密码反馈模式 CFB、输出反馈模式 OFB 和记数模式 CTR。这五种分组密码模式适用于所有的分组密码，包括 DES、AES 和 IDEA 等。

假设密钥为 k，明文为

$$M = m_1 m_2 m_3 \ldots m_n,$$

表 5.14: 固定参数 CK_i

00070e15	1c232a31	383f464d	545b6269
70777e85	8c939aa1	a8afb6bd	c4cbd2d9
e0e7eef5	fc030a11	181f262d	343b4249
50575e65	6c737a81	888f969d	a4abb2b9
c0c7ced5	dce3eaf1	f8ff060d	141b2229
30373e45	4c535a61	686f767d	848b9299
a0a7aeb5	bcc3cad1	d8dfe6ed	f4fb0209
10171e25	2c333a41	484f565d	646b7279

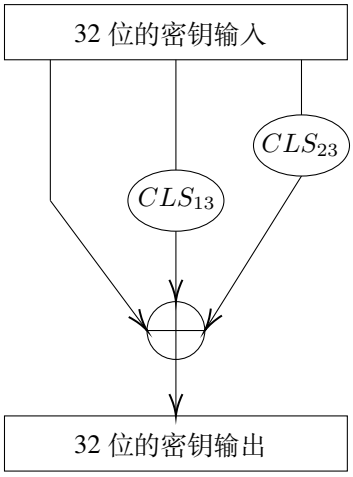


图 5.21: 密钥的 L 线性变换

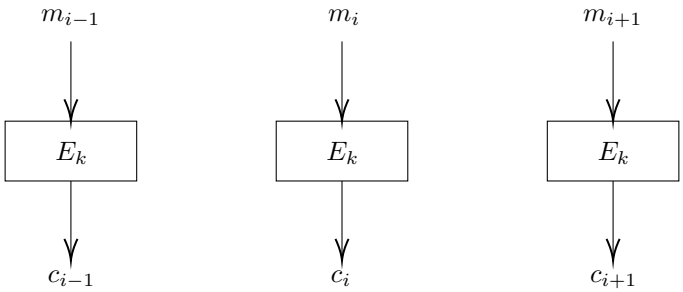


图 5.22: 电子密码本 ECB 模式的加密

密文为：

$$C = c_1 c_2 c_3 \dots c_n,$$

其中 m_i 和 c_i 为 0、1 序列，m_i 和 c_i 的分组长度为 DES，AES，IDEA 等任一分组密码的分组长度。

5.6.1 电子密码本模式（ECB）

在 ECB 模式下，直接用分组密码算法来进行消息的加密和解密。一个明文分组被加密成一个密文分组，相同的明文分组加密成相同的密文分组。如图5.22、5.23所示，电子密码本模式的加密和解密原理。

ECB 模式的算法可表示为：

$$\text{加密：}\ c_i = E_k(m_i),\ i = 1, 2, \dots, n;$$
$$\text{解密：}\ m_i = D_k(c_i),\ i = 1, 2, \dots, n.$$

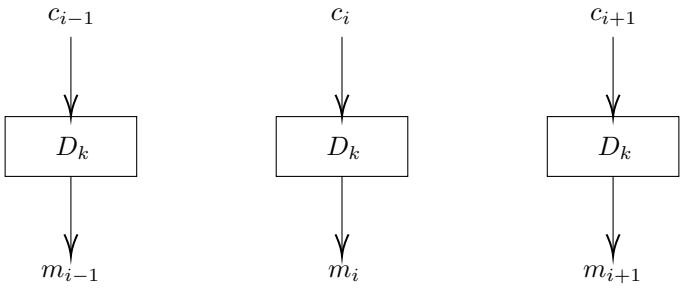


图 5.23: 电子密码本 ECB 模式的解密

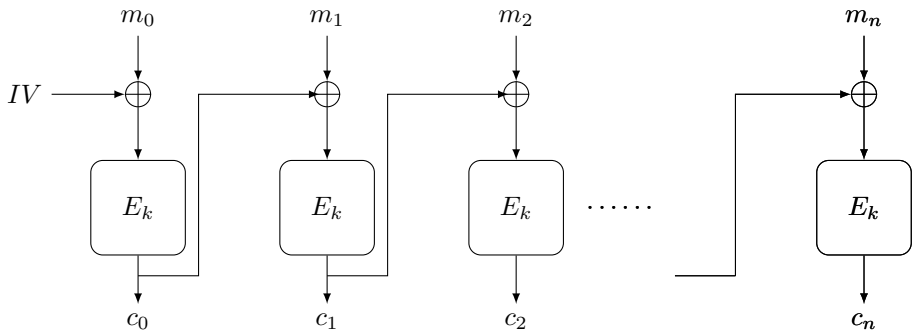


图 5.24: 密码分组链接 CBC 模式的加密

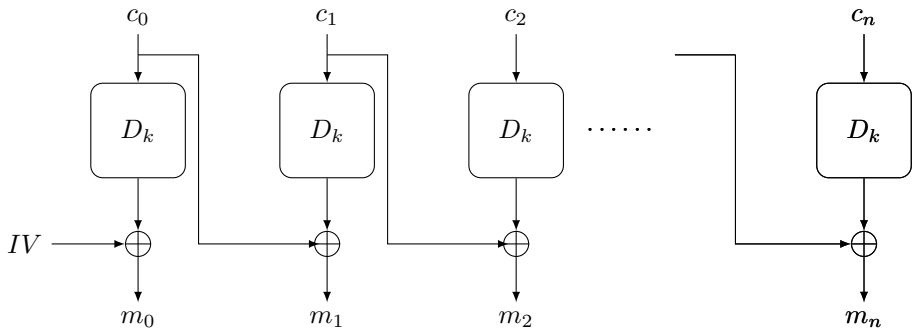


图 5.25: 密码分组链接 CBC 模式的解密

ECB 模式的优点是可以并行运算，速度快，易于标准化。缺点是分组加密不能隐蔽数据模式，即相同的明文蕴含着相同的密文组；不能抵抗分组重放、插入、删除等攻击。

5.6.2 密码分组链接模式（CBC）

CBC 模式的加密和解密如图5.24和5.25所示。在该模式下，每个明文块 m_i 首先与前一个密文块 c_{i-1} 作异或操作，然后再用密钥 k 进行加密。每一分组的加密都依赖于前面所有的分组。在处理第一个分组时要与一个初始向量（IV）进行异或运算。IV 不需要保密，它可以明文的形式与密文一起传送。CBC 模式的算法可表示如下：

$$\text{加密：}\ c_1 = E_k\left(m_1 \oplus IV\right),\ c_i = E_k\left(m_i \oplus c_{i-1}\right),\ i = 2,3,4,\ldots,n;$$
$$\text{解密：}\ m_1 = D_k\left(c_1\right) \oplus IV,\ m_i = D_k\left(c_i\right) \oplus c_{i-1},\ i = 2,3,4,\ldots,n.$$

CBC 模式的优点：引入了随机的初始向量，如果加密算法 E 是伪随机的，则输出具有一定的随机性，避免了 ECB 的缺点，隐蔽了明文的数据模式，在一定程度上等防止数据篡改。

CBC 模式的缺点：如果明文分组中的一位出错，将影响该分组的密文及其以后的所有密文分组。如果密文分组中的一位出现错误，将会影响到错误分组后的第二个分组的明文，其它的分组不受影响；但是如果密文序列中丢失 1 位，那么所有后续分组要移动 1 位，并且解密将全部错误。而且加密的消息的长度只能是分组长度的倍数，不是任意长度的消息。

5.6.3 密码反馈模式（CFB）

如图5.26是密码反馈模式示意图（以 DES 为例），设传送的每个单元是 j 比特长，通常取 $j = 8$，与 CBC 模式相同，明文单元被链接在一起，使得密文是前面所有明文的函数。

加密时，加密算法的输入是 64 比特的移位寄存器，其初值为某个初始初始向量 IV。加密算法输出

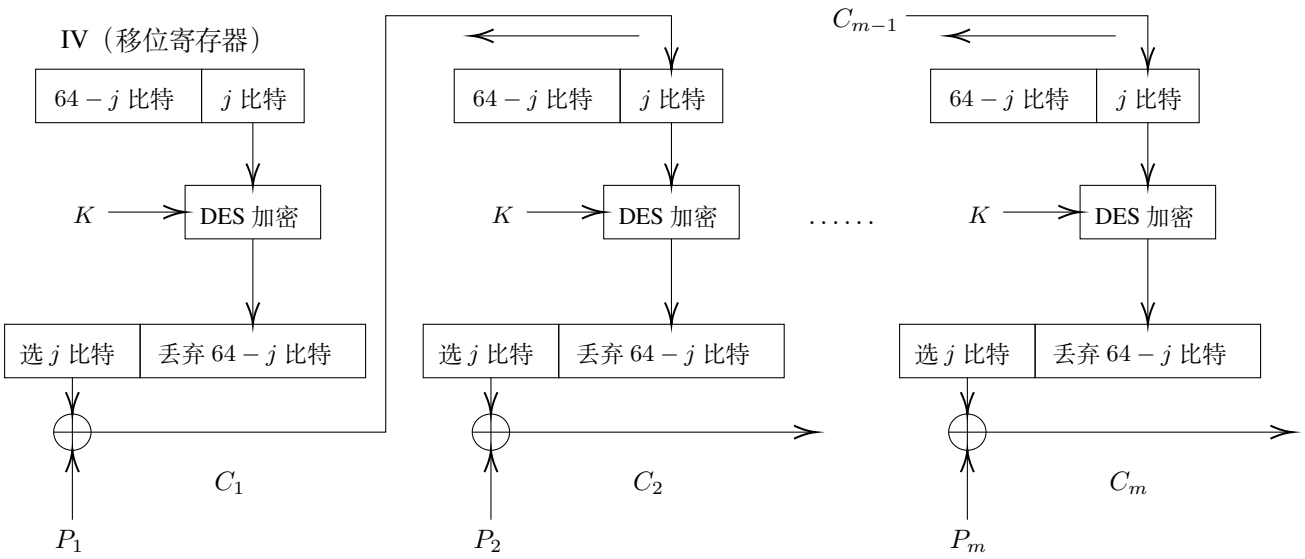


图 5.26: 密码反馈 CFB 模式的加密

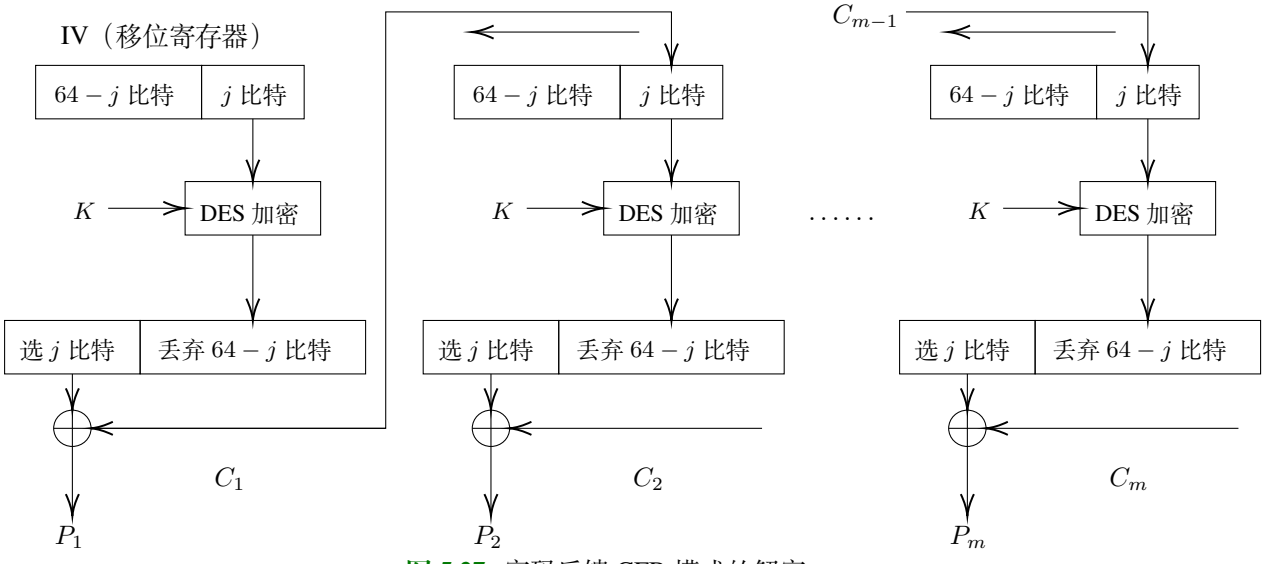


图 5.27: 密码反馈 CFB 模式的解密

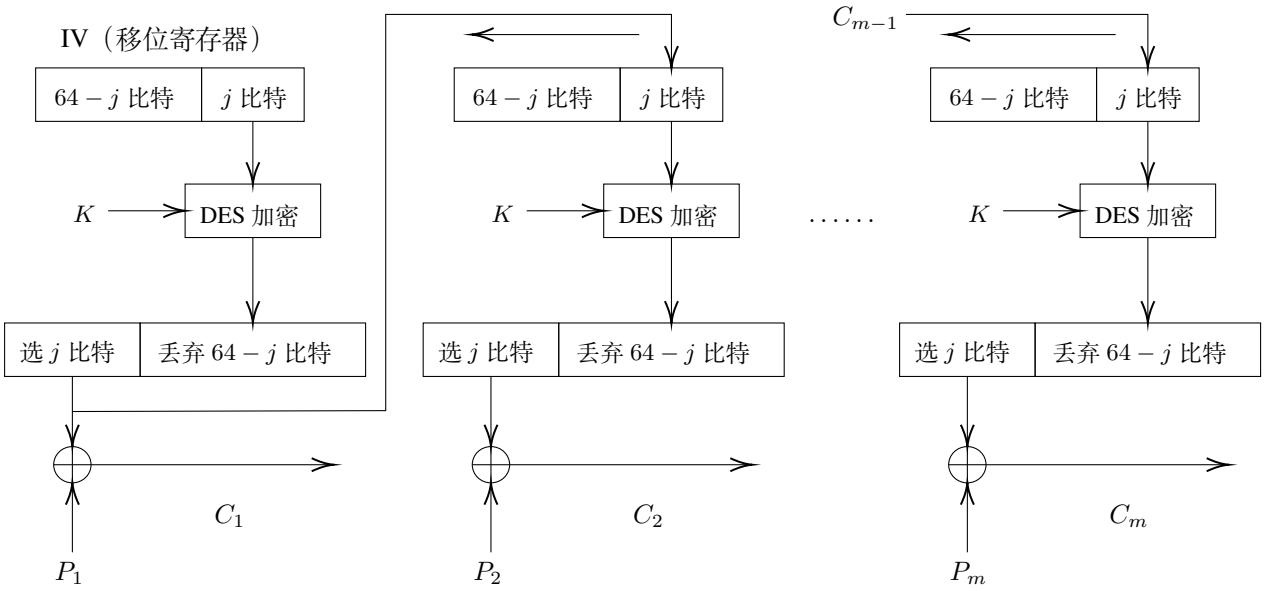


图 5.28: 输出反馈 OFB 模式的加密

的最左边的 j 比特与明文的第一个单元 P_1 进行异或运算，产生出密文的第 1 个单元 C_1，并传送该单元。然后将移位寄存器的内容左移 j 位，并将 C_1 送入移位寄存器的最右边 j 位（最低有效位）。这一过程继续到明文的所有单元都被加密。

如图5.27解密时，将收到的密文单元与加密函数的输出进行异或。这时应注意使用的仍然是加密算法，而不是解密算法。

CFB 模式的优点：自同步能力强，可以处理任意长度的消息。缺点是：明文的 1 个错误就会影响到所有后面的密文，但密文的 1 位错误只会引起明文分组的一个错误。

5.6.4 输出反馈模式（OFB）

如图5.28是输出反馈模式示意图（以 DES 为例），输出反馈模式的结构与 CFB 的接受类似，不同之处在于：OFB 模式将加密算法的输出反馈到移位寄存器，而 CFB 模式中是将加密后的密文单元反馈到移位寄存器。同样设传送的每个单元是 j 比特长，通常取 $j = 8$。

加密时，加密算法的输入是 64 比特的移位寄存器，其初值为某个初始初始向量 IV。加密算法输出的最左边的 j 比特与明文的第一个单元 P_1 进行异或运算，产生出密文的第 1 个单元 C_1，并传送该单元；同时将加密算法输出的最左边的 j 比特移位至移位寄存器的最右边 j 位。这一过程继续到明文的所有单元都被加密。

如图5.29解密时，将收到的密文单元与加密函数的输出进行异或。同样应注意使用的是加密算法，而不是解密算法。

OFB 模式的主要优点是：错误传播小，当前明文分组的错误不会影响后继的密文分组；且密文中的 1 比特错误只导致明文中的 1 个错误；消息长度是任意的。

缺点是：通信的双方必须同步，否则难于解密。

5.6.5 记数器模式（CTR）

CTR 模式使用与明文分组规模相同的计数器长度，但要求加密不同的分组所用的计数器值必须不同。典型地，计数器从某一初值开始，依次递增 1。计数器值经加密函数变换的结果再与明文分组异或，从而

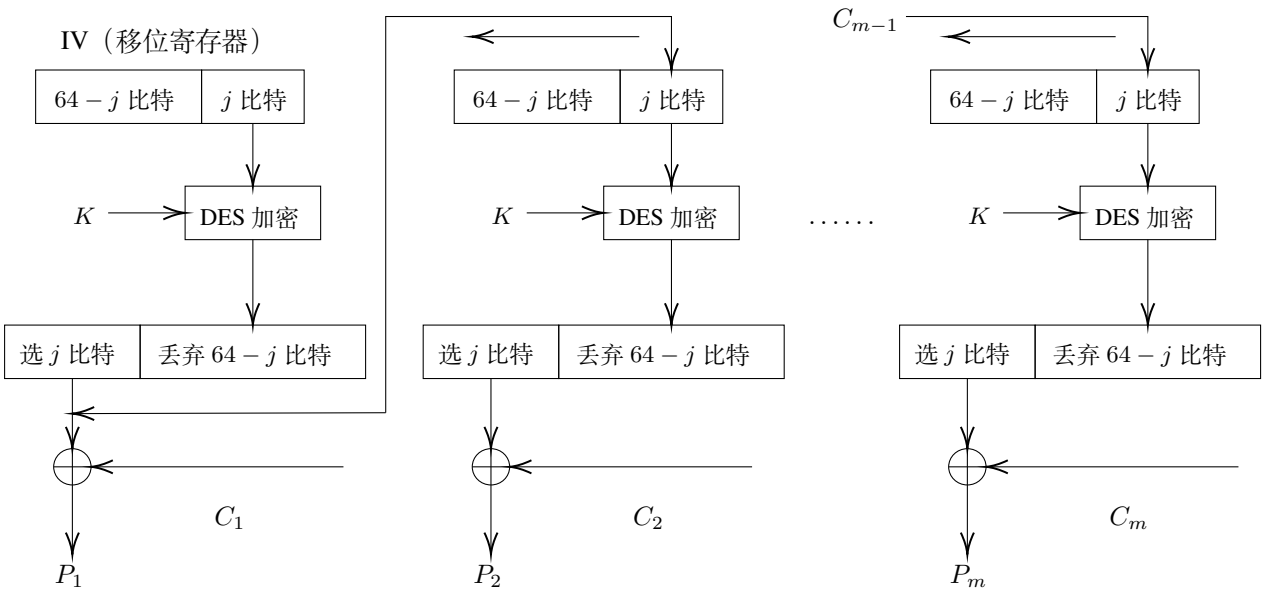


图 5.29: 输出反馈 OFB 模式的解密

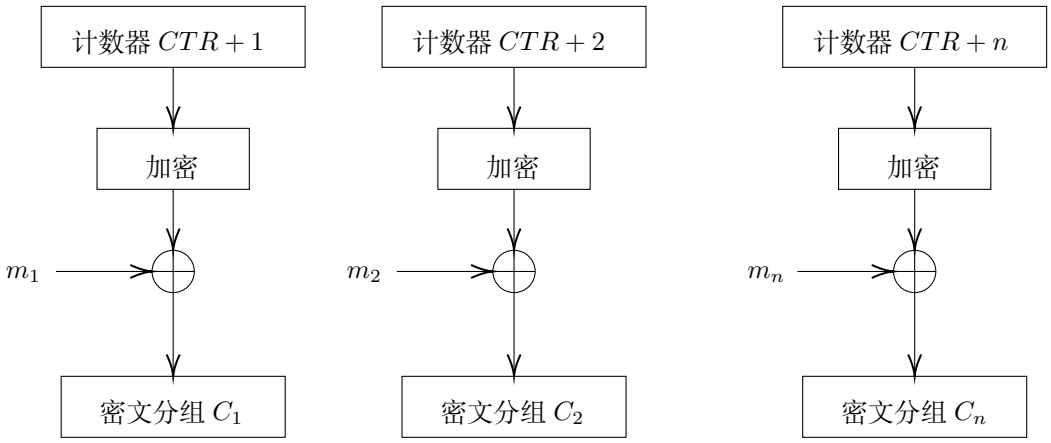


图 5.30: 计数器 CTR 模式的加密

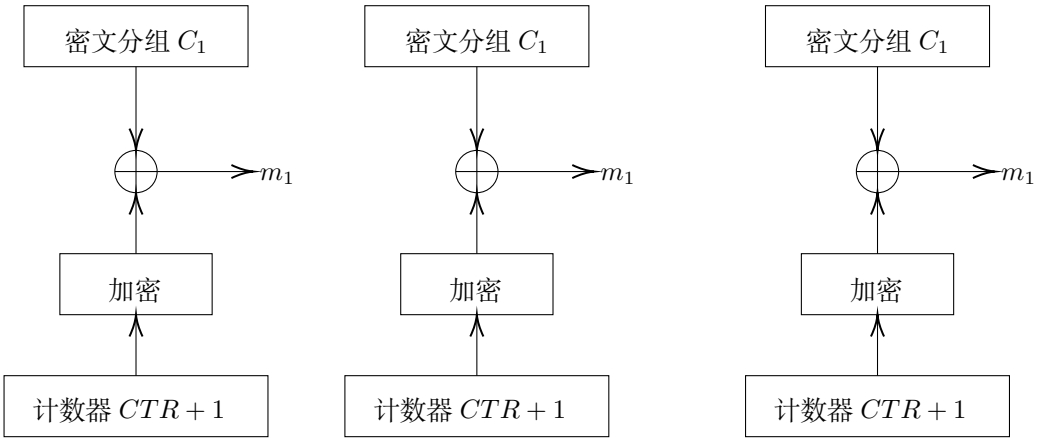


图 5.31: 计数器 CTR 模式的解密

表 5.15: S 置换

input	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
output	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

得到密文。解密时使用相同的计数器值序列，用加密函数变换后的计数器值与密文分组异或，从而恢复明文。如图5.30、5.31所示。

计数器模式的加密和解密算法为：

$$\text{加密：}\ c_i = m_i \oplus E_k(CTR + i),\ i = 1, 2, \ldots, n;$$
$$\text{解密：}\ m_i = c_i \oplus E_k(CTR + i),\ i = 1, 2, \ldots, n.$$

CTR 表示计数器的初值。

CTR 模式的优缺点与 OFB 模式相同，错误传播小，且需要保持同步。

5.7 分组密码分析技术

密码分析和密码设计是相互对立、相互依存的。伴随着对任何一种密码的分析，分析者千方百计从该密码算法中寻找漏洞和缺陷，进而进行攻击。对现代分组密码的分析更是如此，如 DES 自从诞生以来，对它的分析工作就没停止过。归纳起来，分组密码的分析方法主要有以下几种类型：

(1) 穷举攻击；
(2) 线性分析方法；
(3) 差分分析方法；
(4) 相关密钥密码分析方法；
(5) 中间相遇攻击。

在本节中我们典型的以一个 SPN 加密网络为基础，介绍两种典型的分析方法：线性分析方法和差分分析方法。

5.7.1 代换-置换网络

在本小节将介绍密码设计和分析中的基本的代换-置换（SPN）加密网络，如图5.32所示。图5.32所示加密算法，算法的输入为 16 比特的数据块，并且重复四次相同的操作处理数据块。每一轮包括（1）S-box 置换（2）比特变换（3）密钥混合。这个基本结构 Feistel 网络，最初在 1973 年提出，且其中的操作与 DES 中的操作相似，也与其它的像 Rijndael 算法等现代加密算法相似。虽然如此，我们引入这样一个稍微有点简单的结构，通过对这样一个加密算法的分析来分析更复杂密码算法的安全性。

(1) S-box 置换

在给出的加密算法中，将 16 比特的数据块分成 4 个 4 比特的子块。每个子块形成一个 4×4 的 S-box（一种置换装置，输入 4 比特输出也为 4 比特）的输入。S-box 可以很容易的用一个由输入的 4 个比特进行编号的、包含 16 个 4 比特数据的查找表格实现（类似 DES 的 S 盒查找表）。S-box 的最基本的性质是其非线性映射，S-box 的输出不能通过对输入的线性变换而得到。

在如图5.32加密算法中，对所有的 S-box 可以使用相同的非线性映射，也可是使用不同的映射（DES 每一轮中的 S-box 是不相同，但每一轮重复利用这些 S-box）。是否所有的 S-box 都使用不同的非线性映射还是只使用一个映射，对线性密码分析和差分密码分析来说是等价的。我们在密码分析中使用的映射如表5.15所示，是从 DES 的 S-box 中选取的 (是第一个 S-box 的第一行) 。在该表中，这些十六进制的符号表示图5.32的 S-box 的输入与输出。

(2) P 置换

表 5.16: P 置换

input	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
output	1	5	9	13	2	6	10	14	3	7	11	15	4	8	12	16

每一轮的 P 置换只是简单将比特进行替换或者变换比特的位置。图5.32中的 P 变换如表5.16所示（其中的数字表示比特的位置，1 表示最左边的比特，16 表示最右边的比特）。可以简单的描述 P 置换如下：第 j 个 S-box 的第 i 个输出与第 i 个 S-box 的第 j 个输入相关联。由于在加密的最后一轮使用 P 置换是没有用处的，因此我们的加密算中在最后一轮没有使用 P 置换。

(3) 密钥混合

密钥混合是将每一轮比特输入和子密钥进行简单的 OR 运算。最后一轮也需要一个子密钥进行密钥混合运算，如果没有子密钥混合运算，则分析者可以对最后一轮的 S-box 做逆运算，从而忽略最后一轮的加密作用。一般地，一个密码算法中，每一轮的子密钥都是通过加密主密钥变换产生的。在我们的加密算法中，假设所有的子密钥比特都是独立产生的而且没有相互联系。

(4) 解密

解密时，数据以实际上以与加密相反的方向通过 SPN 加密网络。因此，解密的形式和图5.32所示的加密网络的形式相似。但是，解密过程中 S-box 的解密映射应该是加密过程中 S-box 的加密映射的逆映射（例如：输入变成输出，输出变成输入）。也就是说，为了使得一个 SPN 网络既可用于加密又可用于解密，所有的 S-box 必须都是双射的，且对于相同的输入和输出是一个一对一的映射。同时，为了保证加密网络可以正确的解密，子密钥必须以与加密相反的次序使用，并且子密钥的使用必须与 P 置换一致，如果与图5.32中的代换-置换网络相似，必须注意到在最后一轮没有 P 置换，以保证加密和解密的 SPN 网络在结构上的一致（如果在最后一轮有一个 P 置换，则在解密的第一轮的 S 置换之前需要一个 P 置换）。

5.7.2 线性密码分析

在这一部分中，我们以前面的 SPN 网络为基础，介绍利用线性密码分析的方法攻击加密算法的方法。

5.7.2.1 基本的分析概述

线性分析的分析者利用了包含明文、密文和子密钥的线性表达式发生的较大可能性（在该例子中，我们将利用倒数第 2 轮的输出作为密文）。线性分析一种已知明文攻击：假设攻击者已经知道了一系列的明文和相对应的密文。在许多情况下，假设攻击者知道一系列随机的明文和其相应的密文是合理的。

线性分析最基本的思想就是用一个线性表达式来近似表示加密算法的一部分，该线性表达式是关于模 2 的操作（例如用 $\oplus$ 表示 XOR 操作）。表达式具有如下的形式：

$$X_{i_1} \oplus X_{i_2} \oplus \cdots \oplus X_{i_u} \oplus Y_{j_1} \oplus Y_{j_2} \oplus \cdots \oplus Y_{j_v} = 0, \tag{5.8}$$

其中：X_i 表示输入 $X = [X_1, X_2, \ldots]$ 的第 i 个比特，Y_j 表示输出 $Y = [Y_1, Y_2, \ldots]$ 的第 j 个比特。该表达式是 u 比特的输入与 v 比特的输出进行 XOR 操作的结果。

线性密码分析的方法就是测定上述形式的线性表达式发生的可能性的大小。如果一个密码算法使得等式(5.8)成立的可能性非常大或者非常小，这说明该密码算法的随机性比较差。一般情况下，假如我们随即选择 $u+v$ 个比特值，并且将它们代入等式(5.8)，等式成立的可能性应该为 $1/2$。在线性密码分析中，一个线性表达式成立的可能性与 $1/2$ 之间的差值定义为偏移量（或偏差）。一个线性表达式成立的可能性与 $1/2$ 的距离越大，密码分析者利用线性密码分析就越有效果。在下面的叙述中，我们将一个表达式成立的可能性与 $1/2$ 的差值称为线性可能性偏移量，简称偏移量或偏差。因此，如果对于随机选择的明文

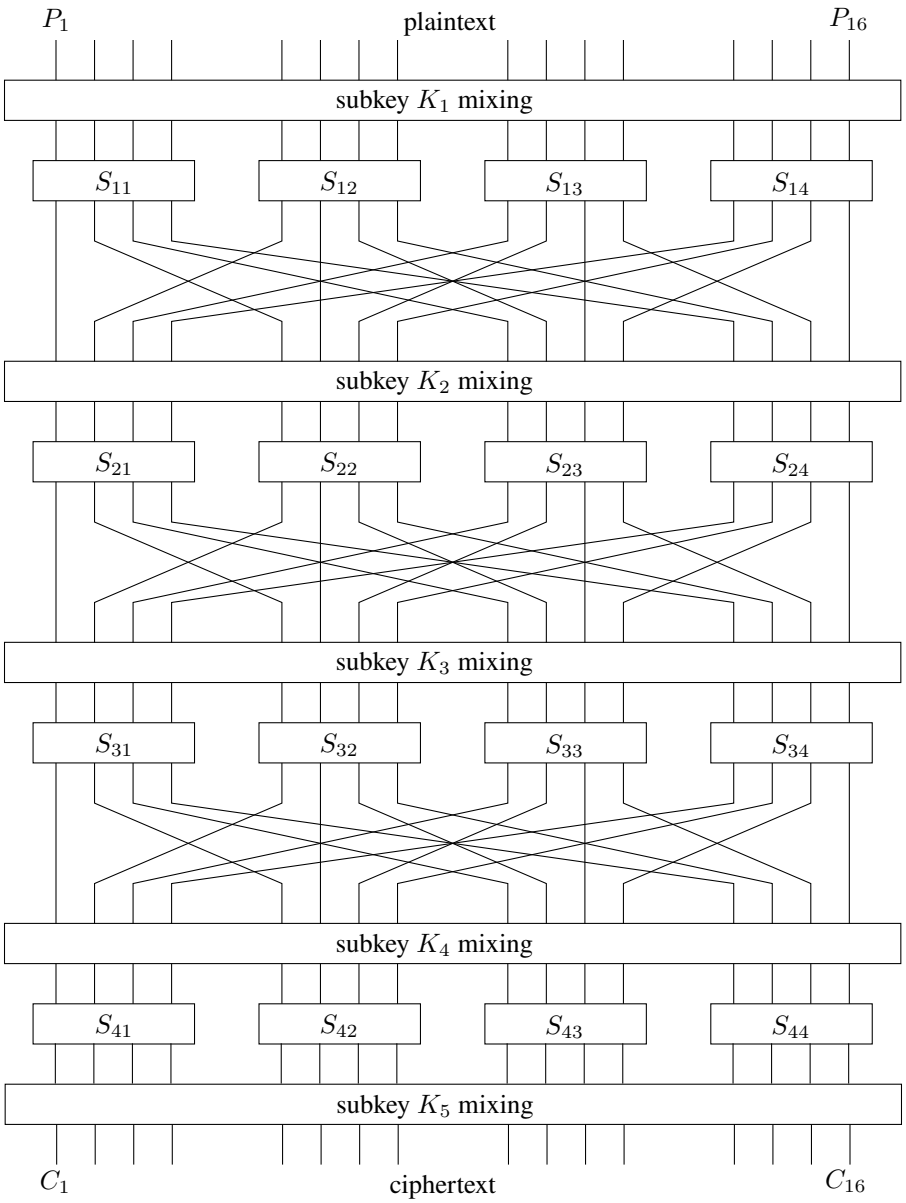


图 5.32: 基本的 SPN 加密网络

找到相应的密文，使得上述表达式成立的可能性为 P_L，则线性可能性偏移量是 $P_L - 1/2$。线性可能性偏移量的绝对值 $|P_L - 1/2|$ 越大，线性密码分析对于知道较少明文情况下的攻击越有效。

有多种方法来设计线性分析攻击。我们将利用 Matsui 的模 2 运算法则。这样安排线性近似表达式的各个部分：将明文用表达式(5.8)中的 X 表示，加密算法最后一轮的输入作为表达式(5.8)中的 Y（也就是倒数第 2 轮的输出）。这样明文比特和最后一轮的输入都是随机的。

可以将子密钥的和作为表达式的右边，对表达式进行改写。当等式(5.8)的右边为 0 时，等式表示含有这样的子密钥：这些密钥比特是固定的、未知的（因为它们由攻击所希望得到的密钥决定），满足表达式的右侧为零，并且线性表达式成立的可能性为 P_L。如果涉及到的子密钥的和为 0，表达式(5.8)的偏移量的符号与改写后的包含子密钥的和表达式的偏移量的符号相同；如果涉及到的子密钥的和为 1，则偏移量的符号相反。

应该注意：当 $P_L = 1$ 表示线性表达式(5.8)具有较好的加密性能同时具有致命的弱点。如果 $P_L = 0$，则表达式(5.8)表示在加密算法中含有仿射关系，同时也表明该加密算有灾难性的弱点。对于一个模 2 加法系统，一个仿射函数仅仅就是一个线性函数的补充。对于线性和仿射近似，无论 $P_L > 1/2$ 还是 $P_L < 1/2$，同样都会受线性密码分析的影响，当我们提及线性时，通常既指线性关系也指仿射关系，对指线性关系与仿射关系不加以区分。

那么接下来的问题就是：如何构造一个具有高线性性质的表达式，又如何使用？这仅需要考虑加密算法中的非线性的元素 S-box 的性质。找出 S-box 的非线性的特性，就有可能找到 S-box 的输入与输出之间的近似线性表达式。因此，连接所有 S-box 的近似线性表达式，就可以省略加密过程的中间数据，这样我们就得到了一个具有较高偏移量且仅仅包含明文和最后一轮输入的加密算法的线性近似表达式。

5.7.2.2 堆积引理

在考虑构造所给加密算法例子的近似线性表达式之前，需要介绍几个基本的理论工具。考虑两个二进制变量 X_1，X_2。我们通过计算简单的关系开始：$X_1 \oplus X_2 = 0$ 是一个线性表达式，等价与 $X_1 = X_2$；$X_1 \oplus X_2 = 1$ 是一个仿射表达式，等价与 $X_1 \neq X_2$。

假设，给出如下的概率分布：

$$\Pr(X_1 = i) = \begin{cases} p_1, & i = 0; \\ 1 - p_1, & i = 1. \end{cases}$$

$$\Pr(X_2 = i) = \begin{cases} p_2, & i = 0; \\ 1 - p_2, & i = 1. \end{cases}$$

如果两个随机变量相互独立，则

$$\Pr(X_1 = \mathrm{i}, X_2 = \mathrm{j}) = \begin{cases} p_1 p_2, & i = 0,\ j = 0; \\ p_1(1 - p_2), & i = 0,\ j = 1; \\ (1 - p_1) p_2, & i = 1,\ j = 0; \\ (1 - p_1)(1 - p_2), & i = 1,\ j = 1. \end{cases}$$

而且可以等价表示为：

$$\begin{aligned} \Pr(X_1 \oplus X_2 = 0) &= \Pr(X_1 = X_2) \\ &= \Pr(X_1 = 0, X_2 = 0) + \Pr(X_1 = 1, X_2 = 1) \\ &= p_1 p_2 + (1 - p_1)(1 - p_2). \end{aligned}$$

另一种表示方法是：令 $p_1 = 1/2 + \varepsilon_1$，$p_2 = 1/2 + \varepsilon_2$，$\varepsilon_1$、$\varepsilon_2$ 是线性可能性偏移量，而且 $-1/2 \leq$

$\varepsilon_1, \varepsilon_2 \leq 1/2$。因此，

$$\Pr\left(X_1 \oplus X_2 = 0\right) = \frac{1}{2} + 2\varepsilon_1\varepsilon_2.$$

并且 $X_1 \oplus X_2 = 0$ 的线性可能性偏移量 $\varepsilon_{1,2}$ 为 $\varepsilon_{1,2} = 1/2 + 2\varepsilon_1\varepsilon_2 - 1/2 = 2\varepsilon_1\varepsilon_2$。

这可以扩展到多个随机二进制变量的情况，若有随机变量 X_1，...，X_n 且概率为 $p_1 = 1/2 + \varepsilon_1$，...，$p_n = 1/2 + \varepsilon_n$。假设 n 个随机相互独立，$X_1 \oplus \cdots \oplus X_n = 0$ 成立的概率可以由 Piling-Up 引理（堆积引理）得到。Piling-Up 引理假定几个随即变量相互独立。

定理 5.1 (Piling-Up 引理)

对于 n 个相互独立的二进制随机变量，X_1，X_2，...，X_n，可得

$$\Pr\left(X_1 \oplus \cdots \oplus X_n = 0\right) = \frac{1}{2} + 2^{n-1}\prod_{i=1}^{n}\varepsilon_i.$$

或者等价表示为 $\varepsilon_{1,2,\ldots,n} = 2^{n-1}\prod_{i=1}^{n}\varepsilon_i$，其中 $\varepsilon_{1,2,\ldots,n}$ 表示 $X_1 \oplus \cdots \oplus X_n = 0$ 的线性可能性偏移量。

从 $\Pr(X_1 \oplus \cdots \oplus X_n = 0) = 1/2 + 2^{n-1}\prod_{i=1}^{n}\varepsilon_i$ 可以看出，如果对于所有的 i 都有 $p_i = 0$（或者 $p_i = 1$），则 $\Pr(X_1 \oplus \cdots \oplus X_n = 0)$ 可能为 0 也可能为 1。但是，只要有一个 $p_i = 1/2$，那么 $\Pr(X_1 \oplus \cdots \oplus X_n = 0) = 1/2$。

为得到一个加密算法的线性近似表达式，将随机变量 X_i 作为 S-box 的线性近似的输入。例如，考虑有四个相互独立的随机变量 X_1，X_2，X_3 和 X_4。令 $\Pr(X_1 \oplus X_2 = 0) = 1/2 + \varepsilon_{1,2}$ 且 $\Pr(X_2 \oplus X_3 = 0) = 1/2 + \varepsilon_{2,3}$，而且 $X_1 \oplus X_3$ 可以由 $X_1 \oplus X_2$，$X_2 \oplus X_3$ 相加得到。因此，可得，

$$\Pr\left(X_1 \oplus X_3 = 0\right) = \Pr\left([X_1 \oplus X_2] \oplus [X_2 \oplus X_3] = 0\right).$$

所以我们可以通过连接多个线性表达式构成新的线性表达式。因为我们可以认为随机变量 $X_1 \oplus X_2$，$X_2 \oplus X_3$ 也是相互独立的，应用 Piling-Up 引理可得：

$$\Pr\left(X_1 \oplus X_3 = 0\right) = \frac{1}{2} + 2\varepsilon_{1,2}\varepsilon_{2,3}.$$

相应的 $\varepsilon_{1,3} = 2\varepsilon_{1,2}\varepsilon_{2,3}$。

正如我们将要看到的，表达式 $X_1 \oplus X_2 = 0$，$X_2 \oplus X_3 = 0$ 和 S-box 的线性近似表达式相似，表达式 $X_1 \oplus X_3 = 0$ 和去掉比特 X_2 的表达式相似。当然实际的线性分析要更复杂，涉及更多的 S-box 线性近似表达式。

5.7.2.3 分析加密部件

在更详细的讨论攻击细节之前，我们首先需要知道 S-box 盒的线性脆弱性。如图5.33所示的 S-box 的输入为 $X = [X_1X_2X_3X_4]$，相应的输出为 $Y = [Y_1Y_2Y_3Y_4]$。所有的线性表达式都可以通过计算可能性偏移量来检测该表达式是否有用。因此，将 X，Y 作为 S-box 的输出，我们检测所有具有表达式(5.8)形式的表达式。

例如，在上面的 S-box 中，考虑表达式 $X_2 \oplus X_3 \oplus Y_1 \oplus Y_3 \oplus Y_4 = 0$ 或等价形式 $X_2 \oplus X_3 = Y_1 \oplus Y_3 \oplus Y_4$。可知，对于 16 种可能的输入和其相应的输出 Y，有 12 种情况可以使得上面的等式成立，因此线性可能性偏移量是 $12/16 - 1/2 = 1/4$。如表5.17所示。相似的，对于等式 $X_1 \oplus X_4 = Y_2$ 其线性可能性偏移量接近于 0，而等式 $X_3 \oplus X_4 = Y_1 \oplus Y_4$ 的线性可能性偏移量是 $2/16 - 1/2 = -3/8$。对于最后一种情况，通过负号可知，其最佳的近似表达式是一个仿射近似表达式。但是，攻击成功的大小取决于线性可能性偏移量的大小，正像我们将要看到的一样，仿射近似表达式与线性近似表达式的效果相同。

表5.18给出了我们使用的加密算法的 S-box 的所有线性近似表达式的线性可能性偏移量。其中，Input

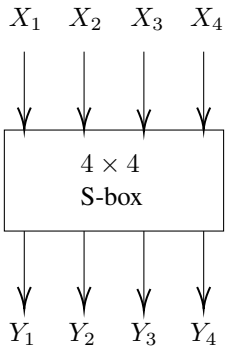


图 5.33: S-box 映射

表 5.17: S-box 线性近似采样

X1	X2	X3	X4	Y1	Y2	Y3	Y4	X2⊕X3	Y1⊕Y3⊕Y4	X1⊕X4	Y2	X3⊕X4	Y1⊕Y4
0	0	0	0	1	1	1	0	0	0	0	1	0	1
0	0	0	1	0	1	0	0	0	0	1	1	1	0
0	0	1	0	1	1	0	1	1	0	0	1	1	0
0	0	1	1	0	0	0	1	1	1	1	0	0	1
0	1	0	0	0	0	1	0	1	1	0	0	0	0
0	1	0	1	1	1	1	1	1	1	1	1	1	0
0	1	1	0	1	0	1	1	0	1	0	0	1	0
0	1	1	1	1	0	0	0	0	1	1	0	0	1
1	0	0	0	0	0	1	1	0	0	1	0	0	1
1	0	0	1	1	0	1	0	0	0	0	0	1	1
1	0	1	0	0	1	1	0	1	1	1	1	1	0
1	0	1	1	1	1	0	0	1	1	0	1	0	1
1	1	0	0	0	1	0	1	1	1	1	1	0	1
1	1	0	1	1	0	0	1	1	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1	0	1	0
1	1	1	1	0	1	1	1	0	0	0	1	0	1

表 5.18: 线性近似偏移量

		output														
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
	0	+8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	0	-2	-2	0	0	-2	+6	+2	+2	0	0	+2	+2	0	0
	2	0	0	-2	-2	0	0	-2	-2	0	0	+2	+2	0	0	-6	+2
I	3	0	0	0	0	0	0	0	0	+2	-6	-2	-2	+2	+2	-2	-2
	4	0	+2	0	-2	-2	-4	-2	0	0	-2	0	+2	+2	-4	+2	0
n	5	0	-2	-2	0	-2	0	+4	+2	-2	0	-4	+2	0	-2	-2	0
	6	0	+2	-2	+4	+2	0	0	+2	0	-2	+2	+4	-2	0	0	-2
p	7	0	-2	0	+2	+2	-4	+2	0	-2	0	+2	0	+4	+2	0	+2
	8	0	0	0	0	0	0	0	0	-2	+2	+2	-2	+2	-2	-2	-6
u	9	0	0	-2	-2	0	0	-2	-2	-4	0	-2	+2	0	+4	+2	-2
	A	0	+4	-2	+4	-4	0	+2	-2	+2	+2	0	0	+2	+2	0	0
t	B	0	+4	0	-4	+4	0	+4	0	0	0	0	0	0	0	0	0
	C	0	-2	+4	-2	-2	0	+2	0	+2	0	+2	+4	0	+2	0	-2
	D	0	+2	+2	0	-2	+4	0	+2	-4	-2	+2	0	+2	0	0	+2
	E	0	+2	+2	0	-2	-4	0	+2	-2	0	0	-2	-4	+2	-2	0
	F	0	-2	-4	-2	-2	0	+2	0	0	-2	+4	-2	-2	0	+2	0

表示表达式的输入系数，而 Output 表示表达式的输出系数，行和列交集处的值表示以此行列值代表线性表达式成立的数量减去 8。因此，一个元素被 16 整除得到由特定输入和输出组合而成的表达式的线性可能性偏移量。将行和列的 16 进值看作二进制的组合时，其表示表达式中所涉及的变量。例如一个输入变量的线性近似表达式 $a_1 \cdot X_1 \oplus a_2 \cdot X_2 \oplus a_3 \cdot X_3 \oplus a_4 \cdot X_4$，其中，$a_i \in \{0,1\}$。“$\cdot$”为二进制的“与”运算，输入行的 16 进制的值是 $a_1a_2a_3a_4$ 的组合。相似的，对于一个输出变量的线性近似表达式 $b_1 \cdot Y_1 \oplus b_2 \cdot Y_2 \oplus b_3 \cdot Y_3 \oplus b_4 \cdot Y_4$，其中 $b_i \in \{0,1\}$，输出行的 16 进制的值是 $b_1b_2b_3b_4$ 的组合。因此，线性表达式 $X_3 \oplus X_4 = Y_1 \oplus Y_4$（输入行标是 3，输出列标是 9）的线性可能性偏移量是 $-6/16 = -3/8$，且线性表达式成立的概率是 $1/2 - 3/8 = 1/8$。

线性近似表达式有着许多明显的特性。例如，任何非空输出组合等于没有比特输入组合的概率是 $1/2$，因为对于一个双射的 S-box 来说，任何线性组合的输出比特的中，0 和 1 的数量必须相等。而且，没有比特输出的线性组合等于没有比特输入的线性组合的线性可能性偏移量是 $1/2$，反映在表中的值是 $+8$，位于表格的左上角。因此，最上面的一行除了最左端的值以外都是 0。相似的，第一列除了最上面的值以外也都是 0。同样可以得到，任何一行或者一列相加的和等于 8 或者 -8。

5.7.2.4 构造加密函数的线性近似表达式

如果一个 SPN 网络中的 S-box 的线性近似信息被找出来，我们可以将数据用形如等式(5.8)关于整个加密算法的近似线性表达式进行运算。可以通过连接 S-box 的适当线性表达式构造整个算法的线性近似表达式。通过构造一个关于明文和倒数第二轮输入的线性表达式，就有可能恢复出最后一轮加密使用的子密钥的一个子集，以达到攻击的目的。下面通过一个例子来说明：

考虑如图5.34所示的关于 S_{12}、S_{22}、S_{32} 和 S_{34} 的线性近似表达式。实际上是建立了一个关于前 3 轮的表达式，而不是整个 4 轮加密过程。在下一小节，我们将会了解到如何通过这些表达式得到最后一轮的子密钥。

首先对于图5.34，有如下的 S-box 线性近似表达式：

$$\begin{aligned}
S_{12}:&\quad X_1 \oplus X_3 \oplus X_4 = Y_2 \quad \text{概率为}\tfrac{12}{16}\text{，偏移量为} +\tfrac{1}{4};\\
S_{22}:&\quad X_2 = Y_2 \oplus Y_4 \qquad\quad \text{概率为}\tfrac{4}{16}\text{，偏移量为} -\tfrac{1}{4};\\
S_{32}:&\quad X_2 = Y_2 \oplus Y_4 \qquad\quad \text{概率为}\tfrac{4}{16}\text{，偏移量为} -\tfrac{1}{4};\\
S_{34}:&\quad X_2 = Y_2 \oplus Y_4 \qquad\quad \text{概率为}\tfrac{4}{16}\text{，偏移量为} -\tfrac{1}{4}。
\end{aligned}$$

令 U_i（V_i）作为第 i 轮 S-box 的 16 比特的输入（输出），并且 U_{ij}（V_{ij}）作为第 U_i 块的第 j 比特（在图中这些比特按照从左到右的顺序，从 1 到 16 顺序标记）。类似地，令 K_i 表示与第 i 轮输出进行 XOR 运算的子密钥。可知，K_5 表示与第四轮的输出进行 XOR 运算的子密钥。

因此，$U_1 = P \oplus K_1$，其中 P 表示 16 比特的明文，$\oplus$ 表示比特之间的 XOR 运算。利用第一轮的线性近似表达式，我们可得

$$V_{1,6} = U_{1,5} \oplus U_{1,7} \oplus U_{1,8} \quad = (P_5 \oplus K_{1,5}) \oplus (P_7 \oplus K_{1,7}) \oplus (P_8 \oplus K_{1,8}), \tag{5.9}$$

其表达式成立的概率为 $3/4$。对于第 2 轮的线性近似表达式，我们有

$$V_{2,6} \oplus V_{2,8} = U_{2,6},$$

表达式成立的可能性为 $1/4$。因为 $U_{2,6} = V_{1,6} \oplus K_{2,6}$，我们可以得到如下形式的线性近似表达式：

$$V_{2,6} \oplus V_{2,8} = V_{1,6} \oplus K_{2,6},$$

表达式成立的可能性为 $1/4$，则联合成立可能性为 $3/4$ 的表达式(5.9)可得到如下表达式：

$$V_{2,6} \oplus V_{2,8} \oplus P_5 \oplus P_7 \oplus P_8 \oplus K_{1,5} \oplus K_{1,7} \oplus K_{1,8} \oplus K_{2,6} = 0. \tag{5.10}$$

由 Piling-Up 引理可得其成立的概率为 $1/2 + 2(3/4 - 1/2)(1/4 - 1/2) = 3/8$（也就是，其线性可能性偏移量为 $-1/8$）。应当注意我们假设 S-box 的所有线性近似表达式都是相互独立的，尽管这样做是不完全正确，但在大多数的密码分析中都工作的很好。

对于第 3 轮，我们得到

$$V_{3,6} \oplus V_{3,8} = U_{3,6},$$

且等式成立的可能性为 $1/4$；

$$V_{3,14} \oplus V_{3,16} = U_{3,14},$$

等式成立的概率为 $1/4$。因此，又由 $U_{3,6} = V_{2,6} \oplus K_{3,6}$，$U_{3,14} = V_{2,8} \oplus K_{3,14}$ 可得：

$$V_{3,6} \oplus V_{3,8} \oplus V_{3,14} \oplus V_{3,16} \oplus V_{2,6} \oplus K_{3,6} \oplus V_{2,8} \oplus K_{3,14} = 0, \tag{5.11}$$

且等式成立的概率为 $12 + 2(1/4 - 1/2)^2 = 5/8$（也就是其线性可能性偏移量为 $+1/8$）。同样应用 Piling-Up 引理。联合表达式(5.10)和(5.11)，构成四个 S-box 的线性近似表达式：

$$V_{3,6} \oplus V_{3,8} \oplus V_{3,14} \oplus V_{3,16} \oplus P_5 \oplus P_7 \oplus P_8 \oplus K_{1,5} \oplus K_{1,7} \oplus K_{1,8} \oplus K_{2,6} \oplus K_{3,6} \oplus K_{3,14} = 0.$$

计算 $U_{4,6} = V_{3,6} \oplus K_{4,6}$，$U_{4,8} = V_{3,16} \oplus K_{4,8}$，$U_{4,14} = V_{3,8} \oplus K_{4,14}$ 和 $U_{4,16} = V_{3,16} \oplus K_{4,16}$，然后我们可以得到

$$U_{4,6} \oplus U_{4,8} \oplus U_{4,14} \oplus U_{4,16} \oplus P_5 \oplus P_7 \oplus P_8 \oplus \sum k = 0.$$

其中，

$$\sum k = K_{1,5} \oplus K_{1,7} \oplus K_{1,8} \oplus \ K_{2,6} \oplus K_{3,6} \oplus K_{3,14} \oplus K_{4,6} \oplus K_{4,8} \oplus K_{4,14} \oplus K_{4,16}.$$

并且 $\sum k$ 是确定的（或者为 0 或者为 1，取决于加密算法的密钥）。利用 Piling-Up 引理，可以得出上述表达式成立的概率为 $1/2 + 2^3(3/4 - 1/2)(1/4 - 1/2)^3 = 15/32$（也就是说，其线性可能性偏移量为 $-1/32$）。

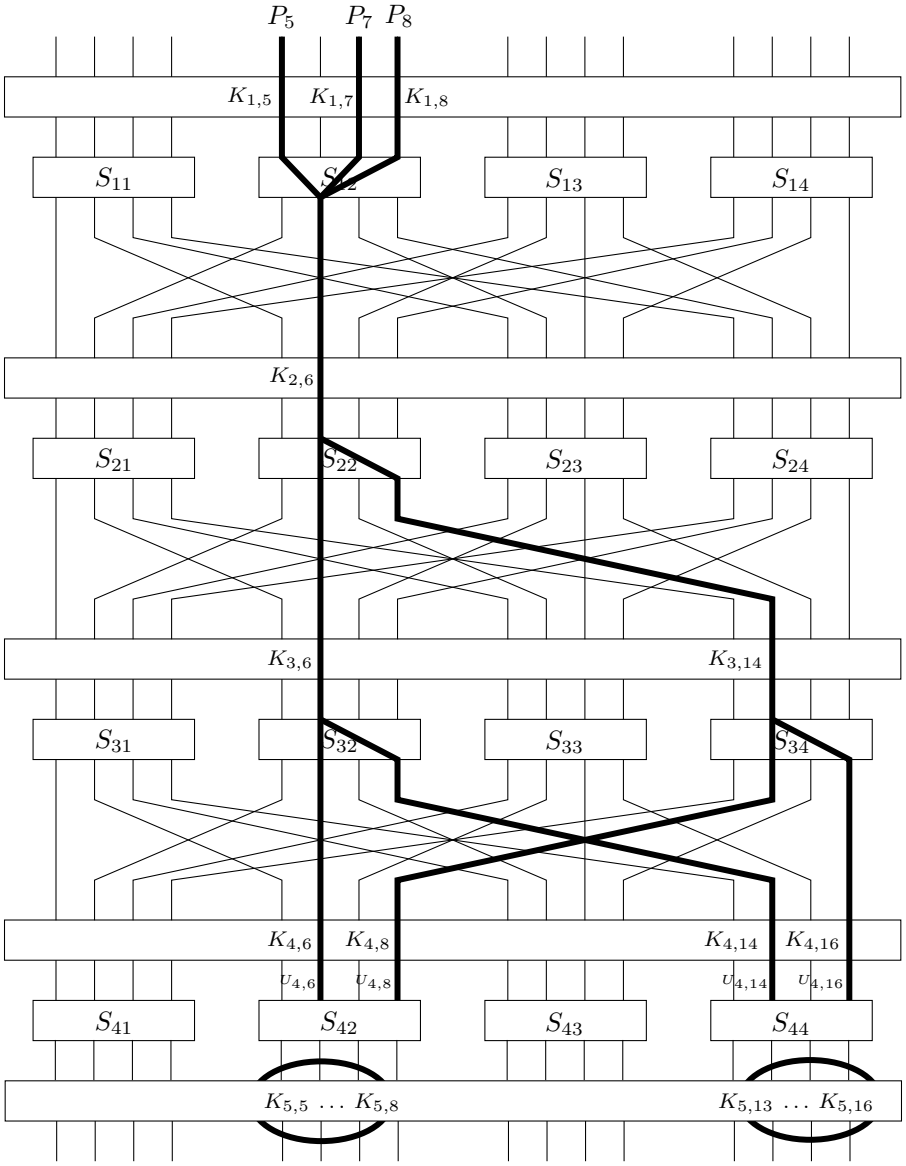

P_5
P_7
P_8
$K_{1,5}$
$K_{1,7}$
$K_{1,8}$
S_{11}
S_{12}
S_{13}
S_{14}
$K_{2,6}$
S_{21}
S_{22}
S_{23}
S_{24}
$K_{3,6}$
$K_{3,14}$
S_{31}
S_{32}
S_{33}
S_{34}
$K_{4,6}$
$K_{4,8}$
$K_{4,14}$
$K_{4,16}$
$U_{4,6}$
$U_{4,8}$
$U_{4,14}$
$U_{4,16}$
S_{41}
S_{42}
S_{43}
S_{44}
$K_{5,5}$ ... $K_{5,8}$
$K_{5,13}$ ... $K_{5,16}$

图 5.34: 线性分析

因为 $\sum k$ 的值是确定的，我们可以得到

$$U_{4,6} \oplus U_{4,8} \oplus U_{4,14} \oplus U_{4,16} \oplus P_5 \oplus P_7 \oplus P_8 = 0. \tag{5.12}$$

成立的概率为 15/32 或者 $(1 - 15/32) = 17/32$（取决于 $\sum k$ 的值是 0 还是 1）。换句话说，我们现在有前三轮加密过程的一个线性近似表达式，其线性偏移量是 1/32。下面将讨论如何利用这样一个偏移量来确定加密子密钥的某些密钥比特。

5.7.2.5 获取密钥比特

一旦获得了有 R 轮加密过程的加密算法的前 $R-1$ 轮的线性近似表达式，且该表达式具有足够大的线性可能性偏移量，则恢复最后一轮的子密钥来攻击该密码算法是可行的。在我们所给的例子中，通过前 3 轮循环的线性近似表达式获得子密钥 K_5 是有可能的。我们把从最后一个子密钥中恢复出的密钥的一部分称为局部目标子密钥（或部分目标子密钥）。局部目标子密钥来自与最后一轮的 S-box 相关联的子密钥。

接下来的步骤包括部分的破解最后一轮的加密。特别地，对于所有可能的局部目标子密钥，相应的密文比特与其进行 XOR 运算，运算的结果向后通过最后一轮相应的 S-box 进行运算。对所有的明文/密文对进行这种运算过程，并且设置一个计数器对每个局部目标子密钥进行计数。若对于输入到最后一轮 S-box 的（密文）比特（由对最后一轮的破解得到）和已知明文，线性近似表达式成立，则计数器增加。如果某个局部目标子密钥对于明文/密文对的计数值与明文/密文对数目的一半相差最大，则该子密钥被假定为正确的子密钥。这样做的原因是一个正确的子密钥将使得线性近似表达式成立的概率远离 1/2（至于大于 1/2 还是小于 1/2，则取决于式线性表达式还是仿射表达式作为近似表达式最合适，且取决于线性表达式中涉及的未知的子密钥比特）。一个不正确的子密钥被认为导致一个向最后一轮 S-box 得随机猜测输入，因而线性表达式成立的可能性非常接近 1/2。

将该理论应用于我们的例子。线性表达式(5.12)影响 S-box 中 S_{42} 和 S_{44} 的输入。对于每个明文/密文对，我们尝试部分目标子密钥 $[K_{5,5}, \ldots, K_{5,8}, K_{5,13}, \ldots, K_{5,16}]$ 的 256 种可能。其中 $[U_{4,5}, \ldots, U_{4,8}, U_{4,13}, \ldots, U_{4,16}]$ 是通过将相应的密文与子密钥 $[K_{5,5}, \ldots, K_{5,8}, K_{5,13}, \ldots, K_{5,16}]$ 进行 XOR 运算，然后将运算结果向后经过 S_{42}，S_{44} 运算得到的。对每个局部目标子密钥，当表达式(5.12)成立时，我们增加计数。若一个子密钥的计数值与明文/密文的数目的一半相差最多，被假定为正确的子密钥。至于差值符号是正还是负，取决与所设计到的子密钥的和 $\sum K$。当 $\sum K = 0$ 时，表达式(5.12)作为一种估计，且成立的概率小于 1/2；当 $\sum K = 1$ 时，表达式(5.12)成立的概率大于 1/2。

我们产生 10000 个已知明文/密文对，来模拟前面描述的攻击，取部分子密钥为 $[K_{5,5}, \ldots, K_{5,8}] = [0010]$，$[K_{5,13}, \ldots, K_{5,16}] = [0100]$ 来模拟前面描述的攻击。正如我们所期望的，计数远离 5000 的符合条件的子密钥正是 $[2, 4]$（16 进制表示）。表5.19给出了部分子密钥对应的偏移量计数值（完整的应该有 256 条记录，每个目标子密钥对应一个），从中可以确认我们找到了正确的子密钥。结合表种的数据，可以计算出线性可能性偏移量，公式如下：

$$|\text{bias}| = |\text{count} - 5000|/10000,$$

其中 count 表示对应的子密钥的计数值。

从表5.19中，可以看出偏移量最大的子密钥是 $[K_{5,5}, \ldots, K_{5,8}, K_{5,13}, \ldots, K_{5,16}] = [2, 4]$，实际上，这个结果对于所有可能子密钥产生的结果也是正确的。

试验测定的偏移量是 0.0336 非常接近预期得线性可能性偏移量 $1/32 = 0.0315$。注意到，尽管正确的局部子密钥具有明显的最大的偏移量，其它的较大的偏移量的出现说明对不正确的目标子密钥的检测不等价于把一个随机的数据看作一个线性表达式（可能偏移量非常接近 0）的输入。S-box 的特性可能会影响不同局部目标子密钥的解密，利用 Piling-Up 定理时不严密的随机变量相互独立的假设，以及线性累积

表 5.19: 线性攻击的实验数据

partial subkey $[K_{5,5},\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}]$	|bias|	partial subkey $[K_{5,5}\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}]$	|bias|
1C	0.0031	2A	0.0044
1D	0.0078	2B	0.0186
1E	0.0071	2C	0.0094
1F	0.0170	2D	0.0053
20	0.0025	2E	0.0062
21	0.0220	2F	0.0133
22	0.0211	30	0.0027
23	0.0064	31	0.0050
24	**0.0336**	32	0.0075
25	0.0106	33	0.0162
26	0.0096	34	0.0218
27	0.0074	35	0.0052
28	0.0224	36	0.0056
29	0.0054	37	0.0048

（在下一小节将会提到）等原因都会导致实验时得到的偏移量不准确性。

5.7.2.6 攻击的复杂度

我们把线性近似表达式中包含的 S-box 称为活跃 S-box。在图5.34中，从第 1 轮到第 3 轮中被粗线条标出的四个 S-box 都是活跃的。一个线性近似表达式成立的可能性，与 S-box 的线性可能性偏移量的大小，活跃 S-box 的数目有关。一般情况下，这些活跃 S-box 的线性可能性偏移量越大，整个线性近似表达式的线性可能性偏移量越大。同样，活跃 S-box 的线性可能性偏移量越小，整个线性表达式的线性可能性偏移量越小。

令 ε 表示整个线性近似表达式成立的概率与 $1/2$ 之间的偏移量。Matsui 给出了攻击所需要的明文的数量与 ε^{-2} 成正比。令 N_L 表示需要的已知明文的数量，用公式 $N_L \approx 1/\varepsilon^2$ 来近似估计 N_L 是合理的。

在实际中，期望得到 ε^{-2} 几倍的明文是合理的假设。尽管严格意义上说，密码分析的复杂性需要从时间和空间（或者存储器）进行衡量，我们用攻击时所需要的明文数量来考虑密码分析的复杂性。那就是说，假设如果我们能获得密码分析所需要的 N_L 明文，那么我们有足够的时间和空间处理它，即假定分析者的计算能力和存储能力是无限的。

既然线性偏移量是由对 S-box 的线性近似表达式应用 Piling-Up 引理得到的，那么可以很容易推出，偏移量取决于 S-box 的线性可能性偏移量和活跃 S-box 的数目。一般地提高算法安全性抵抗线性分析的方法集中在优化 S-box（例如，减小最大偏移量）和增加活跃 S-box 的数目。5.3节介绍的 Rijndael 算法就是应用这种方法的一个极好的例子。

然而，需要知道的是，证明一个算法对线性分析是安全的，其前提是不存在高可能性的线性近似。但是，对这样的线性近似的可能性的计算，是以下两个假设为前提的：每个 S-box 的线性近似是相互独立的（这样才可以应用 $Piling-Up$ 引理）；一个线性近似足够确定关于明文和最后输入数据之间的一个最佳线性近似表达式。实际情况是 S-box 的线性近似之间往往都不是相互独立的，这是计算线性可能性的一个重大障碍。还有，对于包含相同的明文和最后一轮输入的线性近似，具有不同种活跃 S-box 的组合可能比只有一种活跃 S-box 的线性近似具有更高的线性可能性，这个概念被称为线性累积。一个最明显的例子，许多具有较小偏移量的线性近似部件，从自身的角度看可能都是对线性分析攻击是免疫的。但当把这些部件组合在一起时，构成的线性近似表达式可能具有较大的线性可能性偏移量。但是，我们在此提出的方法，在许多密码分析中都是正确的，因为独立性假设是一个合理的近似，并且当一个近似线

性部分具有较高的偏移量时，则由它支配线性累积。

5.7.3 差分密码分析

在这一部分，我们将讨论差分分析在对 SPN 加密网络攻击中的应用。差分分析和线性分析在许多方面都是相似，它与线性密码分析的主要区别是差分分析包含了将两个输入的异或与两个输出的异或相比较。

5.7.3.1 攻击概述

差分密码分析利用了明文差分与最后一轮输入差分的某种特定情况发生的高可能性。若一个加密算法的输入表示为 $X = [X_1X_2 \dots X_n]$，输出表示为 $Y = [Y_1Y_2 \dots Y_n]$。令算法的两个输入为 X' 与 X''，相应的输出为 Y' 和 Y''。则输入差分为 $\Delta X = X' \oplus X''$，其中 $\oplus$ 表示比特组之间的 XOR 运算，因此 $\Delta X = [\Delta X_1 \Delta X_2 \dots \Delta X_n]$，其中 $\Delta X_i = X'_i \oplus X''_i$，$X'_i$ 与 X_i" 分别表示 X' 与 X'' 的第 i 个比特。输出差分公式为 $\Delta Y = Y' \oplus Y''$，且 $\Delta Y = [\Delta Y_1 \Delta Y_2 \dots \Delta Y_n]$，其中 $\Delta Y_i = Y'_i \oplus Y''_i$。

在理想的随机化加密算法中，给定一个输入差分 ΔX 使其对应的输出差分为特定的 ΔY 的概率是 $1/2^n$，n 是输入 X 的比特数目。差分分析利用这样一种情况：给定一个输入差分 ΔX，特定输出差分为 ΔY，该情况发生的概率很高（例如远大于 $1/2^n$，概率记为 P_D）。$(\Delta X, \Delta Y)$ 被作为一个差分对。

差分分析是一种选择明文攻击方法，意味着攻击者可以选择明文输入，检测相应输出以得到密钥。对于差分分析，攻击者选择输入的数据对为 X' 与 X''，得到一个特定的 ΔX，对于这个已知的 ΔX，一个特定的 ΔY 发生的概率很高。

在此，我们把差分结构 $(\Delta X, \Delta Y)$ 中的明文用 X 表示，最后一轮的输入用 Y 表示。我们可以通过检测高概率的差分特征来实现。在加密算法中，前一轮的输出差分对应着下一轮相应的输入差分，一系列每一轮的输入和输出差分就构成了差分特征。利用高概率的差分特征，我们有可能利用输入到最后一轮的信息来获取最后一轮子密钥的子集。

和线性密码分析相同，为了构造高可能性的差分特征，我们观察每个单独的 S-box 的性质，并且用这些特征来得到完整的差分特征。特别地，我们考察 S-box 的输入、输出的差分，以得到一个高可能性的差分对。按照加密的轮次结合各个 S-box 的差分对，且非零的输出差分对应下一轮相应的非零的输入差分。这样，使得我们可以找到一个由明文差分和最后一轮输入的差分组成高可能性差分对。在差分分析中，加密所用的子密钥消失了，因为它们与所有的数据集合有关，当考虑它们对差分表达式的影响时，由其与自身进行 XOR 运算，可知结果为零。

5.7.3.2 分析加密部件

我们测试 S-box 的所有差分对。如图5.33所示的 4×4 的 S-box，其输入为 $X = [X_1X_2X_3X_4]$，输出为 $Y = [Y_1Y_2Y_3Y_4]$。一个 S-box 的所有差分对 $(\Delta X, \Delta Y)$ 可以被找出，而且给定 ΔX 对应特定 ΔY 的可能性也可以得到（对于输入对 (X', X'')，其输入差分 $\Delta X = X' \oplus X''$）。因为输入对的顺序是不相关的，则对一个 4×4 的 S-box，我们只需要考虑 X' 的值，已知 ΔX 值的情况下利用公式 $X'' = X' \oplus \Delta X$ 可以计算出对应的 X'' 的值。

考察加密算法中的 S-box，对于每个输入对 $(X', X'' = X' \oplus \Delta X)$ 我们可以得到其对应的输出差分 ΔY。例如，对于二进制值 X 和 Y，给定输入对 $(X, X \oplus \Delta X)$ 以及其相应 ΔY。如表5.20所示，表格的最后三列给出了在 ΔX 的值分别为 1011（16 进制 B），1000（16 进制 8）和 0100（16 进制 4）的情况下，对应的 ΔY 的值。从这个表格中，我们可以知道对于 $\Delta X = 1011$，$\Delta Y = 0010$ 情况的概率是 $8/16$，$\Delta X = 1000$，$\Delta Y = 1011$ 的情况的概率是 $4/16$，$\Delta X = 0100$，$\Delta Y = 1010$ 情况发生的次数是 0。如果

表 5.20: S-box 的差分对采样

X	Y	ΔY		
		ΔX = 1011	$\Delta X = 1011$	ΔX = 0100
0000	1110	0010	1101	1100
0001	0100	0010	1110	1011
0010	1101	0111	0101	0110
0011	0001	0010	1011	1001
0100	0010	0101	0111	1100
0101	1111	1111	0110	1011
0110	1011	0010	1011	0110
0111	1000	1101	1111	1001
1000	0011	0010	1101	0110
1001	1010	0111	1110	0011
1010	0110	0010	0101	0110
1011	1100	0010	1011	1011
1100	0101	1101	0111	0110
1101	1001	0010	0110	0011
1110	0000	1111	1011	0110
1111	0111	0101	1111	1011

S-box 是理想化的情况，每一个差分对发生的次数是 1，也就是给定 ΔX 对应特定 ΔY 的概率是 $1/16$（已经被证明这样的 S-box 在数学理论上是不存在的）。

我们可列出一个 S-box 的输入输出差分分布表格，其中 ΔX 作为行，ΔY 作为列。表格5.20所表示的 S-box 的差分分布表如表5.21所示。表格中的数字表示对于给定的输入差分 ΔX，相应的输出差分为 ΔY 时，发生的次数。从表5.21中可以得知：除了特殊情况 $(\Delta X = 0, \Delta Y = 0)$ 以外表格中的最大值是 8，对应的是 $\Delta X = \mathrm{B}, \Delta Y = 2$。因此，对于 $\Delta Y = 2$ 时，给定一个随机的输入对满足 $\Delta X = \mathrm{B}$ 的概率是 $8/16$。表格中值是最小值为 0，且许多差分对的对应值为 0。在这种情况下，给定 ΔX，对应的输出差分为 ΔY 的概率是 0。

差分分布表有许多基本的特征。首先，每一行的元素相加的和是 $2^n = 16$；类似的，每一列的元素相加的和也是 $2^n = 16$。其次，所有的元素都是偶数：这是因为对于输入 (X', X'') 和 (X'', X') 的差分值 $(\Delta X = X' \oplus X'' = X'' \oplus X')$ 相同。还有，对于一个一对一映射的 S-box，如果输入差分为 $\Delta X = 0$，则必须导致输出差分 $\Delta Y = 0$。因此，表格最右上角的值为 $2^n = 16$，而且第 1 行和第 1 列的其它值均为 0。最后，如果我们能构造一个理想的 S-box，该 S-box 的输入与输出之间没有明显的差分信息，差分分布表的每一项元素都是 1，且给定 ΔX 对应的特定输出差分为 ΔY 的概率应该是 $1/2^n = 1/16$。但是，将上面讨论的性质都要达到，这显然是不可能的。

在讨论联合 S-box 的差分对构成一个差分特性，而且选择一个攻击中应用的合适差分对之前。我们必须讨论密钥对 S-box 的差分性质的影响。一般假设不涉及密钥的 S-box 的输入为 X，且输出为 Y。但是，在实际的加密算法中，我们必须考虑应用到每个 S-box 的密钥。如图5.35所示，在这种情况下，我们令涉及密钥的 S-box 的输入为 $W = [W_1 W_2 W_3 W_4]$，则可如下表示 S-box 的输入差分表达式

$$\Delta W = [W_1' \oplus W_1'' W_2' \oplus W_2'' \ldots W_n' \oplus W_n''],$$

其中，$W' = [W_1' W_2' \ldots W_n']$，$W'' = [W_1'' W_2'' \ldots W_n'']$ 表示 S-box 的两个输入。

由于 W'，W'' 的加密密钥是相同的，则

$$\begin{aligned} \Delta W_i &= W_i' \oplus W_i'' \\ &= (X_i' \oplus K_i) \oplus (X_i'' \oplus K_i) \\ &= X_i' \oplus X_i'' = \Delta X_i. \end{aligned}$$

表 5.21: 差分分布表

In\Out	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	2	0	0	0	2	0	2	4	0	4	2	0	0
2	0	0	0	2	0	6	2	2	0	2	0	0	0	0	2	0
3	0	0	2	0	2	0	0	0	0	4	2	0	2	0	0	4
4	0	0	0	2	0	0	6	0	0	2	0	4	2	0	0	0
5	0	4	0	0	0	2	2	0	0	0	4	0	2	0	0	2
6	0	0	0	4	0	4	0	0	0	0	0	0	2	2	2	2
7	0	0	2	2	2	0	2	0	0	2	2	0	0	0	0	4
8	0	0	0	0	0	0	2	2	0	0	0	4	0	4	2	2
9	0	2	0	0	2	0	0	4	2	0	2	2	2	0	0	0
A	0	2	2	0	0	0	0	0	6	0	0	2	0	0	4	0
B	0	0	8	0	0	2	0	2	0	0	0	0	0	2	0	2
C	0	2	0	0	2	2	2	0	0	0	0	2	0	6	0	0
D	0	4	0	0	0	0	0	4	2	0	2	0	2	0	2	0
E	0	0	2	4	2	0	0	0	6	0	0	0	0	0	2	0
F	0	2	0	0	6	0	0	0	0	4	0	2	0	0	2	0

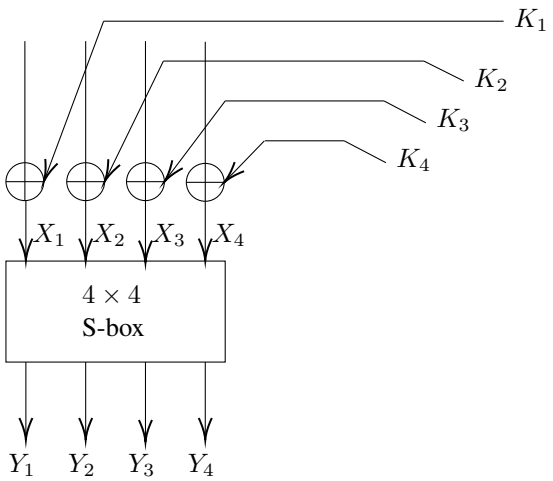


图 5.35: 带有密钥的 S-box

因为，$K_i \oplus K_i = 0$。可知，密钥比特对于 S-box 的输入差分没有影响，可以忽略密钥的影响。换句话说，涉及密钥的 S-box 与不涉及密钥的 S-box 的差分分布相同。

5.7.3.3 构造差分特征

如果找到了一个 SPN 加密网络的 S-box 差分信息，可以利用整个加密算法的差分特征进行数据处理。构造每一轮的特定 S-box 的输入、输出差分构成差分特征，然后连接这些差分特征，构成整个加密算法的差分特征。这样整个加密算法的差分特征涉及了明文比特和加密算法的最后一轮的输入。通过恢复最后一轮子密钥的子集进行密码分析是有可能的。我们用一个例子来说明如何构造一个差分特性。

考察一个包含的 S_{12}、S_{23}、S_{32} 和 S_{33} 的差分特性。和线性分析的情况相同，将差分特征形象化成如图5.36的形式是非常有用的。该图表示了非零的差分通过加密网络时的影响，其中以加粗线条模式标出的 S-box 是活跃 S-box。应该注意，这样是构造了一个前三轮加密的差分特征，而不是整个加密过程的四轮。在下面我们将会看到，这样的差分特性在从最后一个子密钥中获取部分密钥比特的过程中的作用。

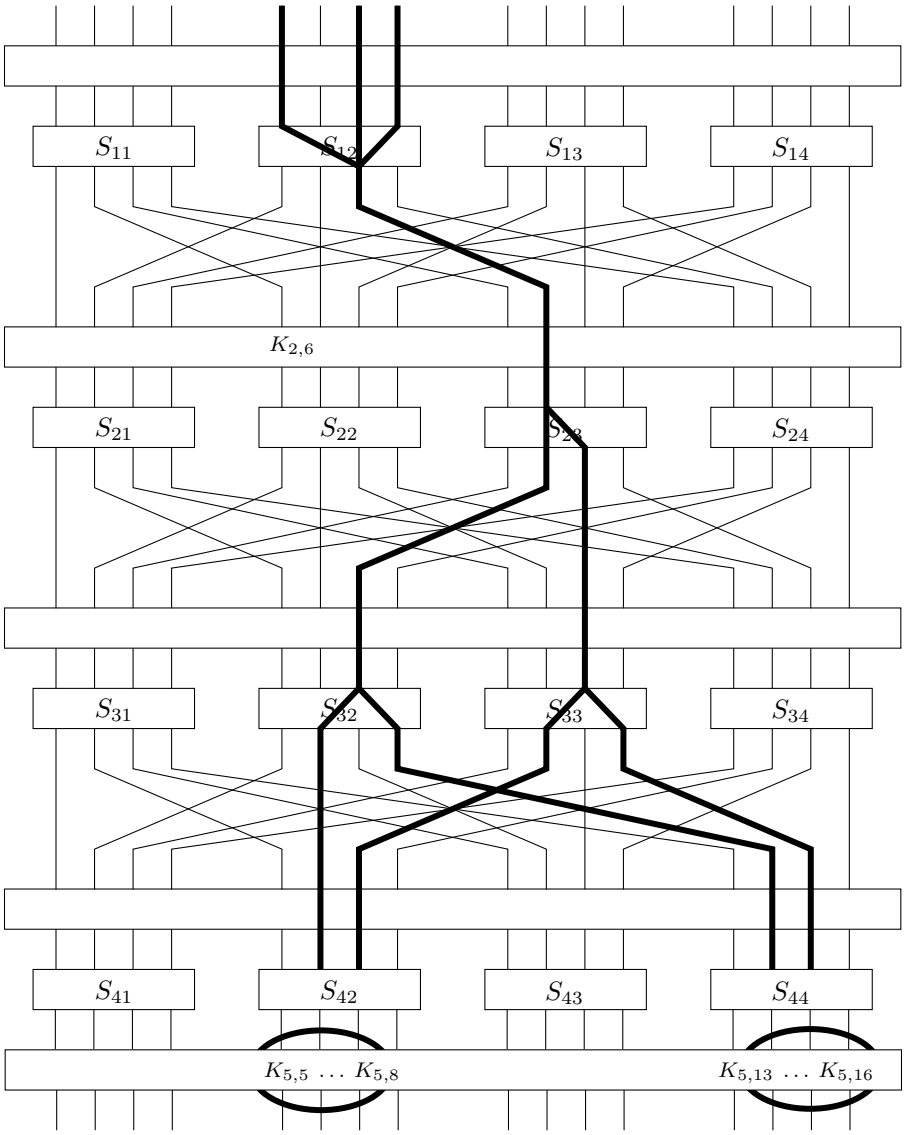

S_{11}
S_{12}
S_{13}
S_{14}
$K_{2,6}$
S_{21}
S_{22}
S_{23}
S_{24}
S_{31}
S_{32}
S_{33}
S_{34}
S_{41}
S_{42}
S_{43}
S_{44}
$K_{5,5}$ ... $K_{5,8}$
$K_{5,13}$ ... $K_{5,16}$

图 5.36: 差分分析

我们利用下面的 S-box 差分对：

$$\begin{aligned} S_{12}: &\quad \Delta X = B \to \Delta Y = 2 \quad \text{成立的概率为} \frac{8}{16}; \\ S_{23}: &\quad \Delta X = 4 \to \Delta Y = 6 \quad \text{成立的概率为} \frac{6}{16}; \\ S_{32}: &\quad \Delta X = 2 \to \Delta Y = 2 \quad \text{成立的概率为} \frac{6}{16}; \\ S_{33}: &\quad \Delta X = 2 \to \Delta Y = 5 \quad \text{成立的概率为} \frac{6}{16}. \end{aligned}$$

所有其它的 S-box 会有零输入差分和零输出差分。

加密算法的输入差分等于第一轮的输入差分，以下式给出：

$$\Delta P = \Delta U_1 = [0000101100000000].$$

同样，和我们讨论线性分析一样，用 U_i 表示第 i 轮的 S-box 的输入，V_i 表示第 i 轮的 S-box 的输出。因此，ΔU_i 与 ΔV_i 分别表示相应的差分。可知 $\Delta V_1 = [0000001000000000]$，上面所列 S_{12} 的差分对和接下来的第 1 轮 P 置换使得 $\Delta U_2 = [0000000001000000]$，可知给定明文差分 ΔP 的情况下 ΔU_2 的概率是 $8/16 = 1/2$。

第 2 轮的差分特征：利用 S_{23} 的差分对

$$\Delta V_2 = [0000000001100000]$$

和第 2 轮的 P 置换给出

$$\Delta U_3 = [0000001000100000],$$

给定 ΔU_2 的条件下 ΔU_3 发生的概率为 $6/16$，给定 ΔP，ΔU_3 发生的概率为 $8/16 \times 6/16 = 3/16$。在给定明文差分 ΔP 确定概率时，我们假设第 1 轮的差分特征与第 2 轮的差分特征是相互独立的。因此，两者同时发生的概率是两者概率的乘积。

随后，我们利用第三轮的 S_{32}、S_{33} 的差分以及第三轮 P 置换得

$$\Delta V_3 = [0000010101010000]$$

以及

$$\Delta U_4 = [0000011000000110].$$

在给定 ΔU_3 的情况下 ΔU_4 发生的概率为 $(6/16)^2$，因此，给定明文差分 ΔP 的情况下 ΔU_4 的概率是 $8/16 \times 6/16 \times (6/16)^2 = 27/1024$，其中我们假设所有轮的 S-box 的差分对相互独立。

在密码分析的过程中，许多差分为 $\Delta P = [0000101100000000]$ 的明文对被加密。图5.36中所示的差分特征将以 $27/1024$ 的高概率发生。我们称这样的 ΔP 对为正确对。差分特征中不发生的明文差分对为错误对。

5.7.3.4 获取密钥比特

如果得到了一个具有 R 轮加密过程的加密算法的前 $R-1$ 轮差分特性，且该差分特性具有较高的概率，通过恢复最后一轮的子密钥的子集来攻击密码算法是可能的。在我们使用的加密算法的例子中，从获取子密钥 K_5 的比特是可能的。接下来的步骤包括穷举攻击最后一轮的加密和检测对最后的一轮输入是否有一个正确对。我们把与最后一轮加密相关联的，且对应的 S-box 受差分特征影响的子密钥称为局部目标子密钥（或部分目标子密钥）。对最后一轮的解密包括：将密文与局部目标子密钥进行 XOR 运算，将运算结果返回最后一轮的 S-box 进行运算，且所有可能的部分目标子密钥都要进行这种运算。

对所有可能的局部目标子密钥，依照其明文对的差分是否为 ΔP，处理每一个密文对。每个局部目标子密钥设置一个计数器，当最后一轮的输入的差分符合差分特性的值时，计数器的计数增加。具有最大计数的局部目标子密钥被作为子密钥的真实值。这样做的原因是，正确的局部目标子密钥会使得最后

表 5.22: 差分分析的实验结果

partial subkey $[K_{5,5},\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}]$	prob	partial subkey $[K_{5,5},\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}]$	prob
1C	0.0000	2A	0.0032
1D	0.0000	2B	0.0022
1E	0.0000	2C	0.0000
1F	0.0000	2D	0.0000
20	0.0000	2E	0.0000
21	0.0136	2F	0.0000
22	0.0068	30	0.0004
23	0.0068	31	0.0000
24	**0.0244**	32	0.0004
25	0.0000	33	0.0004
26	0.0068	34	0.0000
27	0.0068	35	0.0004
28	0.0030	36	0.0000
29	0.0024	37	0.0008

一轮的差分的概率像差分特性预计的一样，因为差分特性具有较高的概率。(如果一个错误对产生，且此时的局部目标子密钥是正确的子密钥，正确局部目标子密钥的计数不会增加。）一个不正确的子密钥被看作导致对输入到 S-box 的最后一轮的数据的随机猜测，因此得到的差分与预计的差分特性一致的可能性非常小。

在对我们所给例子进行攻击时,差分分析影响最后一轮的 S-boxS_{42} 和 S_{44}。对于每个密文对,我们将尝试密钥 $[K_{5,5},\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}]$ 的 256 种所有可能。对于每个局部目标子密钥值,如果由对最后一轮解密得到的输入差分符合 ΔU_4，我们增加局部目标子密钥的计数值。其中这样计算 $[\Delta U_{4,5},\ \ldots,\ \Delta U_{4,8},\ \Delta U_{4,13},\ \ldots,\ \Delta U_{4,16}]$ 的值，将密文与子密钥进行 XOR 运算，然后将运算的结果逆向经过 S_{24} 和 S_{44} 运算。对于每个局部目标子密钥来说，计数表示与正确对一致的差分发生的次数（假设局部目标子密钥是正确的)。计数值最大的作为正确的子密钥是因为我们假设正确对发生的概率最大。

并不需要处理所有的密文对。因为最后一轮的输入差分仅仅对应两个 S-box，当满足差分特性时，S_{41}、S_{43} 对应的密文差分必定为 0。因此我们可以过滤许多错误的对，若一个密文差分的 S_{41}、S_{43} 对应子块不出现 0，则不处理该密文对。在这些情况下，由于密文对不对应正确的对，所没有必要去检测 $[\Delta U_{4,5},\ \ldots,\ \Delta U_{4,8},\ \Delta U_{4,13},\ \ldots,\ \Delta U_{4,16}]$。

我们可以模拟攻击带有密钥的基本密码算法，例如利用随机产生的 5000 个明文/密文对（例如，含有符合明文差分为 $\Delta P = [0000101100000000]$ 的 10000 个密文)，下面描述详细的过程。正确的部分目标子密钥应该是 $[K_{5,5},\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}] = [0010,0100] = [2,4]_{\text{hex}}$。正如预计的一样，具有最大计数的子密钥是 $[2,4]_{\text{hex}}$，证明攻击成功的获取了局部目标子密钥。表5.22中，给出了一部分局部目标子密钥的计数（完整的数据需要 256 项，每个部分子密钥一项)，表中的数据表明了对于局部目标子密钥，利用如下公式对正确对发生的可能性的估计:

$$\text{prob} = \text{count}/5000,$$

其中，count 是局部目标子密钥的计数。

如同从表5.22中所看到的结果一样,子密钥可能性最大的是 $[K_{5,5},\ldots,K_{5,8},K_{5,13},\ldots,K_{5,16}] = [2,4]_{\text{hex}}$，实际上在整个部分子密钥中，这个结果也是正确的。

在我们的例子中，我们希望那个正确的对发生的可能性是 $P_D = 27/1024 = 0.0264$，而且我们通过实验得到的正确密钥 $[2,4]$ 的概率是 $P_D = 0.0244$。注意，有时候会有由错误的局部目标子密钥产生的其

它较大的计数。这表明，对于不正确目标子密钥的检测不等价于将一个随机差分与预计的差分进行对比。有许多因素使得理论的计数与我们实际所得到的计数不同，S-box 的性质对于不同的局部目标子密钥在对最后一轮解密的影响，构造差分特性时的对独立性的不严密的假设，差分包括的多种差分性质等（将在下一小节介绍）。

5.7.3.5 攻击的复杂度

对于差分密码分析，我们把包含在差分的特性中的、且有非零输入差分的（因此输出差分也不为零）S-box 称为活跃 S-box。一般的，活跃 S-box 的差分的可能性越大，则整个加密算法的差分特性的可能性越大。而且，活跃 S-box 的较少，则整个加密算法的差分特性的可能性越大。与线性分析一样，我们用攻击时需要的明文的数量来衡量攻击的复杂性。那就是说，如果我们能获得 N_D 数量的明文，则我们有足够的能力处理它。

一般的，确定准确的攻击需要的明文对的数量是非常复杂的。但是，从以往的经验可知，在尝试局部目标子密钥时，可以辨别出正确对，所需要的明文对的数量 N_D 为

$$N_D \approx c/P_D, \tag{5.13}$$

其中，P_D 是一个 $R-1$ 轮的差分特性的概率，c 是一个小的常量。假设每个 S-box 的差分发生的概率是相互独立的，则差分特性的概率可以如下公式给出：

$$P_D = \prod_{i=1}^{r} \beta_i, \tag{5.14}$$

其中，r 表示活跃 S-box 的数量，β_i 表示第 i 个活跃 S-box 对特定的差分对发生的概率。

证明(5.13)是合理的并不困难。它简单的表明，少量正确对的产生就可以给一个正确的局部目标子密钥足够大的计数，且显然的多于对不正确的局部目标子密钥的计数。既然当大约 $1/P_D$ 明文对被检测，就会得到一个正确对。因此，假设用几倍于 $1/P_D$ 数量的明文，来进行密码分析是合理的。

抵抗差分分析的方法一般是优化 S-box 的特性（例如，减小每个 S-box 的差分的概率）以及寻找一种扩大活跃 S-box 的数目的结构。Rijndael 就是一种高抗差分分析的加密算法。

与线性分析一样，必须谨慎证明加密算法对差分分析的免疫性。差分特性的概率的计算是以 S-box 的近似相互独立为前提条件的，在一个真正的加密算法中，输入到 S-box 的数据是相互依赖的。因此，P_D 是一个估计值。在实际应用中，这样的假设被证明是正确的。

有着相同的输入差分和输出差分的的差分特性结合在一起，所构成的差分特性的概率要远大于只考虑一个单独的差分特性时的概率（类似线性累积的概念）。要证明对差分分析是安全的，需要证明所有独立的差分的概率都小于某个极限值，而不是仅仅证明所有的差分特性的值都小于某个极限值。然而，下面的假设是合理的，一般当一个差分特性具有较高的概率时，其决定着差分发生的概率，而且其差分特征的概率是差分发生的概率的近似。

第 5 章 习题

1. 分组密码有哪些工作模式，试叙述其过程。在 8 位的 CFB 模式中，若传输中密文字符发生了一位错误，这个错误将传播多远?
2. DES 算法通过哪些环节来完成扩散和混乱准则的?
3. 假设 DES 算法的明文输入 M = FEFEFEFEFEFEFEFE，$K_1 = 555555555555$（均为 16 进制表示），求第一轮的输出。
4. 什么是 $3DES$? 为什么要使用 $3DES$? 在 $3DES$ 中用到多少密钥? 为什么在 $3DES$ 的中间部分采用了解密而不是加密?

5. 使用 AES 进行加密，若待加密的明文是 {000102030405060708090A0B0C0D0E0F}，加密的密钥为 {01010101010101010101010101010101}，
   (1) 用 $4 * 4$ 的矩阵来描述 State 的最初内容。
   (2) 给出初始化轮密钥加密后的值。
   (3) 给出字节代换后的值。
   (4) 给出行移位后的值。
   (5) 给出列混和变换后的值。
6. 设 X_1、X_2、X_3 是定义在 $\{0,1\}$ 上的独立离散的随机变量。用 ε_i 表示 X_i 的偏移量，$i = 1,\ 2,\ 3$。证明，$X_1 \oplus X_2$ 与 $X_2 \oplus X_3$ 相互独立当且仅当 $\varepsilon_1 = 0$，$\varepsilon_3 = 0$ 或 $\varepsilon_2 = \pm 1/2$。
7. 计算 DES 算法的某一个 S 盒，计算随机变量 $X_2 \oplus Y_1 \oplus Y_2 \oplus Y_3 \oplus Y_4$ 的偏差。
8. 用差分分析和线性分析的方法分析 SMS4 算法，观察 SMS4 是否对两种分析方法免疫。

第 6 章 Hash 函数

Hash 函数 (也称散列函数或散列算法) 的输入为任意长度的消息，而输出为某一固定长度的消息。即 Hash 函数是一种将任意长度的消息串 M 映射成一个较短的定长消息串的函数，记为 H。称 $h = H(M)$ 为消息 M 的 Hash 值或消息摘要（message digest），有时也称为消息的指纹。通常 Hash 函数应用于数字签名、消息完整性等方面。

设 H 是一个 Hash 函数，x 是任意长度的二元串，相应的消息摘要为 $y = H(x)$，通常消息摘要是一个相对较短的二元串（例如 160 比特）。假设我们已经计算出了 y 的值，那么如果有人改变 x 的值为 x'，则通过计算消息摘要 $y' = hash(x')$，验证 y' 与 y 不相等就可以知道原来的消息 x 已被改变。

通常，Hash 函数可以分为两类：不带密钥的 Hash 函数和带密钥的 Hash 函数。不带密钥的 Hash 函数只需要有一个消息输入；带密钥的 Hash 函数规定要有两个不同的输入，即一个消息和一个秘密密钥。

6.1 Hash 函数的性质

6.1.1 Hash 函数的性质

Hash 函数的目的是为指定的消息产生一个消息“指纹”，Hash 函数通常具有以下这些性质：

(1) 压缩性：Hash 函数将一个任意比特长度的输入 x 映射为固定长度为 n 的输出 $H(x)$。

(2) 正向计算简单性：给定 Hash 函数 H 和任意的消息输入 x，计算 $H(x)$ 是简单的。

(3) 逆向计算困难性：对所有预先给定的输出值，找到一个消息输入使得它的 Hash 值等于这个输出在计算上是不可行的。即对给定的任意值 y，求使得 $H(x) = y$ 的 x 在计算上是不可行的。通常这一性质也称为 Hash 函数的单向性。

(4) 弱无碰撞性：对于任何的输入，找到一个与它有相同输出的第二个输入，在计算上是不可行的。即给定一个输入 x，找到一个 x'，使得 $H(x) = H(x')$ 成立是计算不可行的，如果单向 Hash 函数满足这一性质，则称其为弱单向 Hash 函数。

(5) 强无碰撞性：找出任意两个不同的输入 x 与 x'，使得 $H(x) = H(x')$ 在计算上是不可行的，如果单向 Hash 函数满足这一性质，则称其为强单向 Hash 函数。

攻击者可以对 Hash 函数发起两种攻击。第一种就是找出一个 x'，使得 $H(x) = H(x')$。例如，在一个使用 Hash 函数的签名方案中，假设 s 是签名者对消息 x 的一个有效签名，$s = \mathrm{sig}(H(x))$。攻击者可能会寻找一个与 x 不同的消息 x' 使得 $H(x) = H(x')$。如果能找到一个这样的 x'，则攻击者就可以伪造对消息 x' 的签名，这是因为 s 也是对消息 x' 的有效签名。Hash 函数的弱无碰撞性可以抵抗这种攻击。

攻击者可以发起另一种攻击。同样一个应用 Hash 函数的签名方案中，对手可能会寻找两个不同的消息 x 和 x'，使得 $H(x) = H(x')$。然后说服签名者对消息 x 签名，得到 $s = \mathrm{sig}(H(x))$。由于 $s = \mathrm{sig}(H(x'))$，所以攻击者得到了一个对消息 x' 的有效签名。Hash 函数是强无碰撞性可以抵抗这种攻击。

6.1.2 生日攻击

在本节我们介绍两类生日问题以及基于这两类生日问题的攻击方法。

第 1 类生日问题：假设已经知道 A 的生日为某一天，问至少有多少个人在一起时，至少有 $1/2$ 的概率使有一个人和 A 的生日相同？在此，我们假定一年有 365 天，且所有人的生日均匀分布于 365 天中。下面我们求解所需的最少人数。

首先，有 1 人和 A 有相同生日的概率为 $1/365$，有不同生日的概率则为 $1-1/365=364/365$；k 个人与 A 生日不同的概率应为 $(364/365)^k$；k 个人至少有 1 个人与 A 的生日相同，且概率不小于 $1/2$ 应为 $1-(364/365)^k \geq 1/2$，所以 $(364/365)^k \leq 1/2$。即 $k \geq -\ln 2/\ln(364/365) \geq 0.6931471/0.027370 \geq 253$，即至少为 253 人。若已知 A 的生日，则当至少有 253 个人时，才能保证有 $1/2$ 的概率使有 1 人和 A 的生日相同。

那么有多少个人在一起，至少有 $1/2$ 的概率存在两个人有相同的生日？这就是第 2 类生日问题。

第 2 类生日问题：假设一年有 365 天，每个人的生日均匀分布于 365 天，那么至少有多少个人在一起是，能保证至少有 $1/2$ 的概率存在 2 个人有相同的生日。第 2 类生日问题也称生日悖论。

令 P_m 为 m 个人在一起，不存在相同生日的概率。根据假定，则 $m-1$ 个人中无相同生日的概率为 P_{m-1}，$m-1$ 个人共有生日 $m-1$ 天。第 m 个人与前面 $m-1$ 人无相同生日的概率为：

$$\frac{365-(m-1)}{365}=\frac{366-m}{365},$$

则可得递推关系：

$$P_m=\frac{366-m}{365}P_{m-1}.$$

所以有：

$$\begin{aligned}
&P_1=1,\\
&P_2=\frac{366-2}{365}P_1=\frac{364}{365},\\
&P_3=\frac{363}{365}P_2=\frac{364}{365}\times\frac{363}{365}=\frac{1}{365^2}\times\frac{364!}{362!},\\
&P_4=\frac{362}{365}P_3=\frac{362}{365}\times\frac{1}{365^2}\times\frac{364!}{362!}=\frac{1}{365^3}\times\frac{364!}{361!},\\
&\quad\vdots\\
&P_m=\frac{1}{365^{m-1}}\times\frac{364!}{(365-m)!}.
\end{aligned}$$

可以验证当 $m \geq 23$ 时，$P_m \leq 1/2$。即 23 个人在一起时，无相同生日的概率小于 $1/2$。反过来就是当 23 个人在一起是，有两个人的生日相同的概率大于 $1/2$，结果有点出人意料。

那么基于这两类生日问题，下面我们介绍两类生日攻击：

(1) 第 1 类生日攻击

如果已知一个 Hash 函数 H 有 n 个可能的输出，其中 $H(x)$ 是一个特定的输出。随机取 k 个输入，则至少有一个输入 y 使得 $H(y)=H(x)$ 的概率为 0.5 时，k 有多大?

第 1 类型生日攻击：与第 1 类生日问题相对比，称对 Hash 函数 h 寻找上述 y 的攻击为第 1 类型生日攻击。

因为 H 有 n 个可能的输出，所以输入 y 产生的输出 $H(y)$ 等于特定输出 $H(x)$ 的概率是 $1/n$，反过来说 $H(y)\neq H(x)$ 的概率是 $1-1/n$。y 取 k 个随机值而函数的 k 个输出中没有一个等于 $H(x)$，其概率等于每个输出都不等于 $H(x)$ 的概率之积，为 $(1-1/n)^k$。所以 y 取 k 个随机值得到函数的 k 个输出中至少有一个等于 $H(x)$ 的概率为 $1-(1-1/n)^k$。

由 $(1+x)^k \approx 1+kx$，其中 $|x| \leq 1$，可得

$$1-\left(1-\frac{1}{n}\right)^k \approx 1-\left(1-\frac{1}{n}\right)=\frac{k}{n}.$$

若使上述概率等于 0.5，则 $k=n/2$。特别地，如果 H 的输出为 m 比特，即可能的输出个数 $n=2^m$，则 $k=2^{m-1}$。假如 $m=80$，则 $k=2^{79}$，可见攻击付出的计算量是很大的。

(2) 第 2 类生日攻击

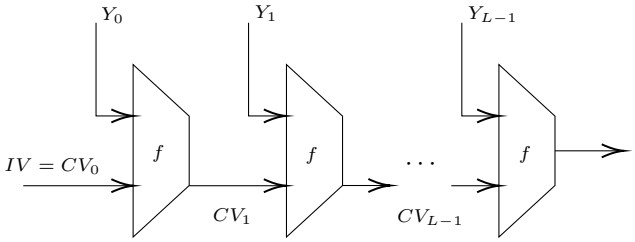


图 6.1: 迭代型杂凑函数的一般结构

第 2 类生日攻击：与第 2 类生日问题相类似，若一文件 m 的 Hash 值 $H(m)$ 为 32 比特，试问至少有多少的文件在一起，有两个文件的 Hash 值以至少 $1/2$ 的概率相同。不同的是在这里将 365 改为 2^{32}。则有：

$$P_m = \frac{2^{32} - m + 1}{2^{32}} P_{m-1},\ P_1 = 1,$$

其中 P_m 表示 m 个长度为 32 比特的 0、1 符号串不存在两个相等的概率。类似可得

$$P_m = \left(\frac{1}{2^{32}}\right)^{m-1} \cdot \frac{(2^{32}-1)!}{(2^{32}-m)!}.$$

验算可知 $m \geq 2^{17}$（$2^{17} = 131072$）时，$P_m \leq 1/2$。即当有 2^{17} 个文件时，则存在两个文件有相同的 Hash 值输出的概率超过 $1/2$。对于现在的计算设备，2^{17} 的计算量不是困难的。

为防止攻击，通常的方法就是增加 Hash 值的比特长度，一般最小的可接受长度为 128 位（此时对于第二类生日攻击需要计算超过 2^{64} 个 Hash 值），例如6.2节将要介绍的 MD5 与 SHA 分别具有 128 比特和 160 比特的消息摘要。

6.1.3 迭代 Hash 函数的结构

目前使用的大多数 Hash 函数如 MD5、SHA-1，其结构都是迭代型的，如图6.1所示。其中函数的输入 M 被分为 L 个分组 Y_0，Y_1，Y_2，...，Y_{L-1}，每一个分组的长度为 b 比特，如果最后一个分组的长度不够，需对其做填充。最后一个分组中还包括整个函数输入的长度值。这样，将使得攻击者的攻击更为困难，即攻击者若想成功地产生假冒的消息，就必需保证假冒消息的 Hash 值与原消息的 Hash 值相同，而且假冒消息的长度也要与原消息的长度相等。

算法中重复使用函数 f（称函数 f 为压缩函数）。f 的输入有两项，一项是上一轮的输出 n 比特值 CV_{i-1}，称为链接变量。另一项是算法在本轮的 b 比特输入分组 Y_i。f 的输出为 n 比特值 CV_i，CV_i 又作为下一轮的输入。算法开始时需对链接变量指定一个初始值 IV，最后一轮输出的链接变量 CV_L 就是最终产生的 Hash 值。算法过程如下：

$$\begin{aligned} &CV_0 = IV = n\text{比特长的初值};\\ &CV_i = f(CV_{i-1}, Y_{i-1}),\ 1 \leq i \leq L;\\ &H(M) = CV_L, \end{aligned}$$

$H(M)$ 就是所产生的 Hash 值。

算法的核心技术是设计无碰撞的压缩函数 f，而攻击者对算法的攻击重点是 f 的内部结构，由于 f 和分组密码一样是由若干轮处理过程组成的，所以对 f 的攻击需通过对各轮之间的位模式的分析来进行，分析过程常常需要先找出 f 的碰撞。由于 f 是压缩函数，其碰撞是不可避免的，因此在设计 f 时就应保证找出其碰撞在计算上是不可行的。

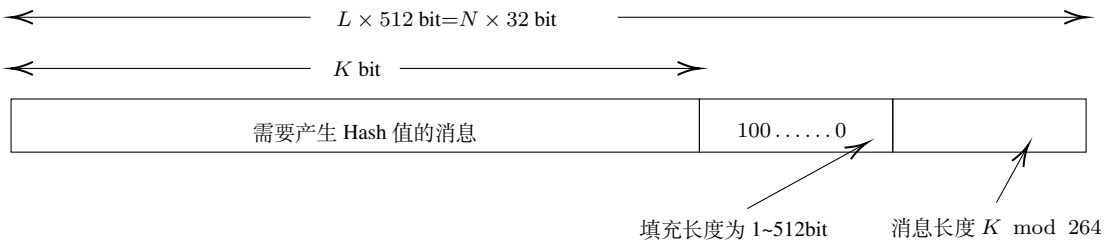


图 6.2: 消息填充

6.2 Hash 函数实例

6.2.1 MD5 散列函数

MD5 是由 Ron Rivest 设计的单向散列函数。MD 表示消息摘要（Message Digest），对输入的任意长度消息，算法产生 128 位的散列值（或消息摘要）。MD5 散列算法的前身是 MD4（1990 年由 Ron Rivest 提出），1992 年 4 月公布的改进后的 MD4 称为 MD5。

MD5 散列函数的处理过程分为如下几步：

(1) 消息填充：对原始消息填充，使得其比特长在模 512 余 448，即填充后消息的长度为 512 的某一倍数减 64。这一步是必须的，即使原始消息的长度已经满足要求，仍需要填充。例如：消息的长度正好为 448 比特，则需要填充 512 比特，使其长度为 960 比特，因此填充的比特数在 1 到 512 之间。填充方式是固定的：第一位为 1，其它位为 0，例如需要填充 100 比特，则填充一个 1 和后面附上 99 个 0。

(2) 添加消息长度：在第 1 步骤填充后，留有 64 个比特位，这 64 比特用来填充消息被填充前的长度。如果消息长度大于 2^{64}，则以 2^{64} 取模。

前两步完成以后，消息的长度为 512 的倍数（设倍数为 L），如图6.2所示填充后的消息。则可将消息表示成分组长为 512 的一系列分组 Y_0，Y_1，...，Y_{L-1}。每一个 512 比特的分组是 16 个（32 比特的）字，因此消息中的总字数为 $N = 16L$。

(3) 初始化 MD 缓冲区：MD5 算法使用 128 比特长的缓冲区以存储中间结果和最终 Hash 值。缓冲区可表示为 4 个 32 位长的寄存器（A，B，C，D），将存储器初始化为以下的 32 位整数：$A = 67452301$、$B =$ EFCDAB、$C =$ 98BADCFE、$D = 10325476$。而且每个寄存器都以 little-endian 方式存储数据，也就是最低有效字节存储在在低地址字节位置，4 个寄存器按如下存储：

$$A = 01234567、B = 89ABCDEF、C = FEDCBA98、D = 7654321.$$

(4) 以分组为单位进行消息处理：每个分组 Y_q 都经过一个压缩函数 H_{MD5} 处理，包括 4 轮处理过程，如图6.3所示，MD5 算法是一种迭代型 Hash 函数，符合上节提到到的迭代型 Hash 函数的一般结构，压缩函数 H_{MD5} 是算法的核心。压缩函数 H_{MD5} 按如下方式工作：

(a). 四个轮运算的结构相同，但各轮使用不同的基本逻辑函数，我们分别称之为 F、G、H 和 I。

(b). 每轮的输入是当前要处理的 512 位的分组 Y_q 和 128 位缓冲区的当前值 A、B、C、D 的内容，输出仍然放在缓冲区中以产生新的 A、B、C、D。

(c). 每轮的处理过程还需要使用常数表 T 中元素的 $1/4$。第 4 轮的输出再与第 1 轮的输入 CV_q 相加，相加时将 CV_q 看作 4 个 32 比特的字，每个字与第 4 轮输出的对应的字按模 2^{32} 相加，相加的结果就是本轮压缩函数 H_{MD5} 的输出。

H_{MD5} 压缩函数要用到常数表 T，表 T 有 64 个元素，如表6.1所示，该表通过正弦函数构建。表中

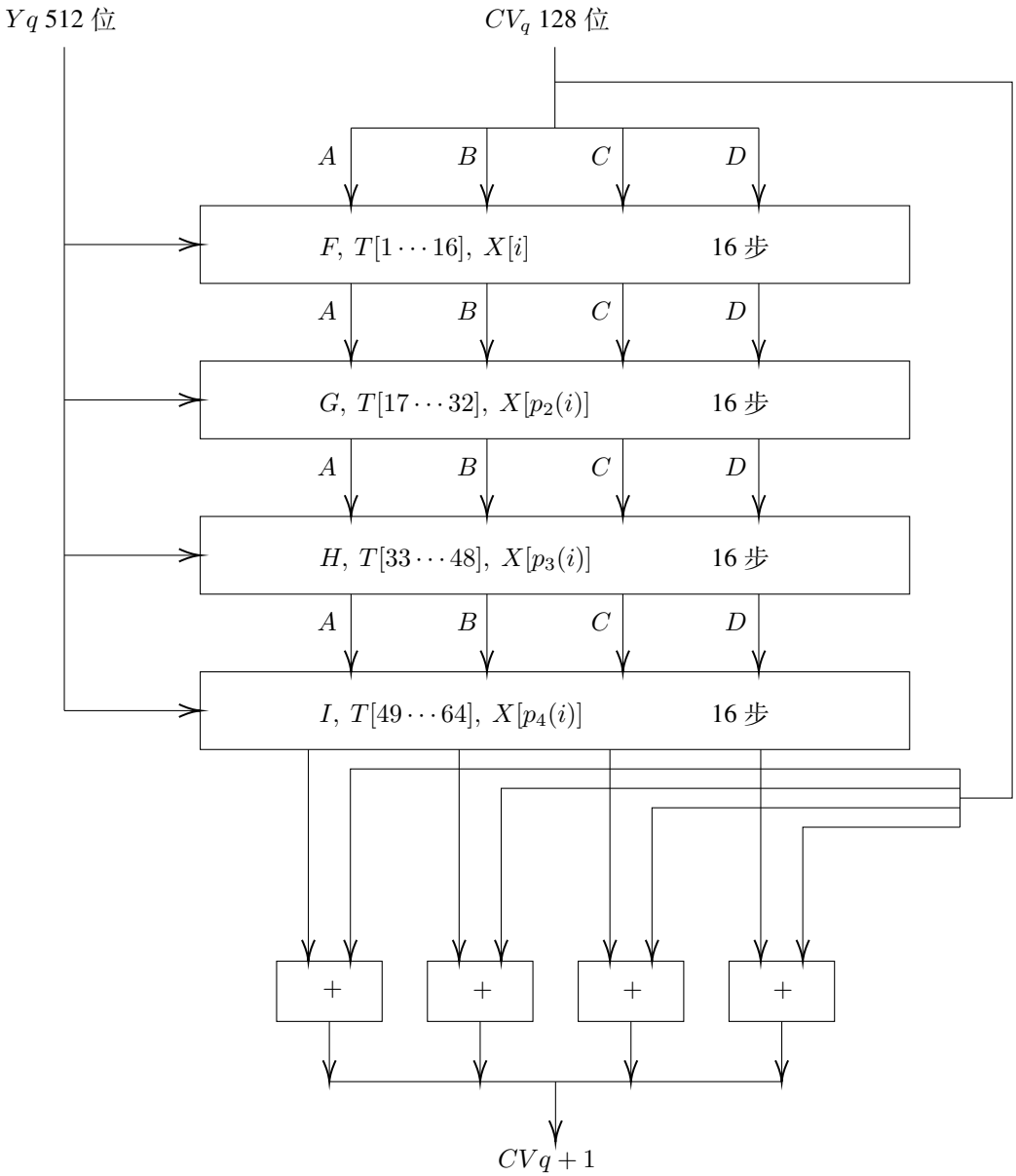


图 6.3: MD5 的分组处理

表 6.1: 常数表 T

$T[1]$=D76AA478	$T[2]$=E8C7B756	$T[3]$=242070DB	$T[4]$=C1BDCEEE
$T[5]$=F57C0FAF	$T[6]$=4787C62A	$T[7]$=A8304613	$T[8]$=FD469501
$T[9]$=698098D8	$T[10]$=8B44F7AF	$T[11]$=FFFF5BB1	$T[12]$=895CD7BE
$T[13]$=6B901122	$T[14]$=FD987193	$T[15]$=A679438E	$T[16]$=49B40821
$T[17]$=F61E2562	$T[18]$=C040B340	$T[19]$=265E5A51	$T[20]$=E9B6C7AA
$T[21]$=D62F105D	$T[22]$=02441453	$T[23]$=D8A1E681	$T[24]$=E7D3FBC8
$T[25]$=21E1CDE6	$T[26]$=C33707D6	$T[27]$=F4D50D87	$T[28]$=455A14ED
$T[29]$=A9E3E905	$T[30]$=FCEFA3F8	$T[31]$=676F02D9	$T[32]$=8D2A4C8A
$T[33]$=FFFA3942	$T[34]$=8771F681	$T[35]$=699D6122	$T[36]$=FDE5380C
$T[37]$=A4BEEA44	$T[38]$=4BDECFA9	$T[39]$=F6BB4B60	$T[40]$=BEBFBC70
$T[41]$=289B7EC6	$T[42]$=EAA127FA	$T[43]$=D4EF3085	$T[44]$=04881D05
$T[45]$=D9D4D039	$T[46]$=E6DB99E5	$T[47]$=1FA27CF8	$T[48]$=C4AC5665
$T[49]$=F4292244	$T[50]$=432AFF97	$T[51]$=AB9423A7	$T[52]$=FC93A039
$T[53]$=655B59C3	$T[54]$=8F0CCC92	$T[55]$=FFEFF47D	$T[56]$=85845DD1
$T[57]$=6FA87E4F	$T[58]$=FE2CE6E0	$T[59]$=A3014314	$T[60]$=4E0811A1
$T[61]$=F7537E82	$T[62]$=BD3AF235	$T[63]$=2AD7D2BB	$T[64]$=EB86D391

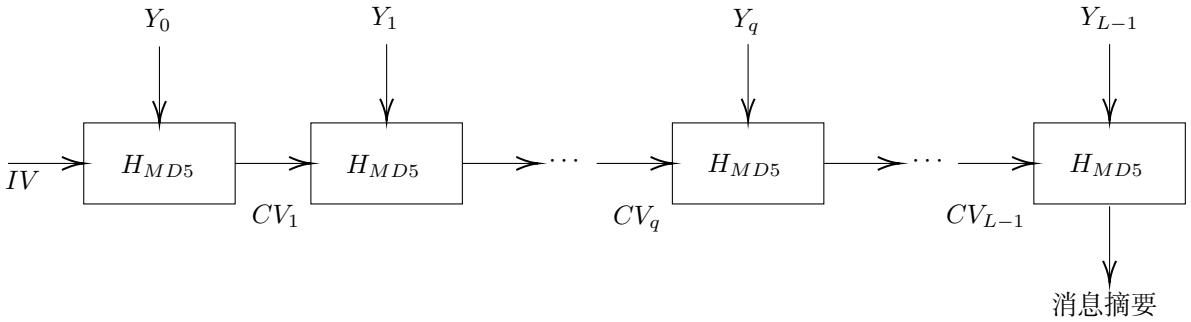


图 6.4: MD5 的消息处理框图

第 i 个元素 $T[i]$ 为 $2^{32} \times |\sin(i)|$ 的整数部分，其中 sin 为正弦函数，i 的单位为弧度。由于 $|\sin(i)|$ 大于 0 小于 1，所以 $T[i]$ 可由 32 比特的字来表示。这个表提供了一个随机化的 32bit 模式集，它将消除输入数据的任何规律性。

(5) 输出：消息的所有 L 个分组被处理完以后，最后一个 H_{MD5} 的输出即为产生的消息摘要（Hash 值）。图 6.4 为 $MD5$ 的消息处理框图。

下面我们具体描述一下 $MD5$ 的压缩函数 H_{MD5}：

压缩函数 H_{MD5} 中有 4 轮处理过程，每轮又对缓冲区 A、B、C、D 进行 16 步迭代运算，每一步的运算形式为，如图6.5所示。

$$a \leftarrow b + (a + g(b,c,d) + X[k] + T[i]) \lll s,$$

其中符号为：

MD5 压缩函数要经过 4 轮处理过程中，每轮又包括 16 步，且每轮以不同的次序使用 16 个字。其中

a，b，c，d	表示缓存中四个字，在不同的步骤中有指名的顺序，在运算完成后再右循环一个字，即得这一步迭代的输出
g	表示函数 F，G，H，I 中的一个
$\lll s$	表示 32bit 的参数循环左移 s 个比特
$X[k]$	表示在第 q 个长度为 512 比特的分组中的第 k 个 32 比特的字
$T[i]$	表示常数表 T 中的第 i 个 32 比特的字
$+$	表示模 2^{32} 的加法

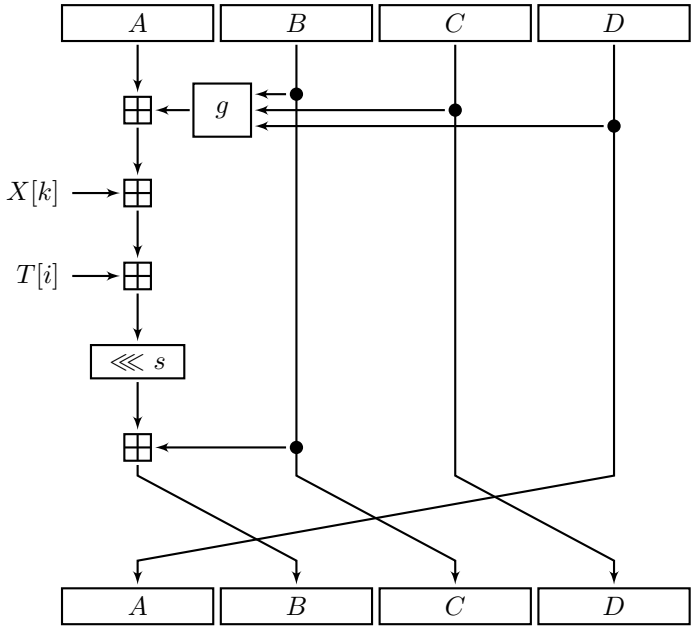


图 6.5: 压缩函数中的一步迭代

表 6.2: 压缩函数每步左循环移位的位数

步数 \ 轮数	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
一	7	12	17	22	7	12	17	22	7	12	17	22	7	12	17	22
二	5	9	14	20	5	9	14	20	5	9	14	20	5	9	14	20
三	4	11	16	23	4	11	16	23	4	11	16	23	4	11	16	23
四	6	10	15	21	6	10	15	21	6	10	15	21	6	10	15	21

在第一轮以字的初始次序使用，第二轮到第四轮，分别对字的次序 i 做置换后得到一个新次序，然后以新次序使用 16 个字。三个置换分别为：

$$P_2(i) = (1 + 5i) \bmod 16,$$

$$P_3(i) = (5 + 3i) \bmod 16,$$

$$P_4(i) = 7i \bmod 16.$$

4 轮处理过程中分别使用基本的逻辑函数 F、G、H、I。每个逻辑函数的输入为 3 个 32 比特的字，输出是一个 32 比特的字，其中的运算为逐比特的逻辑运算，四个逻辑函数的定义如下：

$$F(X, Y, Z) = (X \wedge Y) \vee ((\neg X) \wedge Z),$$

$$G(X, Y, Z) = (X \wedge Y) \wedge (Y \wedge (\neg Z)),$$

$$H(X, Y, Z) = X \oplus Y \oplus Z,$$

$$I(X, Y, Z) = Y \oplus (X \vee (\neg Z)).$$

其中：$\wedge$ 是与，$\vee$ 是或，$\oplus$ 是异或，$\neg$ 是反。

4 轮处理过程中，每一轮的 16 步都一个循环左移，移动的位数用 s 表示，如表6.2所示。

如果设 M_j 表示消息的 512 比特的分组中的第 j 个 32 比特的字（$0 \le j \le 15$），$\lll s$ 表示循环左移

位 s 位，则对应于 4 轮中的 4 种操作：

$FF(a,b,c,d,M_j,s,T[i])$ 表示$a=b+((a+(F(b,c,d)+M_j+T[i])\lll s)$;

$GG(a,b,c,d,M_j,s,T[i])$ 表示$a=b+((a+(G(b,c,d)+M_j+T[i])\lll s)$;

$HH(a,b,c,d,M_j,s,T[i])$ 表示$a=b+((a+(H(b,c,d)+M_j+T[i])\lll s)$;

$II(a,b,c,d,M_j,s,T[i])$ 表示$a=b+((a+(I(b,c,d)+M_j+T[i])\lll s)$.

则 MD5 压缩函数 H_{MD5} 的四轮可表示为：

第一轮的 16 步：

$$FF(a,b,c,d,M0,7,0\text{xd76aa478}),$$
$$FF(d,a,b,c,M1,12,0\text{xe8c7b756}),$$
$$FF(c,d,a,b,M2,17,0\text{x242070db}),$$
$$FF(b,c,d,a,M3,22,0\text{xc1bdceee}),$$
$$FF(a,b,c,d,M4,7,0\text{xf57c0faf}),$$
$$FF(d,a,b,c,M5,12,0\text{x4787c62a}),$$
$$FF(c,d,a,b,M6,17,0\text{xa8304613}),$$
$$FF(b,c,d,a,M7,22,0\text{xfd469501}),$$
$$FF(a,b,c,d,M8,7,0\text{x698098d8}),$$
$$FF(d,a,b,c,M9,12,0\text{x8b44f7af}),$$
$$FF(c,d,a,b,M10,17,0\text{xffff5bb1}),$$
$$FF(b,c,d,a,M11,22,0\text{x895cd7be}),$$
$$FF(a,b,c,d,M12,7,0\text{x6b901122}),$$
$$FF(d,a,b,c,M13,12,0\text{xfd987193}),$$
$$FF(c,d,a,b,M14,17,0\text{xa679438e}),$$
$$FF(b,c,d,a,M15,22,0\text{x49b40821}).$$

第二轮的 16 步：

$$GG(a,b,c,d,M1,5,0\text{xf61e2562}),$$
$$GG(d,a,b,c,M6,9,0\text{xc040b340}),$$
$$GG(c,d,a,b,M11,14,0\text{x265e5a51}),$$
$$GG(b,c,d,a,M0,20,0\text{xe9b6c7aa}),$$
$$GG(a,b,c,d,M5,5,0\text{xd62f105d}),$$
$$GG(d,a,b,c,M10,9,0\text{x02441453}),$$
$$GG(c,d,a,b,M15,14,0\text{xd8a1e681}),$$
$$GG(b,c,d,a,M4,20,0\text{xe7d3fbc8}),$$
$$GG(a,b,c,d,M9,5,0\text{x21e1cde6}),$$
$$GG(d,a,b,c,M14,9,0\text{xc33707d6}),$$
$$GG(c,d,a,b,M3,14,0\text{xf4d50d87}),$$
$$GG(b,c,d,a,M8,20,0\text{x455a14ed}),$$

$GG(a, b, c, d, M13, 5, 0\text{xa9e3e905}),$

$GG(d, a, b, c, M2, 9, 0\text{xfcefa3f8}),$

$GG(c, d, a, b, M7, 14, 0\text{x676f02d9}),$

$GG(b, c, d, a, M12, 20, 0\text{x8d2a4c8a}).$

第三轮的 16 步：

$HH(a, b, c, d, M5, 4, 0\text{xfffa3942})$

$HH(d, a, b, c, M8, 11, 0\text{x8771f681})$

$HH(c, d, a, b, M11, 16, 0\text{x6d9d6122})$

$HH(b, c, d, a, M14, 23, 0\text{xfde5380c})$

$HH(a, b, c, d, M1, 4, 0\text{xa4beea44})$

$HH(d, a, b, c, M4, 11, 0\text{x4bdecfa9})$

$HH(c, d, a, b, M7, 16, 0\text{xf6bb4b60})$

$HH(b, c, d, a, M10, 23, 0\text{xbebfbc70})$

$HH(a, b, c, d, M13, 4, 0\text{x289b7ec6})$

$HH(d, a, b, c, M0, 11, 0\text{xeaa127fa})$

$HH(c, d, a, b, M3, 16, 0\text{xd4ef3085})$

$HH(b, c, d, a, M6, 23, 0\text{x04881d05})$

$HH(a, b, c, d, M9, 4, 0\text{xd9d4d039})$

$HH(d, a, b, c, M12, 11, 0\text{xe6db99e5})$

$HH(c, d, a, b, M15, 16, 0\text{x1fa27cf8})$

$HH(b, c, d, a, M2, 23, 0\text{xc4ac5665})$

第四轮的 16 步：

$II(a, b, c, d, M0, 6, 0\text{xf4292244})$

$II(d, a, b, c, M7, 10, 0\text{x432aff97})$

$II(c, d, a, b, M14, 15, 0\text{xab9423a7})$

$II(b, c, d, a, M5, 21, 0\text{xfc93a039})$

$II(a, b, c, d, M12, 6, 0\text{x655b59c3})$

$II(d, a, b, c, M3, 10, 0\text{x8f0ccc92})$

$II(c, d, a, b, M10, 15, 0\text{xffeff47d})$

$II(b, c, d, a, M1, 21, 0\text{x85845dd1})$

$II(a, b, c, d, M8, 6, 0\text{x6fa87e4f})$

$II(d, a, b, c, M15, 10, 0\text{xfe2ce6e0})$

$II(c, d, a, b, M6, 15, 0\text{xa3014314})$

$II(b, c, d, a, M13, 21, 0\text{x4e0811a1})$

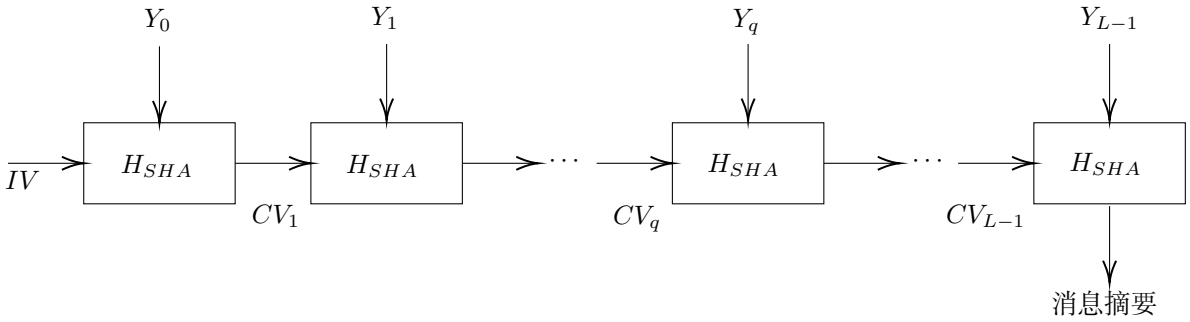


图 6.6: SHA-1 消息处理框图

$$II(a,b,c,d,M4,6,0\text{xf7537e82})$$
$$II(d,a,b,c,M11,10,0\text{xbd3af235})$$
$$II(c,d,a,b,M2,15,0\text{x2ad7d2bb})$$
$$II(b,c,d,a,M9,21,0\text{xeb86d391})$$

6.2.2 安全 Hash 算法

安全 Hash 算法（SHA）是美国国家标准技术研究所 (NIST）设计，并于 1993 年作为联邦信息处理标准（FIPS 180）发布。安全 Hash 标于 1993 年 5 月 11 日正式公布后，NIST 又对其做了一点修改。1995 年 4 月 17 日，NIST 公布了修改后的 SHA 算法，通常称之为 SHA-1。SHA 算法基于 MD4，其结构与 MD4 非常相近。

SHA-1 算法的步骤：SHA-1 算法的输入为小于 2^{64} 比特长的任意消息，输出为 160 比特长的消息摘要，而 MD5 的输入没有长度限制，且输出为 128 比特。与 MD5 算法处理消息的过程一样，SHA-1 算法也将消息按 512 位分组，但 Hash 值的长度和链接变量的长度为 160 比特。图6.6为其消息处理框图。

SHA-1 的算法过程分为如下几步：

(1) 消息填充：与 MD5 的消息填充相同。使得填充后消息的比特长为 512 的某个倍数减 64，同样填充的比特数在 1 到 512 之间。填充的方式也是 100000...0000 的形式。

(2) 附加消息的长度：与 MD5 的第（2）步类似，将填充前消息的长度的二进制表示，填充在第（1）步留出的 64 比特中。第（2）步完成后，消息的长度为 512 的倍数（设倍数为 L），同时消息可表示成分组长为 512 的一系列分组 Y_0，Y_1，...，Y_{L-1}。其消息的填充图和 MD5 的填充图相同。

(3) 初始化 MD 缓冲区：SHA-1 使用 160 比特长的缓冲区存储中间结果和最终 Hash 值，缓冲区可表示为五个 32 比特长的寄存器（A、B、C、D、E），分别将其初始化为 $A=67452301$，$B=$EFCDAB89，C=98BADCFB，$D=10325476$，$E=$C3D2E1F0。其中，前四个值与 MD5 的值相同，但在 SHA-1 中这些值以 big-endian 的方式存储，也就是字的最高有效字节存于低地址字节位置，即按如下方式存储上述的值：

$$A=67452301、B=\text{EFCDAB89}、C=\text{98BADCFB}、D=10325476、E=\text{C3D2E1F0}.$$

(4) 以分组为单位对消息进行处理：每一分组 Y_q 都经过压缩函数处理，压缩函数由 4 轮处理过程构成，如图6.7，每一轮又由 20 步迭代组成。4 轮处理过程结构一样，但所用的基本逻辑函数不同，分别表示为 f_1、f_2、f_3、f_4。

SHA-1 每轮的输入为当前处理的消息分组 Y_q 和缓冲区 A、B、C、D、E 的当前值，输出仍放在缓冲区以替代 A、B、C、D、E 的旧值。每轮处理过程还需加上一个加法常量 K_t，其中 $0\le t\le 79$，

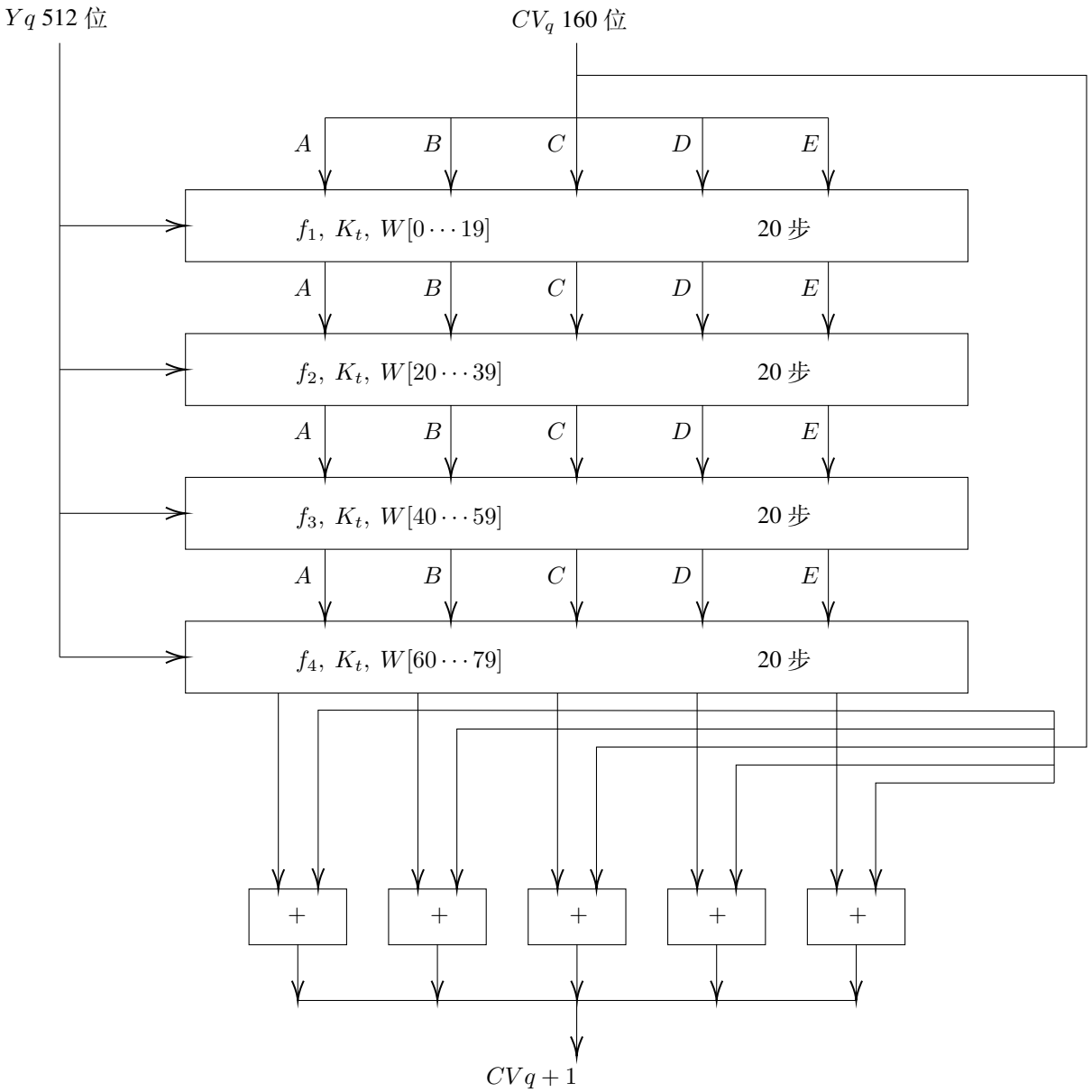


图 6.7: SHA 的分组处理框图

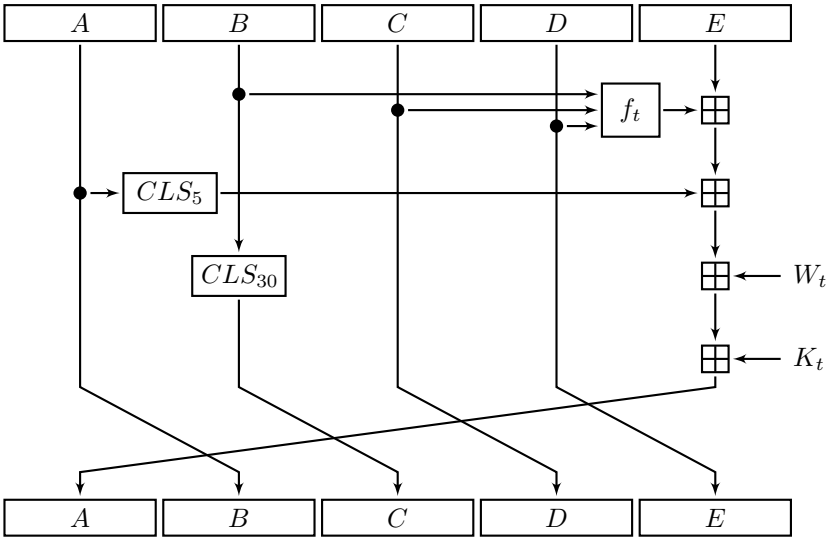


图 6.8: SHA 的压缩函数中一步迭代示意图

t 表示迭代的步数。

第 4 轮的输出（即第 80 步迭代的输出）再与第一轮的输入 CV_q 相加，以产生 CV_{q+1}，其中加法是缓冲区 5 个字中的每一个字与 CV_q 中相应的字模 2^{32} 相加。

(5) 输出消息的 L 个分组都被处理完后，最后一个分组的输出就是 160 比特的消息摘要。

SHA-1 的压缩函数：

SHA 的压缩函数由 4 轮处理过程组成，每轮处理过程又由对缓冲区 $ABCDE$ 的 20 步迭代运算组成，每一步迭代运算的形式如图6.8：

$$A, B, C, D, E \leftarrow (E + f_t(B, C, D) + (A \lll 1), W_t, K_t), A, (B \lll 30), C, D.$$

其中 A、B、C、D、E 为缓冲区的五个字，t 是迭代的步数（$0 \le t \le 79$），f_t 是第 t 步迭代使用的基本逻辑函数，$\lll s$ 为左循环移 s 位，K_t 是加法常量，$+$ 是模 2^{32} 加法，W_t 是由当前 512 比特长的分组导出的一个 32 比特长的字。

基本逻辑函数包括前面提到的函数 f_1，f_2，f_3，f_4 其定义分别为：

$$f_1(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Y),$$
$$f_2(X, Y, Z) = X \oplus Y \oplus Z,$$
$$f_3(X, Y, Z) = (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z),$$
$$f_4(X, Y, Z) = f_2(X, Y, Z) = X \oplus Y \oplus Z.$$

其中：$\wedge$ 是与，$\vee$ 是或，$\oplus$ 是异或，$\neg$ 是反。四轮迭代共需要 80 个常量，实际上只有 4 个不同取值，如表6.3所示。

SHA-1 处理每个 512 的分组都需要 80 个字，前面的 16 个字 W_0，W_1，W_2，...，W_{15}，直接从输入的分组中得到，其余的值由公式 $W_t = (W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \lll 1$ 扩展得到。相比 MD5，其直接使用一个消息分组的 16 个字作为每不迭代的输入，而 SHA-1 则将输入分组的 16 个字扩展成 80 个字以供函数使用，从而使寻找具有相同压缩值得不同消息分组更加困难。

综合上述内容可以将 SHA-1 的处理过程描述为：

第一轮：$0 \le t \le 19$，常数 $K_1 =$ 5a827999，

$$\text{TEMP} = (A \lll 5) + f_1(B, C, D) + E + W_t + K_1,$$
$$E = D,\ D = C,\ C = (B \lll 30),\ B = A,\ A = \text{TEMP};$$

表 6.3: SHA-1 的加法常量

K_1	$0 \le t \le 19$	5a827999
K_2	$20 \le t \le 39$	6ed9eba1
K_3	$40 \le t \le 59$	8f1bbcdc
K_4	$60 \le t \le 79$	ca62c1d6

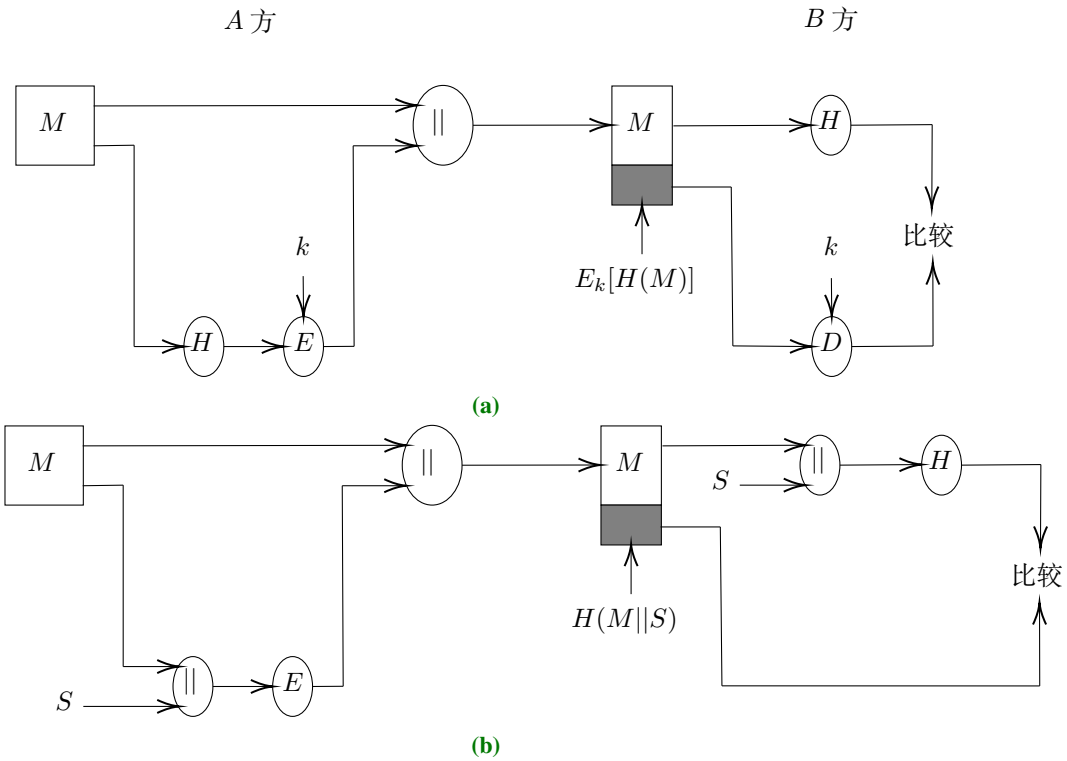


图 6.9: Hash 函数用于消息认证

第二轮：$20 \le t \le 39$，常数 $K_2 =$ 6ed9eba1，

$$\text{TEMP} = (A \lll 5) + f_2(B, C, D) + E + W_t + K_2,$$

$$E = D,\ D = C,\ C = (B \lll 30),\ B = A,\ A = \text{TEMP};$$

第三轮：$40 \le t \le 59$，常数 $K_3 =$ 8f1bbcdc，

$$\text{TEMP} = (A \lll 5) + f_3(B, C, D) + E + W_t + K_3,$$

$$E = D,\ D = C,\ C = (B \lll 30),\ B = A,\ A = \text{TEMP};$$

第四轮：$60 \le t \le 79$，常数 $K_4 =$ ca62c1d6，

$$\text{TEMP} = (A \lll 5) + f_4(B, C, D) + E + W[t] + K_4,$$

$$E = D,\ D = C,\ C = (B \lll 30),\ B = A, A = \text{TEMP}.$$

6.3 Hash 函数的应用举例

Hash 函数可以用于认证，如图6.9所示的两种情况：

(a) 使用分组密码算法仅对 Hash 值进行加密，即 $E_K[H(M)]$ 是变长消息 M 和密钥 K 的函数值，且它是一个定长的输出，对不知道该密钥的攻击者来说是安全的。

(b) 该情况假定通信双方共享一个秘密值 S。发送方 A 对消息 M 和秘密值 S 算出 Hash 值，并将得出

的 Hash 值附加在消息 M 后。因为秘密值本身并不被发送，攻击者无法更改中途截获的消息，也就无法产生假消息。

第 6 章 习题

1. 什么是 Hash 函数？Hash 函数分为哪几类？
2. 不带密钥的 Hash 函数需要的安全性有哪些？两类生日攻击分别对应哪条安全需求？
3. 设 E_K 是一个分组长度为 n 的分组密码的加密算法，密钥为 k。假设密钥的长度也为 n。对于任意消息 x，首先对 x 进行分组，每组的长度为 n。如果 x 的长度不是 n 的倍数，则在 x 的最后适当添加一些数据使得 x 的长度恰好是 n 的倍数。设 $x = x_1x_2 \dots x_l$，其中 $x_i \in GF(2)^n$，$1 \le i \le l$ 最后定义 $H(x) = y_i$。试问按下述两种方式构造的 Hash 函数 H 都是安全的吗？

   (1) $y_i = E_{y_{i-1}}(x_i) \oplus x_i$；

   (2) $y_i = E_{y_{i-1}}(x_i) \oplus x_i \oplus y_{i-1}$。

第 7 章 消息认证码

MAC 全称为 Message Authentication Code（消息认证码），是用来保证数据完整性的一种工具。数据完整性是信息安全的一项基本要求，它可以防止数据未经授权被篡改。随着网络技术的不断进步，尤其是电子商务的不断发展，保证信息的完整性变得越来越重要。特别是双方在一个不安全的信道上通讯的时候，就需要有一种方法能够保证一方所发送的数据能被另一方验证是正确的，即能够防止数据未经授权被篡改。

如果用数学语言来描述，MAC 实质上是一个双方共享的密钥 k 和消息 m 作为输入的函数，如将函数值记为 $\mathrm{MAC}_k(m)$，这个函数值就是一个认证标记，这里用 δ 表示。攻击者发起攻击的时候，能得到的是消息和标记的序列对 $(m_1,\delta_1),(m_2,\delta_2),\ldots,(m_q,\delta_q)$（其中 $\delta_i=\mathrm{MAC}_k(m_i)$）。如果攻击者可以找到一个消息 m，m 不在 $m_1,\ldots,m_q$ 之中，并且能够得到正确的认证标记 $\delta=\mathrm{MAC}_k(m)$ 就说明攻击成功了。攻击者成功的概率就是其攻破 MAC 的概率。

7.1 消息认证码的构造

消息认证码的构造有多种方法，我们主要介绍两种构造方法：一种是基于分组密码的，另一种是基于带密钥的 Hash 函数的。

7.1.1 基于分组密码的 MAC

1. CBC-MAC

   CBC-MAC 是最为广泛使用的消息认证算法之一，同时它也是一个 ANSI 标准（X9.17）。CBC-MAC 实际上就相当于对消息使用 CBC 模式进行加密，取密文的最后一块作为认证码。

   当取 DES 作为加密的分组密码时，称为基于 DES 的 CBC-MAC，若需要产生认证码的消息为 x，加密的 DES 密钥为 k，则生成 MAC 的过程如下（如图7.1所示）：

   (1) 填充和分组。对消息 x 进行填充，将填充得到的消息分成 t 个 n 比特（基于的 DES 的 CBC-MAC 通常 $n=64$）的分组，记为 $x_1,x_2,\ldots,x_t$。

   (2) 密码分组链接。令 E_k 表示以 k 为密钥的加密算法 DES，用以下方式计算 H_i：

   $$H_1 \leftarrow E_k(x_1),$$
   $$H_i \leftarrow E_k(H_{i-1}\oplus x_i),\ 2\le i\le t.$$

   则 H_t 就是 x 的消息认证码，基于的 DES 的 CBC-MAC 的 MAC 为 64 比特。

   CBC-MAC 是一种经典的构造 MAC 的方法，构造方法简单，且底层的加密算法具有黑盒性质，可以方便的进行替换。

2. XOR-MAC

   XOR-MAC 有两种方式：无状态（XMACR）和有状态（XMACC）。这种算法在计算过程中引入索引值使得分组密码每次加密的明文各不相同，最后再将所有的密文异或。具体的构造方法描述如下。假定 $|x|$ 代表消息 x 的长度（即包含多少位），并且它是 32 的倍数。$x=(x_1,x_2,\ldots,x_n)$ 其中 $|x_i|=32$，$i=1,..,n$。假定 n 小于 2^{31}。$<i>$ 是数字 i 的长度为 b 的二进制表示，代表块的索引号。发送者维持一个长度为 63 位的记数 r，在 XMACC 模式下它的初始值为 0，每次增加 1。在 XMACR 模式下，r 是随机选取的一个长度为 63 位的串。他们的具体构造方式如算法7.1和算法7.2所示：

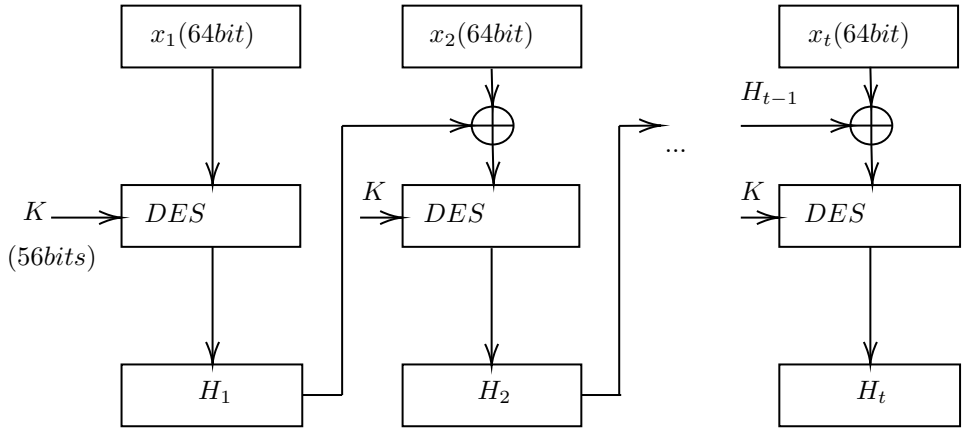


图 7.1: CBC-MAC 的算法过程

算法 7.1： XMACR

Function XMACR(x, k)**:**

填充 x

$r \xleftarrow{R} \{0,1\}^{63}$

$y_0 = F_k(0||r)$

将 x 分割为 $x_1, x_2, \ldots, x_n$

for $i = 1$ *to* n **do**

  $y_i = y_{i-1} \oplus F_k(1 \,||< i >|| \, x_i)$

返回 $(r,\ y_n)$

算法 7.2： XMACC

Function XMACC(x, k)**:**

填充 x

$ctr \leftarrow \text{ctr} + 1$

$y_0 = F_k(0||ctr)$

将 x 分割为 $x_1, x_2, \ldots, x_n$

for $i = 1$ *to* n **do**

  $y_i = y_{i-1} \oplus F_k(1 \,||< i >|| \, x_i)$

返回 $(ctr,\ y_n)$

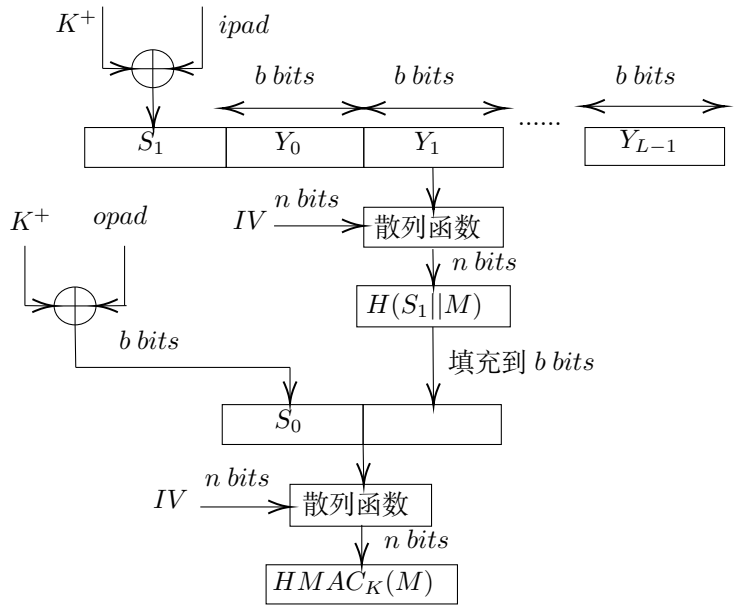


图 7.2: HAMC 的结构

7.1.2 基于带密钥的 Hash 函数的 MAC

前面介绍的是两种基于分组密码的消息认证码算法。下面给出一个由 Hash 函数导出的 MAC。

可以将 HMAC 表示如下：

$$\text{HMACK} = H\left[\left(K^+ \oplus \text{opad}\right) || H\left[\left(K^+ \oplus \text{ipad}\right) || M\right]\right].$$

HAMC 算法过程如图7.2所示。

(1) 对密钥 K 的左端填充一些 0 生成一个 b 比特的串 K^+（例如，如果 K 的长度是 160 比特，而 $b = 512$，那么对 K 填充 44 个 0 字节 0x00）；

(2) 将 K^+ 与 ipad 按比特异或产生一个 b 比特的分组 S_i；

(3) 将消息 M 附加到 S_i 后；

(4) 使用 H 计算第 3 步产生结果的散列值；

(5) 将 K^+ 与 opad 按比特异或产生一个 b 比特的分组 S_0；

(6) 将第 4 步产生的散列值附加到 S_0 后面；

(7) 使用 H 计算第 6 步产生结果的散列值，并输出这个结果。

其中：

H =嵌入的散列函数（如 MD5，SHA-1）；

M =$HMAC$的输入消息（包括嵌入散列函数所需的填充比特）；

Y_i =M中的第i个分组，$0 \le i \le L-1$；

L =M中的分组数；

b =一个分组的比特数；

n =嵌入散列函数产生的散列码长度；

K =密钥；如果密钥的长度大于b，该密钥输入散列函数产生一个n 比特的密钥；

一般情况下密钥长度大于等于n；

K^+ =在K 的左边填充0，使总长度等于b；

ipad =将00110110 重复$b/8$ 次；

opad =将01011010 重复$b/8$ 次。

7.2 MAC 函数的安全性

当使用加密算法加密消息时，无论采用对称加密或不对称加密，其安全性一般依赖于密钥的比特长度。排除加密算法可能有的弱点，攻击者可以使用穷举攻击法来尝试所有可能的密钥。一般情况下，对 k 比特长的密钥，穷绝攻击需要 $2^{(k-1)}$ 次尝试。特别是对于惟密文攻击（ciphertext-only attack），攻击者需要通过获得的密文 C，尝试所有可能的密钥，直到产生适当的明文为止。

对于 MAC，情况则完全不同。MAC 函数一般是一个多对一的函数。攻击者如何用穷举攻击法获得密钥呢？MAC 函数域由任意长度的消息组成，其中域值由所有可能的 MAC 和所有可能的密钥组成。如果使用一个长度为 n 比特的 MAC，那么将有 2^n 个可能的 MAC，而可能有 N 个消息，其中 $N \gg 2^n$。此外，对一个长度为 k 的密钥，还将有 2^k 个可能的密钥。

如果不考虑保密性，就是假设对手已获得消息的明文和相应的 MAC。假定 $k > n$，即密钥长度大于 MAC 长度。那么如果已知 M_1 和 MAC_1，$\text{MAC}_1 = C_{k_1}(M_1)$。密码分析者必须对所有可能的密钥值 K_i 执行 $\text{MAC}_i = C_{k_i}(M_i)$。当 $\text{MAC}_i = \text{MAC}_1$ 时，则至少找到一个密钥。注意，由于总共将产生 2^k 个 MAC，但只有 2^n 个不同的 MAC 值。因此，许多密钥能产生同一个正确的 MAC，但攻击者却无法确认真正的正确密钥。平均说来，总共 $2^k/2^n = 2^{(k-n)}$ 个密钥将产生一个正确的 MAC。因此攻击者必须重复这样的攻击：

第 1 轮：已知 M_1, MAC_1，其中 $\text{MAC}_1 = C_k(M_1)$，对所有可能的密钥计算 $MAC_i = C_{k_i}(M_1)$，将得到 $2^{(k-n)}$ 个可能的密钥。

第 2 轮：已知 $M_2, \text{MAC}_2 = C_k(M_2)$，对上一轮得到的 $2^{(k-n)}$ 个密钥计算 $MAC_i = C_{ki}(M_2)$，得到 $2^{(k-2n)}$ 个可能的密钥。

如此下去，如果 $k = an$，那么需要重复进行 a 轮。例如，如果使用 80bit 的密钥和产生 32bit 的 MAC，那么第 1 轮将产生 2^{48} 可能的密钥。第 2 轮将可能的密钥缩减为 2^{16}。第 3 轮仅产生惟一的一个密钥，一定是发送方使用的那个密钥。

如果密钥的长度小于或等于 MAC 的长度。那么在第 1 轮中就有可能找到正确的密钥，也有能找出多个可能的密钥，如果是后者，则需要执行第 2 轮的搜索。

所以对消息认证码的穷举搜索攻击比对使用相同长度密钥的加密算法的穷举搜索更困难，然而有些攻击方法却不需要寻找产生 MAC 所使用的密钥。

例 7.1 令 $M = (X_1 \,||\, X_2 \,||\, \cdots \,||\, X_m)$ 为由 64bit 分组 X_i 串接而得到的消息。其消息认证码由以下方式得到：

$$\Delta(M) = X_1 \oplus X_2 \oplus \cdots \oplus X_m,$$
$$C_k(M) = E_k[\Delta(M)],$$

其中 $\oplus$ 是异或运算符。而加密算法采用电子密码本模式的 DES 算法。因此，密钥长度是 56 比特，MAC 长度是 64 比特。如果攻击者能窃取到 $\{M \,||\, C_k(M)\}$，通过穷举攻击来确定 K 将至少需要 2^{56} 次加密。但攻击者也可以用以下方式攻击系统：通过用任意的值 Y_1 到 Y_{m-1} 来替代从 X_1 到 X_{m-1} 的值，并用 Y_m 来替代 X_m，其中 Y_m 计算如下：

$$Y_m = Y_1 \oplus Y_2 \oplus \cdots \oplus Y_{m-1} \oplus \Delta(M).$$

现在，攻击者可以成功伪造一个新的消息 $M' = Y_1 \oplus Y_2 \oplus \cdots \oplus Y_{m-1} \oplus Y_m$，$M'$ 的 MAC 与原消息的 MAC 相同。

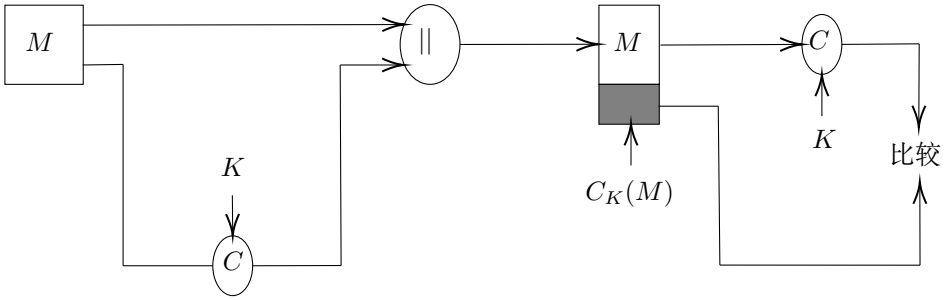


图 7.3: 消息认证码实现认证

考虑能遇到的攻击的类型，MAC 函数必须满足以下要求。其中，假定对手知道 MAC 函数 C 但不知道 K：

(1) 如果攻击者得到 M 和 $C_k(M)$，构造一个消息 M'，使得 $C_k(M') = C_k(M)$，在计算上是不可行的。

(2) $C_k(M)$ 应该在以下意义下均匀分布。随机选择消息 M 和 M'，$C_k(M') = C_k(M)$ 的概率为 2^{-n}，其中 n 为 MAC 的比特长度。

(3) 若 M' 为 M 的某种已知变换，即 $M' = f(M)$，（例如，f 可能为将一个或多个特定的比特取反）那么，$p[C_k(M) = C_k(M')] = 2^{-n}$。

第 1 个需求是针对上述的例子，即在攻击者不知道密钥的情况下，而伪造一个新消息与截获的 MAC 匹配在计算上是不可行的。

第 2 个需求是为了阻止基于选择明文穷举攻击的情况：假定攻击者不知道密钥 K，但能获得 MAC 函数，能对消息产生 MAC。那么攻击者可以对各种消息计算 MAC，知道找到与给定 MAC 相同的消息位置。如果 MAC 函数呈现均匀分布，那么用穷举攻击方法平均需要 $2^{(n-1)}$ 次尝试才能找到匹配给定 MAC 的消息。

第 3 个需求说明消息认证算法对消息的特定部分或比特不应该比别的更脆弱。否则攻击者获得 M 和 MAC 后又可能修改 M 中较弱的部分，从而伪造出一个与原 MAC 匹配的新消息。

7.3 消息认证码的应用

假定通信的双方为 A 和 B，有一个共享密钥 K。当 A 有要发往 B 的消息时，发送方 A 计算 MAC（MAC 是待发送的消息和密钥 K 的一个函数值，MAC $= C_k(M)$），然后将消息和 MAC 一起发往预定的接收者 B。在接收端 B，使用相同的密钥 K 对收到的消息执行相同的计算并得出新的 MAC，将收到的 MAC 与计算得出的 MAC 进行比较，如图7.3所示。如果假定只有接收方和发送方知道密钥 K，同时如果收到的 MAC 与计算得出的 MAC 匹配，那么：

(1) 接收者确信消息未被更改过。如果一个攻击者更改消息而未更改 MAC，那么接收者计算出的 MAC 将不同于接收到的 MAC。因为假定攻击者不知道该密钥，因此攻击者不可能更改 MAC 来对应更改后的消息。

(2) 接收者确信消息来自所谓的发送者。因为没有其他人知道该密钥，因此没有人能够为一个消息准备合适的 MAC。

第 7 章 习题

1. 简述消息认证码有哪些应用?
2. MAC 函数的安全性需求有那些?

3. 用 “MAC 和加密” 实现完整性时，如果 E_k 和 $h_k(x)$ 使用相同的密钥 k，即令 E_k 表示以 k 为密钥的 CBC 加密，$h_k(x)$ 表示以 k 为密钥的 CBC-MAC。对数据 $x = x_1, x_2, \ldots, x_t$，密文 $C = E_k(x \mid\mid h_k(x))$，这种搭配的完整性机制是不可靠，请说明原因。
4. 可以不用 CBC 模式，而用分组密码的 CFB 模式来产生一个消息认证码。给定系列明文分组 x_1，x_2，...，x_n，假定我们定义初始向量 IV 为 x_1。在 CFB 模式中用密钥 K 加密序列 x_2，x_3，...，x_n，得到密文序列 y_1，y_2，...，y_{n-1}（注意只有 $n-1$ 个密文分组）。最后定义 $e_k(y_{n-1})$。证明：该 MAC 与 CBC 模式的 MAC 相同。

第 8 章　公钥密码

几千年来密码通信一直沿用的传统密码体制在加密和解密时，使用的是同一个密钥，或者虽然使用不同的密钥，但是能通过加密密钥方便地导出解密密钥。这类密码体制也叫做对称密码体制，或单密钥密码体制。在这类密码体制中，加密密钥是整个密码通信系统的核心机密，一旦加密密钥被暴露，整个密码体制也就失去了保密作用。

随着信息加密技术应用领域的扩大，尤其是从单纯的军事外交情报领域的使用，扩大到民用领域（商业、金融、计算机通信网络中的信息保密等），传统密码体制在应用中暴露出越来越多的缺陷。

(1) 密钥分配和管理的复杂性：传统密码体制要求双方用相同的密钥要通过秘密信道传输，在一个有 n 个实体的网络中，每个都需要保存其余的 $n-1$ 个实体的密钥，这就需要 $n(n-1)/2$ 个密钥和同样数目的保密信道。而当一个用户要加入到通信网络或一个用户要改变他的密钥时，n 或 $n-1$ 个新的密钥必须通过同样数目的信道进行传输，这样密钥的分配、传送和管理就非常困难。

(2) 签名和认证问题：传统的私钥密码系统可以对数据进行加解密处理，提供数据的机密性，但因为由两个实体共享密钥，所以收方可以伪造原文，发放也可以否认，这使得在没有仲裁者的情况下不能直接提供认证和数字签名服务。而在网络通信发展中，认证和签名的需求是必不可少的，这就限制了传统密码体制的应用范围。

为了对付密钥失窃所带来的损失和危害，最好采用一次一密的密码体制，即每次通信都采用不同的密钥。这要求密钥的分配迅速、保密和经济，而传统密码体制是做不到这一点的。为此，人们希望能设计一种新的密码，从根本上解决传统密码所不能解决的问题，从而适合计算机网络环境的各种应用。

8.1　公钥密码的基本概念

上世纪 70 年代，美国学者 Diffie 和 Hellman，以及以色列学者 Merkle 分别独立地提出了一种全新的密码体制的概念。Diffie 和 Hellman 首先将这个概念公布在 1976 年美国国家计算机会议上，几个月后，他们这篇开创性的论文《密码学的新方向》（“New Directions in Cryptography”）发表在 IEEE 杂志信息论卷上，由于印刷原因，Merk1e 对这一领域的首次贡献直到 1978 年才出版。他们所创造的新的密码学理论，突破了传统密码体制对称密钥的概念，树起了近代密码学的又一大里程碑。

不同于以前采用相同加密和解密密钥的对称密码体制，Diffie 和 Hellman 提出采用双钥体制的每个用户都有一对选定的密钥：一个可以是公开的，另一个则是秘密的。公开的密钥可以象电话号码一样公布，因此称为公钥密码体制（public key cryptosystem，PKC）或双钥体制（two-key cryptosystem）。

8.1.1　公钥密码体制的原理

公钥密码体制的主要特点是将加密和解密的能力分开，因而可以实现多个用户的的信息只能由一个用户解读；或只能由一个用户加密消息而由多个用户解读。前者可用于公共网络中实现保密通信，而后者可以用于认证系统中对消息进行数字签名。

公开密钥密码的基本思想是将传统密码的密钥一分为二，分为加密钥 K_e 和解密钥 K_d，用加密钥 K_e 控制加密，用解密钥 K_d 控制解密。而且由计算复杂性确保由加密钥 K_e 在计算上不能推出解密钥 K_d。这样，即使是将 K_e 公开也不会暴露 K_d，也不会损害密码的安全。于是便可将 K_e 公开，而只对 K_d 保密。由于 K_e 是公开的，只有 K_d 是保密的，所以从根本上克服了传统密码在密钥分配上的困难。

根据公开密钥密码的基本思想，可知一个公开密钥密码应当满足下面三个条件：

(1) 解密算法 D 与加密算法 E 互逆，即对于所有明文 M 都有，$D(E(M,K_e),\ K_d)=M$；

(2) 在计算上不能由 K_e 求出 K_d；

(3) 算法 E 和 D 都是高效的。

条件 (1) 是构成密码的基本条件，是传统密码和公开密钥密码都必须具备的起码条件。

条件 (2) 是公开密钥密码的安全条件，是公开密钥密码的安全基础，而且这一条件是最难满足的。由于数学水平的限制，目前尚不能从数学上证明一个公开密钥密码完全满足这一条件，而只能证明它不满足这一条件。这就是这一条件困难的根本原因。

条件 (3) 是公开密钥密码的工程实用条件。因为只有算法 E 和 D 都是高效的，密码才能实际应用。否则，可能只有理论意义，而不能实际应用。

满足了以上三个条件，便可构成一个公开密钥密码，这个密码可以确保数据的秘密性。然而，如果还要求确保数据的真实性，则还应满足第四个条件。

(4) 对于所有明文 M 都有，$E(D(M, K_d), K_e) = M$。

条件 (4) 是公开密钥密码能够确保数据真实性的基本条件。如果满足了条件 (1)、(2)、(4)，同样可构成一个公开密钥密码，这个密码可以确保数据的真实性。

如果同时满足以上四个条件，则公开密钥密码可以同时确保数据的秘密性和真实性。此时，对于所有的明文 M 都有，$D(E(M, K_e), K_d) = E(D(M, K_d), K_e) = M$。

公开密钥密码从根本上克服了传统密码在密钥分配上的困难，利用公开密钥密码进行保密通信需要成立一个密钥管理结构（KMC），每个用户都将自己的姓名、地址和公开的加密钥等信息在 KMC 登记注册，将公钥记入共享的公开密钥数据库 PKDD。KMC 负责密钥的管理，并且对用户是可信赖的。这样，用户利用公开密钥密码进行保密通信就像查电话号码簿打电话一样方便，再无通信双方预约密钥之苦，因此特别适合计算机网络应用。而且公开密钥密码实现数字签名容易，所以特别受欢迎。

公钥密码算法的最大特点是采用两个相关密钥将加密和解密分开，其中一个密钥是公开的，称为公开密钥，简称公开钥，用于加密；另一个密钥是为用户专用，因而是保密的，称为秘密密钥，简称秘密钥，用于解密。因此公钥密码体制也称为双钥密码体制，它有一个很重要的特性，即已知密码算法和加密密钥，求解密密钥在计算上是不可行的。

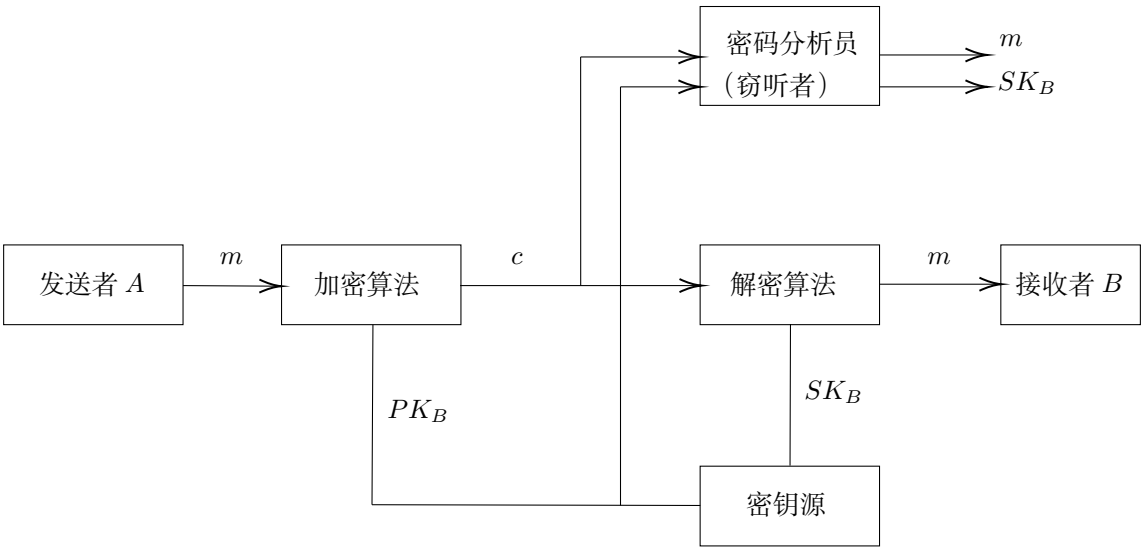


图 8.1: 公钥密码算法

图8.1是公钥体制的框图，主要分为以下几步：

(1) 网络中要求接收消息的端系统，产生一对用来加密和解密的密钥，如图中的接收者 B，产生一对密钥 PK_B ，SK_B，其中 PK_B 是公开钥，SK_B 是秘密钥。

(2) 端系统 B 将加密密钥（图中的 PK_B）在一个公开的寄存器或文件中，另一密钥则被保密（图中的 SK_B）。

(3) A 要想向 B 发送消息 m，则使用 B 的公开钥加密 m，表示为

$$c = E_{PK_B}(m),$$

其中 c 是密文，E 是加密算法。

(4) B 收到密文 c 后，用自己的秘密钥 SK_B 解密，表示为

$$m = D_{SK_B}(c),$$

其中 D 是解密算法。

因为只有 B 知道 SK_B，所以其他人无法对 c 解密。

8.1.2 公钥密码算法应满足的要求

公钥密码算法应满足以下要求：

(1) 接收方 B 产生密钥对（公开钥 PK_B 和秘密钥 SK_B）是计算上容易的。

(2) 发方 A 用收方的公开钥对消息 m 加密以产生密文 c，即 $c = E_{PK_B}(m)$ 在计算上是容易的。

(3) 收方 B 用自己的秘密钥对 c 解密，即 $m = D_{SK_B}(c)$ 在计算上是容易的。

(4) 敌手由 B 的公开钥 PK_B 求秘密钥 SK_B 在计算上是不可行的。

(5) 敌手由密文 c 和 B 的公开钥 PK_B 恢复明文则在计算上是不可行的。

(6) 加、解密次序可换，即

$$E_{PK_B}\left(D_{SK_B}(m)\right) = D_{SK_B}\left(E_{PK_B}(m)\right).$$

这一点虽然非常有用，但不是对所有的算法都作要求。

以上要求的本质在于要求一个单向陷门函数。单向陷门函数是两个集合 X、Y 之间的一个映射，使得 Y 中每一元素 y 都有惟一的一个原像 $x \in X$，且由 x 易于计算它的像 y，由 y 计算它的原像 x 是不可行的。例如，函数的输入是 n 比特，如果求函数值所用的时间是 2^n 的某个倍数，则认为求函数值是不可行的。注意这里的易于计算和不可行两个概念与计算复杂性理论中复杂度的概念极为相似，然而又存在看本质的区别。在复杂性理论中，算法的复杂度是以算法在最坏情况或平均情况时的复杂度来度量的。而在此所说的两个概念是指算法在几乎所有情况下的情形。我们说一个函数是单向陷门函数，是指该函数是易于计算的，但求它的逆是不可行的，除非再已知某些附加信息。当附加信息给定后，求逆可在多项式时间完成。总的来说，单向陷门函数是一组可逆函数 f_k，它满足以下三点要求：

(1) $Y = f_k(X)$ 易于计算（当 k 和 X 已知时）；

(2) $X = {f_k}^{-1}(Y)$ 易于计算（当 k 和 Y 已知时）；

(3) $X = {f_k}^{-1}(Y)$ 计算上是不可行的（当 Y 已知但 k 未知时）。

因此，研究公钥密码算法实际上就是要找出合适的单向门陷函数。

8.1.3 对公钥密码的攻击

和单钥密码体制一样，如果密钥太短，公钥密码也易受到穷举搜索攻击。因此密钥必须足够长才能抗击穷举搜索攻击。然而又由于公钥密码所使用的可逆函数的计算复杂性与密钥长度常常不是呈线性关系，而是增大得更快。所以密钥长度太大又会使得加解密运算太慢而不实用。因此公钥密码目前主要用于密钥管理和数字签名。

对公钥密码算法的第二种攻击按是从寻找公开钥计算秘密钥的方法。目前为止，对常用公钥算法还都未能够证明这种攻击是不可行的。

还有一种仅适用于对公钥密码算法的攻击法，称为可能字攻击。例如对 56 比特的 DES 密钥用公钥密码算法加密后发送，敌手用算法的公开钥对所有可能的密钥加密后与截获的密文相比较。如果一样，则相应的明文即 DES 密钥就被找出。因此不管公钥算法的密钥多长，这种攻击的本质是对 56 比特 DES 密钥的穷举搜索攻击。抵抗方法是在欲发送的明文消息后添加一些随机比特。

8.2 RSA 密码体制

RSA 算法是 1978 年由美国麻省理工学院的三名密码学者 R. Rivest，A. Shamir 和 L. Adleman 提出的一种用数论构造的、基于大合数因子分解困难性的公开密钥密码，也是迄今为止理论上最为成熟完善的公钥密码，简称为 RSA 密码，该体制已得到广泛应用。由于 RSA 密码既可用于加密，又可用于数字签名，安全、易懂，因此 RSA 密码已成为目前应用最广泛的公开密钥密码。

8.2.1 加密算法描述

1. 密钥的产生
   (1) 选两个保密的大素数 p 和 q；
   (2) 计算 $n=p\times q$，$\varphi(n)=(p-1)(q-1)$，其中 $\varphi(n)$ 是 n 的欧拉函数值；
   (3) 选一整数 e，满足 $1<e<\varphi(n)$，且 $\gcd(\varphi(n),e)=1$；
   (4) 计算 d，满足 $de\equiv 1 \bmod \varphi(n)$。即 d 是 e 在模 $\varphi(n)$ 下的乘法逆元，因 e 与 $\varphi(n)$ 互素，由模运算可知，它的乘法逆元一定存在；
   (5) 以 $\{e,n\}$ 为公钥，$\{d,p,q\}$ 为私钥。
2. 加密
   加密时首先将明文比特串分组，使得每个分组对应的十进制数小于 n，即分组长度小于 $\log_2 n$。然后对每个明文分组 m，作加密运算：$c=m^e \bmod n$。
3. 解密
   对密文分组的解密运算为：$m=c^d \bmod n$。

下面对 RSA 算法中解密过程的正确性进行证明。

证明

由加密过程知 $c=m^e \bmod n$，所以

$$c^d \bmod n \equiv m^{ed} \bmod n \equiv m^{1 \bmod \varphi(n)} \bmod n \equiv m^{k\varphi(n)+1} \bmod n.$$

不妨分两种情况讨论：

(1) m 与 n 互素，则由 Euler 定理：$m^{\varphi(n)}\equiv 1 \bmod n$，$m^{k\varphi(n)}\equiv 1 \bmod n$，$m^{k\varphi(n)+1}\equiv m \bmod n$，即 $c^d \bmod n \equiv m$。

(2) $\gcd(m,n)\neq 1$，我们先看 $\gcd(m,n)=1$ 的含义，由于 $n=p\times q$，所以 $\gcd(m,M)=1$ 意味着 m 不是 p 的倍数也不是 q 的倍数。因此 $\gcd(m,n)\neq 1$ 意味着 m 是 p 的倍数或 q 的倍数，不妨设 $m=cp$，其中 c 为一正整数。此时必有 $\gcd(m,q)=1$，否则 m 也是 q 的倍数，从而是 pq 的倍数，与 $m<n=pq$ 矛盾。

根据 $\gcd(m,q)=1$ 以及 Euler 定理可以得到 $m^{\varphi(q)}\equiv 1 \bmod q$，所以 $m^{k\varphi(q)}\equiv 1 \bmod q$，$(m^{k\varphi(q)})^{\varphi(p)}\equiv 1 \bmod q$，$m^{k\varphi(n)}\equiv 1 \bmod q$，因此存在一整数 r，使得 $m^{k\varphi(n)}=1+rq$，两边同乘以 $m=cp$ 得 $m^{k\varphi(n)}=m+rcpq=m+rcn$，即 $m^{k\varphi(n)+1}\equiv m \bmod n$，所以 $c^d \bmod n\equiv m$。

例 8.1 设 $p=7,q=17,n=p\times q,\varphi(n)=(p-1)(q-1)=96$，验证 RSA 解密算法。

解 取 $e=5$，满足 $1<e<\varphi(n)$，且 $\gcd(\varphi(n),e)=1$。确定 $d\cdot e=1 \bmod 96$ 且小于 96 的 d，因为 $77\times 5=385=4\times 96+1$，所以 d 为 77，因此公开钥为 $\{5,119\}$，秘密钥为 $\{77,119\}$。设明文 $m=19$，

则由加密过程得密文为：

$$c = 19^5 \bmod 119 = 2476099 \bmod 119 = 66,$$

解密为：

$$66^{77} \bmod 119 = 19.$$

例 8.2 设 $p = 43$，$q = 59$，$N = pq = 43 \times 59 = 2537$，$\varphi(N) = 42 \times 58 = 2436$，取 $e = 13$，求 e 的逆元 d。

解 解方程：$de \equiv 1 \bmod 2436$，因为

$$2436 = 13 \times 187 + 5,\ 13 = 2 \times 5 + 3,$$
$$5 = 3 + 2,\ 3 = 2 + 1,$$

所以

$$1 = 3 - 2,\ 2 = 5 - 3,\ 3 = 13 - 2 \times 5,$$
$$5 = 2436 - 13 \times 187,$$

即

$$\begin{aligned} 1 &= 3 - 2 = 3 - (5 - 3) = 2 \times 3 - 5 \\ &= 2(13 - 2 \times 5) - 5 = 2 \times 13 - 5 \times 5 \\ &= 2 \times 13 - 5 \times 2436, \end{aligned}$$

即 $937 \times 13 \equiv 1 \bmod 2436$，当 $e = 13$ 时，$d = 937$。

若有明文 public key encryptions，将明文分块为

$$\text{pu}\quad \text{bl}\quad \text{ic}\quad \text{ke}\quad \text{ye}\quad \text{nc}\quad \text{ry}\quad \text{pt}\quad \text{io}\quad \text{ns},$$

再将明文数字化（令 a，b，...，z 分别为 00，01，...，25）得

$$\begin{array}{ccccc} 1520 & 0111 & 0802 & 1004 & 2404 \\ 1302 & 1724 & 1519 & 0814 & 1418, \end{array}$$

利用加密可得密文

$$\begin{array}{ccccc} 0095 & 1648 & 1410 & 1299 & 1365 \\ 1379 & 2333 & 2132 & 1751 & 1289. \end{array}$$

RSA 体制不仅能实现保密通信还能实现数字签名，因而特别适用于现代密码通信的要求。在众多的公钥密码中，RSA 公钥密码被认为是比较完善的一个。自从它问世至今的 20 余年中，尽管有许多密码学家对它进行了长期而深入的分析，但是一直没有发现它有明显的脆弱性，所以 RSA 公钥密码至少有以下一些优点：

(1) 数学表达式简单，在公钥密码中是最容易理解和实现的一个，这个体制也是目前国际上比较流行的公钥密码之一。

(2) RSA 的安全性基于大数分解的困难性，到目前为止除了大数分解之外，人们还没有发现一种其它的方法能够对 RSA 进行有效的密码分析。虽然 RSA 也有一些弱点，但是只要设计密码参数时仔细一点这些弱点是可以避免的，所以说 RSA 公钥密码的安全性比较高。

(3) RSA 公钥密码具有一些传统密码体制不能实现的一些功能，如认证鉴别和数字签名等，特别适合于现代密码通信。

8.2.2 RSA 算法中的计算问题

1. 加解密

RSA 的加、解密过程都为求一个整数的整数次幂，再取模。如果按其含义直接计算，则中间结果非常大，有可能超出计算机所允许的整数取值范围。如上例中解密运算 $66^{77} \bmod 119$，先求 66^{77} 再取模，则中间结果就已远远超出了计算机允许的整数取值范围。而用模运算的性质：

$$(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n,$$

就可减小中间结果。

再者，考虑如何提高加、解密运算中指数运算的有效性。例如求 x^{16}，直接计算的话需做 15 次乘法，即 $x^{16} = x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x \times x$。然而如果重复对每个部分结果做平方运算即求 x, x^2，x^4，x^8，x^{16} 则只需 4 次乘法。

一般，求 a^m 可按照下面的算法进行，其中 a、m 是正整数。

将 m 表示为二进制形式 $b_k b_{k-1} \dots b_n$，即

$$m = \sum_{b_i \neq 0} 2^i,$$

因此

$$a^m = a^{\sum_{b_i \neq 0} 2^i} = \prod_{b_i \neq 0} a^{2^i},$$

$$a^m \bmod n = \left[\prod_{b_i \neq 0} a^{2^i}\right] \bmod n = \prod_{b_i \neq 0} \left[a^{2^i} \bmod n\right].$$

从而可得以下快速指数算法 (算法8.1)。其中 d 是中间结果，d 的终值即为所求结果。c 在这里的作

算法 8.1: 快速指数算法

```
c = 0; d = 1;
for i = k down to 0 do
    c = 2 × c;
    d = (d × d) mod n;
    if b_i = 1 then
        c = c + 1;
        d = (d × a) mod n;
    end
end
返回 d;
```

用是表示指数的部分结果，终值即为指数 m，c 对计算结果无任何贡献，算法中完全可将之去掉。

例 8.3 求 $7^{560} \bmod 561$。

解 将 560 表示为 1000110000，算法的中间结果如表8.1所示。

i	9	8	7	6	5	4	3	2	1	0	
b_i	1	0	0	0	1	1	0	0	0	0	
c	0	1	2	4	8	17	35	70	140	280	560
d	1	7	49	157	527	526	160	241	298	166	1

表 8.1: 快速指数算法示例

所以 $7^{560} \bmod 561 = 1$。

2. 密钥的产生

   产生密钥时，需要考虑两个大素数 p、q 的选取，以及 e 的选取和 d 的计算。因为 $n = pq$ 在体制中是公开的，因此为了防止敌手通过穷搜索发现 p、q 这两个素数，应在一个足够大的整数集合中选取大数。因此如何有效地寻找大素数是第一个需要解决的问题。

   寻找大素数时一般是先随机选取一个大的奇数（例如用伪随机数产生器），然后用素性检验算法检验这一奇数是否为素数，如果不是则选取另一大奇数，重复这一过程，直到找到素数为止。素性检验算法通常都是概率性的，但如果算法被多次重复执行，每次执行时输入不同的参数，算法的检验结果都认为被检验的数是素数，那么我们就可以比较有把握地认为被检验的数是素数。例如 Miller-Rabin 算法。可见寻找大素数是个比较繁琐的工作。然而在 RSA 体制中，只有在产生新密钥时才需要执行这一工作。

   p 和 q 决定出后，下一个需要解决的问题是如何选取满足 1<e<$\varphi(n)$ 和 $\gcd(\varphi(n), e) = 1$ 的 e，并计算满足 $d \cdot e \equiv 1 \bmod \varphi(n)$ 的 d。这一问题可由推广的 Euclid 算法完成。

8.2.3 对 RSA 的攻击

对密码分析者来说攻击 RSA 体制最显而易见的方法就是分解模数 N，因为一旦求出 N 的两个素因子 p 和 q, N 的欧拉数 $\varphi(N) = (p-1)(q-1)$ 立即可得，再求出以 $\varphi(N)$ 为模的公钥 e 的乘逆 d，从而破译 RSA 的秘密密钥。

但是分解大合数是众所周知的数学难题。近 300 多年来许多著名的数学家对这个问题进行了大量研究。虽然近代科学家借助电子计算机和先进的科学技术在这方面取得了很大的进展，数学家已经成功分解了 150 位十进制的大合数，但是至今为止，要分解一个 200 位的大合数还是无能为力的。

1. 公共模数攻击

   如果有一种方法能够不分解 N 而方便地计算出 $\varphi(N)$，则 RSA 体制可轻而易举地被攻破。不过目前还没有什么人能够证明直接计算 $\varphi(N)$ 比分解 N 更容易。从另一个方面也说明为什么 RSA 密码体制的模数 N 必须选择合数而不能选择素数。因为如果模 N 选择素数那么公布了 N 也就公布了 $\varphi(N)$，从 $\varphi(N)$ 和公开钥 e 可以很容易地推导出秘密密钥 d。

   既然秘密密钥 d 不能推导出来，那么它能不能直接计算出来呢？最显而易见的方法当然是穷举搜索法。但是只要密码设计者仔细选取 d，并使它本身足够大就可以挫败密码分析者的穷举搜索攻击。

   如果密码分析者知道了 d，可以方便地按下述方法实现对 N 的分解。首先计算出 $(ed-1)$，由 RSA 体制的协议可知，$(ed-1)$ 必然是 $\varphi(N)$ 的一个倍数。密码学家 Miller 指出，利用 $\varphi(N)$ 的任何倍数都能成功实现对 N 的分解。但是到目前为止，还没有人能够提出这类对 N 进行因子分解的实际方法。因此可以断言，密码分析者不可能用比分解 N 更容易的方法直接计算秘密密钥，否则大合数分解的难题就不存在了。

   但是这并不意味着密码分析者不能通过其它途径来破译 RSA 体制。美国学者 Simmons 指出 RSA 公钥密码存在着一种潜在的弱点，这就是幂剩余变换的周期性。利用这种特殊的周期性，可以不必破译秘密密钥而直接得到明文。其方法如下：

   设 m 是待加密的明文，(N, e) 是 RSA 密码体制的公开密钥，则密文为 $c = m^e \bmod N$。密码分析者从不保密信道上得到密文 c 后，并不设法获取秘密密钥 d，而是利用公开密钥对密文 c 依次进行如

下变换：

$$\begin{aligned} c^e &= c_1 \quad \bmod N, \\ c_1^e &= c_2 \quad \bmod N, \\ &\cdots\cdots \\ c_{k-1}^e &= c_k \quad \bmod N, \\ c_k^e &= c_{k+1} \quad \bmod N. \end{aligned}$$

上述一组变换经过 $k+1$ 步迭代之后，幂剩余变换的结果恰好等于密文 c。对照式 $c = m^e \bmod N$ 可知第 k 步的变换结果 c_k 就是明文 m。这样利用众所周知的公开密钥 (N, e) 和截获的密文成功地破译了 RSA 公开密钥体制。

例 8.4 设 RSA 公开密钥体制的一组参数为 $p = 383$，$q = 563$，$N = pq = 215629$，$e = 49$，$d = 56957$，加密和解密变换式分别为：

$$\begin{aligned} m^{49} &= c \bmod 215629, \\ c^{56957} &= m \bmod 215629. \end{aligned}$$

如果明文信息 $m = 123456$，则相应的密文为

$$123456^{49} = 1603 \bmod 215629.$$

密码分析者利用公钥 $(215629, 49)$ 和密文 1603 按所示的一组变换进行运算得到 $c_1 = 180661$，$c_2 = 109265, c_3 = 131172, c_4 = 98178, c_5 = 56372, \ldots, c_8 = 85978, c_9 = 123456, c_{10} = 1603 \bmod 215629$。由此可见仅仅经过 10 步变换就得到 $c_{10} = c$，于是立即推知 $c_9 = m = 123456$。

这是由于幂剩余函数满足下述周期性定理，即：

定理 8.1

若 N 为素数或不同素数之积，且 $\gcd(e, \varphi(N)) = 1$，则存在某一正整数 k，使得

$$a^{ek} = a \bmod N.$$

♡

当上述例子中模数的欧拉数为 $\varphi(N) = 214684$ 时，49 的指数是 10。也就是说周期 k 仅为 10，这固然是加密指数 e 的特定选择所决定的。这种结果也很容易理解，因为 $\gcd(e, \varphi(N)) = 1$，所以每次变换均为一一映射。由于信息空间是有限的，连续映射最终一定会恢复到初始状态。

要避免密码分析者利用幂剩余的周期性破译 RSA 体制，就要使 k 足够大以至于密码分析者在合理的时间内不能够破译 RSA 密文。事实上幂剩余函数的周期 k 与模数 N 的两个素因子的欧拉数 $\varphi p)$ 和 $\varphi(q)$ 的最小公倍数 l 有关，l 越大幂剩余函数的周期就越长，反之 l 越小，周期就越小。显然，为了使 RSA 公钥密码实际上是不可能破译的，理当选择 l 较大的素因子 p 和 q。

还有一个值得注意的问题是在以 RSA 公钥密码作为加密标准的通信网中，每个用户的密钥不能有相同的模值。这里最显而易见的问题是同一明文用相同的模数但用不同的指数加密，而且这两个指数是互素的。那么无需任何一个解密密钥就可恢复出明文，一般情况下这很有可能。

假设 m 是明文信息，有两个不同用户的加密密钥分别是 e_1 和 e_2 ，他们有共同的模数 N，两个密文信息分别为：

$$\begin{aligned} c_1 &= m^{e_1} \bmod N, \\ c_2 &= m^{e_2} \bmod N. \end{aligned}$$

密码分析者知道 N, e_1，e_2，c_1 和 c_2，他们用下述方法恢复出明文 m。

由于 e_1 和 e_2 和互素，由欧几里德算法能够找出 r 和 s 使之满足 $re_1 + se_2 = 1$，在 r 和 s 中，有一个是负数。假定 r 是负数，那么用欧几里德算法计算出 c_1^{-1}，然后有 $(c_1^{-1})^{-r} c_2{}^s = m^{e_1 r + e_2 s} \equiv m \bmod N$。

这样，无需破译秘密密钥 d，就可以得到明文 m。这种方法叫做 RSA 的共模攻击。避免这种攻击的措施是不在一组用户之间共享 N。

2. 选择密文攻击

RSA 的模幂运算有保持输入的乘法结构的特性，即有 $E_K(ab) \equiv E_K(a)E_K(b) \pmod{n}$，这将影响 RSA 算法协议的安全性。

(1) 破译密文

用户甲发布了他的公钥 (e, n)，攻击者监听到一段发给甲的密文 c，$c = m^e \bmod n$。为了解析出明文 m，攻击者随机选取了一个小于 n 的数 r，计算 $y = r^e \bmod n$，$t = yc \bmod n$，并且令 $k = r^{-1} \bmod n$，则 $k = y^{-d} \bmod n$。现在攻击者拿 t 给用户甲签名（甲得用私钥签名，而非计算 Hash 值，当甲把签名返回给攻击者时，攻击者就得到了 $s = t^d \bmod n$。这样攻击者就可以计算：

$$ks \bmod n \equiv y^{-d} \times t^d \bmod n \equiv y^{-d} \times y^d \times c^d \bmod n \equiv c^d \bmod n = m.$$

于是攻击者获得了明文 m。

(2) 骗取签名

同样对于用户甲，攻击者有非法消息 m_i 要获得甲的签名，但 m_i 直接给甲不能被接受，于是攻击者选择一个 r，算得 $y = re \bmod n$，再计算 $m = ym_i \bmod n$，将 m 给甲签字，获得 $s = md \bmod n$，这样就可由计算

$$sr^{-1} \bmod n \equiv y^d m_i^d r^{-1} \bmod n \equiv r \times r^{-1} \times m_i^d \bmod n \equiv m_i^d \bmod n.$$

于是攻击者得到了 m_i 的签名。所以，安全起见，不要给陌生人提交的随机性文件签名，并且最好先采用单向的散列函数。ISO9796 的分组格式可用来防止这样的攻击。

3. 低加密指数攻击

在 RSA 系统中，使用小的加密公钥可以加快加密和验证签名的速度，但过小的公钥易受到攻击。如果用户组中 3 个用户都使用 3 作为公钥，当用这种公钥对同一个明文 m 加密，则 $c_1 = m^3 \bmod n_1$，$c_2 = m^3 \bmod n_2$，$c_3 = m^3 \bmod n_3$，n_1、n_2、n_3 各不相同，且 $m < n_1$，$m < n_2$，$m < n_3$。一般安全性考虑，n_1、n_2、n_3 是互素的，这样由中国余数定理可从 c_1，c_2，c_3 计算出 c，且 $c = m^3 \bmod (n_1 n_2 n_3)$，显然 $m^3 < n_1 n_2 n_3$，所以 $m = c^{1/3}$。

Hastad 证明如果采用不同的模 n，相同的公钥 e，则对 $e(e+1)/2$ 个线性相关的消息加密，系统就会受到威胁。所以一般选取 16 位以上素数，速度也快，并且可防止这样的攻击。而对短的消息，使用独立随机数填充以保证 $m^e \bmod n \neq m^e$，从而杜绝低加密指数攻击。

4. 定时攻击

定时攻击主要针对于 RSA 核心运算是非常耗时间的模乘，只要能够精确监视 RSA 的解密过程，获得解密的时间，从而估算出私有密钥 d。模幂是通过一位一位来计算的，每次迭代执行一次模乘，并且如果当前位是 1，则还需要进行一次模乘。对于有些密码，后一次模乘执行速度会极慢，攻击者就可以在观测数据解密时，根据执行时间判断当前位是 1 或者 0。不过这种方法只是理论上可以考虑，实际操作很困难。如果在加密前对数据做盲化处理，再进行加密，使得加密时间具有随机性，最后进行去盲，这样可以抵抗定时攻击，不过增加了数据处理步骤。

综上所述，在使用 RSA 公钥密码时必须注意以下问题：

(1) 选择素数 p 和 q 时，应使这两个素数的欧拉数 $\varphi(p)$ 和 $\varphi(q)$ 的最小公倍数 l 尽可能大。l 越大，幂剩余函数的周期就越长。这样就可以避免密码分析者利用剩余函数的周期性破译该体制。

(2) 密钥中的各项参数应选得足够大，以避免密码分析者利用穷举搜索来破译该体制。

(3) 在同一个通信网络中，不同的用户不应该使用共同的模数。

8.2.4 RSA-OAEP 加密标准

最佳非对称加密填充（OAEP）是对消息编码的一种方法，由 Mihir Bellare 和 Phil Rogaway 提出。首先对消息使用 OAEP 编码，然后再使用 RSA 加密，该方法是可证明安全的。

1. 加密操作

(1) 长度检查

a 如果附加在消息上的标签 L 的长度大于 Hash 函数（SHA-1 为 $2^{61}-1$ 字节）的输入限制，那么输出“标签太长”并停止。

b 如果消息 M 的长度 mLen 大于 $k-2\text{hLen}-2$，其中 k 是密文长度，hLen 表示 Hash 函数输出的长度，那么输出“消息太长”并停止。

(2) EME-OAEP 编码（见下图8.2）

a 如果没有提供标签 L，则让 L 为空字符串，且 lHash $=$ hash(L)，hLen 表示其长度。

b 生成包括 $k-\text{mLen}-2hLen-2$ 个 0 的字节串 PS，PS 的长度可能为 0。

c 连接 lHash、PS、十六进制值 0x01 以及消息 M，得到长度为 $k-\text{hLen}-1$ 的数据块 DB：DB $=$ lHash $||$ PS $||$ 0x01 $||$ M。

d 生成随机字节串 seed，其长度为 hLen。

e 使用掩码生成函数 MGF 得到 dbMask $=\text{MGF}(\text{seed}, k-\text{hLen}-1)$。

f maskedDB $=$ DB $\oplus$ dbMask。

g seedMask $=\text{MGF}(\text{maskedDB}, \text{hLen})$。

h maskedSeed $=$ seed $\oplus$ seedMask。

i 连接十六进制值为 0x00 的字节串、maskedSeed 和 maskedDB，得到长度为 k 的编码消息 EM：EM $=$ 0x00 $||$ maskedSeed $||$ maskedDB。

(3) RSA 加密

a 把编码消息 EM 转换成整数消息 m：$m=\text{OS2IP}(EM)$。

b 对 RSA 公钥 (n,e) 和消息 m 使用 RSAEP 加密得到整数密文 c：$c=\text{RSAEP}((n,e),m)$。

c 把 c 转换成长度为 k 字节的密文 C：$C=\text{I2OSP}(c,k)$。

(4) 输出密文 C。

2. 解密操作

hLen 表示 Hash 函数输出的字节长度，MGF 表示掩码生成函数，K 表示 RSA 私钥，C 待解密的密文，其长度为 k，且 $k \geq 2\text{hLen}+2$。消息 M 的长度为 mLen，且 $\text{mLen} \leq k-2\text{hLen}-2$。

(1) 长度检查

a 如果附加在消息上的标签 L 的长度大于 Hash 函数（SHA-1 为 $2^{61}-1$ 字节）的输入限制，那么输出“解密错误”并停止。

b 如果密文 C 的长度不是 k，那么输出“解密错误”并停止。

c 如果 $k<2\text{hLen}+2$，其中 hLen 表示 Hash 函数输出的字节长度，则停止。

(2) RSA 解密

a 把密文 C 转换成整数密文 c：$c=\text{OS2IP}(C)$。

b 对 RSA 私钥 K 和密文 c 使用 RSADP 解密得到整数消息 m：$m=\text{RSADP}(K,c)$。
如果 RSADP 输出“密文超出长度”则输出“解密错误”并停止。

c 把消息 m 转换成长度为 k 字节的编码消息 EM：EM $=\text{I2OSP}(m,k)$。

(3) EME-OAEP 解码

a 如果没有提供标签 L，则让 L 为空字符串，且 lHash $=\text{hash}(L)$，hLen 表示其长度。

b 把编码消息 EM 分离为字节 Y，长度为 hLen 的 maskedSeed 以及长度为 $k-\text{hLen}-1$ 的

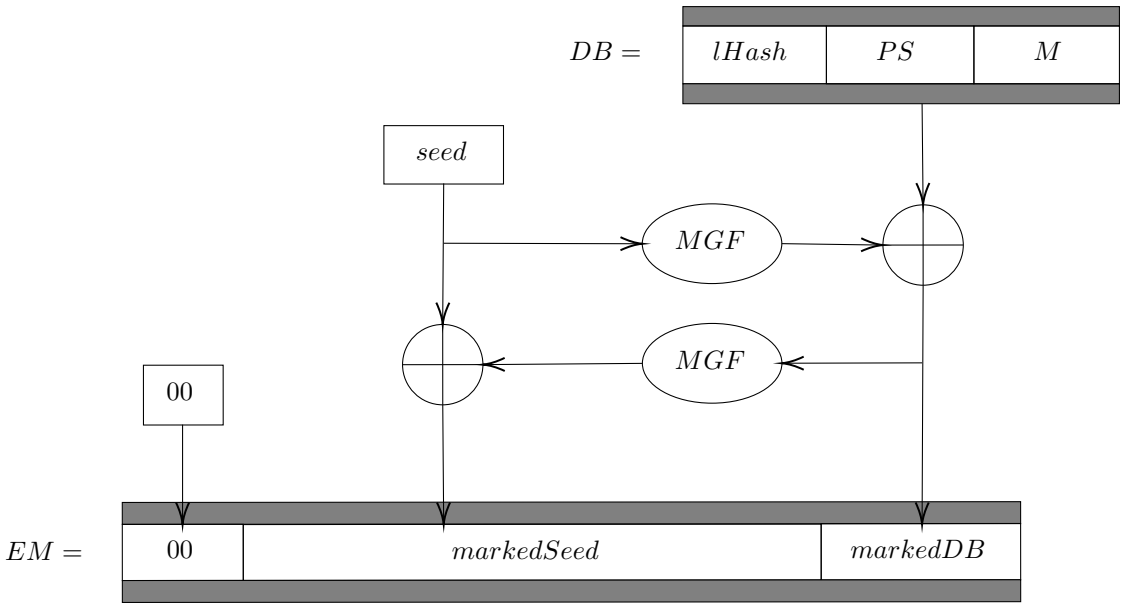


图 8.2: EME-OAEP 编码操作

maskedDB：EM $= Y$ || maskedSeed||maskedDB。

c seedMask $=$ MGF(maskedDB, hLen)。

d seed $=$ maskedSeed $\oplus$ seedMask。

e dbMask $=$ MGF(seed, $k -$ hLen $- 1$)。

f DB $=$ maskedDB $\oplus$ dbMask。

g 分离 DB 得到长度为 hLen 的 lHash（可能为空的）填充字节 PS、十六进制值 0x00 以及消息 M：DB $=$ lHash' || PS || 0x01 || M。

如果分离 M 和 PS 得到的十六进制值不是 0x01，或者 lHash 不等于 lHash'，或者 Y 非零，输出“解密错误”并停止。

(4) 输出消息 M。

8.3 ElGamal 密码体制

ElGamal 密码体制是 T. ElGamal 于 1984 年提出的，它至今仍然是一个安全性良好的公钥密码体制。

8.3.1 ElGamal 算法

ElGamal 算法的安全性是建立在有限域上的离散对数的难解性这一数学难题的基础上的。

(1) 密钥产生过程

选择一素数 p 以及两个小于 p 的随机数 g 和 x，计算 $y \equiv g^x \bmod p$。以 (y, g, p) 作为公开密钥，x 作为秘密密钥。

(2) 加密过程

设加密明文消息 M，随机选取一个与 $p-1$ 互素的整数 k，计算 $C_1 \equiv g^k \bmod p, C_2 \equiv y^k M \bmod p$，密文为 $C = (C_1, C_2)$。

(3) 解密过程

计算 $C_2 C_1^{-x} \bmod p$ 就可以得到正确的解密结果。

解密过程的正确性证明如下：

证明

$$\begin{aligned} C_2C_1^{-x} \bmod p &= (y^kM)(g^k)^{-x} \bmod p \\ &= (g^{xk}M)(g^k)^{-x} \bmod p \\ &= M \bmod p. \end{aligned}$$

下面的这个例子可以简单说明在 ElGamal 密码体制中所进行的计算。

例 8.5 设 $p=11$，$g=7$，在 $GF(11)$ 上有 $7^0=1$，$7^1=7$，$7^2=5$，$7^3=2$，$7^4=3$，$7^5=10$，$7^6=4$，$7^7=6$，$7^8=9$，$7^9=1$，$7^{10}=1$，g 是 $GF(11)$ 的本原元素。

设 A 的私钥 $x_A=3$，公钥 $y_A=2$；B 的私钥 $x_B=5$，公钥 $y_B=10$。

假定 A 欲将消息 $m=6$ 保密寄给 B。A 取 $x=7$，计算：

$$\begin{aligned} C_1 &\equiv g^7 \bmod 11 && \equiv 6, \\ K &\equiv (10)^7 \bmod 11 && \equiv 10, \\ C_2 &\equiv Km \bmod 11 && \equiv 60 \equiv 5 \bmod 11 \end{aligned}$$

A 将 $(6,5)$ 作为密文寄给 B。B 收到后计算

$$\begin{aligned} K &\equiv (g)^5 \bmod 11 && \equiv 10, \\ K^{-1} &\equiv g^{10-5} && \equiv g^5 \equiv 10, \\ M &\equiv K^{-1}C_2 && \equiv 50 \bmod 11 \equiv 6. \end{aligned}$$

故恢复 $m=6$。

8.3.2 ElGamal 公钥密码体制的安全性

为了确保 ElGamal 算法的安全性，通常 p 至少应该具有 150 位以上的十进制数字，大约 512 位的二进制数。而且，$p-1$ 至少有一个大的素因子。在满足上述条件下，根据 p、g、b 来计算离散对数是相当困难的。

设 p 是一个素数，$\alpha \in \mathbb{Z}_p{}^*$，$\alpha$ 是一个本原元，$\beta \in \mathbb{Z}_p{}^*$。已知 α 和 β，求满足 $\alpha^n \equiv \beta \bmod n$ 的唯一整数 n，$0 \le n \le p-2$，称为有限域上的离散对数问题，常将 n 记为 $\log_\alpha \beta$。

ElGamal 公钥密码体制可以在计算离散对数困难的任何群中实现，不过通常使用的是有限域，但不局限于有限域。

关于有限域上的离散对数问题，已进行了许多深入的研究，取得了许多重要成果。但到目前为止，还没有找到一个非常有效的多项式时间算法来计算有限域上的离散对数。一般而言，只要素数 p 选取适当，有限域 $\mathbb{Z}_p$ 上的离散对数问题是难解的。反过来，已知 α 和 n，计算 $\beta=\alpha^n \bmod p$ 是容易的。因此，对于适当的素数 p，模 p 指数运算是一个单向函数。

在 ElGamal 公钥密码体制中，$\beta=\alpha^d \bmod p$，从公开的 α 和 β，求保密的解密密钥 d，就是计算一个离散对数。因此，ElGamal 公钥密码体制的安全性主要是基于有限域 $\mathbb{Z}_p$ 上离散对数问题的难解性。

为了抵抗目前已知的一些对 ElGamal 公钥密码体制的攻击，素数 p 按十进制表示至少应该有 150 位数字，并且 $p-1$ 至少应该有一个大的素因子。

8.4 椭圆曲线密码体制

人们对椭圆曲线的研究已有一百多年的历史，而椭圆曲线密码（Elliptic Curve Cryptography，ECC）是 Koblitz 和 Miller 于 20 世纪 80 年代提出的。ElGamal 密码是建立在有限域 $GF(p)$ 之上的，其中 p 是一个

大素数，这是因为有限域 $GF(p)$ 的乘法群中的离散对数问题是难解的。受此启发，在其他任何离散对数问题难解的群中，同样可以构成 ElGamal 密码。于是人们开始寻找其他离散对数问题难解的群。研究发现，有限域 $GF(p)$ 上的椭圆曲线上的一些点构成交换群，而且离散对数问题是难解的。于是可在此群上定义 ElGamal 密码，并称为椭圆曲线密码。目前，椭圆曲线密码已成为除 RSA 密码之外呼声最高的公钥密码之一。

为保证 RSA 算法的安全性，它的密钥长度需一再增大，使得它的运算负担越来越大。相比之下，椭圆曲线密码体制可用短得多的密钥获得同样的安全性，因此具有广泛的应用前景。它的密钥短、签名短，软件实现规模小、硬件实现电路省电。普遍认为，160 位长的椭圆曲线密码的安全性相当于 1024 位的 RSA 密码，而且运算速度也较快。正因如此，一些国际标准化组织已把椭圆曲线密码作为新的信息安全标准。如，IEEE N363/D4，ANSI F9.62，ANSI G9.63 等标准，分别规范了椭圆曲线密码在 Internet 协议安全、电子商务、Web 服务器、空间通信、移动通信、智能卡等方面的应用。

为使用椭圆曲线构造密码体制，需要找出椭圆曲线上的数学困难问题。在椭圆曲线构成的 Abel 群 $E_p(a,b)$ 上考虑方程 $Q=kP$，其中 P，$Q\in E_p(a,b)$，$k<p$，则由 k 和 P 易求 Q，但由 P、Q 求 k 是困难的，这就是椭圆曲线上的离散对数问题，可应用于公钥密码体制。

8.4.1 Diffie-Hellman 公钥系统

已知有限城 $GF(q)$，g 是 $GF(q)$ 上一固定元素。用户 A 任选一个整数 a（$1\le a\le q-1$），a 保密，$g^a\in GF(q)$ 公开；用户 B 任选一个整数 b（$1\le b\le q-1$），b 保密，$g^b\in GF(q)$ 公开。如果 A 和 B 通信，那么他们间的通信密钥为 g^{ab}，$g^{ab}=(g^a)^b=(g^b)^a$。第三者只能知道 g^a 和 g^b，无法得知 g^{ab}，而从 g^a 和 g^b 要计算 a 或 b 是困难的。

现在来讨论 Diffie-Hellman 体制在椭圆曲线上的实现。椭圆曲线 E 定义在 $GF(q)$ 域上，设 $P\in E$，要求由 P 产生的群的元素足够多（P 的作用相当于上述的 g），A 和 B 分别选 a 和 b 予以保密，但将 aP、$bP\in E$ 公开。A 和 B 的通信密钥为 abP，这是第三者无法知晓的。第三者只能知道 aP 和 bP，无法得知 abP，而从 aP 和 bP 计算 a 或 b 是椭圆曲线上的离散对数问题，是困难的。

8.4.2 Menezes-Vanstone 公钥密码体制

近年来，椭圆曲线上的公钥密码体制越来越受到人们的关注。普遍认为椭圆曲线上的公钥密码体制具有比较好的安全性能并且运算速度也比较快。Menezes-Vanstone 公钥密码体制是 ElGamal 公钥密码体制在椭圆曲线上的一个有效实现。它是由 A. J. Menezes 和 S. A. Vanstone 于 1993 年提出的。

Menezes-Vanstone 公钥密码体制描述如下：

(1) 设 $p>3$ 是一个大素数，E 是有限域 $\mathbb{Z}_P$ 上的椭圆曲线，$\alpha\in E$ 是椭圆曲线上的一个点，并且 α 的阶足够大，使得在由 α 生成的循环子群中离散对数问题是难解的。p 和 E 以及 α 都公开。

(2) 随机选取整数 d，$1\le d\le \mathrm{ord}(\alpha)-1$。计算 $\beta=d\alpha$。β 是公开的加密密钥，d 是保密的解密密钥。

(3) 明文空间为 ${\mathbb{Z}_p}^*\times{\mathbb{Z}_p}^*$，密文空间为 $E\times{\mathbb{Z}_p}^*\times{\mathbb{Z}_p}^*$。

(4) 加密变换：对任意明文 $x=(x_1,x_2)\in{\mathbb{Z}_p}^*\times{\mathbb{Z}_p}^*$，秘密随机选取一个整数 k，$1\le d\le \mathrm{ord}(\alpha)-1$，密文为 $y=(y_0,y_1,y_2)$，其中 $y_0=k\alpha$，$(c_1,c_2)=k\beta$，$y_1=c_1x_1 \bmod p, y_2=c_2x_2 \bmod p$。

(5) 解密变换：对任意密文 $y=(y_0,y_1,y_2)\in E\times{\mathbb{Z}_p}^*\times{\mathbb{Z}_p}^*$，明文为 $x=(y_1c_1^{-1} \bmod p, y_2c_2^{-1} \bmod p)$，其中 $(c_1,c_2)=dy_0$。

现在我们来说明解密变换能正确地从密文恢复相应的明文。

证明 因为

$$(c_1, c_2) = k\beta,$$
$$y_0 = k\alpha,$$
$$\beta = d\alpha,$$

所以

$$(c_1, c_2) = k\beta = kd\alpha = dy_0.$$

又因为

$$y_1 = c_1 x_1 \bmod p,$$
$$y_2 = c_2 x_2 \bmod p,$$

所以

$$\begin{aligned}(y_1 c_1^{-1} \bmod p, y_2 c_2^{-1} \bmod p) =& (c_1 x_1 c_1^{-1} \bmod p, c_2 x_2 c_2^{-1} \bmod p)\\ =& (x_1, x_2)\\ =& x.\end{aligned}$$

因此，解密变换能正确地从密文恢复出相应的明文。

在 Menezes-Vanstone 公钥密码体制中，密文依赖于明文 x 和秘密选取的随机整数 k，因此，明文空间中的一个明文对应密文空间中的许多不同的密文。

在 Menezes-Vanstone 公钥密码体制中，$\beta = d\alpha$。从公开的 α 和 β，求保密的解密密钥 d，就是计算一个离散对数。当 α 的阶足够大时，这是一个目前众所周知的难解的问题. 因此，Menezes-Vanstone 公钥密码体制的安全性主要是基于椭圆曲线上离散对数问题的难解性。

例 8.6 设 $p = 11$，E 是由

$$y^2 \equiv x^3 + x + 6 \bmod 11$$

所确定的有限域 $\mathbb{Z}_{11}$ 上的椭圆曲线。

解 设 $\alpha = (2, 7)$，保密的解密密钥 $d = 7$，计算

$$\beta = 7\alpha = (7, 2).$$

假设明文 $x = (x_l, x_2) = (9, 1)$，我们来计算其对应的密文。首先随机选取 $k = 2$，然后计算

$$y_0 = k\alpha = 6(2, 7) = (7, 9),$$
$$(c_1, c_2) = k\beta = 6(7, 2) = (8, 3),$$
$$y_1 = c_1 x_1 \bmod p = 8 \times 9 \bmod 11 = 6,$$
$$y_2 = c_2 x_2 \bmod p = 3 \times 1 \bmod 11 = 3.$$

$x = (x_l, x_2) = (9, 1)$ 所对应的密文为 $y = (y_0, y_1, y_2) = ((7, 9), 6, 3)$。

假设密文为 $y = (y_0, y_1, y_2) = ((7, 9), 6, 3)$，我们来恢复它所对应的明文。计算 $(c_1, c_2) = dy_0 = 7(7, 9) = (8, 3)$，则明文为

$$\begin{aligned}x &= (y_1 c_1^{-1} \bmod p, y_2 c_2^{-1} \bmod p)\\ &= (6 \times 8^{-1} \bmod 11, 3 \times 3^{-1} \bmod 11)\\ &= (6 \times 7 \bmod 11, 3 \times 4 \bmod 11)\\ &= (9, 1).\end{aligned}$$

因此，解密变换正确地恢复了明文。

表 8.2: ECC 和 RSA/DSA 在保持同等安全条件下所需的密钥长度（单位为比特）

RSA/DSA	512	768	1024	2048	21000
ECC	106	132	160	211	600

8.4.3 椭圆曲线密码体制的优点

与基于有限域上离散对数问题的公钥体制（Diffie-Hellman 密钥交换和 ElGamal 密码体制相比，椭圆曲线密码体制有如下优点：

(1) 安全性高：攻击有限域上的离散对数问题有指数积分法，其运算复杂度为 $O(\exp \sqrt[3]{(\log p)(\log\log p)^2})$，其中 p 是模数，为素数。而它对椭圆曲线上的离散对数问题并不有效。目前攻击椭圆曲线上的离散对数问题只有适合攻击任何循环群上离散对数问题的大步小步法，其运算复杂度为 $O(\exp(\log \sqrt{p_{\max}}))$，其中 $p_{\max}$ 是椭圆曲线所形成的 Abel 群的阶的最大素因子。因此，椭圆曲线密码体制比基于有限域上的离散对数问题的公钥体制更安全。

(2) 密钥量小：由攻击两者的算法复杂度可知，在实现相同的安全性能条件下，椭圆曲线密码体制所需的密钥量远比基于有限域上的离散对数问题的公钥体制的密钥量小。

(3) 灵活性好：有限域 $GF(q)$ 一定的情况下，其上的循环群（即 $GF(q)-\{O\}$ 就定了。而 $GF(q)$ 上的椭圆曲线可以通过改变曲线参数，得到不同的曲线，形成不同的循环群。因此，椭圆曲线具有丰富的群结构和多选择性。

正是由于椭圆曲线具有丰富的群结构和多选择性，并可在保持和 RSA/DSA 体制同样安全性能的前提下大大缩短密钥长度（目前 160 比特足以保证安全性），因而在密码领域有着广阔的应用前景。表8.2给出了椭圆曲线密码体制和 RSA/DSA 体制保持同等安全的条件下各自所需的密钥的长度。

8.5 基于身份的加密体制

1984 年，Shamir 引入了基于身份加密（Identity-based Encryption，IBE）的概念，用另一种方法解决了公钥授权问题。他的想法是确保用户的公钥是与其身份是直接关联的，以省去授权的步骤。确切的讲，用户的公钥直接来自于公共可知的信息，它们对用户而言是唯一的、不可否认的。根据具体的应用情况，这些信息，由用户的（数字）标识来表示，可以是用户的姓名、身份证号、电话号码、Email 地址、IP 地址以及其它可能的个人信息，或这些信息的组合。这样，用户的公钥对所有知道其身份的人都是可读的，于是没有必要在数据库里查找密钥。进一步地，确定的公钥授权使得在 PKI 中存在的证书，在这里没有存在的必要。当然，应当注意到，用户和他们的身份标识之间关联的实现是并不简单的，远比人们想象的复杂得多。

8.5.1 8.5.1 基于身份的密码学概述

在公钥密码体系（Public Key Infrastructure，PKI）中，密钥对是通过随机选择一个私钥然后通过单向函数计算出公钥。基于身份密码学系统中，密钥对是用不同的方法产生的。首先，公钥由用户的标识唯一确定，然后私钥需要从公钥计算产生。由此，私钥的生成不可以由用户自己产生。因为如果某个用户可以生成自己的私钥，那他就可以生成其他人的私钥。因此，我们需要密钥生成中心来生成用户的私钥。因为密钥生成中心知道一些安全信息，主密钥（master-key），所以才可以做到这一点。与安全信息主密钥相对应的公开的信息就是系统参数（system parameters）。这样由密钥生成中心计算一个公钥和主密钥的单向函数来得到用户的私钥。很明显，与用户自己生成密钥对的 PKI 相比，基于身份的公钥密码系统总是需要一个安全通道，用来向用户传送其私钥，Shamir 建议使用智能卡存储用户私钥来解决这个问题。

基于身份的密码学最开始是为了避免使用证书（在公钥授权中需要）而提出的，它和 PKI 都是非对称的。因此，加密、解密、签名、验证在两个系统中都有简单的功能，它们主要不同点在于密钥的管理。事实上，理想的基于身份的密码学系统应当满足下面的属性：

(1) 用户只需要知道他想与之通信的用户的标识；
(2) 不必要保存公共目录比如存有公钥或证书的文件；
(3) 只在系统启动阶段才需要 PKG/TA 的服务。

然而，在实际应用中，这样理想的方案几乎不可实现的。根据实际的应用情况，有许多实用的构建于理想的基于身份的系统基础上的实现版本。下面，我们把基于身份的密码学系统和传统的 PKI 系统作比较。

1. 参数的真实性（Authenticity of system parameters）

   设想在基于 ID 的系统中，一个攻击者生成他自己的主密钥和对应的系统参数，然后欺骗用户，使其相信这些伪造的系统参数是正确的。然后，在他的主密钥之下，对所有公钥他可以生成相应的私钥。于是，他可以解密任何在他的伪造参数下加密的信息。当然，他也可以创建任何姓名名义下的签名，这使得相信他的参数的人接受。攻击者甚至会模仿 PKG/TA 并且向请求的用户发布私钥。然而，此前已经存在的由 PKG/TA 生成的用户的私钥，不会受到破坏。可见，PKG/TA 对于如何保证系统参数的真实性是非常重要的。

   在 PKI 中，存在一个相似的问题，用户需要确认 CA 公钥的确实性。也就是说，如果攻击者能使用户相信某个他所选择的公钥是 CA 公钥，他就可以创建包含伪造公钥，他拥有对应的私钥。然后，攻击者就可以读取用伪造的公钥加密的信息，还可以伪造在哪些密钥下看起来是正确的签名。与基于 ID 的系统的唯一不同点是攻击者不能模仿 CA，因为用户会发现他们所请求的证书包含不正确的公钥。

2. 注册（Registration at the authority）

   在两个系统中，想要参与的用户都需要在注册机构进行注册。在一系列的授权过程之后，RA 发布给用户一个惟一的数字标识，比如以 email 地址的形式。然后，在基于 PKI 系统中，用户可以向 CA 展示他的数字身份标识和公钥，随后就证明他拥有与之对应的私钥。之后，CA 发布一个与数字标识、公钥绑定的证书。

   在基于 ID 的系统中，用户简单地向密钥生成中心展示他的数字身份标识和公钥。RA 只负责数字标识的唯一性以及身份标识与物理用户之间关联的真实性。密钥生成中心从数字标识计算与公钥相对应的私钥。基于身份的系统有一个额外的不足之处就是私钥需要从密钥生成中心传递给用户。因此，它要求一个能保证机密性和真实性的安全通道。

3. 密钥托管（Key escrow）

   基于身份的密码系统天生就带有密钥托管问题。即只要密钥生成中心拥有主密钥，他就可以在任何时候生成任何私钥。在具体的应用情景下，密钥托管有时候并不必然就是坏事。比如，在一个公司内，密钥生成中心的主管部门可能希望保持雇员之间的交流通道。更进一步，密钥托管可被用来恢复用户丢失的密钥。但是对签名方案而言，密钥托管却是非常严重的问题，因为它不能保证不可抵赖性。

   这种密钥托管能力在由中心机构产生密钥对的 PKI 中也是存在的，因为中心机构会存储其产生的私钥。从另一个方面来讲，如果用户自己生成密钥对，或者中心机构不存储其产生的私钥，私钥就不可以恢复，也就不存在密钥托管问题。这跟基于身份的系统是相反的，因为密钥生成中心总是可以再生成私钥。

4. 密钥撤销和更新（Key revocation and key rollover）

   在基于 PKI 的方案中，当一个证书被撤销的时候，其他用户通过证书撤销列表（Certificate Revocation List，CRL）被告知。这种情况可能会在用户离开一个组织或者私钥被破解或泄漏时出现。在后一

种情况中，或者在证书被终止后密钥对需要被替代时，用户可以简单地生成一个新的密钥对并且取得其证书。

撤销列表允许用户检查公钥的有效性，这也可以用在基于 ID 的系统中。然而，因为用户的公钥由其身份标识产生，他不能像在基于 PKI 的系统一样，在撤销之后，简单地生成一个新的密钥对。这样不方便或者说不可能每次需要新的密钥对时就改变用户的身份标识。这个问题的部分解决方案是让公钥不完全从身份标识产生，在身份标识上连上一些通用的信息。比如，可以加上当前的年份，甚至日期等通用的公知的信息。但这种方法在时间跨度大时，一旦用户需要更改密钥对时需要等待太久，不方便使用；时间跨度小时，又会导致 PKG/TA 过中的工作负担。

另一个也同样不令人满意的方法是在身份标识上连上一些特别的密钥的信息，比如索引号，密钥的发布日期等。这样，当一个私钥被破解时，用户可以要求一个附带增大的索引或者更新的日期的新密钥。这种方法的最不好的地方在于，它会破坏基于身份密码学的身份的唯一性属性，公钥可以简单从身份标识产生。因为其他用户需要在某个目录下查看索引号或者公钥的发布日期信息。

5. 密钥发布（Key distribution）

   从用户的角度出发，分发密钥的简单性是引入基于身份密码学的主要原因。所有公钥都可以从用户的身份标识产生，这样一来，获得一个人的公钥用来加密和签名验证变得容易和透明。

   对 PKI 而言，恰恰相反，用户必须查看对应的证书，验证 CA 的签名并且检查证书的有效期。另一个缺点是，用户必须检查证书的有效性，这一般是通过使用一个及时更新的证书撤销列表 CRL 或者向 OCSP（Online Certificate Status Protocol RFC 2560）服务器发送 OCSP 查询包进行证书状态的在线查询。

6. 主密钥安全性（Master-key security）

   在 IDPKC 中，攻击者在得到 PKG/TA 的主密钥 y 之后，就能生成所有私钥，于是也就可以阅读所有消息和伪造任何签名。因此，保持的安全性是至关重要的。一般可以把主密钥分存于多个 PKG/TA，这样还可以防止密钥托管问题。

   在 PKI 系统中，CA 的私钥泄漏之后，攻击者就以 CA 的名义来伪造他所选择公钥的证书，然后欺骗其他用户，使他们相信这些公钥的授权。然而，攻击者无法恢复此前生成的私钥，所以只能影响到系统未来的安全运行。

7. 规模化（Scalability）

   在一个基于 ID 的方案中，PKG/TA 必须为每一个密钥的请求检查和计算一个私钥。尤其是当密钥的请求量很大时，比如，在一个经常终止密钥的系统中，将会使 PKG/TA 的工作负担很重。解决方案之一就是采用多 PKG/TA 的方案，但是，多 PKG/TA 的方案不能用来解决密钥托管问题。另一个方案是通过分发主密钥来提高安全性。也就是说，在这种情况之下，每个 PKG/TA 和在单 PKG/TA 设置中工作量是一样的。同时，基于身份的系统看起来似乎在较小规模下会工作得更好。

8. 商业成熟度（Commercial maturity）

   利用对（Pairing）来实现基于身份的密码学是个很新的技术，2001 年刚刚引入。虽然基于身份的密码学的优势是明显和公认的，但是在它被实际应用时，也发现了存在一些不足。而且，目前它的技术还主要停留在理论阶段，很难在实际应用中检测它。

   这跟基于 PKI 的密码学恰恰相反，后者作为一个被建立起来并广泛使用的技术已经好多年了。而且，基于 PKI 的密码学在很大程度上已经被标准化，而这是实现时很重要的方面。考虑到大多数公司和主要公钥密码学组织的保守性，很难想象短期内基于 ID 的密码学能够取代 PKI。

8.5.2 基于身份的加密方案的定义

基于身份加密的方案通常包括四个算法：设置、密钥产生、加密过程和解密过程。系统中包含一个第三方信任机构 TA（Trust Authority）。

(1) 设置：输入一个安全参数，输出系统参数和主密钥（master）。系统参数包括明文空间 **M**、密文空间 **C**、Hash 函数等，完全公开。主密钥用于计算私钥，由 TA 秘密保存。

(2) 密钥产生：输入系统参数、主密钥和用作公钥的任意字符串 $ID \in \{0,1\}^*$，输出与公钥 ID 对应的私钥 d_{ID}。

(3) 加密过程：输入系统参数、公钥 ID 和明文 $M \in \mathbf{M}$，返回对应的密文。

(4) 解密过程：输入系统参数、公钥 d_{ID} 和密文 $C \in \mathbf{C}$，返回对应的明文。

这些算法必须满足标准的一致性约束，即如果所有的算法被正确地使用，由加密算法加密属于明文空间的任何消息，那么解密算法都可以正确地解密。

IBE 方案要求密钥产生算法和加密算法都是单向的。密钥产生算法的单向性就是说已知任何一对密钥，任何一个攻击者求出主密钥的可能性几乎是可忽略的。

8.5.3 BF-IBE 方案

Boneh 和 Franklin 在 2001 年美洲密码学年会上提出了一个基于身份的加密方案，它是建立在椭圆曲线上双线性配对函数的特性上的。

A. BF-IBE 基础方案

1. 设置：给定安全参数 $k \in \mathbb{Z}^+$，设置的算法如下：
   1) 由输入的 k 值生成一个素数 q，两个 q 阶群 G_1 和 G_2，一个可接受的双线性映射 $\hat{e}$：$G_1 \times G_1 \to G_2$。随机选择生成元 $P \in G_1$。
   2) 取随机数 $s \in {\mathbb{Z}_q}^*$，令 $P_{pub} = sP$。
   3) 选择两个 Hash 函数 H_1：$\{0,1\}^* \to {G_1}^*$，H_2：$G_2 \to \{0,1\}^n$。安全性分析将 H_1, H_2 视为自然随机（oracle random）的。消息空间 $\mathbf{M} = \{0,1\}^n$，密文空间 $\mathbf{C} = G_1^* \times \{0,1\}^n$，系统参数 $\text{params} = \langle q, G_1, G_2, \hat{e}, n, P, P_{\text{pub}}, H_1, H_2 \rangle$，主密钥 $\text{master-key} = s \in {Z_q}^*$。
2. 析出：给定字符串 $\text{ID} \in \{0,1\}$，算法如下：
   1) 计算 $Q_{\text{ID}} = H_1(\text{ID}) \in G_1^*$；
   2) 取私钥 $d_{\text{ID}} = sQ_{\text{ID}}$，其中 s 为主密钥。
   3) 加密：要利用公钥 ID 加密消息 $M \in \mathbf{M}$，步骤如下：
      (1) 计算 $Q_{\text{ID}} = H_1(\text{ID}) \in {G_1}^*$；
      (2) 随机选择 $r \in {Z_q}^*$；
      (3) 求得密文：
      $$C = \langle rP, M \oplus H_2(g_{\text{ID}}^r) \rangle,$$
      其中，$g_{\text{ID}} = \hat{e}(Q_{\text{ID}}, P_{\text{pub}}) \in G_2^*$。
3. 解密：令由公钥 ID 加密的密文 $C = \langle U, V \rangle \in \mathbf{C}$，要用私钥 $d_{\text{ID}} \in G_1^*$ 解密 C，只需计算：$V \oplus H_2(\hat{e}(d_{\text{ID}}, U)) = M$。
4. 验证：由前文交代的可接受的双线性映射的性质知道，$\hat{e}(d_{\text{ID}}, U) = \hat{e}(Q_{\text{ID}}, P)^{sr} = \hat{e}(Q_{\text{ID}}, P_{\text{pub}})^r = g_{\text{ID}}^r$。

B. BF-IBE 完全方案

1. 系统参数选取，在基础方案的基础上，再选 2 个 Hash 函数 H_3：$\{0,1\}^n \times \{0,1\}^n \to {Z_q}^*$，$H_4$：$\{0,1\}^n \to \{0,1\}^n$。用户密钥产生同基础方案。

1) 计算 $Q_{\text{ID}} = H_1(\text{ID}) \in G_1^*$；
2) 随机选取 $\sigma \in \{0,1\}$；
3) 计算 $r = H_3(\sigma, M)$，输出密文

$$C = \langle rP, \sigma \oplus H_2(g_{\text{ID}}^r), M \oplus H_4(\sigma) \rangle ,$$

其中，$g_{\text{ID}} = \hat{e}(Q_{\text{ID}}, P_{\text{pub}}) \in G_2^*$。

2. 要解密 $C = \langle U, V, W \rangle \in \mathbf{C}$，需要计算：
   1) $V \oplus H_2(\hat{e}(d_{\text{ID}}, U)) = \sigma$，由于 $\hat{e}(d_{\text{ID}}, P_{\text{pub}})^r = \hat{e}(d_{\text{ID}}, U)$；
   2) $W \oplus H_4(\sigma) = M$；
   3) 令 $r = H_3(\sigma, M)$；
   4) 验证 $U = rP$，若不成立，拒绝 C（抵抗 CCA2 密文有效性检验）；
   5) 输出 M。

   注意：$W = M \oplus H_4(\sigma)$，可以替换为 $W = E_{H_4(\sigma)}(M)$，其中 $E_{H_4(\sigma)}(\cdot)$ 是语义安全的对称加密方案。

第 8 章 习题

1. 为什么要引入非对称密码体制?
2. 用 RSA 算法加密：
   (1) $p = 3$；$q = 11$，$e = 7$，$M = 5$；
   (2) $p = 5$；$q = 11$，$e = 3$，$M = 9$；
   (3) $p = 7$；$q = 11$，$e = 17$，$M = 8$；
   (4) $p = 11$；$q = 13$，$e = 11$，$M = 7$；
   (5) $p = 17$；$q = 31$，$e = 7$，$M = 2$。
3. 设通信双方使用 RSA 加密体制，接受方的公开密钥是 $(5, 35)$，接收到的密文是 10，求明文。
4. 假设 Alice 如下使用 RSA。她要加密一条由若干字母组成的消息，先对字母赋值 $a = 1$，$b = 2$，...，$z = 26$，然后对每个字母分别加密。例如，如果消息是 cat，她计算 $3^e \bmod n$，$1^e \bmod n$，$20^e \bmod n$，然后发送加密消息给 Bob，说明 Eve 如何不分解 n 就恢复消息。特别的，假设 $n = 8881$，$e = 13$，Eve 截获到 4461，794，2015，2015，3603 不分解 8881 恢复明文消息。
5. 假设 Bob 加密公司制造了两台机器 A 和 B，每一台都是对相同模数 $n = pq$ 的 RSA 的实现，其中 p, q 是未知素数。两台机器还用了相同的加密指数 e。每台机器接收到消息 m，输出密文假定是 $m^e \bmod n$。机器 A 总是给出正确的输出，但是 B 因为实现和硬件的错误，总是输出密文 $c \bmod n$ 满足 $c = m^e \bmod p$ 和 $c = m^e + 1 \bmod q$。如何利用机器 A 和 B 找出 p，q。
6. 加密算法的带宽是指算法能够加密密文的长度与安全参数的比值。假设 RSA-OAEP 的一个实例使用 2084 作为安全参数，160 作为随机输入的长度，那么这个 RSA-OAEP 实例的带宽是多少?
7. 利用椭圆曲线实现 ElGamal 密码体制，设椭圆曲线是 $E_{11}(1, 6)$，生成元 $G = (2, 7)$，接受方 A 的私钥 $n_A = 7$。求：
   (1) A 的公钥是多少?
   (2) 发送方 B 欲发送消息 $P_m = (10, 9)$，选择随机数 $k = 3$，求密文 C_m。
   (3) 显示接收方 A 从密文 C_m 恢复消息 P_m 的计算过程。

第 9 章　数字签名

在人们的工作和生活中，许多事物的处理需要当事者签名。例如，政府部门的文件、命令、证书、商业合同和财务凭证等都需要当事者签名。签名起到确认、核准、生效和负责等多种作用。

实际上，签名是证明当事者的身份和数据真实性的一种信息。既然签名是一种信息，因此签名可以用不同的形式来表示。在传统的以书面文件为基础的事物处理中，采用书面签名的形式，如印章、手印等。书面签名得到司法部门的支持，具有一定的法律意义。在以计算机文件为基础的现代事物处理中，应采用电子形式的签名，即数字签名。

9.1 数字签名的基本概念

数字签名又称之为数字签字、电子签名、电子签章等，主要用于在网络环境中模拟日常生活中的手工签字或印章。与传统签字或印章有根本不同，对每个消息的都是不同的，否则签名就会被获取并复制到另一个文件中。数字签名的基础是公钥密码学，通过数学的手段来达到传统签名的功能。

ISO 对数字签名是这样定义的：数字签名是指附加在数据单元上的一些数据，或是对数据单元所做的密码交换，这种数据或变换允许数据单元的接收者用以确认数据单元来源和数据单元的完整性，并保护数据，防止被人伪造。简单地说，数字签名是指消息的发送者通过某种签名方法产生的别人无法伪造的一段“特殊报文”，该“报文”就是签名。

Diffie 和 Hellman 的公开密钥系统的思想，对实现数字签名提供了一种简单的实现方法（假定 A 向 B 发送一条消息 M）：

(1) A 计算出 $C = D_A(M)$，对 M 签名；

(2) B 通过检查 $E_A(C)$ 是否恢复 M，验证 A 的签名；

(3) 如果 A 和 B 之间发生争端，仲裁者可以用 (2) 中的方法鉴定 A 的签名。

一个签名方案是一个满足下列条件的五元组 (P, A, K, S, V)：

(1) P 是所有可能的消息组成的有限集。

(2) A 是所有可能的签名组成的有限集。

(3) K 是所有可能的密钥组成的有限集。

(4) 对每一个 $k \in K$，有一个签名算法 $S_k \in S$ 和一个相应的验证算法 $V_k \in V$。对每一个消息 $x \in P$ 和每一个签名 $y \in A$，每一个签名算法 S_k：$P \to A$ 和验证算法 V_k：$P \times A \to \{0,1\}$ 都满足：当 $y = S_k(x)$ 时，$V_k(x, y) = 1$，否则 $V_k(x, y) = 0$。

数字签名具备以下主要特点：

(1) 不可伪造性。在不知道签名者私钥的情况下，通常敌手很难有效地伪造一个合法的数字签名。只有掌握签名私钥的签名者才能有效地生成数字签名。

(2) 保证消息的完整性。通常，签名者可对消息的 Hash 函数值进行签名。由于 Hash 函数具备抗碰撞的特点（对于一个安全的 Hash 函数，通常很难找到两个不同的消息使得它们具有相同的 Hash 值），签名便与原消息绑定在一起而形成一个完整的整体。任何对消息的修改都将会导致（消息、签名）对无法通过验证。因此，签名具备保证消息完整性的特点，即其可防止篡改消息。

(3) 不可否认性。对于普通的数字签名，任何人可利用签名者的公钥对签名进行验证，并通过公钥证书确定签名者的身份。这样，签名者便无法否认自己对消息的签名，即签名具有不可否认性。不可否认性使得签名的接收者可以确认消息及相应签名的原始来源，并可用来进行身份认证。

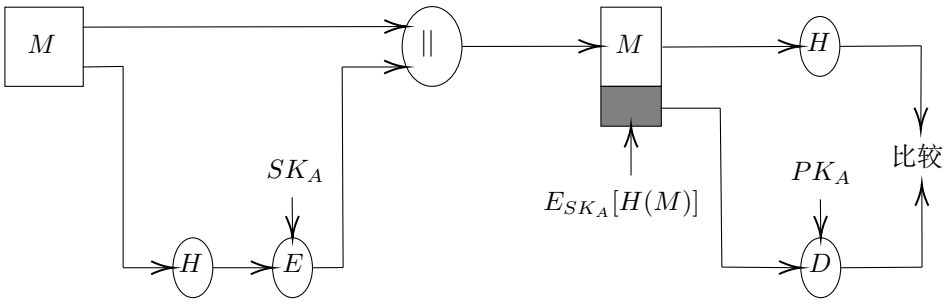


图 9.1: RSA 签名算法过程

9.2 RSA 签名

9.2.1 利用 RSA 密码实现数字签名

Diffie 和 Hellman 给出了公钥密码系统实现数字签名的方法，但他们并没有给出具体的数字签名方案。第一个数字签名方案是 1978 年由美国麻省理工学院的 Rivest、Shamir 和 Adleman 三人提出的，即 RSA 签名方案。

RSA 签名算法的安全性是建立在大数分解问题的困难性上的，具体过程如图9.1所示。

签名方产生一对公开密钥 (n, e) 和私有密钥 (n, d)，签名过程如下：

(1) 首先利用摘要算法计算消息的摘要 $H(M)$；

(2) 用私有密钥 (n, d) 加密消息摘要得到：$s = H(M)^d \bmod n$。

签名完成之后，签名方将 (M, s) 发送给接收方。接收方收到签名 (M_1, s) 之后通过以下步骤进行验证：

(1) 利用摘要算法计算消息的摘要 $H(M_1)$；

(2) 用公开密钥 (n, e) 加密消息摘要得到：$s_1 = H(M_1)^e \bmod n$。如果 $s_1 = s$，那么验证通过，否则拒绝接受签名。因为当且仅当 $M_1 = M$ 时，$s = s_1$。

9.2.2 对 RSA 数字签名的攻击

RSA 的数字签名很简单，但要实际应用还要注意许多问题。

(1) 一般攻击

由于 RSA 密码的加密运算和解密运算具有相同的形式，都是模幂运算。设 e 和 n 是用户 A 的公开密钥，所以任何人都可以获得并使用 e 和 n。攻击者首先随意选择一个数据 Y，并用 A 的公开密钥计算 $X = Y^e \bmod n$，于是便可以用 Y 伪造 A 的签名。因为 X 是 A 对 Y 的一个有效签名。

这种攻击实际上的成功率是不高的。因为对于随意选择的 Y，通过加密运算后得到的 X 具有正确语义的概率是很小的。

可以通过认真设计数据格式或采用 Hash 函数与数字签名相结合的方法阻止这种攻击。

(2) 利用已有的签名进行攻击

假设攻击者想要伪造 A 对 M_3 的签名，他很容易找到另外两个数据 M_1 和 M_2，使得

$$M_3 = M_1 M_2 \bmod n,$$

他设法让 A 分别对 M_1 和 M_2 进行签名：

$$S_1 = (M_1)^d \bmod n,$$

$$S_2 = (M_2)^d \bmod n.$$

于是攻击者就可以用 S_1 和 S_2 计算出 A 对 M_3 的签名 S_3：

$$(S_1S_2) \bmod n = \left((M_1)^d (M_2)^d\right) \bmod n = (M_3)^d \bmod n = S_3.$$

对付这种攻击的方法是用户不要轻易地对其他人提供的随机数据进行签名。更有效的方法是不直接对数据签名，而是对数据的 Hash 值签名。

(3) 利用签名进行攻击获得明文

设攻击者截获了密文 $C, C = M^e \bmod n$，他想求出明文 M。于是，他选择一个小的随机数 r，并计算

$$x = r^e \bmod n,$$

$$y = xC \bmod n,$$

$$t = r^{-1} \bmod n.$$

因为 $x = r^e \bmod n$，所以 $x^d = (r^e)^d \bmod n$，$r = x^d \bmod n$。然后攻击者设法让发送者对 y 签名，于是攻击者又获得

$$S = y^d \bmod n.$$

攻击者计算

$$tS \bmod n = r^{-1}y^d \bmod n = r^{-1}x^dC^d \bmod n = C^d \bmod n = M,$$

于是攻击者获得了明文 M。

对付这种攻击的方法也是用户不要轻易地对其他人提供的随机数据进行签名。最好是不直接对数据签名，而是对数据的 Hash 值签名。

(4) 对先加密后签名方案的攻击

我们已经介绍了先签名后加密的数字签名方案，这一方案不仅可以同时确保据的真实性和秘密性，而且还可以抵抗对数字签名的攻击。

假设用户 A 采用先加密后签名的方案把 M 发送给用户 B，则他先用 B 的公开密钥 e_B 对 M 加密，然后用自己的私钥 d_A 签名。再设 A 的模为 n_A，B 的模为 n_B。于是 A 发送如下的数据给 B：

$$((M)^{e_B} \bmod n_B)^{d_A} \bmod n_A,$$

如果 B 是不诚实的，则他可以用 M_1 抵赖 M，而 A 无法争辩。因为 n_B 是 B 的模，所以 B 知道 n_B 的因子分解，于是他就能计算模 n_B 的离散对数，即他就能找出满足 $(M_1)^x = M \bmod n_B$ 的 x，然后他公布他的新公开密钥为 xe_B。这时他就可以宣布他收到的是 M_1 而不是 M。

A 无法争辩的原因在于下式成立：

$$((M_1)^{xe_B} \bmod n_B)^{dA} \bmod n_A = ((M)^{e_B} \bmod n_B)^{d_A} \bmod n_A.$$

为了对付这种攻击，发送者应当在发送的数据中加入时间戳，从而可证明是用 e_B 对 M 加密而不是用新公开密钥 xe_B 对 M_1 加密。另一种对付这种攻击的方法是经过 Hash 处理后再签名。

这里介绍了四种对数字签名的攻击或利用签名进行攻击获得明文的方法，由此可以得出以下结论：

(1) 不要直接对数据签名，而应对数据的 Hash 值签名。

(2) 要采用先签名后加密的数字签名方案，而不要采用先加密后签名的数字签名方案。

9.2.3 RSA 签名标准 PSS

编码方法由可选的 Hash 函数、掩码生成函数以及盐值长度参数化。对于特定的 RSA 密钥，除了盐值的长度是可变的外，这些可选项应该是固定的。编码方法基于 Bellare 和 Rogaway 的随机签名方案。该方法是随机的，而且包括编码操作和验证操作。

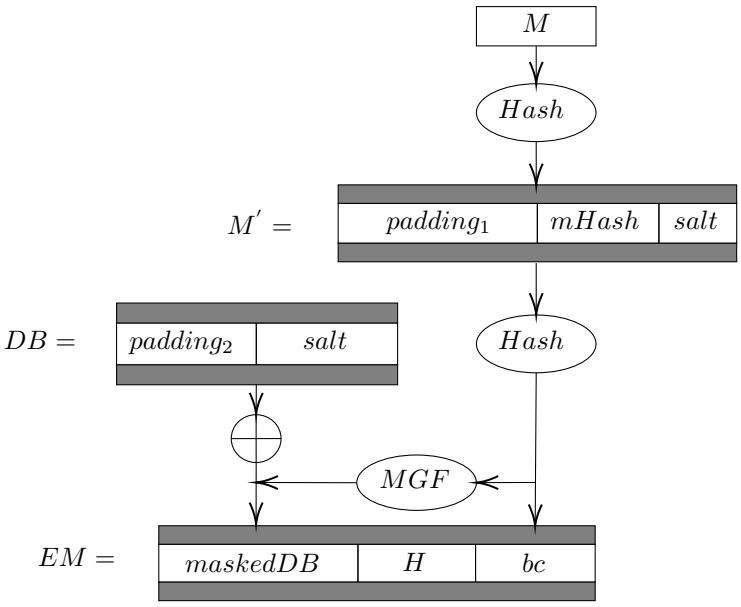


图 9.2: EMSA-PSS 编码操作

图9.2说明了编码操作，它的具体步骤如下：

(1) 如果消息 M 的长度大于 Hash 函数的输入限制（SHA-1 为 $2^{61}-1$），输出“消息太长”并停止。

(2) mHash $=\mathrm{hash}(M)$，长度为 hLen。

(3) 如果 emLen $<$ hLen $+$ sLen $+\,2$，输出“编码错误”并停止。

(4) 生成长度为 sLen 的盐值 salt，如果 sLen $=0$，那么盐值是空字节串。

(5) $M'=$ (0x)0000000000000000 $||$ mHash $||$ salt，M' 的长度为 $8+$ hLen $+$ sLen，其中包括八个初始的八位字节零。

(6) $H=\mathrm{hash}(M')$，长度为 hLen。

(7) 生成字节串 PS ，其中包括 emLen $-$ sLen $-$ hLen $-\,2$ 字节个零。PS 的长度可能为零。

(8) DB $=$ PS $||$ 0x01 $||$ salt，DB 是长度为 emLen $-$ hLen $-\,1$ 的字节串。

(9) dbMask $=\mathrm{MGF}(H,\text{emLen}-\text{hLen}-1)$。

(10) maskedDB $=$ DB $\oplus$ dbMask。

(11) 设置 maskedDB 最左边 8emLen $-$ emBits 比特为零，其中 emBits 是整数 OS2IP(EM) 的最大长度，至少为 8hLen $+$ 8sLen $+\,9$。

(12) EM $=$ maskedDB $||$ H $||$ 0xbc。

(13) 输出 EM，即编码后的消息，长度为 emLen $=$ [emBits$/8$]。

验证过程如下：

(1) 如果 M 的长度大于 Hash 函数的输入限制（SHA-1 为 $2^{61}-1$），输出“长度不一致”并停止。

(2) mHash $=$ hash(M)，长度为 hLen。

(3) 如果 emLen $<$ hLen $+$ sLen $+\,2$，输出“长度不一致”并停止。

(4) 如果 EM 的最右边没有十六进制值 0xbc，输出“不一致”并停止。

(5) 让 EM 的最左边为 maskedDB，长度为 emLen $-$ hLen $-\,1$，H 在其旁边，长度为 hLen。

(6) 如果 maskedDB 最左边的 8emLen $-$ emBits 个比特不等于零，输出“不一致”并停止。

(7) dbMask $=\mathrm{MGF}(H,\text{emLen}-\text{hLen}-1)$。

(8) DB $=$ maskedDB $\oplus$ dbMask。

(9) 让 DB 最左边 8emLen $-$ emBits 比特为零。

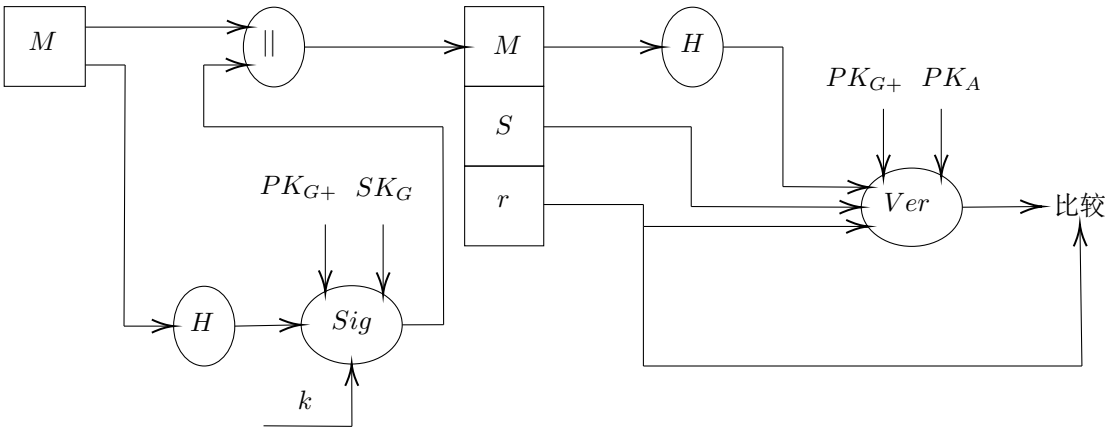


图 9.3: DSS 体制

(10) 如果 DB 最左边 emLen − hLen − sLen − 2 字节不等于零，或者 emLen − hLen − sLen − 1 字节未知处没有十六进制值 0x01，输出“不一致”并停止。

(11) 盐值 salt 为 DB 最后 sLen 字节。

(12) 让 $M' =$ (0x)0000000000000000 || mHash || salt，M' 是长度为 $8 +$ hLen $+$ sLen 的字节串，且有八个初始化的零字节。

(13) $H' = \mathrm{hash}(M')$，长度为 hLen。

(14) 如果 $H = H'$，输出“一致”；否则，输出“不一致”。

9.3 数字签名标准 DSS

9.3.1 DSS 的基本方式

1991 年美国政府颁布了数字签名标准 DSS（Digital Signature Standard），这标志着数字签名已得到政府的支持。和当年推出 DES 时一样，DSS 一提出便引起了一场激烈的争论。反对派的代表人物是 MIT 的 Rivest 和 Standford 的 Hellman 教授。反对的意见主要认为，DSS 的密钥太短，效率不如 RSA 高，不能实现数据加密，并怀疑 NIST 在 DSS 中留有“后门”。尽管争论十分激烈，最终美国政府还是颁布了 DSS。针对 DSS 密钥太短的批评，美国政府将 DSS 的密钥从原来的 512 位提高到 512 ∼ 1024 位，从而使 DSS 的安全性大大增强。从 DSS 颁布至今尚未发现 DSS 有明显缺陷。目前，DSS 的应用已十分广泛，并被一些国际标准化组织采纳作为标准。2000 年 1 月美国政府将 RSA 和椭圆曲线密码引人数字签名标准 DSS，进一步丰富了 DSS 的算法。美国的一些州已经通过了相关法律，正式承认数字签名的法律意义。这是数字签名得到法律支持的一个重要标志。可以预计，从此以后 DSS 数字签名标准将得到广泛的应用。

DSS 使用的算法只能提供数字签名功能，它的整个过程如图9.3所示。

DSS 签名利用一个单向 Hash 函数产生消息的一个 Hash 值，Hash 值连同随机数 k 一起作为签名函数的输入，签名函数还需使用发方的秘密钥 SK_A 和供所有用户使用的一组参数，称这组参数为全局公开钥 PK_G。签名函数的两个输出 s 和 r 就构成了消息的签名（s, r）。接收方收到消息后再产生出消息的 Hash 值，将 Hash 值与收到的签名一起输入验证函数，此外，验证函数还需输入全局公开钥 PK_G 和发方的公开钥 PK_A。验证函数的输出如果与收到的签名成分 r 相等，则证明了签名是有效的。

9.3.2 数字签名算法 DSA

1. 算法参数

DSS 的签名算法称为 DSA，DSA 使用以下参数：

(1) p 为素数，要求 $2^{L-1} < p < 2^L$，其中 $512 \le L \le 1024$ 且 L 为 64 的倍数，即

$$L = 512 + 64j,\ j = 0, 1, 2, \ldots, 8.$$

(2) q 为素数，它是 $(p-1)$ 的因子，$2^{159} < q < 2^{160}$。

(3) $g = h^{(p-1)/q} \bmod p$，其中 $1 < h < p-1$，且满足使 $g = h^{(p-1)/q} \bmod p > 1$。

(4) x 为随机数，$0 < x < q$。

(5) $y = g^x \bmod p$。

(6) k 为随机数，$0 < k < g$。

这里参数 p, q, g 可以公开，且可为一组用户公用。x 和 y 分别为一个用户的私钥和公钥。所有这些参数可在一定时间内固定。参数 x 和 k 用于产生签名，必须保密。参数 k 必须对每一签名都重新产生，且每一签名使用不同的 k。

2. 签名的产生

对数据 M 的签名为数 r 和 s，它们分别如下计算产生：

$$r = (g^k \bmod p) \bmod q,$$

$$s = (k^{-1}(SHA(M)) + xr) \bmod q$$

其中 k^{-1} 为 k 的乘法逆元，即 $k^{-1}k \equiv 1 \bmod q$，且 $0 < k^{-1} < q$。SHA 是安全的 Hash 函数，它从数据 M 抽出其摘要 SHA(M)，SHA(M) 为一个 160 位的二进制数字串。

应该检验计算所得的 r 和 s 是否为零，若 $r = 0$ 或 $s = 0$，则重新产生 k，并重新计算产生签名 r 和 s。

最后，把签名 r 和 s 是附在数据 M 后面发给接收者：$(M \,||\, r \,||\, s)$。

3. 验证签名

为了验证签名，要使用参数 p，q，g，用户的公开密钥 y 和其标识符。令 MP，rP，sP 分别为接收到的 M，r 和 s。

(1) 首先检验是否有 $0 < rP < q, 0 < sP < q$，若其中之一不成立，则签名为假。

(2) 计算：

$$w = \left(sP^{-1}\right) \bmod q,$$

$$u_1 = (\text{SHA}(MP)w) \bmod q,$$

$$u_2 = ((rP)w) \bmod q,$$

$$v = \left((g)^{u1}(y)^{u2} \bmod p\right) \bmod q.$$

(3) 若 $v = rP$，则签名为真，否则签名为假或数据被篡改。

9.4 其他数字签名方案

9.4.1 离散对数签名体制

A. 离散对数签名体制

(1) 体制参数

1. p：大素数；
2. q：$p-1$ 或 $p-1$ 的大素因子；
3. g：$g \in {}_R\mathbb{Z}_p{}^*, g^q \equiv 1 \bmod p$；
4. x：用户秘密，x 为在 $1 < x < q$ 内的随机数；

表 9.1: a、b、c 的可能置换

a	b	c
$\pm r$	$\pm s$	$H(m)$
$\pm rH(m)$	$\pm s$	1
$\pm rH(m)$	$\pm H(m)s$	1
$\pm H(m)r$	$\pm rs$	1
$\pm H(m)s$	$\pm rs$	1

5. y：用户公钥 $y = g^x \bmod p$。

(2) 签名产生过程

1. 计算 m 的杂凑值；
2. 选择随机数 k：$1 < k < q$，计算 $r = g^k \bmod p$；
3. 从签字方程 $ak = b + cx \bmod q$ 中解出 s，(r, s) 为数字签字。系数 a, b 和 c 根据情况而变，表9.1中每行给出了六个可能值。

(3) 签名验证

$$\mathrm{Ver}(y, (r, s), m) = \text{True} \Leftrightarrow r^a \equiv g^b y^c \bmod p.$$

B. ElGamal 数字签名

选 p 是一个大素数，$p-1$ 是有大素数因子。α 是一个模 p 的本原元，将 p 和 α 公开。用户随机地选择一个整数 x 作为自己的秘密的解密钥，$1 \le x \le p-1$，计算 $y \equiv \alpha^x \bmod p$，取 y 为自己的公开的加密钥。公开参数 p 和 α 可以由一组用户共用。

(1) 产生签名

设用户 A 要对明文消息 M 加签名，$0 \le m \le p-1$，其签名过程如下：

1. 用户 A 随机地选择一个整数 k，$1 < k \le p-1$，且 $(k, p-1) = 1$；
2. 计算 $r = \alpha^k \bmod p$；
3. 计算 $s = (m - x_A r)k^{-1} \bmod p - 1$；
4. 取 (r, s) 作为 M 的签名，并以 $\langle m, r, s\rangle$ 的形式发给用户 B。

(2) 验证签名

用户 B 验证 $\alpha^m = {y_A}^r r^s$，是否成立，若成立则签名为真，否则签名为假。签名的可验证性证明如下：

因为 $s \equiv (m - x_A r)k^{-1} \bmod p - 1$，所以 $m \equiv x_A r + ks \bmod p - 1$，从而 $\alpha^m \equiv \alpha^{x_A r + ks} \equiv {y_A}^r r^s \bmod p$，故签名可验证。

对于上述 ElGamal 数字签名，为了安全，随机数 k 应当是一次性的。否则，可用过去的签名冒充现在的签名。

注意，由于取 (r, s) 作为 M 的签名，所以 ElGamal 数字签名的数据长度是明文的两倍，即数据扩展一倍。

例 9.1 取 $p = 11$，生成元 $\alpha = 2$，私钥 $x = 8$。计算公钥

$$y = \alpha^x \bmod p = 2^8 \bmod 11 = 3.$$

取明文 $m = 5$，随机数 $k = 9$，因为 $(9, 11) = 1$，所以 $k = 9$ 是合理的。计算

$$r = \alpha^k \bmod p = 2^9 \bmod 11 = 6.$$

再利用 Euclidean 算法从下式求出 s，过程如下：

$$M = (ks + x_A r) \bmod p - 1,$$
$$5 = (9s + 8 \times 6) \bmod 10,$$
$$s = 3,$$

于是，签名 $(r, s) = (6, 3)$。

为了验证签名，需要验证 $\alpha^M = {y_A}^r r^s$ 是否成立。为此计算

$$\alpha^M = 2^5 \bmod 11 = 32 \bmod 11 = 10,$$
$${y_A}^r r^s \bmod p = 3^6 \times 6^3 \bmod 11 = 729 \times 216 \bmod 11 = 157464 \bmod 11 = 10,$$

因为 $10 = 10$，通过签名验证，这说明签名是真的。

(3) 安全性分析

以下探讨有关 ElGamal 签名体制安全性的几个结果。

1. 在签名验证过程中，检验 $r < p$ 的重要性。Bleichenbacher 发现了下面的攻击，条件是在 Bob 接受的签名中有 $r > p$ 成立。设 (r, s) 表示消息 m 的签名，Malice 可以通过下面的方法伪造一个消息 m' 的一个新签名：
   1) $u = m'm^{-1} \bmod p - 1$；
   2) $s' = su \bmod p - 1$；
   3) 计算 r'，使之满足 $r' = ru \bmod p - 1$ 和 $r' = r \bmod p$；这可以应用中国剩余定理来完成。

   然后，例行的做法是检验下面的同余等式成立：

   $${y_A}^{r'} {r'}^{s'} = {y_A}^{ru} r^{su} = ({y_A}^r r^s)^u = \alpha^{mu} = \alpha^{m'} \bmod p.$$

   如果 Bob 对 $r < p$ 进行验证，就会阻止这样的攻击。这是因为在上面第 3) 步中由中国剩余定理所计算的 r' 是一个 $p(p-1)$ 量级的量。
2. 这也是由 Bleichenbacher 发现的：Alice 应该从 ${F_p}^*$ 中随机地选取公开参数 α。如果该参数不是由 Alice 选取地，那么必须有一个所有用户都知道地公共过程来检验 α 选择的随机性。

   现在假设公共参数 α，p 是由 Malice 选择的。参数 p 可以按如下方法来建立：设 $p - 1 = bq$，其中 q 可以是一个足够大的素数，而 b 可以是平滑的（也就是说，b 仅有一个小的素因子，因此在 b 阶群上计算离散对数是容易的）。

   Malice 可以如下产生 α：

   $$\alpha = \beta^t \bmod p,$$

   对某个 $\beta = cq$，有 $c < b$。

   对 Alice 的公钥 y_A，我们知道，当底为 α 时，求 y_A 的离散对数是困难的。但是，当底为 α^q 时，求取 ${y_A}^q$ 的离散对数是容易的。该离散对数为 $z = x_A \bmod b$，也就是说，下面的同余等式成立：

   $${y_A}^q = (g^q)^z \bmod p.$$

   有了 z，Malice 可以伪造 Alice 的签名如下：

   $$r = \beta = cq,$$
   $$s = t(m - cqz) \bmod p - 1.$$

然后，例行的做法是检验下面的同余等式成立：

$$y_A{}^r r^s \equiv y_A{}^{cq} (\beta^t)^{(m-cqr)} \equiv \alpha^{cqr} \alpha^{rq-cqr} \equiv g^m \bmod p$$

因此，(r, s) 实际上是 m 的一个有效签名，其生成过程没有使用 x_A，而是使用 $x_A \bmod b$。在这个签名伪造攻击中，r 是一个可以被 q 整除的值。而在 p 的标准参数建立阶段，p 满足 $p = bp$，其中 q 是一个大的素数。因此，在验证过程中，如果 Bob 验证 $(q \times r)$，那么就可以防范 Bleichenbacher 所发现的攻击。

3. 最后一个关于短暂密钥 k。类似于 ElGamal 加密：ElGamal 签名的生成过程也是一个随机的算法。它的随机性是由于这个短暂密钥 k 的随机性。
   Alice 永远不要在不同的签名过程中重复使用同一个短暂密钥。如果重复使用一个短暂密钥 k 对两个消息 $m_1 \neq m_2 \bmod p-1$ 进行签名，那么可有

$$K(s_1 - s_2) = m_1 - m_2 \bmod p - 1.$$

   因为 $k^{-1} \bmod p-1$ 存在，$m_1 \neq m_2 \bmod p-1$ 意味着

$$k^{-1} = (s_1 - s_2)/(m_1 - m_2) \bmod p - 1,$$

   即得到了 k^{-1}。依次地，可以计算出 Alice 地私钥 x_A：

$$x_A = (m_1 - ks_1)/r \bmod p - 1.$$

   还应该注意，这个短暂地密钥必须从空间 ${\mathbb{Z}_{p-1}}^*$ 中随机均匀地选取。当一个签名是由小型计算机生成时，比如智能卡或掌上设备等，应该特别注意：必须要确保这些设备配足了足够的、可依赖的随机资源。
   只要保证 k 在每次签名中仅使用一次，并且它是随机均匀产生的，那么签名过程的第二个等式说明它实质上是为签名者的私钥 x 提供了一次性的乘法加密。因此，这两个密钥在信息论安全的意义上彼此互相保护。

C. Schnorr 签名体制

(a). 体制参数

1. p，q：大素数，$q \mid p-1$。q 是大于等于 160bits 的整数，p 是大于等于 512bits 的整数，保证 $\mathbb{Z}_p$ 中求解离散对数困难；
2. g：${\mathbb{Z}_p}^*$ 中元素，且 $g^q \equiv 1 \bmod p$；
3. x：用户密钥 $1 < x < q$；
4. y：用户公钥 $y \equiv g^x \bmod p$。

   空间 $M = {\mathbb{Z}_p}^*$，签字名空间 $S = {\mathbb{Z}_p}^* \times \mathbb{Z}_q$；密钥空间 $K = \{(p, q, g, x, y) \mid y \equiv g^x \bmod p\}$。

(b). 签名过程

令待签消息为 M，对 M 做下述运算：

1. 发送者任选一秘密随机数 $k \in \mathbb{Z}_q$。
2. 计算

$$r \equiv g^k \bmod p,$$
$$s \equiv k + xe \bmod p,$$

   式中 $e = H(r \mid\mid M)$。
3. 签字 $S = \mathrm{Sig}_k(M) = (e, s)$。

(c). 验证过程

收信人收到消息 M 及签字 $S = (e, s)$ 后，执行以下操作：

1. 计算 $r' \equiv g^s y^e \bmod p$ 而后计算 $H(r' \mid\mid M)$；

2. 验证 $\mathrm{Ver}(M,r,s) \Leftrightarrow H(r' \,||\, M) = e$。

因为，若 $(e \,||\, s)$ 是 M 的合法签字，则有

$$g^s y^e \equiv g^{k-xe} g^{xe} \equiv g^k \equiv r \bmod p.$$

9.4.2 利用椭圆曲线密码实现数字签名

相对于 RSA 等密码体制来说，椭圆曲线加密体制是比较新的技术。椭圆曲线密码技术是前几年密码学界的研究热点之一，现在已经逐渐被人们用作基本的数字签名系统（如 ECDSA：Elliptic Curve Digital Signature Algorthm）。ECDSA 和 RSA 与 DSA 的功能相同，但其数字签名产生与认证的速度比 RSA 和 DSA 快。

于 1985 年 N. Koblitz 和 Miller 提出将椭圆曲线用于密码算法，分别利用有限域上椭圆曲线的点构成的群实现了离散对数密码算法。除椭圆曲线外，还有人提出在其他类型的曲线如超椭圆曲线上实现公钥密码算法。椭圆曲线密码是根据有限域上的椭圆曲线上的点群中的离散对数问题（ECDLP）。ECDLP 是比因子分解问题更难的问题，许多密码专家认为它是指数级的难度。从目前已知的最好求解算法来看，160 比特的椭圆曲线密码算法的安全性相当于 1024 比特的 RSA 算法。此后，有人在椭圆曲线上实现了类似 ElGamal 的加密算法. 以及可恢复明文的数字签名方案等。除上述有限域上的椭圆曲线密码算法外，人们还探索了在椭圆曲线上实现 RSA 算法，如 KMOV 等。

1. ECDSA 签名算法

签名的时候，自然有待签署的消息 m；全局参数 $D = (q, \mathrm{FR}, a, b, G, n, h)$，还有签名者的公钥私钥对 (Q, d)。

签名的算法步骤描述如下：

(1) 选择一个随机数 k，$k \in [1, n-1]$；

(2) 计算 $kG = (x_1, y_1)$；

(3) 计算 $r = x_1 \bmod n$；如果 $r = 0$，则回到步骤（1）；

(4) 计算 $k^{-1} \bmod n$；

(5) 计算 $e = \mathrm{SHA1}(m)$；

(6) 计算 $s = k^{-1}(e + dr) \bmod n$，如果 $s = 0$，则回到步骤 (1)；

(7) 对消息的签名为 (r, s)。

最后签名者就可以把消息 m 和签名 (r, s) 发送给接收者。

当接收者收到消息 m 和签名 (r, s) 之后，验证对消息签名的有效性，需要取得如下参数：全局参数 $D = (q, \mathrm{FR}, a, b, G, n, h)$，发送者的公钥 Q。我们建议接收者利用前面的合法性检验算法，也对参数 D 和 Q 的合法性进行检验。

验证算法可以描述如下：

(1) 检验 r、s，要求 r、$s \in [1, n-1]$；

(2) 计算 $e = \mathrm{SHA1}(m)$；

(3) 计算 $w = s^{-1} \bmod n$；

(4) 计算 $u_1 = ew \bmod n$；$u_2 = rw \bmod n$；

(5) 计算 $X = u_1 G + u_2 Q$；

(6) 如果 $X = 0$，表示签名无效；否则，$X = (x_1, y_1)$，计算 $v = x_1 \bmod n$；

(7) 如果 $v = r$，表示签名有效；否则表示签名无效。

下面来证明 $ECDSA$ 算法成立。

证明

如果签名 (r,s) 是消息 m 的合法签名，有 $s=k^{-1}(e+dr) \bmod n$，由此可以得到

$$\begin{aligned} k &\equiv s^{-1}(e+dr) \bmod n \\ &\equiv (s^{-1}e+s^{-1}dr) \bmod n \\ &\equiv (we+wrd) \bmod n \\ &\equiv (u_1+u_2d) \bmod n, \end{aligned}$$

再有 $u_1G+u_2Q=(u_1+u_2d)G=kG$，其中的 kG 横坐标 $x_1=r$，u_1G+u_2Q 的横坐标为 v，即有 $v=r$。

所以 ECDSA 签名于认证算法成立。

2. ElGamal 签名算法

利用椭圆曲线密码可以很方便地实现数字签名，以下给出用椭圆曲线密码实现上节中的 ElGamal 数字签名的算法。

一个椭圆曲线密码系统由下面的六元组描述：

$$T=\langle p,a,b,G,n,h\rangle,$$

其中，p 为大于 3 的素数，p 确定了有限域 $GF(p)$；元素 a，$b\in GF(p)$。a 和 b 确定了椭圆曲线；G 是循环子群 E_1 的生成元，n 为素数且为生成元 G 的阶，G 和 n 确定了循环子群 E_1。d 为用户的私钥。用户的公开钥为 Q 点，$Q=dG$，m 为消息，HASH(m) 是 m 的摘要。

(1) 选择一个随机数 k，$k\in\{0,1,2,\ldots,n-1\}$；

(2) 计算点 $R(x_R,y_R)=kG$，并记 $r=x_R$；

(3) 利用保密的解密钥 d 计算数 $s=(\text{HASH}(m)-dr)k^{-1} \bmod n$；

(4) 以 (r,s) 作为消息 m 的签名，并以 $\langle m,r,s\rangle$ 的形式传输或存储。

验证过程如下：

(1) 计算 $s^{-1} \bmod n$；

(2) 利用公开的加密钥 Q 计算 $U(x_U,y_U)=s^{-1}(HASH(m)G-rQ)$；

(3) 如果 $x_U=r$，则 (r,s) 是用户 A 对 M 的签名。

证明 因为 $s=(\text{HASH}(m)-dr)k^{-1} \bmod n$，所以 $s^{-1}=(\text{HASH}(m)-dr)^{-1}k \bmod n$，所以

$$\begin{aligned} U(x_U,y_U) &= (\text{HASH}(m)-dr)^{-1}k(\text{HASH}(m)G-rQ) \\ &= (\text{HASH}(m)-dr)^{-1}R(\text{HASH}(m)-dr) \\ &= R. \end{aligned}$$

9.5 基于身份的签名方案

1984 年，Shamir 提出了第一个基于身份的数字签名方案，该方案的安全性建立在大数因式分解的困难性假设上。此后，许多基于身份的签名方案被提出，但他们都没有给出方案的安全性证明。2003 年，Cha 和 Cheon 提出了一个基于身份的签名方案，并在定义了基于身份的数字签名方案的安全性模型后给出了方案的安全性证明。

9.5.1 Shamir 的基于身份的数字签名方案

Shamir 的方案包括四个算法：

(1) 初始化：KGC 选择 n、e 和 f，其中 n 为两个大素数 p、q 的乘积，e 是一个与 $\varphi(n)$ 互素的大素数，f 是一个单向函数，则 $\langle n,e,f\rangle$ 为公开的安全参数。

(2) 密钥提取：所有的用户都使用相同的 n、e、f，唯一不同是公开的用户身份值 i，与 i 值相对应的私钥为 g，满足：$g^e = i \bmod n$。由离散对数的困难性可知，其他人计算 g 是困难的。

(3) 签名算法：用户对消息 m 进行签名，首先选择一个随机数 r，计算 $t = r^e \bmod n$，$s = g \cdot r^{f(t,m)}$，则将签名 (s,t,r) 发送给验证者。

(4) 验证算法：验证者收到签名 (s,t,r) 之后，如果

$$s^e = i \cdot t^{f(t,m)}$$

成立，则接受该签名为有效签名；否则，拒绝该签名。

9.5.2 Cha-Cheon 的基于身份的数字签名方案

Cha 和 Cheon 的方案包括 4 个阶段：设置、析出、签名、验证。

(1) 设置：给定安全参数 $k \in \mathbb{Z}^+$，由输入的 k 值生成一个素数 q，两个 q 阶群 $\mathbf{G}_1$ 和 $\mathbf{G}_2$，一个可接受的双线性映射 $\hat{e}$：$\mathbf{G}_1 \times \mathbf{G}_1 \to \mathbf{G}_2$。随机选择生成元 $P \in \mathbf{G}_1$。取随机数 $s \in {\mathbb{Z}_q}^*$ 为主密钥，令 $P_{\text{pub}} = sP$。选择 2 个密码学 Hash 函数 H_1：$\{0,1\}^* \to \mathbf{G}_1^*$ 和 H_2：$\{0,1\}^* \times \mathbf{G}_1^* \to {\mathbb{Z}_q}^*$。系统参数为 $\text{params} = \langle q, \mathbf{G}_1, \mathbf{G}_2, \hat{e}, P, P_{\text{pub}}, H_1, H_2 \rangle$，主密钥为 $s \in {\mathbb{Z}_q}^*$。

(2) 析出：给定串 $\text{ID} \in \{0,1\}^*$，PKG 计算 $Q_{\text{ID}} = H_1(\text{ID})$，计算私钥 $d_{\text{ID}} = sQ_{\text{ID}}$，这里 s 为主密钥。

(3) 签名：给定私钥 d_{ID} 和消息 m，选取随机数 $r \in {\mathbb{Z}_q}^*$，计算 $U = rQ_{\text{ID}}$，$h = H_2(m,U)$ 和 $V = (r+h)d_{\text{ID}}$，输出签名 $\sigma = (U,V)$。

(4) 验证：对于给定的身份 ID，消息 m 的签名 $\sigma = (U,V)$，验证 $\hat{e}(P,V) = \hat{e}(P_{\text{pub}}, U + hQ_{\text{ID}})$ 是否成立，这里 $h = H_2(m,U)$。

Cha 和 Cheon 在预言机模型下证明了其方案的安全性，他们的这个思路被普遍应用于基于身份的签名方案的安全性证明中。

习题 9

1. 选择 $p = 7$，$q = 17$，$e = 5$，试用 RSA 方法对明文 $m = 19$ 进行加密和解密，给出签名和验证结果(给出其过程)，并指出公钥和私钥各为什么?
2. DSA 中，如果签名产生过程中出现 $s = 0$，则必须产生新的 k，并重新计算签名，为什么?
3. 假设 Bob 使用 DSS 对消息 $x = 52$ 签名，$q = 101$，$p = 7879$，$\alpha = 170$，$a = 75$，$\beta = 4567$，随机数 $k = 49$，给出签名结果并验证。
4. 对于 DSS 签名方式来说，每次签名都使用不同的 k 值，即使对同一消息签名两次，其签名结果也是不同的。这和 RSA 签名方式是不同的。这种不同有什么实际意义?
5. 假设在 ElGamal 签名方案中，$p = 31847$，$a = 5$，$y = 25703$。已知 $(23972, 31396)$ 是对消息 $m = 8990$ 的签名，$(23972, 204481)$ 是对消息 $m = 31415$ 的签名，求 k 和 x。
6. ElGamal 签名方案对于一种称为存在伪造的攻击是脆弱的，这里是基本的存在伪造攻击。选择 u, v 使得 $\gcd(v, p-1) = 1$，计算 $r = \beta^r \alpha^u \bmod p$ 并且 $s = -rv^{-1} \bmod p - 1$。
   (a) 证明 (r,s) 数对是消息 $m = su \bmod p - 1$ 的一个有效的签名 (当然 m 很可能不是一个有意义的消息)。
   (b) 假设使用散列函数 h，并签名必须为 $h(m)$ 而不是对 m 有效。解释为什么这个方案能够抵御存在伪造攻击，就是说为什么通过这种过程签名的消息很难产生伪造。

第 10 章　密码学侧信道攻击

目前，密码技术越来越多地嵌入到各种各样的设备中，如智能卡、移动电话、PDA、付费电视解码器等，由于这些持有秘密密钥的设备掌握在个人手中，因此密码设备成为极具吸引力的攻击目标，面临秘密泄露的威胁。例如，如果能够确定加密数字电视传输的密钥，也就能够制造出相应的解码器。

10.1 基本概念

传统的密码分析通常把密码算法看作是抽象的数学函数，密码的具体实现被看作是黑盒子，其内部状态对攻击者来说是不可见的，如利用分组密码输入输出的统计特征发起的线性密码分析、差分密码分析。传统密码分析没有考虑现实中使用密码时所受到的物理方面的影响。密码算法必须在物理设备中实现，并会在操作过程中泄露出与密钥相关的各种各样的信息，这些信息可被敌手获取并被用来推导秘密参数。如智能卡等电子设备，在操作时会消耗能量并产生辐射，在受到温度变化、电磁场影响时，可能会产生错误。侧信道密码分析利用密码系统实现时泄露的额外信息，推导密码系统中的秘密参数。特别地，最近几年，计算错误、执行时间、能量消耗、电磁辐射等侧信道得到了深入研究。如在美国评选 AES 的过程中，密码界达成了这样一种共识：密码算法针对传统密码分析即使是安全的，但如果不能被安全地实现，该算法也是无用的。欧洲 NESSIE 工程在评价密码算法时，考虑了以下评价准则：标准简单侧信道分析、Hamming 重量简单侧信道分析、软件实现的差分侧信道分析、硬件实现的差分侧信道分析。除了计算错误、执行时间、能量消耗、电磁辐射等侧信道之外，更多的侧信道信息也许在将来可以被发现和利用。

侧信道密码分析利用密码具体实现相关的信息，恢复计算中涉及的秘密参数。由于这种攻击与具体的实现有关，因此不是通用的，但比古典密码分析更强大，能够在极少的时间内攻破密码系统，并被认为是对实现密码设备的严重威胁。

从文献上看，侧信道攻击有两种相互交叉的分类：

侧信道攻击可以分为入侵型、非入侵型和半入侵型攻击。入侵型攻击通过特殊的工具对智能卡进行物理篡改。需要打开卡片直接访问芯片表面，如揭开智能卡的保护层，直接在数据总线上连线，观察数据传输。入侵型的攻击可以不干扰芯片的正常操作。非入侵型攻击只利用暴露在外部的可用信息，如运行时间、能量消耗等。半入侵型攻击也需要打开卡片，访问芯片表面，但不去篡改钝化层，也就是对金属表面不需要电接触。

侧信道攻击还可以分为主动攻击和被动攻击。主动攻击是指攻击者篡改芯片的正常操作功能，例如在芯片计算过程中引入错误，发起错误攻击。被动攻击只是观察芯片处理数据的行为，收集可利用的侧信道信息，而不去干扰芯片的操作。被动攻击也可是入侵型攻击，因为可能需要打开芯片，以便于更好地收集信息。

10.2 入侵型攻击

10.2.1 一般的篡改方法

对智能卡的主动攻击主要包括以下几个方面。

(1) 解包装

对智能卡的物理篡改，需要在专门的实验室使用专门的工具，通常需要几个小时甚至几个星期的时间。

对智能卡的入侵攻击首先需要去除芯片的包装，把芯片从塑料卡中分离出来。首先使用锋利的刀削去芯片模块后的塑料，直到环氧树脂可见。加几滴浓硝酸溶解树脂，使用有机溶液丙酮洗去酸和树脂，直到硅表面完全暴露。

(2) 重建线路图

攻击者使用带有 CCD 相机的光学显微镜获得芯片表面的高分辨率图象，可以确定出包括数据与地址总线在内的基本结构。使用一定的技术，能够观察芯片的操作，从而揭示出秘密密钥。

(3) 探针工作站

为了更好地观察分支线路的行为，对攻击者来说，一个可利用的重要工具是探针工作站。探针工作站广泛应用于半导体制造业，它被用来人工检验生产线上的样本。探针工作站的主要组件是一个特殊的光学显微镜，在显微镜手臂上装有弹性探针，探针可以与芯片上的总线建立电接触。

(4) 使用高级光技术

为了探测芯片内部的信息，可以使用离子束、电子束、激光等技术。

通过入侵型攻击，可以完成读取或改写内存推导密钥，如直接读出 ROM 内容、改写 ROM、改写 $EEPROM$、固定寄存器中的某位值、读取内存剩磁、读取总线上的值。

10.2.2 保护措施

智能卡上通常覆盖着钝化层，以防攻击者观察智能卡的操作行为，但这种保护对于一个装备精良的攻击者来说是不够的。还有一些智能卡装有检测器，在实际电路外包裹一层金属层，构成一个不加载敏感数据的检测网。检测网一旦断开或者短路，智能卡就拒绝处理并破坏敏感数据。类似的，对时钟频率进行监测，在不正常的低频率或高频率下，芯片将拒绝操作。但不幸的是，这些保护措施同样具有弱点。

10.3 错误攻击

错误攻击是当前一种强大的密码分析技术，它考虑抗篡改设备上的密码系统（如智能卡），观察密码设备的一些错误操作，期望从错误行为的结果中推导涉及的秘密参数。Boneh、DeMillo 和 Lipton 首次给出了基于计算错误分析秘密密钥的错误攻击，他们的攻击应用在公开密钥密码系统 RSA 中，随后错误攻击被应用到各种安全协议中，如签名方案、身份鉴别方案等。分组密码同样可能遭受错误分析攻击。Biham 和 Shamir 指出在数据加密标准（DES）的实现中，如果寄存器发生错误，DES 很容易被攻破。他们的攻击方法结合了差分分析和错误分析，称之为差分错误分析。Chen 和 Yen 针对 AES 的密钥编排给出了差分错误分析并讨论了相应的对策，Piret 和 Quisquater 给出了代换-置换网络的差分错误分析并应用到 AES。NESSIE 工程中的候选密码也评估了错误攻击的可能。

密码系统出现的错误，既可能是密码实现时无意中存在的漏洞造成的，也可能是攻击者通过改变密码系统的执行环境（改变到不正常的环境）故意引入造成的。例如通过电压瞬变、或高或低的时钟频率、极端温度条件、微波攻击、光照射、涡电流等不同手法均可造成密码设备的故障。最近的结果表明，错误攻击是面向实践的并且是容易实现的。

10.3.1 简单错误分析攻击

错误分析是当前一种强大的密码分析技术，它观察密码设备的一些错误操作，期望从错误行为的结果中推导涉及的秘密参数。

首先我们讨论对使用中国剩余定理（CRT）的 RSA 的简单错误分析攻击。在实现 RSA 模乘幂操作时，为了提高 RSA 的性能，常常利用中国剩余定理。设 m 是待签名的信息，p、q 是两个秘密素数，$n = pq$ 是公开的模数，d 和 e 是私有和公开的指数。使用 CRT 的乘幂操作如算法10.1所示。

算法 10.1: 使用 CRT 的 RSA

1 $m_p = m \bmod p$;
2 $m_q = m \bmod q$;
3 $d_p = d \bmod (p-1)$;
4 $d_q = d \bmod (q-1)$;
5 $x_p = {m_p}^{d_p} \bmod p$;
6 $x_q = {m_q}^{d_q} \bmod q$;
7 $s = \text{chinese}(x_p, x_q) = q(q^{-1} \bmod p)x_p + p(p^{-1} \bmod q)x_q \bmod n$;
8 **返回** s;

假设在计算 x_p 或 x_q 时发生错误，比如在计算 x_p 时，得到一个错误的结果 x_p，那么由 x_p 和 x_q 得到的签名结果 s' 将满足：

$$s'^e \equiv m \bmod q,$$
$$s'^e \not\equiv m \bmod p.$$

于是，计算 $\gcd((s'^e - m) \bmod n, n)$，从而得到秘密指数 q。正如我们看到的，获得一个错误的签名就可以很容易地伪造任何签名。

10.3.2 差分错误分析（DFA）攻击

分组密码同样可能遭受错误分析攻击。Biham 和 Shamir 指出在数据加密标准（DES）的实现中，如果寄存器发生错误，DES 很容易被攻破。他们的攻击方法结合了差分分析和错误分析，因此称之为差分错误分析。

DES 是 16 轮的 Feistel 结构的分组密码，它使用 56 位的密钥，把 64 位的明文分组映射到 64 位的密文分组。DES 的执行过程如算法10.2所示。

算法 10.2: DES 的执行过程

输入: M, K
输出: $C = \text{DES}_K(M)$
1 由密钥 K 导出子密钥 $K_1, K_2, \ldots, K_{16}$；
2 $L_0R_0 \leftarrow IP(M)$;
3 **for** $i = 1, \ldots, 16$ **do**
4 $\quad L_i \leftarrow R_{i-1}$;
5 $\quad R_i \leftarrow L_{i-1} \oplus f(R_{i-1}, K_i)$;
6 **end**
7 $C \leftarrow FP(R_{16}L_{16})$;
8 **返回** C;

16 个子密钥 K_1，K_2，...，K_{16} 由 56 位初始密钥 K 的 48 位子集组成，明文 M 首先进行初始置换 IP，之后划分为两个部分 L_0 和 R_0，在 DES 的每一轮中需要更新两个寄存器 R_i 和 L_i，见图**??**的第 4、5 行。函数 f 定义为：

$$f\left(R_{i-1}, K_i\right) = P\left(S\left(E\left(R_{i-1}\right) \oplus K_i\right)\right),$$

这里，32 位的右半部分 R_{i-1} 通过扩展置换 E 扩展成 48 位，接着与子密钥 K_i 进行异或，异或后的结果分成 8 个 6 位分组，8 个分组经过 8 个表（S-盒）代换，这个代换操作表示成 S。每个 6 位分组经 S-盒代换后成为 4 位分组，所以 S 的输出是 32 位，然后使用 P 进行置换。

在 16 轮结束后，需要进行 FP 置换，FP 是初始置换的逆置换，即 $FP = IP^{-1}$。最终的输出 C 就

表 10.1: S_1 的部分差分分布表

S_1	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7	0x8	0x9	0xA	0xB	0xC	0xD	0xE	0xF
0x1	0	0	0	6	0	2	4	4	0	10	12	4	10	6	2	4
0x2	0	0	0	8	0	4	4	4	0	6	8	6	12	6	4	2
0x4	0	0	0	6	0	10	10	6	0	4	6	4	2	8	6	2
0x8	0	0	0	12	0	8	8	4	0	4	2	8	8	2	2	4
0x10	0	0	0	0	0	0	2	14	0	4	6	12	4	6	8	6
0x20	0	0	0	10	0	12	8	2	0	4	4	4	4	2	0	12

是明文 M 在密钥 K 下的密文。

考虑实现 DES 的智能卡。在 DES 执行过程中，假设存放 R_{i-1} 和 L_{i-1} 的寄存器的单个位出现翻转，这些错误将影响中间结果，并最终得到错误的输出。在以下的讨论中，我们假设错误的加密结果来自 R_{i-1} 的某个位翻转。

为了发起 Biham 和 Shamir 的攻击，敌手需要从智能卡得到一些明文（可能未知）的两个加密结果。一个是在正常的运行环境下得到的正确的密文 C，另一个是在特殊环境下致使寄存器发生错误而得到的错误的密文 C'。

利用 FP 的逆（即初始置换 IP），敌手能够通过 C 得到 $R_{16}L_{16}$，通过 C' 得到 $R'_{16}L'_{16}$。进一步，由于 $L_{16} = R_{15}$。所以敌手也能够获得 R_{15} 和 R'_{15}。如果寄存器错误出现在第 16 轮开始的位置，那么 $R_{15} \oplus R'_{15}$ 将正确地揭示出 R'_{15} 的哪一位翻转了。

由于 $L_{15} = L'_{15}$，并且获得了 $R_{15} \oplus R'_{15}$ 的值，因此能够确定 S-盒的输入差分；根据 $R_{16}L_{16}$ 和 $R'_{16}L'_{16}$ 的值，能够确定相应的输出差分。差分分析可以利用这些输入与输出差分推导出有关 48 位子密钥 K_{16} 的信息。为了更清楚地说明，我们举个例子，假设我们有：

$$E(R_{15} \oplus R'_{15}) = \text{0x100000000000} = 000100 \mid 0 \ldots 0,$$

$$P^{-1}(R_{15} \oplus \tilde{S}R'_{15}) = \text{0xC0000000} = 1100 \mid 0 \ldots 0,$$

这些值意味着，在 16 轮，对于第一个 S-盒 S_1 的输入差分是 000100（或 0x4），输出差分是 1100（或 0xC）。K_{16} 中的 6 位影响 S_1 的输出差分。参考差分分布表，可以发现 64 种 6 位可能密钥值中只有两种能够产生这样的输出差分。因此，我们能够推导出 K_{16} 中的 5 位信息。

表10.1中的差分分布表的每行对应出现一位错误的情况。最左边的列表示输入差分，最上边的行表示输出差分，其余的元素表示在特定输入差分下产生特定输出差分的 6 位密钥的个数。

10.3.3 错误引入

通过改变智能卡的执行环境（不正常的环境）能够在智能卡中引入错误。可以使用以下几种途径：

电压：按 ISO 标准，智能卡工作在 4.5V 到 5.5V 范围内。如果电压发生瞬变（尖峰信号），智能卡的功能将出现问题，事实上，这常常导致计算错误。

时钟：按照标准，智能卡工作在一定的时钟频率下，或高或低的时钟频率将导致处理错误。

温度：把智能卡放在极端温度条件下工作也能够导致处理错误。

辐射：利用微波改变智能卡的操作行为，在智能卡中引入错误，也称微波攻击。

光：光照射也能产生瞬间错误。

涡电流：通过利用涡电流可以产生内存错误。

10.3.4 错误攻击的对策

错误攻击需要密码设备提供错误的输出，为防止这种攻击，密码设备可以首先验证操作的结果，只有当结果正确的时候才输出结果。验证需要额外的操作，也势必损失实现的效率。例如 DES 加密，可以

对明文加密两次，如果两次加密结果相同便认为加密过程没有出现错误，也可以使用解密操作验证 DES 密文正确性。

随机化操作也可以抵抗错误攻击。例如，对于 RSA 算法，首先对信息使用随机位填充，然后再进行加密或签名。

智能卡可以采用入侵检测和自检测对付错误引入，从而避免错误攻击。

10.4 时间攻击

密码实现需要的执行时间是评价不同密码方案的一个因素，加密一条信息或产生数字签名需要的时间是衡量密码效率的标准，速度快的方案被认为是高效的。

计算一个密码函数需要的时间不仅依赖于函数功能，也依赖于函数输入。即不同的函数通常具有不同的计算时间，同一个函数的不同输入也通常具有不同的计算时间。密码函数使用秘密密钥作为输入，密钥的值可能会影响到可观察的时间特征。

1995 年 Kocher 描述了密码系统如 RSA、DSS 和 Diffie-Hellman 的时间特征如何与私有密钥相关的，论述了攻击者通过测量时间推导签名者私钥，此后密码学界认识到现实使用的一些产品与协议（如 SSL）对于时间攻击是脆弱的，这种新的密码分析方法甚至刊登到了纽约《时代》周刊的首页。

10.4.1 对平方-乘算法的时间攻击

许多公开密钥密码系统的基本操作是模乘幂，模乘幂被用来加密、解密、签名、验证签名一个信息分组，如 RSA、椭圆曲线密码系统等。在 RSA 算法中，为了有效实现模乘幂操作，RSA 发明者建议使用平方-乘算法。一些 RSA 实现采用了这种算法。图 9.4 描述了平方-乘算法，输出 S 可看作是数字签名，私有指数 d 为最多 n 位的 2 进制表示，n 是 RSA 模 N 的位长度。

算法 10.3: 模乘幂的平方-乘算法

输入: M, N, $d = (d_{n-1}d_{n-2}\dots d_1d_0)_2$
输出: $S = M^d \bmod N$

1 $S \leftarrow 1$;
2 **for** $j = n-1, \dots, 0$ **do**
3 $\quad S \leftarrow S^2 \bmod N$;
4 $\quad$ **if** $d_j = 1$ **then**
5 $\quad\quad S \leftarrow S \times N$;
6 **返回** S;

对于平方-乘算法，Kocher 进行了仔细观察，算法10.3第 4 行的条件表达式根据不同的指数值算法会出现不同的执行路径。在每个循环中，如果相关位 d 是 1，将执行模平方和模乘（第 3、5 行）；如果相关位是 0，只执行模平方。因此，n 步循环迭代的执行时间都受到指数值的影响。

如果攻击者能够观察并比较平方-乘算法中循环迭带的执行时间，他将能够推导出对应的指数位。将这种技巧应用到 RSA 签名操作，便能揭示出签名者的私有密钥。如何观察到每次循环迭代的时间特征这里还不是很清楚，在后边介绍的能量分析中，可以看到如何通过能量分析获得每步的时间信息的。Kocher 的时间攻击描述了攻击者利用算法的全部执行时间推导私有指数位，一个被动攻击者可以很容易观察到全部执行时间。

假设一个恶意用户 Marvin，向使用平方-乘算法实现 RSA 的 PC 发送一系列已知消息 M_1，M_2，...，$M_k \in Z_N$ 请求签名，Marvin 记录 PC 返回签名的时间 T_1，T_2，...，T_k。Marvin 能够恢复指数 d 的位。

因为 $d < N$，n 是 N 的位长度，d 的 2 进制表示可能包括许多高位 0，为简化讨论，假设 $d_{n-1} = 1$。参考算法10.3中的伪码，在第 2 次循环迭代的开始，$S = M \bmod N$，接着计算模平方 $S = M^2 \bmod N$。如果 $d_{n-2} = 1$，PC 计算模乘 $M \cdot M^2 \bmod N$，否则将不计算。

利用目标 PC 的物理技术规范，Marvin 配置一台与目标 PC 同样的仿真 PC（也就是相同的处理器、RAM 等），对每个已知的明文 M_i 在仿真机上计算模乘 $M \cdot M^2 \bmod N$，所需的时间记为 t'_i。模乘 $M \cdot M^2 \bmod N$ 的计算时间 t'_i 受明文 M_i 的影响。

Kocher 注意到，当 $d_{n-2} = 1$，两个样本集 $\{t'_i\}$ 和 $\{T_i\}$ 是相关的。即当 t'_i 比它的数学期望大的多时，T_i 也比它的数学期望大的多。如果 $d_{n-2} = 0$，两个样本的行为表现为独立的随机变量。通过样本的表现是否相关，Marvin 能够确定 d_{n-2} 的值。现在 Marvin 知道了第 3 次迭代开始时 S 的值。为了得到 d_{n-3}，Marvin 在仿真机上计算 $S \cdot M \bmod N$，构造新的样本集 $\{t'_i\}$，再比较样本集 $\{t'_i\}$ 和 $\{T_i\}$ 是否相关，表现相关则 $d_{n-3} = 1$，否则为 0。利用同样的方法可以确定指数 d 其他的值。

Kocher 的时间攻击利用了不同的操作数模乘的计算时间不同。假定平方和乘运算采用 Montgomery 算法。模乘的 Montgomery 算法运行需要的时间几乎是一个常数，但条件减带来一些细微的时间变化，从而面临时间攻击的弱点。

我们这里不描述 Montgomery 算法，对于固定的模，Montgomery 乘的时间是独立于乘数因子的，除此之外，当乘积的中间结果大于模数时，需要执行额外的减操作（称为约减 reduction），因此需要更多的计算时间。

对一些消息明文 M，在 Montgomery 乘过程中，需要执行额外的约减，而其他一些信息，则不需要约减。于是，可以将样本集划分成 2 个子集：一个是计算 $M \cdot M^2 \bmod N$ 需要约减的样本集合，一个是不需要约减的样本集合。如果 d_{n-2} 确实为 1，我们可以期望第一个集合中信息的计算时间将稍微大与第二个集合中信息的计算时间。相反，如果 d_{n-2} 为 0，则 $M \cdot M^2 \bmod N$ 将不需要计算。在这种情况下，我们的划分标准将没有意义，从而划分的两个样本集合看起来是随机的，两个集合的计算时间将不会有明显的差别。考察划分后的两个集合的时间差别，Marvin 可以确定 d_{n-2} 的值。

确定了 d_{n-2} 的值之后，Marvin 对每个已知的明文 M_i 在仿真机上计算前 2 次循环所花的时间，利用同样的方法就推导出 d_{n-3} 及之后的位。

时间攻击具有错误检测的性质，如果指数中的某位 d_b 计算错误，则所有的 $d_{k \geq b}$ 都将错误。在猜测错误的指数位之后，将不能再观察到相关性。也就是说，如果相关性不再存在，则攻击者猜测的指数位意味着是错误的。这些错误检测的性质，能够用来修正错误。

10.4.2 对多位窗口平方-乘算法的时间攻击

RSAREF 2.0 由 RSA Laboratories 在 1994 发布的一些通用密码方案的参考实现，其中模乘幂采用更一般形式的多位窗口平方-乘算法，算法10.4的伪码描述了使用 2 位窗口的平方-乘算法。

假设 Alice 和 Marvin 使用他们自己的 PC 参与签名协议。Marvin 向 Alice 发送消息，Alice 使用 RSAREF 程序和她的私钥对 $\langle N, d \rangle$ 对消息进行签名，随后将签名送回 Marvin。Marvin 记录他发送消息 M_i 到 Alice 响应的时间 T_i。

有多种因素对值 T_i 有贡献。参看算法10.4，我们用 c_i 表示执行第 1 到 4 行需要的时间，在算法10.4的循环中，对特定的值 j，执行第 6、7 和 9 的时间相应地表示为 r_i，j、s_{ij} 和 t_{ij}。r_{ij} 和 s_{ij} 是严格的正值，t_{ij} 可以为 0。其他因素，如测量误差和传输距离同样对 T_i 有贡献，这可看作是源误差，记成 e_i。于是 T_i 可以表示成：

$$\begin{aligned} T_i &= e_i + c_i + (r_{i,n-1} + s_{i,n-1} + t_{i,n-1}) + (r_{i,n-3} + s_{i,n-3} + t_{i,n-3}) + \cdots + (r_{i,1} + s_{i,1} + t_{i,1}) \\ &= e_i + c_i + \sum_j (r_{i,j} + s_{i,j} + t_{i,j}). \end{aligned}$$

算法 10.4: 2 位窗口的平方-乘算法

输入: $M,\ N,\ d=(d_{n-1}d_{n-2}\dots d_1d_0)_2$
输出: $S=M^d \bmod N$

```
m_1 ← M mod N;
m_2 ← m1 · M mod N;
m_3 ← m2 · M mod N;
S ← 1;
for j = n − 1, ..., 0 by 2 do
    S ← S^2 mod N;
    S ← S^2 mod N;
    S ← S · m_{(d_j d_{j−1})_2} mod N;
返回 S;
```

Alice 的秘密指数影响 T_i 中几乎所有的部分。对于特定的 j,两个平方操作中的操作数完全由 $d_{n-1}d_{n-2}\dots d_{j+2}d_{j+1}$ 确定，乘操作中的操作数受 $d_{n-1}d_{n-2}\dots d_jd_{j-1}$ 的影响。c_i 只受值 M_i 的影响。

考虑算法的第 1 次循环迭代，Marvin 利用一台与 Alice 机器相同的仿真机，使用 $d_{n-1}d_{n-2}$ 的 4 种可能的值（00、01、10、11）执行信息签名的第 1 次循环迭代，生成 $c_i+(r_{i,n-1}+s_{i,n-1}+t_{i,n-1})$ 的 4 个侯选值 $T'_{i,n-1,0}$、$T'_{i,n-1,1}$、$T'_{i,n-1,2}$ 和 $T'_{i,n-1,3}$。Marvin 构造如下的表:

00	01	10	11
$T_1-T'_{1,n-1,0}$	$T_1-T'_{1,n-1,1}$	$T_1-T'_{1,n-1,2}$	$T_1-T'_{1,n-1,3}$
$T_2-T'_{2,n-1,0}$	$T_2-T'_{2,n-1,1}$	$T_2-T'_{2,n-1,2}$	$T_2-T'_{2,n-1,3}$
$T_3-T'_{3,n-1,0}$	$T_3-T'_{3,n-1,1}$	$T_3-T'_{3,n-1,2}$	$T_3-T'_{3,n-1,3}$
…	…	…	…

其中:

$$\begin{aligned} T_i-T'_{i,n-1,g} &= e_i+c_i+\sum_j (r_{i,j}+s_{i,j}+t_{i,j})-c_i-(r_{i,n-1}+s_{i,n-1}+t'_{i,n-1,g}) \\ &= e_i+\sum_{j<n-1}(r_{i,j}+s_{i,j}+t_{i,j})+(t_{i,n-1}-t'_{i,n-1,g}), \end{aligned}$$

这里，$t_{i,n-1g}$ 是 $t_{i,n-1}$ 候选值，如果 $g=0$，那么 $t_{i,n-1,g}=0$，否则 $t_{i,n-1,g}>0$。

在表的 4 列中有一列，Marvin 的仿真操作将与 Alice 实际执行到第一次循环迭代结束的操作相同。在这列中，Marvin 的候选值比其他 3 个候选值更接近。$c_i+(r_{i,n-1}+s_{i,n-1}+t_{i,n-1})$ 经过分析可知，这个相关列的样本方差具有较高的概率比其他列低。通过比较 4 个样本方差，$Marvin$ 能够确定 $d_{n-1}d_{n-2}$ 的值。

确定 $d_{n-1}d_{n-2}$ 的值之后,为了推导出 $d_{n-3}d_{n-4}$ 的值,Marvin 使用 4 个指数 $d_{n-1}d_{n-2}00$、$d_{n-1}d_{n-2}01$、$d_{n-1}d_{n-2}10$ 和 $d_{n-1}d_{n-2}11$ 在仿真机对每一个信息 M_i 进行签名，记录从开始到第 2 次循环迭代结束之间的执行时间，这 4 个候选值记为 $T'_{i,n-3,0}$、$T'_{i,n-3,1}$、$T'_{i,n-3,2}$ 和 $T'_{i,n-3,4}$。$Marvin$ 构造如下的表:

$T_i-T'_{i,n-3,0}$	$T_i-T'_{i,n-3,1}$	$T_i-T'_{i,n-3,2}$	$T_i-T'_{i,n-3,3}$

接着 Marvin 计算每一列的样本方差以确定 $d_{n-3}d_{n-4}$ 的真实值。私有指数的其他位可以使用同样的方法得到。

10.4.3 时间攻击的对策

1. 隐藏时间差别

   最明显的对策就是保证所有的加密、解密操作都消耗常量时间，但达到这个目标是十分困难的。编译优化、内存查找能够引起意外的时间变化，而这是实现者不能控制的。为了保证操作具有常量时间，可以通过增加延迟达到这个目的，但延迟的时间信息能通过能量消耗泄露出来。另外，消耗的这个常量时间是最坏情形的消耗时间，因此这种方法将大大降低系统性能。

   增加随机延迟隐藏操作的时间差别是另一种对策。但如果随机延迟的平均值很大，这将严重降低密码系统的效率，另外通过增加样本数也能够过滤掉这种时间噪音。

2. 隐藏内部状态

   隐藏内部状态从而使得攻击者不能仿真密码设备的内部计算过程。例如，Kocher 建议在 RSA 中使用盲因子，如果 Marvin 不知道模乘幂中基的值，那么对应的时间信息在时间攻击中就不可用。Alice 在对信息 $M \in \mathbb{Z}_N{}^*$ 签名前，选择一个随机数 $r \in \mathbb{Z}_N{}^*$，对信息 $M' = r^e g M \bmod N$ 签名，结果记成 S'。Alice 现在利用 S' 计算出信息 M 的签名：$r^{-1}S' = r^{-1}r^{ed}M^d = r^{-1}rM^d = M^d \bmod N$。

10.5 能量攻击

除了运行时间，密码设备的能量消耗也能够提供与操作有关的秘密参数信息，这就是 Kocher 等人于 1998 年提出的简单能量分析（SPA）、差分能量分析（DPA）的思想，他们的研究也再次登上了纽约《时代》周刊。

电子设备运行的时候需要电源，为了测量电路的能量消耗，一个大约 $10-50\Omega$ 的电阻器与电源（或大地）串联，示波器能够测量出通过电阻器的电压的变化，从而利用 Ohm 定律计算出电流。数字示波器能够以很高的频率样本化一段时间内的电压，由此可以得到电流的迹（trace）。由于为密码设备提供的电压是恒定的，因此密码设备消耗的功率正比于电流，如果能量消耗使用功率这个指标，则功率分析攻击与电流分析攻击只相差一个常数因子。

10.5.1 简单能量分析（SPA）攻击

简单能量分析通过直接分析密码设备操作时的能量消耗，从单个能量消耗曲线中推导出涉及到的秘密参数信息。由于 SPA 能够揭示出执行的指令序列，所以能够用来破解执行路径依赖于处理数据的密码系统。例如：实验表明，DES 操作的能量消耗迹能清楚地表示出初始置换、16 轮 DES 轮函数操作和最后的一次置换。

当不同的操作具有不同的能量消耗，或者同一种操作的不同操作数具有不同的能量消耗时，密码系统将容易遭受 SPA 攻击。如密码实现常涉及如下一些容易遭受 SPA 攻击过程：

密钥循环移位：通常实现循环移位的做法是首先移去寄存器的最后一位，在另一端增加一个 0，如果移去的位是 1，则增加的 0 位需要翻转，这个条件操作在能量迹中能够被检测出来。DES 密钥编排涉及到 28 位密钥寄存器的循环移位，利用 SPA 能够揭示密钥全部的 56 位。

模乘幂：模乘幂常常采用平方-乘算法。在平方-乘算法中如果指数位为 1，则需要执行额外的乘操作。如果平方操作与乘操作具有不同的能量消耗特征，则从平方-乘算法的能量迹中能够分辨出条件分支，从而确定出全部的指数位。例如我们得到平方操作(S)、乘(M)操作的部分序列是 $SMSSSMSMSSSSMSMS$，则对应的指数位是 10011000110。

位置换：在软件或微代码实现位置换时，“0”、“1”位的不同将带来能量消耗的不同。

比较：在串比较操作中，当相应位不匹配时，将执行一个条件分支。这个条件分支容易遭到 SPA 攻击。

模乘：泄露与具体的实现有关，但模乘操作易于泄露有关操作数的信息，如操作数的 Hamming 重量。操作数的 Hamming 重量是指操作数的非 0 位个数。

实验发现在访问操作数时，能量消耗与操作数的 Hamming 重量相关。对于敌手来说，操纵密钥位时的能量消耗具有特别的意义，从一条能量消耗曲线中久能够推出密钥的某些部分的 Hamming 重量。这里我们给出一个例子说明敌手如何通过操作数的 Hamming 重量暴露秘密参数的。

平均来说，56 位 DES 密钥的 Hamming 重量传达的信息（熵）大约是 3.95 位。

$$-\sum_{i=0}^{56}\frac{\binom{56}{i}}{2^{56}}\lg\frac{\binom{56}{i}}{2^{56}}\approx 3.95.$$

在许多智能卡中微处理器使用 8 位分组处理数据，如果知道了 DES 密钥的每个字节的 Hamming 重量，则这个字节的 Hamming 重量传达的信息大约是 2.54 位。

$$-\sum_{i=0}^{8}\frac{\binom{8}{i}}{2^{8}}\lg\frac{\binom{8}{i}}{2^{8}}\approx 2.54.$$

全部 7 个字节的 Hamming 重量传达的信息约是 $7\times 2.54=17.8$ 位。这些信息使得 DES 易于遭到暴力攻击，因为密钥空间已经从 2^{56} 降低到大约 2^{38}。当然如果使用足够长的密钥（如 $3DES$），则利用 Hamming 重量分析的暴力攻击仍然是不可行的。

依赖于特定的目标密码，使用更有效的 Hamming 重量分析是可能的。在 DES 中，包括奇偶校验位在内的 64 位 DES 密钥记成 $k_1k_2\ldots k_{64}$。第 1 轮子密钥包括的位如下：

10	51	34	60	49	17	33	57
2	9	19	42	3	35	26	25
44	58	59	1	36	27	18	41
22	28	39	54	37	4	47	30
5	53	23	29	61	21	38	63
15	20	45	14	13	62	55	31

这个子密钥的第一个字节可以用如下的方程描述：

$$k_{10}+k_{51}+k_{34}+k_{60}+k_{49}+k_{17}+k_{33}+k_{57}=w1.$$

通过以上方法把所有的密钥字节的 Hamming 重量表示出来，得到 $16\times 6=96$ 个方程，其中含有 56 个未知密钥位。对于任意的 Hamming 重量值来说，这个方程组具有唯一解。

通过仔细研究 DES 的密钥编排方案，96 个方程包含的变量分别属于两个 28 位密钥位的子集，于是方程组可以分解成两个独立的方程组，每个方程组包括 48 个方程和 28 个未知位。在实际操作中，通过能量迹得到的 Hamming 重量可能会出现一些错误，但方程组中包含了大量的冗余，这些冗余方程能够推导出的密钥，从而能够获得最可能的密钥值。

10.5.2 差分能量分析（DPA）攻击

差分能量分析是一种比简单能量分析更具有威胁的一种密码分析方法。为了发起 DPA 攻击，敌手需要涉及到两个阶段：数据收集与数据分析。数据收集就是收集密码设备使用同一个密钥和不同的输入时执行加密操作的能量迹，同时敌手还需要截获最终生成的密文。DPA 攻击使用统计分析和错误相关技巧推导与密钥相关的信息。

这里我们介绍一个对 DES 的 DPA 攻击例子：

(1) 收集数据构成样本集合。测量 1000 次 DES 操作的最后几轮的能量消耗，每次包含 100000 个数据点。收集到的数据表示成 2 维数组 $S[0\ldots 999][0\ldots 99999]$，这里第一个指标表示第几次收集的样本，第

二个指标表示该样本的数据点。对这个例子来说，假设攻击者也获得了加密后的密文 $C[0\ldots999]$。

(2) 接着攻击者选择一个密钥依赖的选择函数 D，攻击者利用这个选择函数把原来的样本集合划分成两个样本子集合。在这个例子中，选择函数具有形式 $D(K_i,C)$，这里 K_i 是一些候选的密钥信息，C 是一个密文。例如，攻击者的目的是找到提供给 DES 的第 16 轮作为 S-盒 4 输入的 6 位密钥，所以 K_i 是 6 位的输入，对应 64 种可能的密钥，并且其中只有一种对应实际的密码设备的密钥（设为 K_s）。现在我们确定选择函数 $D(K_i,C)$ 的值：对任意一种可能的 6 位密钥 K_i，利用初始置换 IP，敌手能够通过 C 得到 $R_{16}L_{16}$。对 L_{16} 执行扩展置换 E，取出作为第 16 轮 S-盒 4 输入的 6 位，与 K_i 进行异或运算，然后通过 S-盒 4 进行代换，选择 S-盒 4 结果中的 1 位（如最高位），对这个位执行 P 置换操作，置换后该位所在的位置称为目标位（也就是 L_{15} 中的某位作为目标位），如果目标位与 R_{16} 中对应位置的位相同则 $D(K_i,C)$ 定义为 1，否则定义为 0。

(3) 利用数据集 S 和选择函数 D 构造差分平均迹 $T[0\ldots63][0\ldots99999]$，即

$$T\left[K_i\right][j]=\frac{\left(\sum_{l=0}^{999}D\left(K_i,C\left[l\right]\right)\right)S\left[l\right][j]}{\sum_{l=0}^{999}D\left(K_i,C\left[l\right]\right)}-\frac{\left(\sum_{l=0}^{999}\left(1-D\left(K_i,C\left[l\right]\right)\right)\right)S\left[l\right][j]}{\sum_{l=0}^{999}\left(1-D\left(K_i,C\left[l\right]\right)\right)}.$$

(4) 对 K_i 来说，攻击者知道只有 64 个可能值中的一个与设备的密钥值相关，其他的值都是不相关的，攻击目标就是确定正确的密钥值。当目标设备执行 DES 操作时，目标位的值将存储在寄存器里，“0”与“1”的不同将产生可检测的能量消耗差分。所以，当 $T[K_i=K_s]$，样本集 $T[K_i]$ 将显示能量消耗偏差，当 $T[K_i\neq K_s]$ 时，$D(i,C[l])$ 将不对应设备的任何操作，平均值将是 0。

(5) 对剩下的 S-盒重复以上步骤可以确定最后一轮的 48 位子密钥。余下的 8 位密钥可以使用暴力攻击找到。如果不能利用暴力攻击，在可以使用类似的方法确定前一轮的子密钥。

对乘幂系统的 DPA，Messerges 等人指出可被攻破的三种类型的场景：

SEMD：单指数、多数据。攻击者需要设备使用秘密指数和一个攻击者已知的指数对许多随机信息执行乘幂运算。

MESD：多指数、单数据。攻击者需要设备使用秘密指数和多个攻击者选择的指数对单个信息执行乘幂运算。

ZEMD：零指数、多数据。这是一种最现实的攻击，攻击者只需要设备使用秘密指数对多个信息执行乘幂运算。

高阶 DPA 是 DPA 的重要进展。一阶（古典）DPA 依据的思想是目标位不同在某些点能量消耗将产生差异。高阶 DPA 的基本思想是目标位不同则某几个点的联合能量消耗将表现出差异。

10.5.3 能量攻击的对策

可以通过软件也可以通过硬件抵抗能量攻击。

在密码算法中使用与秘密参数相关的条件分支能够带来 SPA 脆弱性，消除这种条件分支可以降低 SPA 攻击的风险。这种策略可以应用到平方-乘算法，见图**??**。

即使对所有的秘密参数都使用固定的执行路径，但如果操作的能量消耗与操作数相关，也存在能量攻击的弱点。对付这个问题的一种方法是使用秘密共享中的门限方案，把原操作数分解成多个“影子”，分别处理这些“影子”，在这种情况下，可以使用高阶 DPA 进行攻击，但多个“影子”有效地增加了噪音，从而增加了攻击者难度。这种方法也同时降低了系统的性能。

在密码操作的执行过程中插入随机的计算是一种对付 DPA 的通用方法。例如，在执行加密操作时，随机地插入虚假的运算，从而每次加密都产生不同的能量迹，加大了 DPA 的难度。

硬件组件（如电容、感应器）可以增加到智能卡的能源线上，使得外部电源不直接连接内部芯片，从而降低能量消耗与内部操作的相关性，以此过滤、平滑能量消耗特征，减低能量消耗偏差，增加 DPA 攻击所需的能量迹。

算法 10.5: 抗 SPA 的平方-乘算法

输入: M, N, $d=(d_{n-1}d_{n-2}\dots d_1d_0)_2$
输出: $S=M^d \bmod N$

```
S ← 1;
for j = n − 1, ..., 0 do
    S_0 ← S^2 mod N;
    S_1 ← S_0 · S^2 mod N;
    S ← S_{d_j} ;
返回 S;
```

10.6 电磁攻击

电荷的任何运动都伴随着电磁场。基于电磁的分析为远距离攻击密码设备提供了一条途径。

时间攻击、能量攻击和电磁攻击可看成时不同维数的侧信道。时间攻击是一维的，在每个测量单位内只测量运行时间；能量攻击是两维的，在每个测量单位内测量一系列的能量消耗值；电磁攻击是多维的，在每个测量单位内可以在不同的位置测量一系列的电磁辐射信息。

测量的电磁辐射信息可采用与能量分析相同的方式,如简单电磁分析(SEMA)、差分电磁分析(DEMA)。电磁辐射包含多样信号，针对密码设备的计算，每种信号泄露一些不同的信息。

如同能量攻击一样，电磁攻击需要样本采集设备，如数字示波器或基于 PC 的数字样本卡。电磁攻击设备的关键组成部分是电磁接受/调制器。

第 10 章 习题

1. 入侵型侧信道攻击是主动攻击还是被动攻击?
2. 简述差分错误分析（DFA）攻击及其对策。
3. 举出典型时间攻击的例子。
4. 差分能量分析（DPA）攻击涉及哪两个阶段？举例说明。
5. 怎样理解时间攻击、能量攻击和电磁攻击可看成时不同维数的侧信道?

第 11 章　密码协议

密码协议是解决网络安全问题最有效的手段之一，它可以有效地解决以下一些重要的安全问题：源认证和目标认证，消息的完整性，匿名通信，抗拒绝服务，抗抵赖，授权等等。

11.1 什么是密码协议

所谓协议就是两个或两个以上的参与者为完成某项任务所采取的一系列步骤。这些步骤是有序的，必须依次执行，只有前一步骤完成了，才能执行后一步骤。协议至少需要两个参与者。一个人也可以通过执行一系列步骤来完成某项任务，但它不构成协议。另外，通过执行协议必须能够完成某项任务。某些看似协议，但它没有完成任何任务，也不能成为协议。

协议具有下述特点：

· 协议的每个参与者都必须了解协议，事先知道所要完成的所有步骤。

· 协议的每个参与者都必须同意并遵循协议。

· 协议必须是清楚的，也就是说，每一步骤必须明确定义，不会引起误解。

· 协议必须是完整的，对每种可能的情况都要规定具体的操作。

所谓密码协议就是应用密码技术构造的协议。参与密码协议的各方可能是朋友或可信任的人，也可能是敌人或相互不信任的人。密码协议的目的就是在完成某项任务的同时，不仅能够发现或防止协议参与者彼此之间的欺骗行为。还要能够避免敏感信息被窃听者窃取或篡改。密码协议必包含某种密码算法，其研究包括两个方面：协议的安全性分析和各种实用密码协议的设计与分析。

密码学的用途是解决种种难题。当我们考虑现实世界中的应用时，常常遇到以下安全需求：机密性、完整性、认证性、匿名性、公平性等，密码学解决的各种难题围绕这些安全需求。密码协议（cryptographic protocol）是使用密码学完成某项特定的任务并满足安全需求的协议，又称安全协议（security protocol）。常用的密码协议还包括：

(1) 密钥建立协议

通常的密码技术是使用对称密码算法用单独的密钥对每一次单独的会话加密，这个密钥称为会话密钥。密钥建立协议的目的是在两个或者多个实体之间建立会话密钥。协议可以采用对称密码体制，也可以采用非对称密码体制。有时通过一个可信的服务器为用户分发密钥，这样的密钥建立协议称为密钥分发协议；也可以通过两个用户协商，共同建立会话密钥，这样的密钥建立协议称为密钥协商协议。

(2) 认证协议

当某用户登录进入计算机时，计算机怎么知道他是谁呢？计算机怎么知道不是其他人伪造的身份呢？认证协议主要防止假冒攻击。将认证和密钥建立协议结合在一起，是网络通信中最普遍应用的安全协议。

(3) 电子商务协议

该类协议中主体往往代表交易的双方，其利益目标不一致。因此，电子商务协议最关注公平性，即协议应保证交易双方都不能通过损害对方利益而得到它不应该得到的利益。常见的电子商务协议有 SET 协议、iKP 协议。

(4) 安全多方计算协议

安全多方计算协议的目的是保证分布式环境中各参与方以安全的方式来共同执行分布式的计算任务。安全多方计算协议包括：抛币协议、广播协议、选举协议、电子投标和拍卖协议、电子现金协议、合同签署协议、匿名交易协议、保密信息检索、保密数据库访问、联合签名、联合解密等协议。

密码协议在网络通信中有着重要的应用价值。目前已有许多各种各样的用于不同目的的密码协议。例如保密选举协议、数字现金协议、多方计算协议等密码协议。

11.2 密码协议的安全性

密码协议是许多分布式系统安全的基础，确保这些协议的安全运行是极为重要的。大多数密码协议只有为数不多的几个消息传递，其中每一个消息都是经过巧妙设计的，消息之间存在着复杂的相互作用和制约；同时，密码协议中使用了多种不同的密码体制，密码协议的这种复杂情况导致目前许多密码协议存在安全缺陷。造成协议存在安全缺陷的原因主要有两个：一是协议设计者误解或者采用了不恰当的技术；二是协议设计者对环境要求的安全需求研究不足。因此，对协议的安全性进行分析和研究是一个重要的课题。

由于实际应用的密码协议产生缺陷的原因是多种多样的，所以很难有一种通用的分类方法对密码协议的安全缺陷进行分类。根据安全缺陷产生的原因和相应的攻击方法可以对安全缺陷进行如下分类：

(1) 基本协议缺陷：
是指在密码协议的设计中没有或者很少防范攻击者的攻击。

(2) 口令/密钥猜测缺陷：
这类缺陷产生的原因是用户往往从一些常用的词中选择其口令，从而导致攻击者能够进行口令猜测攻击；或者选取了不安全的伪随机数生成算法构造密钥，使攻击者能够恢复该密钥。

(3) 陈旧消息缺陷：
主要是指协议设计中对消息的新鲜性没有充分考虑，从而使攻击者能够进行消息重放攻击，包括消息源的攻击、消息目的的攻击等等。

(4) 并行会话缺陷：
在攻击者的特意安排下，一个协议的两个或更多的运行并发执行，并发的多个运行使得攻击者能够从一个运行中得到另外某个运行中困难问题的答案。包括并行会话单角色缺陷、并行会话多角色缺陷等等。

(5) 内部协议缺陷：
协议的可达性存在问题，协议的参与者中至少有一方不能够完成所有必须的动作而导致的缺陷。

(6) 密码系统缺陷：
协议中使用的密码算法和密码协议导致协议不能完全满足所要求的机密性、认证等需求而产生的缺陷。

密码协议的安全性是一个很难解决的问题，许多广泛应用的密码协议后来都被发现存在安全缺陷。因此，从密码协议的分析和设计角度来看，我们都不能存在轻信和盲从心理，而应当对协议的安全性作认真分析。

目前，对密码协议进行分析的方法主要有两大类：一类是攻击检验方法；一类是形式化的分析方法。

所谓攻击检验方法就是搜集目前使用的对协议的有效攻击方法，逐一对密码协议进行攻击，检验密码协议是否具有抵抗这些攻击的能力。在分析的过程中主要使用自然语言和示意图，对密码协议所交换的消息进行剖析。这种分析方法往往是非常有效的，关键在于攻击方法的选择。

形式化的分析方法是采用各种形式化的语言或者模型，为密码协议建立模型，并按照规定的假设和分析、验证方法证明协议的安全性。目前，形式化的分析方法是研究的热点，但是就其实用性来说，还没有什么突破性的进展，其应用也不是非常广泛，主要原因是密码协议的安全性的形式化过程比较困难。

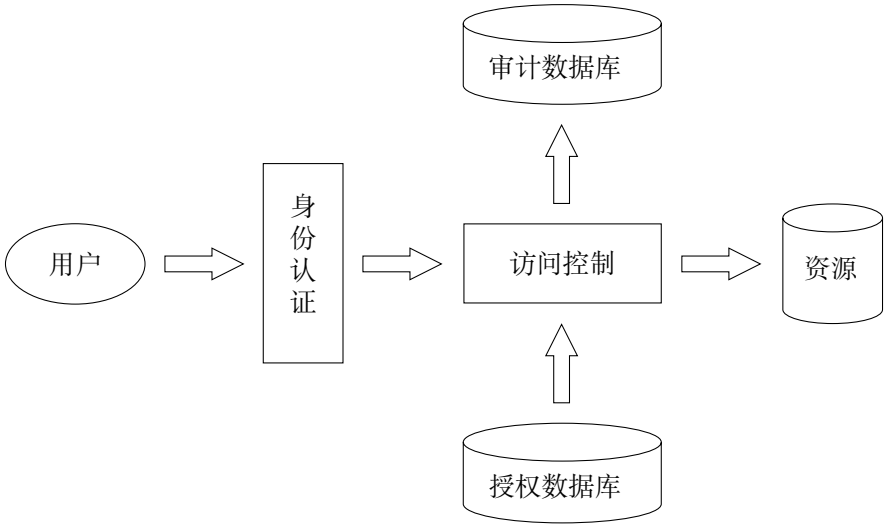


图 11.1: 用户对资源的访问过程

需要指出的是，由于密码协议本身的复杂性，目前并没有一种方法能够给出密码协议安全性的充分而且必要的理论证明。上述每一类方法都有不同的侧重点，或者说或多或少地存在不足之处，我们在使用上述方法分析密码协议的时候，应当仔细分析协议的特点、应用环境和需求，综合使用这些分析方法，以得到一个比较合理的结果。

11.3 身份认证协议

今天，许多商业活动、交易以及政府服务已经越来越多地出现在开放且易受攻击的通信网络上，如Internet。与期望的通信伙伴真诚地通信是至关重要地。这里需要的安全服务是认证，它可通过应用密码技术来获得。以下讲述认证协议的基本工作原理，分析认证协议中的典型错误并了解其起因。

11.3.1 身份认证概述

为了保护网络资源及落实安全政策。需要提供可追究责任的机制，这里涉及三个概念：认证、授权以及审计。

(1) 认证（Authentication）：在做任何动作之前必须要有方法来识别动作执行者的真实身份。认证又称为鉴别、确认。身份认证主要是通过标识和鉴别用户的身份，防止攻击者假冒合法用户获取访问权限。

(2) 授权（Authorization）：授权是指当用户身份被确认合法后，赋予该用户进行文件和数据等操作的权限。这种权限包括读、写、执行及从属权等。

(3) 审计（Auditing）：每一个人都应该为自己所做的操作负责，所以在做完事情之后都要留下记录，以便核查责任。

在现实生活中，我们个人的身份主要是通过各种证件来确认的，比如：身份证、户口本等。计算机网络信息系统中，各种计算资源（如：文件、数据库、应用系统）也需要认证机制的保护，确保这些资源被合法使用者使用。在大多数情况下，认证机制与授权和记账也紧密结合在一起。身份认证是对网络中的主体进行验证的过程，用户必须提供他是谁的证明，即证实客户的真实身份与其所声称的身份是否相符的过程。身份认证往往是许多应用系统中安全保护的第一道设防，它的失败可能导致整个系统的失败。身份认证的依据包括：用户所知道的信息，例如密码、口令等；用户拥有的东西，例如身份证、护照、密

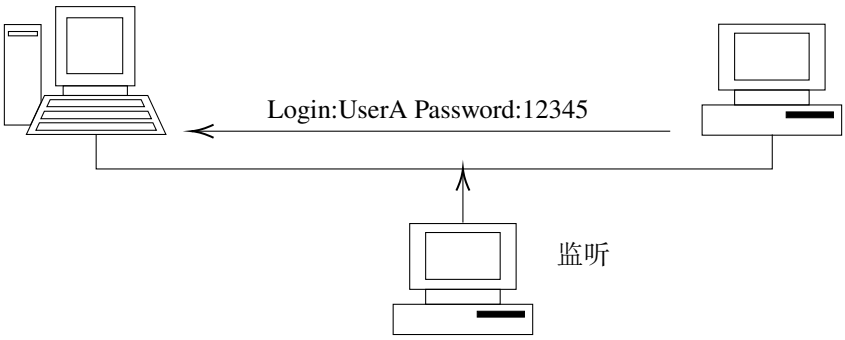


图 11.2: 窃听

钥盘等；用户特征，例如指纹、笔迹、声纹、虹膜、DNA 等。

身份认证分为单向认证和双向认证。如果通信的双方只需要一方被另一方鉴别身份，这样的认证过程就是一种单向认证。在双向认证过程中，通信双方需要互相认证对方的身份。该协议使得通信各方互相认证鉴别各自的身份，然后交换会话密钥。

从用户角度来看，敌手常采用以下手段对身份认证进行攻击：

数据流窃听：由于认证信息要通过网络传递，并且很多认证系统的口令是未经加密的明文，攻击者通过窃听网络数据，就很容易分辨出某种特定系统的认证数据，并提取出用户名和口令。

拷贝/重传：非法用户截获信息，然后再传送给接收者。

修改或伪造：非法用户截获信息，替换或修改信息后再传送给接收者，或者非法用户冒充合法用户发送信息。

11.3.2 基于口令的认证

口令是与特定实体相关联的、且能够证明实体身份的信息。口令验证的安全性虽然不如其他几种方法，但是口令验证简单易行，因此口令验证是目前应用最为广泛的身份认证方法之一。通过用户 ID 和口令进行认证是操作系统或应用程序通常采用的。如果非法用户获得合法用户身份的口令，他就可以自由访问未授权的系统资源，所以需要防止口令泄露。易猜的口令或缺省口令也是一个很严重的问题，但一个更严重的问题是有的帐号根本没有口令。实际上，所有使用弱口令、缺省口令和没有口令的帐号都应从系统中清除。另外，很多系统有内置的或缺省的帐号，这些帐号在软件的安装过程中口令通常是不变的。攻击者通常查找这些帐号。因此，所有内置的或缺省的帐号都应从系统中移出。

在一些简单的系统中，用户的口令以口令表的形式存储。当用户要访问系统时，系统要求用户提供其口令，系统将用户提供的口令与口令表中存储的相应用户的口令进行比较。若相等则确认用户身份有效，否则确认用户身份无效，拒绝访问。

目前各类计算资源主要靠固定口令的方式来保护。这种以固定口令为基础的认证方式存在很多问题，对口令的攻击包括以下几种：

(1) 网络数据流窃听：攻击者通过窃听网络数据，如果口令使用明文传输，则可被非法截获。大量的通讯协议比如 Telnet、Ftp、基本 HTTP 都使用明文口令，这意味着它们在网络上是以未加密格式传输于服务器端和客户端，而入侵者只需使用协议分析器就能查看到这些信息，从而进一步分析出口令。

(2) 认证信息截取/重放（Record/Replay）：有的系统会将认证信息进行简单加密后进行传输，如果攻击者无法用第一种方式推算出密码，可以使用截取/重放方式，需要的是重新编写客户端软件以使用加密口令实现系统登录。

(3) 字典攻击：根据调查结果可知，大部份的人为了方便记忆选用的密码都与自己周遭的事物有关，例如：身份证字号、生日、车牌号码、在办公桌上可以马上看到的标记或事物、其他有意义的单词或

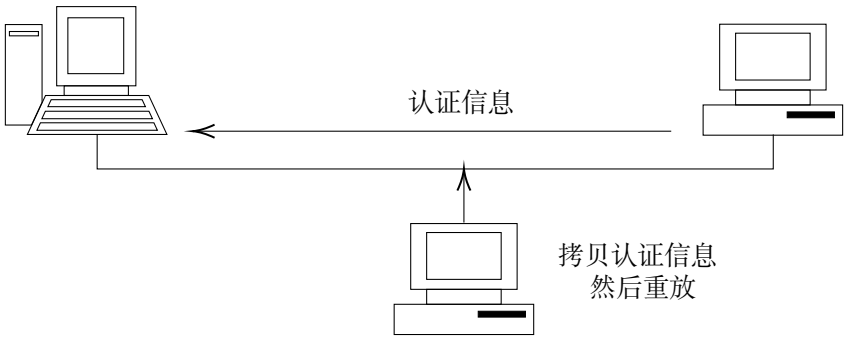


图 11.3: 截取/重放

数字，某些攻击者会使用字典中的单词来尝试用户的密码。所以大多数系统都建议用户在口令中加入特殊字符，以增加口令的安全性。

(4) 穷举攻击：也称蛮力破解。这是一种特殊的字典攻击，它使用字符串的全集作为字典。如果用户的密码较短，很容易被穷举出来，因而很多系统都建议用户使用长口令。

(5) 窥探：攻击者利用与被攻击系统接近的机会，安装监视器或亲自窥探合法用户输入口令的过程，以得到口令。

(6) 社交工程：社会工程就是指采用非隐蔽方法盗用口令等，比如冒充是处长或局长骗取管理员信任得到口令等等。冒充合法用户发送邮件或打电话给管理人员，以骗取用户口令等。比如，在终端上可能发现如下信息：

Please enter your user name to login:

Your password:

这很可能是一个模仿登录信息的特洛伊木马程序，他会记录口令，然后传给入侵者。

(7) 垃圾搜索：攻击者通过搜索被攻击者的废弃物，得到与攻击系统有关的信息，如果用户将口令写在纸上又随便丢弃，则很容易成为垃圾搜索的攻击对象。

在口令的设置过程中，有许多个人因素在起作用，攻击者可以利用这些因素来解密。由于口令安全性的考虑，人们会被禁止把口令写在纸上，因此很多人都设法使自己的口令容易记忆，而这就给攻击者提供了可乘之机。为防止攻击猜中口令。安全口令具有以下特点：

(1) 位数 >6 位。

(2) 大小写字母混合。如果用一个大写字母，既不要放在开头，也不要放在结尾。

(3) 可以把数字无序的加在字母中。

(4) 系统用户一定用 8 位口令，而且包括 $\sim$!@#\$%&$*$ $<>?:"\{\}$ 等特殊符号。

不安全的口令主要有如下几种情况：

(1) 使用用户名（帐号）作为口令。这种方法便于记忆，可是在安全上几乎是不堪一击。几乎所有以破解口令为手段的黑客软件，都首先会将用户名作为口令的突破口。

(2) 用用户名（帐号）的变换形式作为口令。将用户名颠倒或者加前后缀作为口令，比如说著名的黑客软件 John，如果用户名是 fool，那么它在尝试使用 $fool$ 作为口令之后，还会试着使用诸如 fool123、fool1、loof、loof123、lofo 等作为口令。

(3) 使用自己或者亲友的生日作为口令。这种口令有着很大的欺骗性，因为这样往往可以得到一个 6 位或者 8 位的口令，但实际上可能的表达方式只有 $100 \times 12 \times 31 = 37200$ 种，即使再考虑到年月日三者共有六种排列顺序，一共也只有 $37200 \times 6 = 223200$ 种。

(4) 使用常用的英文单词作为口令。这种方法比前几种方法要安全一些。如果选用的单词是十分偏僻的，那么黑客软件就可能无能为力了。

为了加强口令的安全性，可以采取以下措施：

(1) 在创建口令时执行检查功能。如检查口令的长度。

(2) 强制使口令周期性过期。也就是定期更换口令。

(3) 保持口令历史记录，使用户不能循环使用旧口令。

基于口令的认证主要有以下几种方式：

(1) 基于单向函数

计算机存储口令的单向函数值而不是存储口令。在这种验证机制中，用户的口令在系统中以密文的形式存储，并且对用户口令的加密应使得从口令的密文恢复出口令的明文在计算上是不可行的。也就是说，口令一旦加密，将永不可能以明文形式在任何地方出现。这就要求对口令加密的算法是单向的，即只能加密，不能解密。用户访问系统时提供其口令，系统对该口令用单向函数加密，并与存储的密文相比较。若相等，则确认用户身份有效，否则确认用户身份无效。例如：Alice 将口令传送给计算机，计算机使用单向函数计算，然后把单向函数的运算结果和它以前存储的单向函数值进行比较。由于计算机不再存储口令表，所以敌手侵入计算机偷取口令的威胁就减少了。

(2) 掺杂口令

如果敌手获得了存储口令的单向函数值的文件，采用字典攻击是有效的。敌手计算猜测的口令的单向函数值，然后搜索文件，观察是否有匹配的。

$Salt$ 是使这种攻击更困难的一种方法。$Salt$ 是一随机字符串，它与口令连接在一起，再用单向函数对其运算。然后将 $Salt$ 值和单向函数运算的结果存入主机中。$Salt$ 只防止对整个口令文件采用的字典攻击，不能防止对单个口令的字典攻击。

(3) SKEY

Lamport 提出了一个简单的思想来错败在线的口令窃听。该技术可以看作是一次性口令机制。这里，“一次性”的意思是 A 发送给 H 的口令不会重复，但是这些口令在计算上是相关的。这样，因为从协议的某次运行中窃听的口令以后不能再用，所以也就成功地防止了口令窃听。

在用户初始化时，A 的口令记录设置为 $(\mathrm{ID}_A, f^n(P_A))$，其中

$$f^n(P_A) \stackrel{\text{def}}{=} f(\ldots(\ldots f(P_A)\ldots)\ldots),$$

n 是一个大整数。Alice 只需记住 P_A。

A 和 H 首次运行口令认证协议，在要求输入口令时，A 的计算设备，比如客户平台或计算器，会要求 A 输入 P_A，然后重复计算 f 函数 $n-1$ 次，得到 $f^{n-1}(P_A)$。即使当 n 比较大时也能够很有效的完成。

在接收到 $f^{n-1}(P_A)$ 以后，H 会对接收到的口令执行一次 f 运算，得到 $f^n(P_A)$，然后执行正确性检验。如果通过检验，H 就认为接收到的值是 $f^{n-1}(P_A)$，并且是从 P_A 计算得到的，而该 P_A 是在初始化时设定的，因此通信对方必然是 A。这样，A 就被允许进入该系统。另外，H 将会更新 A 的口令记录：用 $f^{n-1}(P_A)$ 替换 $f^n(P_A)$。

在下次运行这个协议时，A 和 H 会分别使用 $f^{n-2}(P_A)$ 和 $f^{n-1}(P_A)$，就像前面一次使用 $f^{n-1}(P_A)$ 和 $f^n(P_A)$ 一样。所以该协议是有状态的，它使用了计数器从 n 递减到 1。当计数器的值是 1 时，A 和 H 应该重新设置口令。

这种方法要求 A 和 H 在口令的状态上是同步的：当 H 在 $f^i(P_A)$ 状态时，A 必须处于 $f^{i-1}(P_A)$ 状态。这种同步可能会丢失，例如因为“不可靠”的通信链路或者主机系统“死机”。应该注意，这里的“不可靠”和“死机”可能是 Malice 造成的。

Lamport 提出了一种简单的方法用于同步丢失时重建同步。本质上，该方法要求系统“向前跳”：如果 A 在 $f^j(P_A)$ 状态而用户在 $f^k(P_A)$ 状态，并且 $j \neq k+1$，那么同步就丢失了，此时我们要求系统“向前跳”，H 到达 $f^i(P_A)$ 状态，A 到达状态 $f^{i-1}(P_A)$，其中 $i \leq min(j,k)$。很明显，这种重新

同步的机制需要 H 和 A 的双方认证通信。然而，在 Lamport 很短的技术论文中没有给出这个必要性的细节。

Lamport 基于口令的远端访问机制已经被修补并实现为“一次性口令”系统，称为 S/KEY。S/KEY 机制的修补是为解决“不可靠通信”问题而提出的，该修补要求 H 为 A 维护一个计数器。在用户初始化时，H 保存用户 A 的口令记录 $(\mathrm{ID}_A, f^c(P_A), c)$，其中 c 初始化为 n。表11.1描述了 S/KEY 机制。

表 11.1: S/KEY 协议

前提：用户 A 和主机 H 已经设定 A 的初始口令记录 $(ID_A, f^n(P_A), n)$，其中 f 是密码杂凑函数；A 记住口令 P_A；H 中 A 的当前口令记录是 $(\mathrm{ID}_A, f^n(P_A), c)$，其中 $1 \le c \le n$。
目标：A 向 H 认证同时不以明文形式传输 P_A。
1. $A \to H$：ID_A；
2. $H \to A$：c，“输入口令”；
3. $A \to H$：$Q = f^{n-1}(P_A)$；
4. H 从其口令文档中查找记录 $(\mathrm{ID}_A, f^c(P_A), c)$；如果 $f(Q) = f^c(P_A)$，就允许接入，并把 A 的口令记录更新为 $(ID_A, Q, c-1)$。

(4) Bellovin-Merritt 的简化 EKE 协议

EKE 的思想是传送一个使用口令加密的临时公钥，该密钥可作为共享密钥。只有知道口令的通信双方才能完成协议。该思想可应用于各种不同的公钥方案。在此只考虑 $Diffie-Hellman$ 密钥交换方案，该方案中，通信的双方都选择一个临时的公钥，口令用于加密临时密钥。

Bellovin-Merritt 的 EKE 协议的思想是协议发起者选择一个临时公钥，并使用共享口令加密。响应者解密该公钥，并用其将会话密钥将发送至初始者。假设共钥是随机字符串，试图暴力查找口令的敌手不可能知道使用了哪一个临时公钥。此外，即使找到了正确公钥，也不可能根据公钥而得到会话密钥，因为不可能从公钥得到私钥。可将基于口令的协议和提供后向安全的协议相比较。为实现认证和消息交换，二者都只有通过使用临时公钥，更普遍的做法是使用长期密钥。很多基于口令的协议也提供后向安全。

1) 基于口令的协议中使用的符号

π：A 所拥有的长度较小的密钥，例如口令；

p：一个大素数（通常至少为 1024 比特）；

q：素数，且 $q \mid p-1$；

G：$\mathbb{Z}_p{}^*$ 的子群。通常 G 是以 p 为阶的子群，但有时等于 $\mathbb{Z}_p{}^*$；

g：G 的生成元；

r_A，r_B：分别由 A 和 B 选择的随机数；

t_A，t_B：在 $\mathbb{Z}_p$ 中分别计算 g^{r_A} 和 g^{r_B}；

x_A，x_B：分别为 A 和 B 的私钥；

y_A，y_B：分别为 A 和 B 的公钥，即 g^{x_A} 和 g^{x_B}。公钥将以标准方式认证，本文不考虑此问题；

$H_i(\cdot)$：单向 Hash 函数。

2) 协议过程（如图11.4所示）

(a) A 选择随机数 r_A，$r_A \in_R \mathbb{Z}_p$，计算 $t_A = g^{r_A}$，用共享口令 π 加密 t_A，发送 A，$\{t_A\}_\pi$ 给 B。

(b) B 选择随机数 r_B，$r_B \in_R \mathbb{Z}_p$。B 用共享口令 π 解密 $\{t_A\}_\pi$ 得到 t_A，于是可确认消息发送者是 A。计算 $Z_{AB} = {t_A}^{r_B}$，$t_B = g^{r_B}$。选择随机数 n_B，$n_B \in_R \{1, \ldots, 2^L\}$。用共享口令

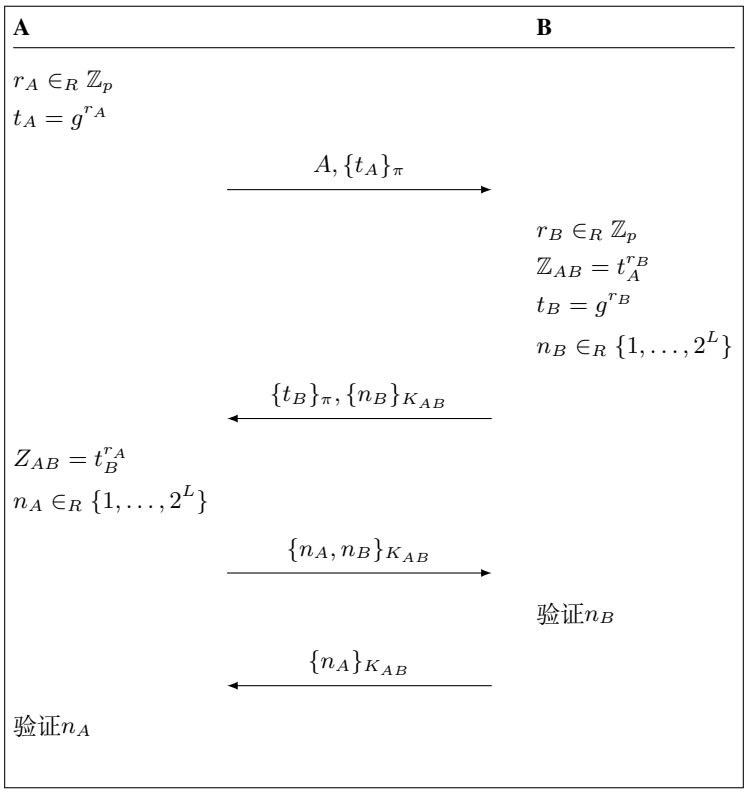


图 11.4: Bellovin-Merritt 的简化 EKE 协议

π 加密 t_B，用会话密钥 Z_{AB} 加密 n_B，将加密后的结果发送给 A。

(c) A 选择随机数 n_A，$n_A \in_R \{1, \ldots, 2^L\}$。用会话密钥 Z_{AB} 加密 n_A 和 n_B，将加密后的结果 $\{n_A, n_B\}_{K_{AB}}$ 发送给 B。

(d) B 解密收到的消息，检查 n_B 是否相等。若相等，用会话密钥 Z_{AB} 加密 n_A，将加密后的结果 $\{n_A\}_{K_{AB}}$ 发送给 A，A 解密后验证 n_A 是否相等。

11.3.3 基于对称密码的认证

1978 年出现了著名的 Needham-Schroeder 认证协议，该协议完成了双向认证，并同时实现了秘密通信。

这里需建立一个称为认证服务器的可信权威机构（密钥分发中心 KDC），拥有每个用户的秘密密钥。若用户 A 欲与用户 B 通信，则用户 A 向认证服务器申请会话密钥。在会话密钥的分配过程中，双方身份得以认证。认证过程如下：

(1) $A \rightarrow$KDC：$A \,||\, B \,||\, R_a$；

(2) KDC$\rightarrow A$：$E_{K_a}[Ra \,||\, B \,||\, K_s \,||\, E_{K_b}[Ks \,||\, A]]$；

(3) $A \rightarrow B$：$E_{K_b}[K_s \,||\, A]$；

(4) $B \rightarrow A$：$E_{Ks}[R_b]$；

(5) $A \rightarrow B$：$E_{Ks}[R_b - 1]$。

其中，KDC 是密钥分发中心，R_a、R_b 是一次性随机数，保密密钥 K_a 和 K_b 分别是 A 和 KDC、B 和 KDC 之间共享的密钥，K_s 是由 KDC 分发的 A 与 B 的会话密钥，E_X 表示使用密钥 X 加密。

Needham-Schroeder 认证协议使用了多次呼叫/响应协议。

(1) A 告诉 KDC，A 想与 B 通信，明文消息中包含一个大的随机数 R_a。

(2) KDC 发送一个使用 A 和 KDC 之间共享的密钥 K_a 加密的消息，消息包括由 KDC 分发的 A 与 B 的会话密钥 K_s、A 的随机数 R_a、B 的名字以及一个只有 B 能看懂的许可证。A 的随机数 R_a 保证了该消息是新的而不是攻击者重放的，B 的名字保证了第一条明文消息中的 B 未被更改，许可证 $E_{K_b}[K_s \parallel A]$ 使用 B 和 KDC 之间共享的密钥 K_b 加密。

(3) A 将许可证 $E_{K_b}[K_s \parallel A]$ 发给 B。

(4) B 解密许可证 $E_{K_b}[K_s \parallel A]$ 获得会话密钥 K_s，然后产生随机数 R_b，B 向 A 发送消息 $E_{K_s}[Rb]$。

(5) A 向 B 发送消息 $E_{K_s}[Rb-1]$ 以证明是真正的 A 与 B 通信。

11.4 秘密共享

秘密共享的概念是由 Shamir 和 Blakley 于 1979 年分别独立提出的。他们的初衷旨在保护密钥不被发现，使得即使部分或全部密钥被窃也难以找出真正的关键钥。自动秘密共享被提出后，许多研究人员对其做了大量的研究，并取得了不少成果。

11.4.1 秘密共享的思想

所谓秘密共享就是将一个密钥分成许多份额，然后秘密的分配给一些有关人员，使得某些有关人员在同时给出他们的份额后可以重建密钥，而另外一些有关人员在同时给出他们的份额后不能重建密钥。

例如某个银行有三位出纳，他们每天都要开启保险柜，为防止每位出纳可能出现的监守自盗行为，银行规定至少有两位出纳在场才能开启保险柜，银行的这个开启保险柜的问题可以利用秘密共享方案来实现。

假设你中了几十亿的彩票，而且你想把这些财产留给你的亲戚。你的钱被锁进了只有你知道的安全的地方，你并不想把秘密告诉你的 5 个孩子中的任何一个，因为他们并不可信。你想把秘密拆开来，从而让他们中的任何 3 个拼在一起才能重建真实。那样的话，如果有人想得到遗产就必须和其余两个孩子合作。

将密钥 K 分成 n 块，每部分叫做它的“影子”，知道任意 m 个或更多的块就能够计算出密钥 K，知道任意 $m-1$ 个或更少的块都不能够计算出密钥 K，这叫做 (m,n) 门限（阈值）方案。目前，人们基于拉格朗日内插多项式法、射影几何、线性代数、孙子定理等提出了许多秘密共享方案。

11.4.2 Shamir 门限秘密共享方案

Shamir 于 1979 年提出了一个门限秘密共享方案，该方案是基于拉格朗日插值多项式构造的，具体算法如下：

(1) 初始化阶段

秘密分发者 D 随机地从 $GF(q)$（q 为素数且 $q>n$）中选取 n 个不同的非零元素 x_1，x_2，...，x_n。D 将 x_i 分配给 $U_i(i=1,2,...,n)$，且 x_i 的值是公开的。

(2) 秘密分发阶段

如果 D 打算让 n 个参与者 U_1，U_2，...，U_n 共享秘密 $s \in GF(q)$，D 随机地选取 $GF(q)$ 中的 $t-1$ 个元素 a_1，a_2，...，a_{n-1}。构造 $t-1$ 次多项式 $f(x)=s+a_1x+a_2x^2+\cdots+a_{t-1}x^{t-1}$，$D$ 计算 $y_i=f(x_i)$，$1 \le i \le n$，并将 y_i 安全地分配给参与者 U_i 最为他的子秘密。

(3) 秘密恢复阶段

n 个参与者中的任意 t 个参与者，不妨设为 U_1，U_2，...，U_t，出示他们的子秘密，从而得到 t 个点

对：(x_1, y_1)，(x_2, y_2)，...，(x_t, y_t)，这样就可以重构多项式 $f(x)$ 和共享的秘密 s 值如下：

$$f(x) = \sum_{i=1}^{t} y_i \prod_{j=1, j\neq i}^{t} \frac{x - x_j}{x_i - x_j},$$

$$s = \sum_{i=1}^{t} y_i \prod_{j=1, j\neq i}^{t} \frac{x_j}{x_j - x_i}.$$

例 11.1

设 $K = 11$，构造 $(3, 5)$ 门限方案，在这个方案中 5 个人中任意 3 个人都能重构 K，产生一个二次多项式：$(ax^2 + bx + K) \bmod p$，其中系数 a 和 b 为随机选择，p 是一个比所有系数都大的随机素数，其中 $a = 7$，$b = 8$ 且 $p = 13$，则 $F(x) = (7x^2 + 8x + 11) \bmod 13$。

5 个影子是：

$$k_1 = F(1) = 7 + 8 + 11 \equiv 0 \bmod 13,$$
$$k_2 = F(2) = 28 + 16 + 11 \equiv 3 \bmod 13,$$
$$k_3 = F(3) = 63 + 24 + 11 \equiv 7 \bmod 13,$$
$$k_4 = F(4) = 112 + 32 + 11 \equiv 12 \bmod 13,$$
$$k_5 = F(5) = 175 + 40 + 11 \equiv 5 \bmod 13.$$

为了从 3 个影子（比如 k_2，k_3 和 k_5）重构 K，解线性方程组：

$$3 = a \times 2^2 + b \times 2 + K \bmod 13,$$
$$7 = a \times 3^2 + b \times 3 + K \bmod 13,$$
$$5 = a \times 5^2 + b \times 5 + K \bmod 13,$$

解为 $a = 1$，$b = 8$ 和 $K = 11$，这样就恢复了 M。

秘密共享解决了两个问题：一是若密钥偶然或有意地被暴露，整个系统就易受攻击；二是若密钥丢失或损坏，系统中的所有信息就不能用了。

11.5 阈下信道

假设 Alice 和 Bob 被捕入狱。他将去男牢房，而她则进女牢房。看守 Walter 愿意让 Alice 和 Bob 交换消息，但他不允许他们加密。Walter 认为他们可能会商讨一个逃跑计划，因此，他想能够阅读他们说的每个细节。

Walter 也希望欺骗 Alice 和 Bob，他想让他们中的一个将一份欺诈的消息当作来自另一个的真实消息。Alice 和 Bob 愿意冒这种欺骗的危险，否则他们根本无法联络，而他们必须商讨他们的计划。为了完成这件事，他们不得不欺骗看守，并找出一个秘密通讯的方法。他们不得不建立一个阈下信道，即完全在 Walter 视野内的他们之间的一个秘密通信信道，即使消息本身并不包含秘密信息。通过交换完全无害的签名的消息，他们可以来回传送秘密信息，并骗过 Walter，即使 Walter 正在监视所有的通信。

11.5.1 阈下信道的基本原理

一个简易的阈下信道可以是句子中单词的数目。句子中奇数个单词对应 “1”，而偶数个单词对应 “0”。因此，当读这种仿佛无关的段落时，已将消息 “101” 送给了在现场的我方的人。这种技术的问题在于它仅仅是密码；没有密钥，安全性依赖于算法的保密性。

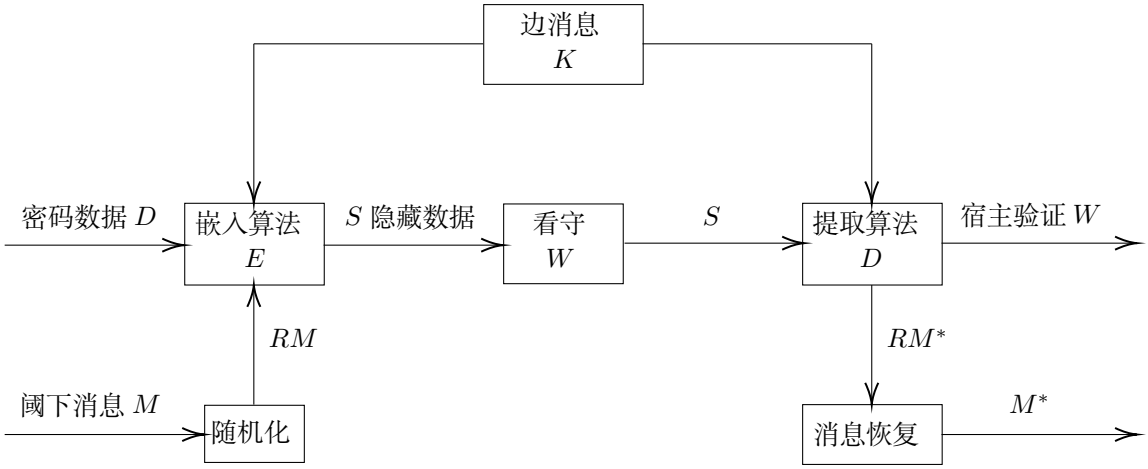


图 11.5: 使用阈下信道的基本过程（其中，边信息是指收发双方共享的信息）

Gustavus Simmons 发明了传统数字签名算法中阈下信道的概念。由于阈下消息隐藏在看似正常数字签名的文本中，这是一种迷惑人的形式。Walter 看到来回传递的签名的无害消息，但他完全看不到通过阈下信道传递的信息。事实上，阈下信道签名算法与通常的签名算法不能区别，至少对 Walter 是这样。Walter 不仅不能读阈下信道消息，而且他也不知道阈下信道消息已经出现：

一般地，使用阈下信道的基本过程如下：

（1）Alice 产生一个无害消息，最好随机；

（2）用与 Bob 共享的秘密密钥，Alice 对这个无害信息这样签名：她在签名中隐藏她的阈下信息；

（3）Alice 通过 Walter 发送签名消息给 Bob；

（4）Walter 读这份无害的消息并检查签名，没发现什么问题，他将这份签了名的消息传递给 Bob；

（5）Bob 检查这份无害消息的签名，确认消息来自于 Alice；

（6）Bob 忽略无害的消息，而用他与 Alice 共享的秘密密钥，提取阈下消息。

阈下信道的最显见的应用是在间谍网中。如果每人都收发签名消息，间谍在签名文件中发送阈下信息就不会被注意到。当然，敌方的间谍也可以做同样的事。

用一个阈下信道，Alice 可以在受到威胁时安全地对文件签名。她可以在签名文件时嵌入阈下消息，说“我被胁迫”。别的应用则更为微妙，公司可以签名文件，嵌入阈下信息，允许它们在文档整个文档有效期内被跟踪。政府可以“标记”数字货币。恶意的签名程序可能泄露其签名中的秘密信息。其可能性是无穷的。

11.5.2 ElGamal 签名的阈下信道

公钥：p 是素数，$g < p$（p，g 可由一组用户共享）$K = g^r \bmod p$。

私钥：$r < p$，除 Alice 之外，Bob 也知道 r，这个密钥用来发送和阅读阈下消息，另外还用来签署无害消息。

签名：使用无害消息 M' 来发送阈下消息 M 时，M'，M 和 p 都必须相互完全互素，同时 M 和 $p-1$ 也必须互素。Alice 计算：

$$X = g^M \bmod p,$$

并对 Y 解下列方程：

$$M' = rX + MY \bmod p - 1,$$

则签名是 X 和 Y。

验证：Walter 能验证签名，确认：

$$K^X X^Y \equiv gM' \bmod p.$$

Bob 能恢复阈下消息，首先确认：

$$(g^r)^X X^Y \equiv g^{M'} \bmod p.$$

如果上面的确认成立，那么他接收到的消息就是真实的。

恢复消息时计算：

$$M = (Y^{-1}(M' - rX)) \bmod p - 1.$$

例 11.2 设 $p = 11$ 和 $g = 2$，秘密密钥 r 选为 8，这意味着 Walter 能用来验证签名的公开密钥是 $g^r \bmod p = 2^8 \bmod 11 = 3$。

使用无害消息 $M' = 5$ 来发送阈下消息 $M = 9$ 时，Alice 确认 9 和 11 互素，同时 5 和 11 互素，她还证实 9 和 $11 - 1 = 10$ 也互素。因此 Alice 计算：

$$X = g^M \bmod p = 2^9 \bmod 11 = 6,$$

然后，她对 Y 解下列方程：

$$5 = 8 \times 6 + 9 \times Y \bmod 10,$$

得 $Y = 3$，所以签名是数对 6 和 3。

Bob 进一步确认：

$$(g^r)^X X^Y \equiv g^{M'} \bmod p,$$
$$(2^8)^6 6^3 \equiv 2^5 \bmod 11.$$

Bob 通过计算下式恢复阈下消息：

$$\begin{aligned} M &= \left(Y^{-1}\left(M' - rX\right)\right) \bmod p - 1 \\ &= 3^{-1}(5 - 8 \times 6) \bmod 10 \\ &= 7 \times (7) \bmod 10 = 49 \bmod 10 = 9. \end{aligned}$$

11.6 比特承诺

比特承诺技术也称为委托技术，是密码学协议的重要组成部分。自从 Blum 于 1982 年提出首个比特承诺方案以来，已经有很多的承诺方案先后被提出来。其中的大部分方案是以数论中的一些困难问题为基础的。

11.6.1 什么是比特承诺

比特承诺的基本思想是这样的：承诺者 Alice 向接收者 Bob 承诺一个消息，承诺过程要求，Alice 想 Bob 承诺时，Bob 不可能获得关于被承诺消息的任何信息；经过一段时间后，Alice 能够向 Bob 证实她所承诺的消息，但是 Alice 无法欺骗 Bob。举一个例子：Alice 把消息 m 放在一个箱子里并锁住（只有 Alice 有钥匙可以打开箱子）送给 Bob；当 Alice 决定向 Bob 证实消息时，Alice 会把消息 m 及钥匙给 Bob；Bob 能够打开箱子并验证箱子里的消息与 Alice 出示的消息相同，并且 Bob 确信箱子里的消息在他的保管期间没有被篡改。

Alice 要向 Bob 承诺一个消息 m，简单的承诺方法如下：

（1）Alice 随机选择 r_1，利用 Hash 函数 h 计算：$H = h(r_1, m)$，然后把 H 发给 Bob；

（2）当 Alice 向 Bob 出示消息 m 时，她把 r_1 和 m 发给 Bob。

做个简单的分析，由于 Hash 函数的性质，在 Alice 向 Bob 发送 H 之后，Bob 无法知道 m；同样 Alice 不能找到 $r_1{}'$ 和 m' 满足 $h(r_1,\ m) = h(r_1{}', m')$，这使得 Alice 无法欺骗 Bob。

比特承诺方案具有两个重要性质：一是隐蔽性，即接收者不能通过接受的加密消息来确定承诺值；二是约束性，发送者能打开加密消息，但发送者打开后得到的承诺值只能是原来承诺的数值。

比特承诺方案是许多密码协议的一个基本组成部分。在计算机网络中，彼此互不信任的通信双方进行通信时，各用户希望通过安全的比特承诺方案互相传递可靠的数据，同时也希望能保持各自数据的安全性和完整性。在比特承诺过程中，一方 Alice 向另一方 Bob 承诺比特 b，在比特承诺时 Bob 不知道 b 真实值，等时机成熟后，Alice 能揭开 b 的值，并且 Bob 也能验证 b 的值确实是 Alice 承诺的。一个对安全比特承诺机制的直观描述是：Alice 把比特 b 放入一个箱子，用一把只有她自己才能开启的锁锁上这个箱子，然后把这个箱子给 Bob，当时机成熟时，她把比特 b 和打开箱子的钥匙给 Bob，通过打开箱子验证，Bob 知道比特 b 的内容没有改动，因为箱子在他控制之下。比特承诺有很强的应用背景，特别是在电子信息时代。如网上电子投标（拍卖）、网上商业谈判、电子投票、电子现金、在线游戏等问题。还可以用于零知识证明、身份认证、多方协议等等。如果我们能安全承诺任何比特，就掌握了一个很强大的工具，实际上任何一个密码协议都可以分解成一系列安全比特承诺方案。

11.6.2 使用对称密码算法的比特承诺

这个比特承诺协议使用对称密码：

(1) Bob 产生一个随机比特串 R，并把它发送给 Alice：R；

(2) Alice 生成一个由她想承诺的比特 b 组成的消息（b 实际上可能是几个比特），以及 Bob 的随机串。她用某个随机密钥 K 对它加密，并将结果送回给 Bob：$E_K(R, b)$。

这是这个协议的承诺部分，Bob 不能解密消息，因而不知道比特为何。

当到了 Alice 揭示她的比特的时候，协议继续：

(3) Alice 发送密钥给 Bob；

(4) Bob 解密消息以揭示比特。他检测他的随机串以证实比特的有效性。

如果消息不包含 Bob 的随机串，Alice 能够秘密地用一系列密钥解密她交给 Bob 的消息，直到找到一个给她的 bit，而不是她承诺的比特。由于比特只有两种可能的值，她只需试几次肯定可以找到一个。Bob 的随机串避免了这种攻击，她必须能找到一个新的消息，这个消息不仅使她的比特反转，而且使 Bob 的随机串准确地重新产生。如果加密算法好，她发现这种消息的机会是极小的。Alice 不能在她承诺后改变她的比特。

11.6.3 使用单向函数的比特承诺

本协议利用单向函数：

(1) Alice 产生两个随机比特串：R_1 和 R_2。

(2) Alice 产生消息，该消息由她的随机串和她希望承诺的比特（实际上可能是几比特）组成：(R_1, R_2, b)。

(3) Alice 计算消息的单向函数值，将结果以及其中一个随机串发送给 Bob：$H(R_1, R_2, b)$，R_1。

这个来自 Alice 的传送就是承诺证据。Alice 在第 (3) 步使用单向函数阻止 Bob 对函数求逆并确定这个比特。

当到了要 Alice 出示她的比特的时候，协议继续：

(4) Alice 将原消息发给 Bob：(R_1, R_2, b)。

(5) Bob 计算消息的单向函数值，并将该值及 R_1 与原先第 (3) 步收到的值及随机串比较。如匹配，则比特有效。

这个协议较前面一个的优点在于 Bob 不必发送任何消息。Alice 送给 Bob 一个对比特承诺的消息，以及另一揭示该 bit 的消息。

这里不需要 Bob 的随机串，因为 Alice 承诺的结果是对消息进行单向函数变换得到的。Alice 不可能欺骗，并找到另一个消息 (R_1, R_2', b')，满足

$$H(R_1, R_2', b') = (R_1, R_2, b).$$

通过发送 R_1 给 Bob，Alice 对 b 的值作了承诺。如果 Alice 不保持 R_2 是秘密的，那么 Bob 能够计算 $H(R_1, R_2, b')$ 和 (R_1, R_2, b)，并比较哪一个等于他从 Alice 那里接收的。

11.7 零知识证明

零知识证明（zero-knowledge proof）的思想是：证明者 Peggy 拥有某些知识（如某些长期没有解决的难问题的解决方法），零知识证明就是在不将该知识的内容泄露给验证者 Victor 的前提下，Peggy 向 Victor 证明自己拥有该知识。首先，我们看下面 Peggy 和 Victor 之间的一段对话：

Peggy：“我可以对密文为 C 的消息进行解密。”

Victor：“我不相信。请证明。”

Peggy（糟糕的回答）：“密钥是 K，您可以看到消息解密成了 M。”

Victor：“哈哈！现在我也知道了密钥和消息。”

这里，Peggy 虽然证明了自己拥有某些知识（密钥 K 及明文 M），却向 Victor 泄露了这些知识。一个更好的对话是：

Peggy：“我可以对加密为 C 的消息进行解密。”

Victor：“我不相信。请证明。”

Peggy（好的回答）：“让我们使用一个零知识协议，我将以任意高的概率证明我的知识（但是不会将关于消息的任何情况泄露给您）。”

Victor：“好”。

Peggy 和 Victor 通过该协议……

可以使用洞穴例子来解释零知识，C 和 D 之间存在一个密门，并且只有知道咒语的人才能打开。Peggy 知道咒语并想对 Victor 证明，但证明过程中不想泄露咒语。

步骤如下：

(1) Victor 站在 A 点；

(2) Peggy 一直走进洞穴，到达 C 点或者 D 点；

(3) 在 Peggy 消失在洞穴中之后，Victor 走到 B 点；

(4) Victor 随机选择左通道或者右通道，要求 Peggy 从该通道出来；

(5) Peggy 从 Victor 要求的通道出来，如果有必要就用咒语打开密门；

(6) Peggy 和 Victor 重复步骤 (1) 至 (5)n 次。

如果 Peggy 不知道这个咒语，那么只能从进去的路出来，如果在协议的每一轮中 Peggy 都能按 Victor 要求的通道出来，那么 Peggy 所有 n 次都猜中的概率是 $1/2^n$。经过 16 轮后，Peggy 只有 65536 分之一的机会猜中。于是 Victor 可以假定，如果所有 16 次 Peggy 的证明都是有效的，那么她一定知道开启 C 点和 D 点间的密门的咒语。

我们来看一个零知识证明的例子。图是否同构是 NP 完全问题，对于一个非常大的图，判断两个图是否同构是非常困难的。对于图 G_1 和 G_2，如果存在一个一一对应的函数 F：F 的定义域是 G_1 的顶点集。F 的值域是 G_2 的顶点集。当且仅当 $[g_1, g_2]$ 是 G_1 中的一条边，$[F(g_1), F(g_2)]$ 才是 G_2 中的一条边，称 G_1 和 G_2 同构的。

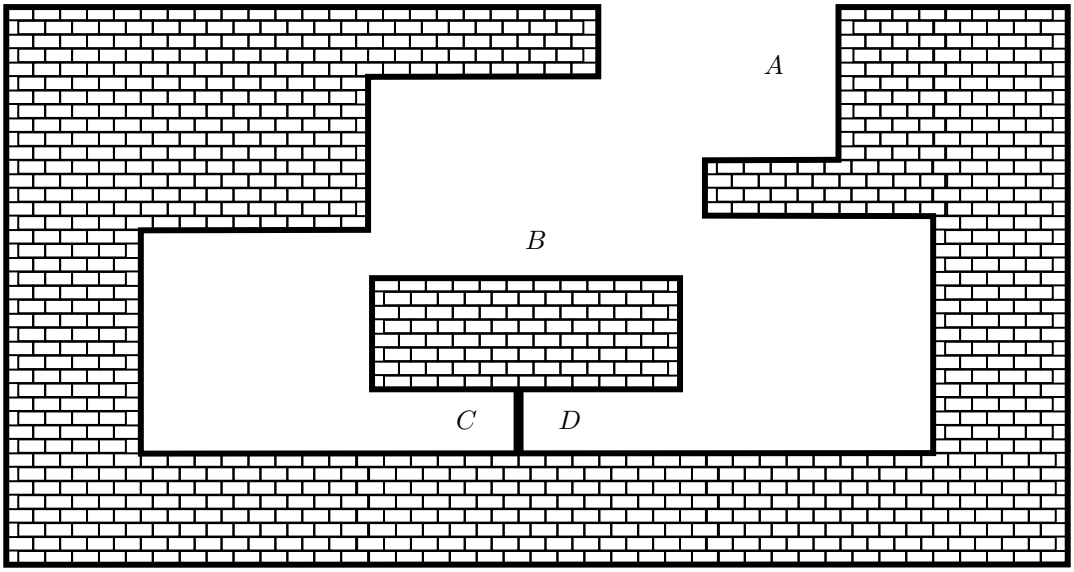


图 11.6: 零知识洞穴

假设 Peggy 知道图 G_1 和 G_2 之间同构，Peggy 使用下面的协议将使 Victor 相信 G_1 和 G_2 同构：

(1) Peggy 随机置换 G_1 产生另一个图 H，并且 H 和 G_1 同构。因为 Peggy 知道 G_1 和 H 同构，也就知道了 H 和 G_2 同构。

(2) Peggy 把 H 送给 Victor。

(3) 对如下两个问题 Victor 选择其中的一个，要求 $Peggy$ 证明：证明 G_1 和 H 同构，或者证明 G_2 和 H 同构。但是，Victor 不要求两者都证明。

(4) Peggy 按 Victor 的要求证明。

(5) Peggy 和 Victor 重复步骤 (1) 至 (4)n 次。

如果 Peggy 不知道 G_1 和 G_2 之间的同构性，Peggy 就只能创造一个图或者与 G_1 同构或者与 G_2 同构。每一轮中 Peggy 只有 50% 的机会猜中 Victor 的选择。又因为 Peggy 在协议的每一轮都产生一个新图 H，故不管经过多少轮协议 Victor 也得不到任何信息，他不能从 Peggy 的答案中了解 G_1 和 G_2 的同构性。图同构的零知识证明只具有理论意义，从实现来看，是不实用的。

这里我们介绍著名的 Feige-Fiat-Shamir 零知识身份认证协议的一个简化方案。

在发放私人密钥之前，可信赖仲裁选定一个随机模数 n，n 为两个大素数乘积，实际中至少为 512 位长，尽量接近 1024 位。n 值可以在一组证明者之间共享。为了产生 Peggy 的公开密钥和私人密钥，仲裁方产生随机数 v，使 $x^2 \equiv v \bmod n$，即 v 为模 n 的平方剩余，且有 $v^{-1} \bmod n$ 存在。以 v 作为 Peggy 的公钥，而后计算最小的整数 s：$s = \text{sqrt}(v^{-1}) \bmod n$ 作为 Peggy 的私钥。实施身份证明的协议如下：

(1) 用户 Peggy 取随机数 r，这里 $r < n$，计算 $x = r^2 \bmod n$，把 x 送给 Victor；

(2) Victor 把一个随机位 b 送给 Peggy；

(3) 若 $b = 0$，则 Peggy 将 r 送给 Victor；若 $b = 1$，则 Peggy 将 $y = rs$ 送给 Victor；

(4) 若 $b = 0$，则 Victor 验证 $x = r^2 \bmod n$，从而证实 Peggy 知道 $\text{sqrt}(x)$；若 $b = 1$，则 Victor 验证 $x = y^2 v \bmod n$，从而证实 Peggy 知道 s。

这是一轮鉴定，Peggy 和 Victor 可将此协议重复 t 次，直到 Victor 相信 Peggy 知道 s 为止。这是一个分割选择协议。如果 Peggy 不知道 s，她可以选取 r 以便在 Victor 送给她 0 时欺骗 Victor，或选取 r 以便在 Victor 送给她 1 时欺骗 Victor，而她不能同时做到上述两点。她欺骗 Victor 一次的可能性为 50%，t 次的可能性为 $1/2^t$。

攻击这个协议的另一种方法是 Victor 试图冒充 Peggy，他和另一个验证者 Valerie 开始进行这个协议。

在第 1 步时，他用他曾看到 Peggy 用过的 r，而不是随机选取 r，然而 Valerie 在第 2 步选取和 Victor 所选取的 b 值相同的可能性为 $1/2$。所以，他欺骗 Valerie 的可能性为 50%。他欺骗她 t 次的可能性为 $1/2^t$。

要避免这点，Peggy 不能重新使用 r。如果她重新使用，在第 2 步，Victor 发给 Peggy 另外一个随机位，那么他能获得 Peggy 的两种应答。从中，Victor 能计算出 s，然后冒充她。

第 11 章 习题

1. 在一个 $(3,5)$ 的 Shamir 秘密共享方案中，用模数 $p=17$ 将 $(1,8)$，$(3,10)$，$(5,11)$ 分别给 Alice，Bob 和 Cherry。计算一下相应的拉格朗日插值多项式，并确定秘密。
2. 密钥分配者用 $(2,20)$ 阈方案将电子保险箱的秘密分配给 20 个参与者
   (1) 如果一个未知的参与者是个骗子，泄漏了一个随机的分享，那么打开保险箱最少需要多少参与者。
   (2) 如果保险箱只允许试一次密码（出现错误，将永久关闭），那么需要多少参与者才能打开保险箱。
3. Alice 想向 Bob 证明他知道 n 的因数分解（$=pq$，p 和 q 大素数），但并不想把因式分解透漏给 Bob 和其他人。他设计了以下方案：
   (1) 选择一个随机数 x 模 n，计算 $y=x^2 \bmod n$，并把 y 发给 Bob。
   (2) 算出 y 模 n 的一个平方根 s，并把它发给 Bob。
   (3) Bob 检验是否 $s^2=y \bmod n$。

   Alice 重复这个过程 20 次。
   a. 描述 Alice 如何计算 s，设 p 和 $q=3 \bmod 4$。
   b. 解释 Bob 是如何运用这一程序高效发现 n 的因数分解的。
   c. 假定 Cherry 正在偷听，并知道了每一个 y 和 s 的值。他是否有可能获得有用的信息？设 y 的值不能重复。

附录 A 密码数学基础

本章用 $\mathbb{Z}$ 表示整数集合，$\mathbb{Z}^+$ 表示正整数集合，$\mathbb{N}$ 表示自然数集合，即 $\mathbb{N}=\{0\}\cup\mathbb{Z}^+$。

A.1 数论基础

数论是研究整数及其性质的一门学科，是现代数学的重要研究分支。本节主要介绍初等数论方面的一些基础知识，它们是一些密码方案，特别是公钥密码方案设计的基本工具。

定义 A.1 (整除、因子)

设 a, b $(b\neq 0)$ 是两个整数，如果存在另一个整数 c，使得 $a=b\cdot c$，则称 b 整除 a，或称 a 被 b 整除，记作 $b\mid a$，并称 b 是 a 的因子。

例 A.1 $4\mid 16$，$3\nmid 7$；3 是 12 的因子，5 不是 7 的因子。

命题 A.1 (整除的性质)

1. 如果 $a\mid b$ 且 $b\mid c$，则有 $a\mid c$。
2. 如果 $a\mid b$ 且 $b\mid c$，则对于任意 $m,n\in\mathbb{Z}$ 有 $a\mid mb+nc$。
3. 如果 $a\mid b$ 且 $|b|<|c|$，则有 $b=0$。
4. 如果 $a\mid b$ 且 $b\mid c$，则有 $a=\pm b$。

素数是数论的主要研究对象，理解素数的分布规律可以说是数论研究历程中最根本性的问题。素数在密码学中也发挥着重要作用。

定义 A.2 (素数)

如果一个大于 1 的正整数只有 1 和它本身两个正因子，则称该整数为素数。

例 A.2 100 以内的素数共有 25 个，它们是：2，3，5，7，11，13，17，19，23，29，31，37，41，43，47，53，59，61，67，71，73，79，83，89，97. 注意 2 是唯一的偶素数。

截至 2022 年 8 月，已被找到的最大素数为 $2^{82589933}-1$，它是一个拥有 24862048 个十进制位的整数，于 2018 年 12 月由一位来自美国佛罗里达州的程序员 Patrick Laroche 利用互联网梅森素数大搜索项目（GIMPS）成功发现。

命题 A.2

如果 $n\in\mathbb{Z}$ 不是一个素数，则 n 必含有一个素因子。

证明 令 n 除了 1 之外的最小正因子是 q。由于 n 不是素数，则有 $1<q<n$。下证 q 一定是素数。

若 q 不是素数，则存在 q 的因子 d 满足 $1<d<q$，根据命题A.1可知 d 也是 n 的因子，这与 q 为 n 的最小正因子矛盾，命题得证。

由命题A.2可以导出如下的算术基本定理，又称为整数的唯一分解定理。这是关于整数最基本的性质，它表明素数是整数的“原子”，关于整数的研究可以归结为关于素数的研究，体现出素数在数论中的重要地位。

定理 A.1 (算术基本定理)

对于任意正整数 $a>1$，一定存在互不相同的素数 $p_1, p_2, \ldots, p_n$ 和正整数 $a_1, a_2, \ldots, a_n$，使得

$$a = p_1^{a_1} p_2^{a_2} \cdots p_t^{a_t}. \tag{A.1}$$

进一步地，在不考虑素因子排列顺序的意义下分解式(A.1)是唯一的。 ♡

证明 先证明存在性。对 a 使用归纳法。当 $a=2$ 时定理显然成立，假设 $a>2$，并且对所有小于 a 的正整数存在定理给出的素因子分解。根据命题A.2可知，存在一个素数 $p \geq 2$ 和正整数 a' 使得 $a = pa'$。显然有 $a' < a$。根据假设，a' 能够分解为素数的乘积，从而 a 也能够分解为素数的乘积，存在性得证。

下证分解式(A.1)的唯一性。同样对 a 使用归纳法。当 $a=2$ 结论显然成立。假设 $a>2$，并且对所有小于 a 的正整数都可使得唯一性成立。令 a 的两种素因子分解为：

$$a = p_1 p_2 \cdots p_k = q_1 q_2 \cdots q_l. \tag{A.2}$$

由于 $p_1 \mid q_1 q_2 \cdots q_l$ 并且 p_1 为素数，则 p_1 必整除某个 q_j。通过重新排序，不妨设 $p_1 = q_1$。进而有

$$a' = p_2 p_3 \cdots p_k = q_2 q_3 \cdots q_l. \tag{A.3}$$

a' 为小于 a 的正整数，根据归纳假设可得 $k=l$ 且对 $i=2,3,\cdots k$ 都有 $p_i = q_i$。唯一性得证。

定理 A.2 (欧几里得定理)

素数有无穷多个。 ♡

证明 反证法。假设仅有有限个素数，其集合为 $\mathbb{P} = \{p_1, p_2, \ldots, p_k\}$。令

$$n = p_1 p_2 \cdots p_k + 1, \tag{A.4}$$

则有 $n>1$。由于 $n \notin \mathbb{P}$，因此 n 不是素数。根据定理A.2，存在素数 q 使得 $q \mid n$。注意到对任意 $1 \leq i \leq k$，$p_i \nmid n$，从而 $q \notin \mathbb{P}$。这与 $\mathbb{P}$ 为所有素数的集合矛盾，因此素数必有无穷多个。

定理A.2最早由欧几里得在公元前 300 年左右发现并证明，目前关于该定理已有很多不同证明方法，历史上著名数学家欧拉（Euler）、库默尔（Kummer）、埃尔德什（Erdős）、福滕伯格（Furstenberg）等人都曾给出过该定理的新证明。尽管已经知道了素数有无穷多个，但关于素数个数的研究仍存在大量悬而未决的、看似初等的公开问题，例如：形如 2^p-1（其中 p 为素数）的素数称为梅森素数，目前被发现的前 10 个最大的素数中有 9 个是梅森素数，是否存在无穷多个梅森素数目前还是未知的；形如 $F_n = 2^{2^n}+1$ $(n \geq 0)$ 的数称为费马数，费马数中是否存在无穷多个素数目前仍是公开难题。此外还有孪生素数猜想：存在无穷多个素数 p 使得 $p+2$ 也是素数。关于素数的其它性质目前也存在很多待解决的难题，例如算术基本定理给出了正整数在乘法意义下的素因子分解，但是在加法意义下的分解则涉及著名的哥德巴赫猜想：每个不小于 6 的偶数均为两个奇素数之和；每个不小于 9 的奇数均为三个奇素数之和。

定义 A.3 (最大公因子)

对于整数 a, b，如果正整数 d 满足 $d \mid a$ 且 $d \mid b$，则称 d 为 a 和 b 的公因子。a 和 b 的所有公因子中最大者称为 a 和 b 的最大公因子，记为 $\gcd(a,b)$，或简记为 (a,b)。特别地，若 $\gcd(a,b)=1$，则称 a 和 b 是互素的。 ♣

定义 A.4 (最小公倍数)

对于整数 a, b，如果正整数 m 满足 $a \mid m$ 且 $b \mid m$，则称 m 为 a 和 b 的公倍数。a 和 b 的所有公倍数中最小者称为 a 和 b 的最小公倍数，记为 $\mathrm{lcm}(a,b)$，或简记为 $[a,b]$。 ♣

例 A.3 $(24,16)=4$, $(99,-55)=11$, $(31,128)=1$, $[3,4]=12$, $[35,-11]=385$。注意两个整数的最大公

因子和最小公倍数都是正整数。

容易证明如下关于整数最大公因子和最小公倍数的性质。

命题 A.3

1. $\gcd(a,b)=\gcd(b,a)=\gcd(-a,b)=\gcd(a,-b)=\gcd(-a,-b)=\gcd(a,a-b)$.
2. $\gcd(0,a)=a$.
3. 设 $a=\prod_p p^{a_p}$，$b=\prod_p p^{b_p}$ 是正整数 a 和 b 的素因子分解。令 $d_p=\min(a_p,b_p)$，$m_p=\max(a_p,b_p)$，则
$$(a,b)=\prod_p p^{d_p},\quad [a,b]=\prod_p p^{m_p}.$$
4. 对 $a,\ b\in\mathbb{Z}^+$，$ab=(a,b)\cdot[a,b]$。

♠

根据命题A.3 (3) 容易得到如下结论：

命题 A.4

设 $a,\ b\in\mathbb{Z}$。

- 对 $a,\ b$ 的任一公因子 c，都有 $c\mid(a,b)$；
- 对 a,b 的任一公倍数 m，都有 $[a,b]\mid m$。

♠

为了得到整数最大公因子和最小公倍数的进一步性质，首先介绍整数的带余除法定理。

定理 A.3 (带余除法)

对任意 $a,b\in\mathbb{Z},\ b\neq 0$，存在唯一的商 q 和余数 r 满足 $a=qb+r$ 且 $0\leq r<|b|$。

♡

证明 （存在性）考虑集合 $S=\{a-kb\mid k\in\mathbb{Z},\ a-kb\geq 0\}\subseteq\mathbb{N}$，显然 S 非空。根据自然数的良序定理，S 中存在最小元，即存在最小的自然数 $r=a-qb\in S$。若 $r\geq|b|$，则 $r\geq r-|b|\geq 0$ 且显然 $r-|b|\in S$，这与 r 为 S 中的最小元矛盾，因此 $r<|b|$。存在性得证。

（唯一性）根据存在性的证明知道 r 为集合 S 中的最小元，从而是唯一的，进而可导出 q 也是唯一确定的。

例 A.4 设 $a=15,\ b=7$，则有 $a=b\times 2+1$，即商为 $q=4$，余数为 $r=3$。

注 定理A.3中 a 除以 b 的余数 r，通常也被称为 a 模 b 的余数，记为 $c=(a \bmod b)$。

推论 A.1

设 $a,\ b\in\mathbb{Z}$。如果 $a=qb+c,\ q\neq 0$，则有 $(a,b)=(b,c)$。

♡

证明 由于 $(a,b)\mid a$，$(a,b)\mid b$，故 $(a,b)\mid c=a-qb$。根据命题A.4可知 $(a,b)\mid(b,c)$。同样的办法可证明 $(b,c)\mid(a,b)$，从而 $(a,b)=(b,c)$。

定理 A.4 (Bézout 等式)

设 $a,\ b\in\mathbb{Z}^+$，$d=\gcd(a,b)$，则存在 $\alpha,\ \beta\in\mathbb{Z}$，使得
$$d=\alpha a+\beta b. \tag{A.5}$$

♡

证明 令 $S=\{ax+by\mid x,y\in\mathbb{Z},\ ax+by>0\}$，则显然 $S\neq\emptyset$（因为 $a\in S$）。设 d' 为 S 中最小的正整数，如下证明 $d'=d$。

首先，由于 $d\mid a,\ d\mid b$，则显然 $d\mid d'$。反过来，证明 d' 是 a 和 b 的一个公因子，从而根据命题A.4可知 $d'\mid d$。若 $d'\nmid a$，则根据定理A.3，存在 q 和 r 使得 $a=qd'+r$ 且 $0<r<d'$。由于 $r=a-qd'\in S$，

这与 d' 为 S 中的最小元矛盾，从而 $d' \mid a$。同样地可以证明 $d' \mid b$。

定理A.4表明，两个整数的最大公因子可以表示成它们的整系数线性组合。式(A.5)中的 α, β 称为 (a,b) 的 Bézout 系数。集合 $S = \{ax + by \mid x, y \in \mathbb{Z}\}$ 称为由 a, b 生成的格。定理A.4说明格 S 可以由一个元素 (a,b) 生成，即其维数为 1，且 (a,b) 是 S 中最“短”（即绝对值最小）的非零元素。从本章后续要介绍的代数结构的观点来讲，定理A.4的含义是：a, b 在整数环 $\mathbb{Z}$ 中生成的理想为主理想，其生成元恰为 (a,b)。

推论 A.2

任给 a, $b \in \mathbb{Z}^+$，a 和 b 互素当且仅当存在 α, $\beta \in \mathbb{Z}$，使得

$$\alpha a + \beta b = 1. \tag{A.6}$$

♡

下面讨论整数最大公因子的计算问题。由推论A.1可以看出，可以利用整数的带余除法将整数 a, b 最大公因子的计算转化为整数 b, c 最大公因子的计算，其中 $c = (a \bmod b)$。将此过程依次进行下去，直到带余除法中不再出现非零的余数，即可求出 a, b 的最大公因子。这种方法称为辗转相除法，又称为欧几里得算法。事实上，中国古代的数学典籍《九章算术》中记载的更相减损术：“子之数，以少减多，更相减损，求其等也”，本质上也是利用辗转相除求整数最大公因子的方法。

定理 A.5 (欧几里得算法)

设 a, $b \in \mathbb{Z}$，$a \geq b > 0$，则存在商序列 $q_1, q_2, \cdots, q_k$ 和余数序列 $r_0, r_1, \cdots, r_k$ 满足：

$$\begin{aligned}
a &= r_0, \\
b &= r_1, \\
r_0 &= r_1 q_1 + r_2, \quad 0 \leq r_2 < r_1, \\
r_1 &= r_2 q_2 + r_3, \quad 0 \leq r_3 < r_2, \\
&\vdots \\
r_{k-2} &= r_{k-1} q_{k-1} + r_k, \quad 0 \leq r_k < r_{k-1}, \\
r_{k-1} &= r_k q_k,
\end{aligned} \tag{A.7}$$

并且有 $\gcd(a,b) = r_k$.

♡

证明 根据推论A.1可得

$$(a,b) = (r_0, r_1) = (r_1, r_2) = \cdots = (r_{k-1}, r_k) = r_k.$$

欧几里得算法的伪代码如下：

算法 A.1： 欧几里得算法

输入： 整数 a, b, $a \geq b \geq 0$

输出： $\gcd(a,b)$

```
Function Euclid(a, b):
    while b ≠ 0 do
        r = (a mod b)
        a = b
        b = r
    返回 a
```

例 A.5 求 $\gcd(68, 28)$.

解

$$
\begin{aligned}
68 = 2 \times 28 + 12 \quad &\to \quad \gcd(28, 12); \\
28 = 2 \times 12 + 4 \quad &\to \quad \gcd(12, 4); \\
12 = 3 \times 4 + 0 \quad &\to \quad \gcd(4, 0).
\end{aligned}
$$

所以 $\gcd(68, 28) = 4$。

定义 A.5 (同余和剩余类)

设 a, b, $n \in \mathbb{Z}$, $n \neq 0$。如果 $(a \bmod n) = (b \bmod n)$，则称 a 和 b 模 n 同余，记为 $a \equiv b \pmod{n}$ 或 $a \equiv_n b$。与 a 模 n 同余的整数全体称为 a 的剩余类，记作 $[a]$。称 a 为这个剩余类的代表元。模 n 的所有剩余类构成的集合记作 $\mathbb{Z}_n$。

例 A.6 $23 \equiv 18 \pmod{5}$, $57 \equiv 1 \pmod{7}$，但 $12 \not\equiv 3 \pmod{7}$

命题 A.5

设 a, b, c, $n \in \mathbb{Z}$, $n \neq 0$。整数的同余关系满足如下性质：

- 自反性：$a \equiv a \pmod{n}$；
- 对称性：若 $a \equiv b \pmod{n}$，则 $b \equiv a \pmod{n}$；
- 传递性：若 $a \equiv b \pmod{n}$，$b \equiv c \pmod{n}$，则 $a \equiv c \pmod{n}$。

命题A.5表明同余关系是整数集合 $\mathbb{Z}$ 上的一个等价关系。

命题 A.6

设 $n \in \mathbb{Z}$, $n \neq 0$。对任意 a_1, b_1, a_2, $b_2 \in \mathbb{Z}$ 都有：

- 若 $a_1 \equiv b_1 \pmod{n}$ 且 $a_2 \equiv b_2 \pmod{n}$，则有 $a_1 + a_2 \equiv b_1 + b_2 \pmod{n}$；
- 若 $a_1 \equiv b_1 \pmod{n}$ 且 $a_2 \equiv b_2 \pmod{n}$，则有 $a_1 a_2 \equiv b_1 b_2 \pmod{n}$；
- 若 $a \equiv b \pmod{n}$，则有 $n \mid (a - b)$。

定义 A.6 (乘法逆)

设 $n \in \mathbb{Z}$, $n \neq 0$。对于给定的 $a \in \mathbb{Z}$，若存在 $b \in \mathbb{Z}$ 使得 $ab \equiv 1 \pmod{n}$，则称 b 是 a 模 n 的乘法逆。

定理 A.6

设 a, b, c, $n \in \mathbb{Z}$, $n \neq 0$。

1. 整数 a 存在模 n 的乘法逆当且仅当 $(a, n) = 1$。
2. 如果 $ab \equiv ac \pmod{n}$ 且 a 存在模 n 的乘法逆，则 $b \equiv c \pmod{n}$。

证明 1. 若整数 a 存在模 n 的乘法逆，则存在 $b \in \mathbb{Z}$ 使得 $ab \equiv 1 \pmod{n}$，即 $n \mid (ab - 1)$，因此存在 $k \in \mathbb{Z}$ 使得 $ab - kn = 1$。若 $d = (a, b) > 1$，则由 $d \mid a$，$d \mid n$ 可得 $d \mid 1$，矛盾。因此有 $(a, n) = 1$。反过来，若 $(a, n) = 1$，根据推论A.2可知存在 α, $\beta \in \mathbb{Z}$ 使得 $1 = \alpha a + \beta n$，从而有 $\alpha a \equiv 1 \pmod{n}$，即 a 存在乘法逆。

2. 由 $ab \equiv ac \pmod{n}$，在同余式两边乘以 a 的乘法逆，即可得到 $b \equiv c \pmod{n}$。

由定理A.6的证明可以看出，求整数 a 模 n 的乘法逆，等价于求 $(a, n) = 1$ 的 Bézout 系数。欧几里得算法能够求出两个整数的最大公因子，若要将相应的 Bézout 系数同时求出来，则需要利用扩展欧几里得算法。

定理 A.7 (扩展欧几里得算法)

设 $a,\ b \in \mathbb{Z},\ a \geq b > 0$，则存在商序列 $q_1,\ q_2,\ \cdots,\ q_k$ 和余数序列 $r_0,\ r_1,\ \cdots,\ r_{k+1}$，以及序列 $s_0,\ s_1,\ \cdots,\ s_{k+1},\ t_0,\ t_1,\ \cdots,\ t_{k+1}$，满足

$$r_0 = a,\ r_1 = b,\ r_{i-1} = r_i q_i + r_{i+1},\ 0 \leq r_{i+1} < r_i$$

$$s_0 = 1,\ s_1 = 0,\ s_{i+1} = s_{i-1} - s_i q_i,$$

$$t_0 = 0,\ t_1 = 1,\ t_{i+1} = t_{i-1} - t_i q_i,$$

其中 $1 \leq i \leq k,\ r_{k+1} = 0$，使得

$$r_i = s_i a + t_i b,\ \ 0 \leq i \leq k+1. \tag{A.8}$$

进而有

$$(a, b) = s_k a + t_k b. \tag{A.9}$$

♡

证明 先证式(A.8)。对 i 使用归纳法。当 $i = 0,\ 1$ 时，结论显然成立。假设对于 $2, 3, \ldots, i-1$ 等式成立，根据递推关系有

$$\begin{aligned} r_i &= r_{i-2} - r_{i-1} q_{i-1} \\ &= (a s_{i-2} + b t_{i-2}) - (a s_{i-1} + b t_{i-1}) q_{i-1} \\ &= a(s_{i-2} - s_{i-1} q_{i-1}) + b(t_{i-2} - t_{i-1} q_{i-1}) \\ &= a s_i + b t_i. \end{aligned}$$

根据定理A.5可知 $(a, b) = r_k$，从而式(A.9)成立。

扩展欧几里得算法的伪代码如下：

算法 A.2： 扩展欧几里得算法

输入： 整数 $a,\ b,\ a \geq b \geq 0$

输出： $\gcd(a, b)$ 及其 Bézout 系数 $(s,\ t)$

```
Function ExtendedEuclid(a, b):
    s = 0, old_s = 1
    t = 1, old_t = 0
    r = b, old_r = a
    while r ≠ 0 do
        q = old_r div r                    /* div 表示求 r 除 old_r 的商 */
        (old_r, r) = (r, old_r − q * r)
        (old_s, s) = (s, old_s − q * s)
        (old_t, t) = (t, old_t − q * t)
    返回 old_r, (old_s, old_t)
```

例 A.7 求 $\gcd(68, 28)$ 的 Bézout 系数。

解

$$s_0 = 1,\ s_1 = 0,\ t_0 = 0,\ t_1 = 1,$$

$$\begin{aligned} 68 = 2 \times 28 + 12 \quad &\rightarrow \quad q_1 = 2,\ s_2 = 1 - 0 \times 2 = 1,\ t_2 = 0 - 1 \times 2 = -2, \\ 28 = 2 \times 12 + 4 \quad &\rightarrow \quad q_2 = 2,\ s_3 = 0 - 1 \times 2 = -2,\ t_3 = 1 - (-2) \times 2 = 5, \\ 12 = 3 \times 4 + 0 \quad &\rightarrow \quad q_3 = 3,\ s_4 = 1 - (-2) \times 3 = 7,\ t_4 = (-2) - 5 \times 3 = -17. \end{aligned}$$

所以 $\gcd(68, 28) = (-2) \times 68 + 5 \times 28 = 4$。

定义 A.7 (欧拉函数)

设 n 为正整数，小于 n 且与之互素的正整数的个数称为 n 的欧拉函数，记为 $\phi(n)$。 ♣

例 A.8 $\phi(7)=6$，$\phi(8)=4$，$\phi(10)=4$。若 p 为素数，则显然有 $\phi(p)=p-1$。

定理 A.8 (欧拉定理)

对于 $a,\ m\in\mathbb{Z}^+$，若 $(a,m)=1$，则

$$a^{\phi(m)}\equiv 1 \pmod{m}. \tag{A.10}$$

♡

例 A.9 $7^{\phi(6)}=7^2\equiv 1 \pmod 6$，$12^{\phi(3)}=12^2\equiv 1 \pmod 3$。

证明 当 $m=1$ 时，结论显然成立，如下假设 $m>1$。设

$$S=\{k_1,k_2,\cdots,k_{\phi(m)}\}$$

表示小于 m 且与 m 互素的正整数全体。定义集合

$$S'=\{(ak \bmod m)\mid k\in S\}.$$

下证 $S=S'$。

定义映射：

$$\begin{aligned} f:\ S &\longrightarrow S' \\ k_i &\longmapsto (ak_i \bmod m) \end{aligned}$$

若存在 $1\le i,j\le\phi(m)$，$i\ne j$，使得 $(ak_i \bmod m)=(ak_j \bmod m)$，则有 $ak_i\equiv ak_j \pmod m$。由于 $(a,m)=1$，进而有 $k_i\equiv k_j \pmod m$。注意到 $0\le k_i,\ k_j<m$，因此必有 $k_i=k_j$。这表明 f 为单射。

由于 $S'\subseteq S$ 且 S 为有限集，由此可知 $S=S'$。从而有

$$a^{\phi(m)}\prod_{1\le i\le\phi(m)}k_i\equiv\prod_{1\le i\le\phi(m)}ak_i\equiv\prod_{1\le i\le\phi(m)}k_i \pmod m.$$

再由 $\left(\prod_{1\le i\le\phi(m)}k_i,m\right)=1$ 即可得到

$$a^{\phi(m)}\equiv 1 \pmod m.$$

注 使用欧拉定理也可求解 a 模 m 的乘法逆。事实上，由 $a^{\phi(m)}\equiv 1 \pmod m$ 显然可以得到 $a\cdot a^{\phi(m)-1}\equiv 1 \pmod m$，因此 $a^{-1}=(a^{\phi(m)-1} \bmod m)$。

欧拉定理的一个特例是如下的费马小定理。

定理 A.9 (Fermat 小定理)

设 p 是一个素数，对任意 $a\in\mathbb{Z}^+$，若 $(a,p)=1$，则有 $a^p\equiv a \pmod p$。 ♡

定理 A.10

设 $m,\ n\in\mathbb{Z}$，$(m,n)=1$，则有

$$\mathbb{Z}_{mn}=\{my+nx\mid x\in\mathbb{Z}_m,\ y\in\mathbb{Z}_n\}.$$

♡

证明 只需要证明集合 $\{my+nx\mid x\in\mathbb{Z}_m,\ y\in\mathbb{Z}_n\}$ 的 mn 个元素互不相同即可。假设对 $x,\ x'\in\mathbb{Z}_m$，$y,\ y'\in\mathbb{Z}_n$，

$$my+nx\equiv my'+nx' \pmod m n,$$

则

$$my \equiv my' \pmod{n},$$
$$nx \equiv nx' \pmod{m}.$$

由于 $(m,n)=1$，所以有

$$y \equiv y' \pmod{n},$$
$$x \equiv x' \pmod{m}.$$

定理 A.11

设 $m,\ n \in \mathbb{Z}^+$，$(m,n)=1$，则 $\phi(mn)=\phi(m)\phi(n)$.

♡

证明 设 $\mathbb{Z}_m^*$，$\mathbb{Z}_n^*$ 分别表示小于 m 或 n 且与它们分别互素的正整数集合。下证 $\mathbb{Z}_{mn}^* = \{my+nx \mid x \in \mathbb{Z}_m^*,\ y \in \mathbb{Z}_n^*\}$。

一方面，对任意 $x \in \mathbb{Z}_m^*$，$y \in \mathbb{Z}_n^*$，显然有 $(my+nx, mn)=1$。这是因为 $(m,n)=1$，从而

$$(my+nx, mn) = (my+nx, m)\cdot(my+nx, n) = (nx, m)\cdot(my, n) = 1,$$

即 $my+nx \in \mathbb{Z}_{mn}^*$。

反过来，设 a 是小于 mn 且与之互素的整数。首先，根据定理A.11，必存在整数 $x \in \mathbb{Z}_m$，$y \in \mathbb{Z}_n$ 使得 $a \equiv my+nx \pmod{m}n$。由 $(m,n)=1$ 和 $(my+nx, mn) = (nx, m)\cdot(my, n) = 1$ 可得 $(nx, m) = (my, n) = 1$，进而可得 $(x, m) = (y, n) = 1$，即 $x \in \mathbb{Z}_m^*$，$y \in \mathbb{Z}_n^*$。

设 f 为定义在正整数集合上的函数，称为数论函数。若对任意 m，n，$(m,n)=1$，都有 $f(mn) = f(m)\cdot f(n)$，则称 f 为乘性函数。定理A.11表明欧拉函数为乘性函数。定理A.11还给出了在给定正整数 n 的素因子分解的条件下计算其欧拉函数值的方法。

命题 A.7

设 $n \in \mathbb{Z}$，$n>2$，其素因子分解为

$$n = \prod_{i=1}^{k} p_i^{e_i},$$

其中 p_i 是互不相同的素数，$e_1 \geq 1$，则

$$\phi(n) = \prod_{i=1}^{k}(p_i^{e_i} - p_i^{e_i-1}) = n\prod\left(1-\frac{1}{p_i}\right). \tag{A.11}$$

♠

设 $a,\ b \in \mathbb{Z}$，$a \neq 0$，$m \in \mathbb{Z}^+$。形如

$$ax + b \equiv 0 \pmod{m}$$

的方程称为线性同余方程，也称模线性方程，其中 x 为未定元。

定理 A.12

若 $(a,m) \mid b$，则线性同余方程 $ax+b \equiv 0 \pmod{m}$ 有 (a,m) 个模 m 互不同余的解。否则，同余方程无解。

♡

证明 若 $(a,m)=1$，则存在 $\alpha,\ \beta$ 使得

$$a\alpha + m\beta = 1.$$

故 $x = -b\alpha$ 即为同余方程的一个解。下证在模 m 同余的意义下解的唯一性。若存在 $x, x' \in \mathbb{Z}$ 使得

$$ax' + b \equiv 0 \pmod{m},$$
$$ax + b \equiv 0 \pmod{m},$$

则有

$$a(x - x') \equiv 0 \pmod{m}.$$

由于 $(a, m) = 1$，所以

$$x \equiv x' \pmod{m}. \tag{A.12}$$

若 $(a, m) = d > 1$，容易看出若同余方程 $ax + b \equiv 0 \pmod{m}$ 有解则 d 必整除 b。由此可得

$$\frac{a}{d}x + \frac{b}{d} \equiv 0 \pmod{\frac{m}{d}}. \tag{A.13}$$

注意到 $(a/d, m/d) = 1$，根据上面的讨论，方程(A.13)有唯一解 x_1 满足 $0 \le x_1 < \frac{m}{d}$。而对任意整数 t，

$$x = x_1 + \frac{m}{d}t$$

显然皆为方程(A.13)的解。故在模 m 的意义下，

$$x_1,\ x_1 + \frac{m}{d},\ x_1 + 2\frac{m}{d},\ \ldots,\ x_1 + (d-1)\frac{m}{d}$$

两两不同余，给出了方程 $ax + b \equiv 0 \pmod{m}$ 的所有解。

下面再考虑线性同余方程组的求解。所谓线性同余方程组，是指由一组线性同余方程 $a_i x + b_i \equiv 0 \pmod{m_i}$ 构成的方程组。根据定理A.12，对每个同余方程的研究都可以归结为 $(a_i, m_i) = 1$ 的情况，因此本质上只需考虑形如 $x \equiv b_i \pmod{m_i}$ 的方程构成的线性同余方程组即可。

关于线性同余方程组的研究最早可见于中国南北朝时期（公元 500 年左右）的数学典籍《孙子算经》，该书的卷下第二十六题是如下的“物不知数”问题：

> 今有物不知其数，三三数之剩二，五五数之剩三，七七数之剩二，问物几何？

使用现代数学语言描述，“物不知数”问题即为求正整数 x，使得下列同余方程组成立：

$$\begin{cases} x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{5} \\ x \equiv 2 \pmod{7} \end{cases}$$

南宋数学家秦九韶在《数书九章》一书中对“物不知数”问题给出了完整、系统的解答，他提出的“大衍求一术”事实上给出了求解线性同余方程组的一般算法，这一方法国际上通常称作“孙子定理”或“中国剩余定理”。对“物不知数”问题，明朝数学家程大位还将其解法编成了易于上口的《孙子歌诀》：

> 三人同行七十稀，五树梅花廿一支，
> 七子团圆正半月，除百零五使得知。

其含义是：将用 3 除所得的余数乘以 70，加上用 5 除所得的余数乘以 21，加上用 7 除所得的余数乘以 15，其总和用 105 除，所得余数即为答案。

定理 A.13 (中国剩余定理)

设 $m_1, m_2, \ldots, m_k$ 是 k 个两两互素的正整数，$M = m_1 m_2 \cdots m_k$。线性同余方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \quad \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

则存在唯一小于 M 的解

$$x = \frac{M}{m_1} \cdot e_1 a_1 + \frac{M}{m_2} \cdot e_2 a_2 + \cdots + \frac{M}{m_k} \cdot e_k a_k \bmod m, \tag{A.14}$$

其中 $\frac{M}{m_i} \cdot e_i \equiv 1 \pmod{m_i}$，$1 \le i \le k$。

♡

证明 设

$$M_i = \frac{M}{m_i} = \prod_{\substack{l=1 \\ l \neq i}}^{k} m_l, \quad 1 \le i \le k.$$

由 M_i 的定义可知 $(M_i, m_i) = 1$，所以 M_i 在模 m_i 下有唯一的乘法逆元 e_i，即

$$\frac{M}{m_i} \cdot e_i \equiv 1 \pmod{m_i}.$$

由于对任意 $i \neq j$，都有 $m_j \mid M_i$，所以容易看出式(A.14)给出的 x 满足

$$x \equiv a_i \pmod{m)_i}, \quad 1 \le i \le k$$

因此只需证明在模 M 的意义下解是唯一的即可。假设有另外一个解 y，即

$$y \equiv a_i \pmod{m)_i}, \quad 1 \le i \le k,$$

则有

$$x - y \equiv 0 \pmod{m)_i}, \text{ 即 } m_i \mid x - y, \quad 1 \le i \le k.$$

由于对任意 $i \neq j$，$(m_i, m_j) = 1$，从而有 $M \mid (x - y)$，即 $x \equiv y \pmod{M}$。

例 A.10 求下列同余方程组的解：

$$\begin{cases} x \equiv 2 \pmod{3} \\ x \equiv 3 \pmod{5} \\ x \equiv 2 \pmod{7} \end{cases}$$

解 $M = 3 \times 5 \times 7 = 105$，$M_1 = 35$，$M_2 = 21$，$M_3 = 15$。易知

$$e_1 = M_1^{-1} \equiv 2 \pmod{3},$$
$$e_2 = M_2^{-1} \equiv 1 \pmod{5},$$
$$e_3 = M_3^{-1} \equiv 1 \pmod{7}.$$

所以

$$x \equiv 35 \times 2 \times 2 + 21 \times 1 \times 3 + 15 \times 1 \times 2 \equiv 23 \pmod{105}.$$

A.2 抽象代数初步

A.2.1 群

设 S 是一非空集合，我们把 $S \times S \to S$ 的一个映射 $\circ$ 称为 S 上的 (二元) 运算, 而把 $a, b \in S$ 的像 $\circ(a, b)$ 记做 $a \circ b$，或省略 $\circ$，只简单地写作 ab.

定义 A.8 (群)

我们说一个非空集合 G 对于 G 上的一个二元运算 $\circ$ 来说作成一个群，如果：

1) $\circ$ 是结合的，即对任何 $a, b, c \in G, a \circ (b \circ c) = (a \circ b) \circ c$

2) G 中存在一个元素 e 满足：$\forall a \in G, a \circ e = e \circ a = a$。这个元素称为 G 中的单位元

3) 对 $\forall a \in G$, 存在一个元素 $a^{-1} \in G$ 满足 $a \circ a^{-1} = a^{-1} \circ a = e$, 称为 a 的逆元

如果 G 中的元素还满足:

4) 对 $\forall a, b \in G, a \circ b = b \circ a$, 则 G 称为交换群或 Abel 群。(这时也把 $\circ$ 表示成 $+$)，因此也称为加法群。

♣

容易证明在一个群中，单位元和元素的逆元是唯一的，而且对 $\forall a, b \in G, (a \circ b)^{-1} = b^{-1} \circ a^{-1}$. 我们约定：

$a^0 = e$

$a^n = \underbrace{a \circ a \circ \ldots \circ a}_{\text{(n 个)}}$ 或 $na = \underbrace{a + a + \ldots + a}_{\text{(n 个)}}$

$a^{-n} = (a^{-1})^n$

显然有:

$a^n a^m = a^{m+n}, (a^n)^m = a^{mn}$

例 A.11 所有整数的集合在加法运算下构成一个交换群。

例 A.12 只含有一个元素 e 的集合在运算 $e * e = e$ 下构成一个群。

定义 A.9 (有限群、子群、循环群)

1) 一个群 G 称作有限群（无限群）如果 G 中含有有限多个元素（无限多个元素）。有限群中元素的个数称为 G 的阶，用 $|G|$ 表示。

2) 群 G 的一个子集 H，如果在 G 的运算下，仍构成一个群, 则称 H 为 G 一个子群。

3) 一个群 G 称为循环群，如果存在一个元素 $a \in G$ 使得 $\forall b \in G$, 存在整数 t 满足 $b = a^t$。这样的元素 a 称为 G 的生成元。记作 $G = \langle a \rangle$.

♣

定义 A.10 (元素的阶)

$\forall a \in G$, 定义 $\langle a \rangle = \{a^i | i \in \mathbf{Z}\}$. 则 $\langle a \rangle$ 为子群。如果 $\langle a \rangle$ 是有限群则 $\langle a \rangle$ 的阶也称为 a 的阶，否则称 a 为无限阶的。

显然一个元素 a 的阶 k 是满足 $a^k = e$ 的最小的正整数。

♣

定义 A.11 (等价关系)

$R \subset S \times S$ 称为等价关系，如果：

a) $(s, s) \in R$ （自反性）；

b) $(s, t) \in R \Rightarrow (t, s) \in R$ （对称性）；

c) $(s,t),(t,u)\in R\Rightarrow(s,u)\in R$ （传递性）；

显然模整数 n 的同余关系是一个等价关系。将 **Z** 分成 n 个等价类 $[0],[1],[2],...,[n-1]$.

例 A.13 $\{[0],[1],[2],...,[n-1]\}$ 在运算 $[a]+[b]=[a+b]$ 下构成一个加法群，称为模 n 的整数群。记作 $\mathbf{Z}_n$.

显然，$\mathbf{Z}_n$ 是一个循环群。$\mathbf{Z}_n=\langle[1]\rangle$，而且任何一个与 n 互素的整数 t，$[t]$ 都是 $\mathbf{Z}_n$ 的生成元。

定理 A.14 (lagrange 定理)

对于一个有限群 G, 其任何一个子群 H 的阶整除群 G 的阶，任一元素的阶也整除 G 的阶。

定理 A.15

1) 任意循环群的子群仍是循环群。
2) 在一个阶为 m 的有限阶循环群 $\langle a\rangle$ 中，a^k 生成一个阶为 $\frac{m}{gcd(k,m)}$ 的子群。
3) 设 f 是有限循环群 $\langle a\rangle$ 的阶的正因子，则 $\langle a\rangle$ 中有 $\phi(f)$ 个阶为 f 的元素。$\phi(f)$ 是 Euler 函数即与 f 互素的整数的个数。
4) 阶为 m 的有限循环群 $\langle a\rangle$ 正好有 $\phi(m)$ 个生成元，即 a^r 满足 $\langle a^r\rangle=\langle a\rangle$. 这些生成元形为 a^r, 其中 $(r,m)=1$.

定义 A.12

设 $f:G\to H$ 是群 G 到群 H 的一个映射。如果 $\forall a,b\in G, f(a\cdot b)=f(a)\cdot f(b)$ 则称 f 为 G 到 H 的同态。进一步如果 $f(G)=H$, 则称 f 为满同态 (或称 f 为 G 到 H 上的同态), 这时 H 称为 G 的同态像；如果 f 是单射，则称 f 为单同态映射；如果 f 是一一对应，则称 f 为同构映射，这时称 G 和 H 是同构的，记做 $G\cong H$。G 到 G 自身的同构映射称为自同构。

同态映射总是把单位元映到单位元，把逆元素映射成像的逆元素。

例 A.14 内自同构 设 G 是群。$\forall a\in G$，定义 f_a: $f_a(b)=aba^{-1}, b\in G$. 则 f_a 称为群 G 的由 a 定义的内自同构，而把一个元素在内自同构下的像称为该元素的共轭元。显然群中元素的共轭关系是一个等价关系。

A.2.2 环和域

定义 A.13

一个集合 R 称为一个环，如果 R 有两个运算 + 和 · 满足：

1. $(R,+)$ 是一个交换群。
2. · 是结合的，即 $(a\cdot b)\cdot c=a\cdot(b\cdot c)$.
3. 满足分配率：即对任何 $a,b,c\in R, a(b+c)=ab+ac, (b+c)a=ba+ca$.

定义 A.14

1. 一个环称为有单位元的，如果它有乘法单位元。
2. 一个环称为交换环，如果其中的乘法运算是交换的。
3. 一个环称为整环，如果它是一个交换的有单位元的环且 $e\neq 0$, 对于任意的 $a\neq 0, b\neq 0,\Rightarrow$

$ab \neq 0$.

4. 一个环称为除环，如果所有非零元在乘法运算 $\cdot$ 下构成一个群。
5. 一个交换的除环称为域。

♣

定义 A.15

环 R 的一个子集 S 称为 R 的一个子环，如果 S 关于 $+$ 和 $\cdot$ 是封闭的并且在这两种运算下形成一个环。

♣

定义 A.16

环 R 的一个子集 J 称为一个理想，如果 J 是 R 的一个子环并且对所有 $a \in J$，$r \in R$, 有 $ar \in J$ 和 $ra \in J$.

♣

例 A.15

1) 设 R 为有理数域，则整数集 $\mathbf{Z}$ 是 R 的子环，但不是理想。因为 $1 \in \mathbf{Z}, \frac{1}{2} \in Q$, 但是 $\frac{1}{2} \cdot 1 = \frac{1}{2} \notin \mathbf{Z}$.
2) 设 R 为交换环，$a \in R$, 则包含 a 的最小理想 $(a) = \{ra + na : r \in R, n \in \mathbf{Z}\}$. 如果 R 包含一个单位元，则 $(a) = \{ra : r \in R,\}$

定义 A.17

R 是一个交换环，R 的一个理想 J 称为主理想，如果存在 $a \in R$ 使得 $J = (a)$. 这时，J 也称为由 a 生成的主理想。

♣

定义 A.18

设 R 是一整环。如果 R 中的每一个理想都是主理想，则称 R 是主理想整环。

♣

定义 A.19

设 $P \neq R$ 是 R 的一个理想，如果 $ab \in P \Rightarrow a \in P$ 或 $b \in P$, 则称 P 为素理想。
$M \neq R$ 是 R 的一个理想。如果 J 是 R 的理想且 $M \subset J \Rightarrow J = R$ 或 $J = M$, 则称 M 是极大理想。

♣

例 A.16

1) R 为任意运算为 $+$ 的 Abel 群，对 $a, b \in R$, 定义 $ab = 0$. 则 R 为环。
2) 所有的整数在通常数的加法和乘法运算下形成一个整环，称为整数环。整数环是一主理想整环。
3) 所有偶数在通常数的加法和乘法运算下形成一个没有单位元的交换环。
4) 所有从实数到实数的映射按照运算 $(f + g)(x) = f(x) + g(x), (fg)(x) = f(x)g(x), \forall x \in R.$ 形成一个有单位元的交换环。
5) $\{\begin{pmatrix} a & b \\ c & d \end{pmatrix}\}, a, b, c, d \in R$ 按照矩阵的加法和乘法构成有单位元的环。
6) 所有的有理数在通常数的加法和乘法运算下构成一域，称为有理数域。
7) 模 n 的剩余类环 $\mathbf{Z}_n = \mathbf{Z}/(n)$.

设 J 是环 R 的一个理想，则我们可以定义环 R 中元素的等价关系 $\sim$:

$$a \sim b \Longleftrightarrow a - b \in J.$$

该等价关系将环 R 分成了一些互不相交的等价类的并，每个等价类叫作模 J 的剩余类。剩余类中的元素记为 $[a] = a + J$(因为里面的元素都是具有形式 $a + c, c \in J$).

可以直接验证，环 R 模 J 的剩余类按运算

$(a+J)+(b+J)=(a+b)+J$

$(a+J)(b+J)=ab+J$

可以形成一个环。

定义 A.20 (商环)

按上述运算所作成的剩余类环称为 R 模 J 的剩余类环 (或商环)，记作 R/J.

♣

定义 A.21 (同态、同构)

设 R,S 为环，$a,b\in R$. 一个映射 $\varphi: R\to S$ 称为环同态, 如果:

1) $\varphi(a+b)=\varphi(a)+\varphi(b)$

2) $\varphi(ab)=\varphi(a)\varphi(b)$

而集合 $ker\varphi=\{a\in R:\varphi(a)=0\in S\}$ 称为同态映射 φ 的核. 一个同态映射，如果既是单的又是满的，则称为同构映射。如果两个环之间存在同构映射，我们则说这两个环是同构的，用 $\cong$ 表示。

♣

定理 A.16 (同态基本定理)

设 R 和 S 是环。如果 $\varphi: R\to S$ 是满同态，则 $ker\varphi$ 为 R 的理想，且 $S\cong R/ker\varphi$. 反过来，如果 J 为 R 的理想，定义映射 $\varphi(a)=a+J, a\in R$. 则映射 $\varphi: R\to R/J$ 是满同态且 $ker\varphi=J$.

♡

定理 A.17

设 R 是一个有单位元的交换环，则:

1) 理想 M 是极大理想 $\Leftrightarrow$ R/M 是域。

2) 理想 P 是素理想 $\Leftrightarrow$ R/P 是整环。

3) 每个极大理想都是素理想。

4) 如果 R 是主理想整环，则 $R/(c)$ 是域 $\Leftrightarrow$ c 是 R 中的素元。

♡

证明:

1) $\Rightarrow$，如果 M 是极大理想。$\forall a\notin M$，则 $J=\{ar+m|r\in R, m\in M\}$ 是一个理想且 $J\supsetneq M$, 从而 $J=R$. 所以 $\exists r\in R, m\in M$ 使得 $ar+m=1$, 因此 $\overline{a}\cdot\overline{r}=\overline{1}$. 所以 R/M 是域。

$\Leftarrow$, 假设 R/M 是域，假设 $J\supseteq M$, $J\neq M$, 则存在 $a\in J$, $a\notin M$, 所以 $\overline{a}\neq 0$. 设 $\overline{a}$ 的逆元 $\overline{r}$，即 $(a+M)(r+M)=1+M$. 所以 $ar+m=1$ 对某些 $m\in M$. 又因为 $m\in M\subseteq J, a\in J$, 所以 $ar+m\in J$, 所以 $1\in J$, 从而 $J=R$.

2) $\Rightarrow$，假设 P 是素理想，则 R/P 是一个交换环。单位元为 $\overline{1}=1+P\neq 0+P$. 假设 $(a+P)(b+P)=0+P\Rightarrow ab+P=0+p$, 所以 $ab\in P$, 从而 $a\in P$ 或 $b\in P$ 即 $\overline{a}=0$ 或 $\overline{b}=0$. 所以 R/P 没有零因子。

$\Leftarrow$, 设 $ab\in P$, 所以 $\overline{a}\overline{b}=0$. 所以 $\overline{a}=0$ 或 $\overline{b}=0$, 即 $a\in P$ 或 $b\in P$。

3) 由 1) 和 2) 即得。

4) $\Rightarrow$，如果 $R/(c)$ 是域，则 c 不是单位（否则，$R/(c)=0$, 不是域)。假设 c 不是素元，则 c 有一因子 a 既不是单位也不是 c 的相伴元素。($a\neq 0$, 否则 $c=0$, 从而 a 和 c 相伴). 记 $c=a\cdot b, b\in R$. 我们有 $a\notin(c)$, 否则 $a=c\cdot d=abd$. 所以 $a(1-bd)=0$, 由于 R 是整环。所以 $(1-bd)=0$，即 b 和 d 为单位，所以 a 和 c 相伴，矛盾。从而 $(a)\supset(c)$, 这与由 1) 得出的 (c) 是极大理想矛盾。

$\Leftarrow$, 假设 c 是素的，则 $(c) \neq R$。假设 $J \supset (c)$ 是 R 的一个理想，由于 R 是主理想整环，所以 $\exists a \in R, J = (a)$. 所以 a 或者是单位或者与 c 相伴。所以 $J = R$ 或 $J = (c)$，所以 (c) 是极大的。从而 $R/(c)$ 是域。

推论 A.3

$\mathbf{Z}_p = \mathbf{Z}/(p)$ 是域 $\Leftrightarrow$ p 是素数。$\mathbf{Z}_p$ 也称为阶为 p 的 Galois 域

♡

定义 A.22 (环的特征)

设 R 是任一环，如果存在正整数 n 使得对任何 $r \in R$,$n \cdot r = 0$. 且对任何小于 n 的正整数 n', $n' \cdot r \neq 0$, 则称 n 为环 R 的特征，否则则称 R 的特征为 0。

♣

定理 A.18

域的特征或者是 0, 或者是一素数。特别，有限域的特征一定是一素数。

♡

定义 A.23 (子域、扩域)

设 F 是域，K 是 F 的子集。如果 K 在 F 的运算下也构成一个域则称 K 为 F 的子域，而 F 则称为 K 的扩域。如果 $K \neq F$ 则称 K 为 F 的真子域。

一个域如果不包含任何子域，则称为素域。如果一个域 F 的子域作为域是素域，则称该子域为 F 的素子域。

♣

A.2.3 多项式环

定义 A.24

由系数在 R 中的多项式组成的环称为 R 上的多项式环，记为 $R[x]$.

♣

定理 A.19

假设 R 是一个环，则：

1) $R[x]$ 是交换的 $\Leftrightarrow$ R 是交换的。

2) $R[x]$ 有单位元 $\Leftrightarrow$ R 有单位元。

3) $R[x]$ 是一个整环 $\Leftrightarrow$ R 是整环。

特别当 R 是一个域 F 时，$F[x]$ 是一个整环。

♡

定理 A.20 (除法)

$g \neq 0, g \in F[x]$, 则对任何 $f \in F[x], \exists q, r \in F[x]$, 满足 $f = g \cdot q + r, \deg(r) < \deg(g)$.

♡

例 A.17 $f(x) = 2x^5 + x^4 + 4x + 3 \in F_5[x], g(x) = 3x^2 + 1 \in F_5[x]$.

答案：$q(x) = 4x^3 + 2x^2 + 2x + 1, r(x) = 2x + 2$.

定理 A.21

$F[x]$ 是主理想整环。事实上对任何 $J \neq (0) \subset F[x]$, 存在唯一的首一多项式 $g \in F[x]$ 使得 $J = (g)$.

♡

定理 A.22

假设 $f_1, f_2, ..., f_n$ 是 $F[x]$ 中不全为零的一组多项式，则存在唯一的首一多项式 d 满足：

1) d 整除每一个 f_i.

2) 整除每个 f_i 的 c 一定整除 d. 更进一步，$\exists b_i \in F[x]$ 使得 $d = \sum b_i f_i$.

♡

证明：$J = \{\sum c_i f_i | c_i \in F[x]\}$, 则 J 是 $F[x]$ 中的理想，则 $f_i \in J$. 由于 $F[x]$ 是主理想整环，所以 $\exists d \in F[x]$, 使 $J = (d)$(d 是某一首一多项式)。所以 d/f_i 且 $d = \sum c_i f_i$(因为 $d \in J$). 所以 2) 成立。

假设有另一个 d_1 满足 1) 和 2), 则 d_1 和 d 是相伴的, 因此 $d_1 = d$.

上述定理中的首一多项式 d 称为 $f_1, \cdots, f_n$ 的最大公因子，表示为 $d = \gcd(f_1, \cdots, f_n)$. 如果 $d = 1$, 则称 $f_1, \cdots, f_n$ 是互素的。

定义 A.25 (不可约多项式)

一个多项式 $p \in F[x]$ 称为不可约的或素的，如果 p 是 $F[x]$ 中的素元，即 p 的次数不为零且 p 不能分成两个正次数多项式的乘积。

♣

定理 A.23 (唯一分解)

$deg(f) \geq 1, f \in F[x]$ 可以唯一写成 $f = ap_1^{e_1}...p_k^{e_k}$. 其中 $a \in F, p_i$ 是不同的不可约多项式，$e_i \geq 1$ 而且在不计次序的情况下，这种分解是唯一的。

♡

定理 A.24

$f \in F[x]$，则 $F[x]/(f)$ 是域 $\Leftrightarrow$ f 是不可约多项式。

♡

证明：作为练习。

推论 A.4

如果 $F[x]$ 中的不可约多项式 p 整除 $f_1, f_2, ..., f_m$ 则 p 整除至少其中的一个因子。

♡

证明：考虑 $F[x]/(p)$, 由题设知 $\overline{f_1 f_2}...\overline{f_m} = 0$. 由于 p 是不可约多项式，所以 $F[x]/(p)$ 是域。所以至少有一个 $\overline{f_j} = 0$，即 $f_j \in (p)$. 所以 $p \mid f_j$.

A.3 有限域初步

A.3.1 有限域的特征性质

定理 A.25

假设 F 是一个有限域，K 为其子域。如果 K 有 q 个元素，则 F 有 q^m 个元素，其中 $m = [F : K]$。

♡

证明：F 可以看作 K 上的向量空间。如果 $m = [F : K]$，则 F 有由 m 个元素组成的基底 $b_1, b_2, ..., b_m$, 且每一个元素唯一的表示成 $\sum a_i b_i$ 的形式, 其中 $a_i \in K$，所以 F 有 q^m 个元素。

定理 A.26

假设 F 是一个有限域，则 F 有 p^n 个元素。其中 p 是 F 的特征，n 是 F 关于其素域的扩张次数。

♡

证明：由于 F 的特征为 p，则 F 的素子域同构于 F_p。因此含有 p 个元素。由定理A.25知 F 有 p^n 个元素。

定理 A.27

假设 F 是有 q 个元素的有限域，则 $\forall a \in F, a^q = a$。

证明：首先 $a^q = a$ 对 $a = 0$ 成立。F 中所有非零元素组成一个 $q-1$ 阶有限群，所以 $a^{q-1} = 1$ 对所有 $a \neq 0$ 成立。所以 $a^q = a$。

定理 A.28

如果 F 是有 q 个元素的有限域。K 为一子域，则 $K[x]$ 中的多项式 $x^q - x$ 在 $F[x]$ 中可分解为：$x^q - x = \prod_{a \in F}(x - a)$ 且 F 是 K 上多项式 $x^q - x$ 的分裂域。

证明：我们知道 $x^q - x$ 在 F 中至多含有 q 个根，而根据定理A.27，F 中的 q 元素都是这个多项式的根。所以 $x^q - x$ 在 F 中是分裂的，而且不能在任何更小的域中分裂。

定理 A.29 (存在，唯一性定理)

对任何素数 p 和正整数 n，存在一个有限域含有 p^n 个元素。且任何具有 $q = p^n$ 个元素的有限域同构于 $x^q - x$ 在 F_p 上的分裂域。

证明：（存在性）对 $q = p^n$，考虑 F_p 上的多项式 $x^q - x$，假设 F 是 F_p 上 $x^q - x$ 的分裂域。我们知道 $x^q - x$ 有 q 个不同的根，令 S 是 F 中多项式 $x^q - x$ 的所有根组成的集合。则：

1) $0, 1 \in S$.

2) $a, b \in S \Rightarrow a - b \in S$, 因为 $(a-b)^q = a^q - b^q = a - b$.

3) $a, b \in S \Rightarrow (ab^{-1}) \in S$, 因为 $(ab^{-1})^q = a^q \cdot b^{-q} = ab^{-1}$. 所以 S 为 F 的一子域。另一方面 $x^q - x$ 在 S 中分裂，所以 $S = F$，所以 F 具有 q 个元素。

（唯一性）假设 F 是具有 $q = p^n$ 个元素的有限域。则 F 的特征为 p 且以 F_p 为其子域。所以 F 是 F_p 上多项式 $x^q - x$ 的分裂域。由于多项式的分裂域是唯一的（在同构意义下）。所以具有 q 个元素的有限域唯一，且同构于 $x^q - x$ 在 F_p 上的分裂域。

定理 A.30 (子域准则)

假设 F_q 是一个具有 $q = p^n$ 个元素的有限域，则 F_q 的每一个有限域含有 p^m 个元素，则 $m \mid n$。反之，对 n 的任一正因子 m，存在唯一 F_q 的子域含有 p^m 个元素。

证明：假设 K 是 F_q 的子域。显然有 $|K| = p^m$。对某一 $m \in N^+$。由定理A.25，知 q 是 $|K|$ 的某一幂次。所以 $(p^m)^r = p^n$。所以 $m \mid n$. 反之，假设 $m \mid n$，则 $p^m - 1 | p^n - 1$。所以 $x^{p^m-1} - 1 \mid x^{p^n-1} - 1$。因此 $x^{p^m} - x \mid x^{p^n} - x$。从而 $x^{p^m} - x$ 的分裂域为 F_q 的子域且此子域含有 p^m 个元素。假设 F_q 有两个不同的含有 p^m 个元素的子域则这两个子域中元素都是 $x^{p^m} - x$ 的根。从而这两个子域相等。

定理 A.31

对每一个有限域 F_q，其乘法群 F_q^* 是 $q-1$ 阶的循环群。

证明：假设 $q \geq 3$ 且 $h = q - 1 = p_1^{r_1} p_2^{r_2} \cdots p_m^{r_m}$ 下面构造阶为 $p_i^{r_i}$ 的元素 b_i 如下：对每一个 i, 多项式 $x^{h/p_i} - 1$ 最多只有 h/p_i 个根在 F_q 中，所以 F_q 中至少有一个 a_i 不是 $x^{h/p_i} - 1$ 的根，即 $a_i^{h/p_i} \neq 1$ 令

$b_i = a_i^{h/p_i^{r_i}}$ 则 $b_i^{p_i^{r_i}} = a_i^h = 1$. 所以 b_i 的阶整除 $p_i^{r_i}$, 但 $b_i^{p_i^{r_i-1}} = a_i^{h/p_i} \neq 1$。所以 b_i 的阶为 $p_i^{r_i}$。下面证明: $b = b_1 b_2 ... b_m$ 的阶为 h. 否则则 b 的阶至少为某一 h/p_i 的因子。假设为 h/p_1, 则得到 $1 = b^{h/p_1} = b_1^{h/p_1} ... b_m^{h/p_1} = b_1^{h/p_1}$。所以 $p^r \mid h/p_1$, 矛盾。所以 b 的阶为 h。

定义 A.26

F_q^* 中的生成元称为 F_q 的本原元。

♣

显然 F_q 中有 $\phi(q-1)$ 个本原元, 其中 ϕ 是欧拉函数。

定理 A.32

设 F_q 是一个有限域, F_r 是一个有限扩域, 则 F_r 为 F_q 的一个单扩张, 且任一 F_r 的本原元都是 F_r 在 F_q 上的定义元。

♡

证明: 假设 ξ 为 F_r 的本原元。显然 $F_q(\xi) \subset F_r$。同时有 $F_r \subset F_q(\xi)$。所以 $F_r = F_q(\xi)$

定理 A.33

对任意的有限域 F_q 和正整数 n, 存在 n 次不可约多项式 $\in F_q[x]$。

♡

证明: 根据定理A.29, 存在有限域 F_{q^n}. 显然 F_{q^n} 是 F_q 的 n 次扩张, 即 $[F_{q^n} : F_q] = n$。则由定理A.32, $F_{q^n} = F_q(\xi)$ 对某个 $\xi \in F_{q^n}$ 从而知 ξ 的极小多项式的次数 n.

A.3.2 不可约多项式的根

定理 A.34

设 $f(x) \in F_q[x]$, 是一个不可约多项式。α 是 f 在 F_q 的某一个扩域上的根, 则 $h \in F_q[x], h(\alpha) = 0 \Leftrightarrow f \mid h$。

♡

证明: 假设 a 是 f 的首项次数。定义 $g(x) = a^{-1} f(x)$。显然 g 是一个首一的不可约多项式且 $g(\alpha) = 0$。所以是 α 在 F_q 上的极小多项式。所以 $h(\alpha) = 0 \Leftrightarrow g(x) \mid h \Leftrightarrow f \mid h$。

定理 A.35

设 $f \in F_q[x]$ 是 m 次不可约多项式。则 $f(x) \mid x^{q^n} - x \Leftrightarrow m \mid n$。

♡

证明: 假设 $f(x) \mid x^{q^n} - x$。α 是 f 在某一个分裂域中的根, 则 $\alpha^{q^n} = \alpha$。所以 $\alpha \in F_{q^n}$。这说明 $F_q(\alpha) \subset F_{q^n}$ 但 $[F_q(\alpha) : F_q] = m, [F_{q^n} : F_q] = n$。所以 $m \mid n$。反之, 如果 $m \mid n$。则 F_{q^m} 可以看作 F_{q^n} 的子域。如果 α 是 f 在某一个分裂域中的一个根, 则 $[F_q(\alpha) : F_q] = m$, 所以 $F_q(\alpha) = F_{q^m}$。因此 $\alpha \in F_{q^n}$。所以 $\alpha^{q^n} = \alpha$。即 α 是 $x^{q^n} - x$ 的根。所以 $f(x) \mid x^{q^n} - x$。

定理 A.36

设 f 是 $F_q[x]$ 中次数为 m 的不可约多项式。则 f 有根 α 在 F_{q^m} 中。进一步, f 的所有根正好为 F_{q^m} 中如下 m 个元素: $\alpha, \alpha^q, \alpha^{q^2}, ..., \alpha^{q^{m-1}}$。

♡

证明: 假设 α 是 f 在某一分裂域中的根则 $[F_q(\alpha) : F_q] = m$。所以 $F_q(\alpha) = F_{q^m}$。所以 $\alpha \in F_{q^m}$。因为如果 $\beta \in F_{q^m}$ 是 f 的一个根, 则 β^q 也是 f 的根, 所以 $\alpha, \alpha^q, ..., \alpha^{q^{m-1}}$ 都是 f 的根。下面证明这些元素互不相同。假设 $\alpha^{q^j} = \alpha^{q^k}, 0 \leq j < k \leq m-1$, $(\alpha^{q^j})^{q^{m-k}} = (\alpha^{q^k})^{q^{m-k}}$。所以 $\alpha^{q^{m-k+j}} = \alpha^{q^m} = \alpha$。从

而 $f(x) \mid x^{q^{m-k+j}} - x$。由定理A.35，得 $m \mid m-k+j$ 与 $k > j$ 矛盾。

推论 A.5

设 f 是 F_q 上的 m 次不可约多项式，则：f 在 F_q 上的分裂域为 F_{q^m}。 ♡

证明：首先，定理A.36说明 f 在 F_{q^m} 中是分裂的。另一方面 $F_q(\alpha, \alpha^q, ..., \alpha^{q^{m-1}}) = F_q(\alpha) = F_{q^m}$。

推论 A.6

$F_q[x]$ 中同次不可约多项式有同构的分裂域。 ♡

定义 A.27

假设 F_{q^m} 是 F_q 的扩域，$\alpha \in F_{q^m}$，则 $\alpha, \alpha^q, ..., \alpha^{q^{m-1}}$ 称为 α 相对于 F_q 的共轭元。 ♣

定理 A.37

$\alpha \in F_q^*$ 相对于任意子域的共轭在 F_q^* 中有相同的阶。 ♡

证明：因为 $\circ(\alpha) \mid q-1$。但特征的任一次幂都与 $q-1$ 互素。所以 α^{q^i} 的阶都为 $\circ(\alpha)$。

推论 A.7

如果 α 是 F_q 的本原元，则 α 相对于任一子域的共轭元也是本原元。 ♡

例 A.18 $f(x) = x^4 + x + 1 \in F_2[x]$，$\alpha \in F_{16}$，则共轭元为 $\alpha, \alpha^2, \alpha^4 = \alpha + 1, \alpha^8 = \alpha^2 + 1$。但相对于 F_4 的共轭元的 $\alpha, \alpha^4 = \alpha + 1$。

A.3.3 元素的迹

定义 A.28 (迹的定义)

设 $\alpha \in F = F_{q^m}, K = F_q$，元素 α 的迹定义为 α 所有共轭元素之和，即：$Tr_{F/K}(\alpha) = \alpha + \alpha^q + , ..., + \alpha^{q^{m-1}}$. 若 K 是 F 的素子域，则称为绝对迹。 ♣

定理 A.38

设 $K = F_q, F = F_{q^m}$，则迹函数 $Tr_{F/K}$ 满足：

1) $\forall \alpha \in F$，$Tr_{F/K}(\alpha) \in K$.
2) $Tr_{F/K}(\alpha + \beta) = Tr_{F/K}(\alpha) + Tr_{F/K}(\beta)$.
3) $Tr_{F/K}(c\alpha) = cTr_{F/K}(\alpha), c \in K, \alpha \in F$.
4) $Tr_{F/K}$ 是 F 到 K 上的（作为 K 上向量之间的）线形变换。
5) $Tr_{F/K}(a) = ma, a \in K$.
6) $Tr_{F/K}(\alpha^q) = Tr_{F/K}(\alpha), \forall a \in F$

♡

证明 1) 不难验证 $(Tr_{F/K}(\alpha))^q = Tr_{F/K}(\alpha)$，所以 $Tr_{F/K}(\alpha)$ 是 $x^q - x$ 的根。

2) 和 3) 保证了 $Tr_{F/K}$ 为一线形变换，因此只需要证 $Tr_{F/K}$ 是满的，而这只要证明 $\exists \alpha \in F, Tr_{F/K}(\alpha) \neq 0$ 即可。考虑到 $Tr_{F/K}(\alpha) = 0 \Leftrightarrow \alpha$ 是 $x^{q^{m-1}} + ... + x^q + x$ 的根，而该多项式最多只有 q^{m-1} 个根而 F 中 q^m 个元素。因而命题得证。

定理 A.39 (迹的传递性)

假设 K 是一个有限域，F 是 K 的有限扩张，E 是 F 的有限扩张，则 $\forall \alpha \in E, Tr_{E/K}(\alpha) = Tr_{F/K}(Tr_{E/F}(\alpha))$。

♡

证明 设 $K = F_q, [F:K] = m, [E:F] = n$，则 $[E:K] = mn$。$\forall \alpha \in E$，我们有：$Tr_{F/K}(Tr_{E/F}(\alpha)) = \sum_{i=0}^{m-1}(Tr_{E/F}(\alpha))^{q^i} = \sum_{i=0}^{m-1}(\sum_{j=0}^{n-1}\alpha^{q^{jm}})^{q^i} = \sum_{i=0}^{m-1}\sum_{j=0}^{n-1}\alpha^{q^{jm+i}} = \sum_{k=0}^{mn-1}\alpha^{q^k} = Tr_{E/K}(\alpha)$

定理 A.40

设 F 是有限域 K 的有限扩张。把它们都看作 K 上的向量空间，则 F 到 K 的所有线形变换正好是如下形式的映射 $L_\beta, \beta \in F$。其中 $L_\beta(\alpha) = Tr_{F/K}(\beta\alpha)$。进一步，如果 β 和 γ 是不同的元素，则 $L_\beta \neq L_\gamma$。

♡

证明 根据定理A.38(3), L_β 是线形变换是显然的。假设 $\beta \neq \gamma$，由于 $Tr_{F/K}$ 是 F 到 K 上的线形变换，所以 $\exists \alpha \in F$ 使得 $Tr_{F/K}((\beta-\gamma)\alpha) \neq 0$。因此 $L_\beta(\alpha) - L_\gamma(\alpha) = Tr((\beta-\gamma)\alpha) \neq 0$。从而 $L_\beta \neq L_\gamma$。这样 $\{L_\beta : \beta \in F\}$ 共给出 q^m 个线形变换。但 $F \to K$ 仅有 q^m 个不同的线形变换。所以 $\{L_\beta : \beta \in F\}$ 就是线形变换的全体。

A.3.4 多项式的阶和本原多项式

引理 A.1

设 $f \in F_q[x]$ 是一次数大于 0 的多项式，$f(0) \neq 0$，则存在正整数 $e \leq q^m - 1$ 使得 $f(x) \mid x^e - 1$

♡

证明 剩余类环 $F_q[x]/(f)$ 含有 $q^m - 1$ 个非零元素。而 q^m 个剩余类 $x^j + (f), j = 0, 1, 2, ..., q^m - 1$ 是 q^m 个非零元素。所以存在 $0 \leq r < s \leq q^m - 1$，使得 $x^s \equiv x^r \ mod\ f(x)$，$x^{s-r} \equiv 1 \ mod\ f(x)$，即 $f(x) \mid x^{s-r} - 1$，$0 < s - r \leq q^m - 1$。

定义 A.29 (多项式的阶（周期）)

设 $f(x) \in F_q[x]$ 是一非零多项式。如果 $f(x) \neq 0$，则 $f(x)$ 的阶（order）定义为满足 $f(x) \mid x^e - 1$ 的最小的正整数 e。并记作 $ord(f)$。如果 $f(0) = 0$，则 $f(x) = x^h \cdot g(x)$。其中 $g(0) \neq 0$。这时 $f(x)$ 的阶定义为 $g(x)$ 的阶。

♣

定理 A.41

设 $f(x) \in F_q[x]$ 是一不可约多项式，阶数为 m。$f(0) \neq 0$，则 $ord(f)$ 等于 $f(x)$ 任一根在乘法群 $F_{q^m}^*$ 中的阶。

♡

证明 我们知道 F_{q^m} 是 $f(x)$ 的分裂域，而且 $f(x)$ 的所有根都有相同阶（因为所有的根都是共轭的）. 设 $\alpha \in F_{q^m}^*$ 是 $f(x)$ 的根，则根据定理A.34, $\alpha^e = 1 \Leftrightarrow f(x) \mid x^e - 1$，所以 α 的阶就是 $f(x)$ 的阶。

推论 A.8

设 $f(x) \in F_q[x]$ 是 F_q 上次数为 m 的不可约多项式，则 $ord(f) \mid q^m - 1$。

♡

证明：如果 $f(x) = c \cdot x, c \in F_q^*$，则 $ord(f) = 1$，所以定理成立。不然我们有 $ord(f)$ 等于 $f(x)$ 的根在 F_{q^m} 中的阶，所以应整除 $q^m - 1$。

第 A 章 习题

1. 编写一段程序实现扩展欧几里得算法。
2. 设 a_1，…，a_n，m 为非零整数，$m>1$，b 为任意整数。证明同余方程

$$a_1x_1+a_2x_2+\cdots+a_nx_n\equiv b \pmod{m}$$

有解 $(x_1,x_2,\ldots,x_n)$ 当且仅当

$$(a_1,a_2,\ldots,a_n,m)\mid b.$$

进一步地，在该条件满足时，其模 m 不同余的解的个数为

$$m^{n-1}\cdot(a_1,a_2,\ldots,a_n,m).$$

3. 二数余一，五数余二，七数余三，九数余四，问本数?

参考文献

1. 林东岱. 代数学基础与有限域. 北京：高等教育出版社 2006
2. 万哲先. 代数和编码（第三版）. 北京：高等教育出版社 2007
3. 冯登国，裴定一. 密码学导引. 北京：科学出版社 1999
4. 曹天杰，张永平，毕方明. 计算机系统安全（第二版）. 北京：高等教育出版社 2007
5. 冯登国，吴文玲. 分组密码的分析和设计. 北京：清华大学出版社 2000
6. [美]Wenbo Mao 著，王继林，伍前红等译. 现代密码学理论与实践. 北京：电子工业出版社 2004
7. [加]Alfred J.Menezes，Paul C.van Oorschot、Scott A.Vanstone 著，胡磊，王鹏等译. 应用密码学手册. 北京：电子工业出版社 2005
8. Douglas R.Stinson 著，冯登国译. 密码学原理与实践（第二版）. 北京：电子工业出版社 2003
9. 闵嗣鹤，严士健. 初等数论（第二版）. 北京：高等教育出版社 1982
10. 陈鲁生，沈世镒. 现代密码学. 北京：科学出版社 2002
11. [美]Bruce Schneier 著，吴世忠，祝世雄，张文政等译. 应用密码学协议、算法与 C 程序. 北京：机械工业出版社 2000
12. 孙淑玲. 应用密码学. 北京：清华大学出版社 2004
13. 杨波. 现代密码学. 北京：清华大学出版社 2003
14. [以色列]Oded Goldreich. *Foundations of Cryptography Basic Tools*. 北京：电子工业出版社 2003